

SUMMARIZED BY NOAM KIMHI
noam.kimhi@mail.huji.ac.il

Introduction to COMMUNICATION NETWORKS

YEAR 3 SEMESTER A

COURSE NO. 67594

2025-2026



Course Summary

מרצה: פרופ' יוסי גלעד

החומר המחייב הוא זה שנלמד בהרצאות ובתרגולים. סגל הקורס אינו אחראי לתוכן הסיכום או לנכונותו.

4	הרצאה 1
4	הקדמה
5	רשת טלפוניה
8	הרצאה 2
8	Packet Switching
13	תרגול 1
13	חזרה על הסתברות
18	הרצאה 3
18	מודל השכבות
22	תרגול 2
22	OSI Model
23	Random Access Protocols
24	ALOHA
29	שאלות תרגול
34	הרצאה 4
34	Random Access Protocols – ALOHA
36	Random Access Protocol – CSMA
39	כתובות MAC
41	תרגול 3
41	Random Access Protocols
41	ALOHA – Poisson Approach
44	שאלות תרגול
51	הרצאה 5
51	Wi-Fi ברשתות CSMA/CA
53	Switch
57	STP (Spanning Tree Protocol)
61	תרגול 4
61	CSMA/CD (Carrier Sense Multiple Access/Collision Detection)
66	CSMA/CD – Questions
69	Errors Detection and Recovery
74	הרצאה 6
74	IP Networks – IP רשתות
80	פרוטוקול DHCP
83	תרגול 5
83	Spanning Tree
85	Spanning Tree Protocol
87	שאלות בנושא Spanning Tree Protocol
96	הרצאה 7
97	DNS - Domain Name System
100	בעיות אבטחה
103	תרגול 6
103	Internet Protocol (IP)
106	Dynamic Host Configuration Protocol – DHCP
107	Address Resolution Protocol (ARP)

108	שאלות תרגול ARP ו-DHCP
109	Routers and Broadcast Domains
112	Domain Name System (DNS)
115	הרצאה 8
115	בעיות אבטחה – המשך
118	ARP (Address Resolution Protocol)
120	תרגול 7
120	Routing
121	Distance Vector Protocol
127	Link State Protocol
129	שימושים
130	הרצאה 9
130	Interconnecting IP subnets
134	Link-State Routing (LS) & Distance Vector Routing (DV)
138	Traffic Management
142	תרגול 8
142	Modeling Network as a Linear Problem
144	Equal Cost Multi-Path (ECMP)
147	שאלות לדוגמה
152	הרצאה 10
152	A closer look at ECMP
153	Transport Layer
154	Multiplexing
156	פרוטוקול TCP
160	תרגול 9
160	Reliable Transport
160	Stop & Wait Protocol
164	Go Back N
170	Selective Repeat
177	הרצאה 11
177	TCP – Timeout and ACKs
181	Congestion Control
189	תרגול 10
189	שכבת ה-Transport
190	User Datagram Protocol – UDP
190	Transmission Control Protocol – TCP
193	TCP Sliding Window
197	שאלות תרגול – TCP
200	Network Address Translation (NAT)
202	הרצאה 12
202	Congestion Avoidance
206	Queueing at Routers
206	Interdomain Routing
210	Border Gateway Protocol – BGP
214	תרגול 11
214	BGP
218	BGP Stability

222	הרצאה 13
222	A Closer Look at BGP
225	Inter vs. Intra
226	BGP Security

הקדמה

מצגת 1

מה נלמד בקורס?

קונספטים של רשתות, רעיונות ועקרונות. נתעסק במורכבות שבתקשורת מגישות של פרטיות, אבטחת מידע, ביצועים וצדדים אחרים.

העלינו כדוגמה את נושא בעיית טעינת סרטונים ברשת. דרך להתמודד עם הבעיה היא פרוטוקול שבו הלקוח מספר לשרת שהוא לא מצליח לטעון את הסרטון ולכן הלקוח יוכל לבקש מהשרת להוריד את איכות הסרטון, זה נקרא Quality of Experience (QoE).

אם הלקוח מנסה לקבל וידאו, השרת לפעמים לא יודע שהקליינט מסוגל להוריד מהר יותר. כשהלקוח עובר מקום, השרת צריך להתאים חזרה את קצב השידור שלו.

חלוקה לשכבות

אפיינו חלוקה ל-3 שכבות: אפליקציות, פרוטוקולי תקשורת, וטכנולוגיות חומרה.

יותר קל לבנות אפליקציות חדשות ולשנות חומרה (כמו שדרוגים לראוטרים) מאשר לשנות פרוטוקולי תקשורת. לוקח זמן לפרוס פרוטוקולים חדשים, ושינוי פרוטוקולים חדשים (כדוגמת השינוי בתקופה האחרונה מ-IPv4 ל-IPv6) אורך שנים רבות, אבל שינויים קורים. המטרה המרכזית היא להעביר מידע בין משתמשי קצה דרך רשת. אנחנו לא יכולים לבצע חיבור בין משתמשי קצה בצורה שהיא 1:1, אז יהיו צמתים שינתבו מידע והם מחוברים בלינקים (ראוטרים/סוויצ'ים). לינק יכול להיות פייבר אופטי, wireless, כבל Ethernet ומשתמשי קצה יכולים להיות שרת, פלאפון נייד, מחשב נייד וכד'.

ברטיס רשת – נותן יכולת לנקודת קצה להתחבר לרשת.

שקף 36

Jitter – האם יש שוני ב-latency בין הודעות (ראשונה נמסרה ב-50ms, אחריה ב-70ms,...)

שקף 37

מערכת מבוזרת – כמה מכשירי קצה שמחוברים לשרת.

שקף 40

Circuit Switching

שני משתמשים רוצים לתקשר. השולח מקבל מסלול ברשת שמוקצה לשיחה שלו. הוא יתקשר עם צמתיים ברשת, כל אחד יקצה לו משאבים ויאפשר לו להשתמש בלינק הבא. ייתכן מצב שאין משאבים פנויים ואז נקבל סטטוס "Busy". אם הצלחנו לייצר מעגל, השולח יכול להתחיל לשלוח מסרים מעל המעגל הקיים אל משתמש הקצה. הוא לא צריך לכתוב לאן הוא הולך, איפה שהמעגל מסתיים – לשם התקשורת הולכת. בסיום יש פירוק של המעגל שעליו נוצר הקשר.

שקף 42

בעבר היה חיבור אנושי לתקשורת טלפונית.

שקף 44-46

ב-Circuit Switching מתג = switch. כל ציר זו ישות אחרת ברשת, ציר γ מתאר את הזמן. מתחילים בניסיון להקים מעגל. יש כמה סוגי דילאי שהצגנו:

Propagation Delay – פער הזמן מהרגע שהביט הראשון של ההודעה נשלח, ועד שהוא התקבל ביעד.

מדובר במאפיין של התווך שדרכו המידע זורם (פייבר, Ethernet).

ההודעה היא לא ביט בודד, היא רצף של ביטים. לכן נדרשת הגדרה של דילאי נוסף:

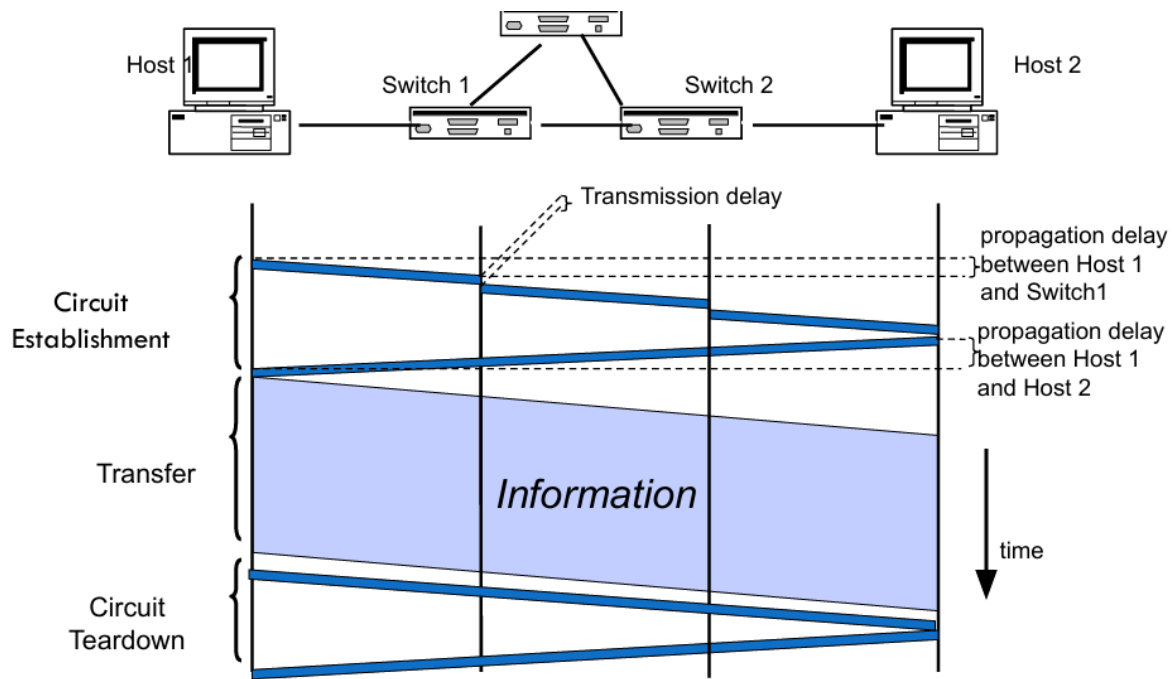
Transmission Delay – זמן שעובר מקבלת הביט הראשון ועד קבלת הביט האחרון.

זה מאפיין ששייך גם לתווך אבל גם לכרטיס הרשת.

שקף 47-50

עבשיו Host2 יכול לאשר ל-Host1 הודעה שהמעגל נוצר בהצלחה, ואז מידע עובר בחופשיות.

בסוף התקשורת צריך להרוס את המעגל כדי לשחרר את משאבי הרשת, לספר להם שהפורטים למעשה כבר לא בשימוש.

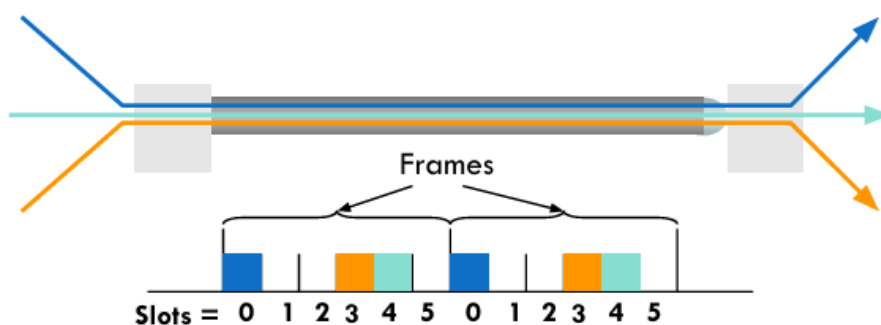


שקף 51-53

ייתכן שנרצה לחלוק לינקים בין כמה שיחות. ישנן כמה אפשרויות:

1. חלוקה בזמן – יש לינק, זמן השידור עליו רציף. נחלק את הזמן למקטעים. בכל מקטע נשדר קטע משיחה אחרת וכך נמשיך במעגל. איפה נמצאת השיחה האדומה בזמן שמוקצה לשיחה הכתומה? נדרש סנכרון בין כולם, או buffer שיחזיק את המידע.
2. להקצות מרחבים שונים בתדר

ב-Time Slotting אם למשל יש 6 משתמשים אז כל אחד מקבל חתך בזמן (מודולו 6). חייב לסנכרן כאן את השליחות, בעיקר לאור העובדה שאנחנו לא כותבים לאן המידע ממוען. השיטה הזו לא מתחשבת בכמות המידע של כל שיחה, ייתכן שהשיחה הכחולה למשל עמוסה יותר ותתעכב בלי סיבה מוצדקת בעקבות השימוש בשיטה זו.



יתרונות

- ברור כמה slots שמורים לי, וכמה זמן ייקח להודעה להגיע לצד השני.
- אין אובדן של מידע.
- ברגע שהקמתי מעגל השיחה תתקיים ללא תלות בעומס.

חסרונות

- **עמידות נמוכה בפני כשלים** – כשהקצו מעגל, חייב לעבור בו. אם לינק/צומת נופל בדרך, אין חלופה. הדרך היחידה לטפל בבעיה היא לבקש לסגור את המעגל הנוכחי ולהקים מעגל חדש.
- **בזבזי** – משלמים על סלוטים ומבזבזים סלוטים שלא נעשה בהם שימוש מוצדק. למשל, אפליקציה שבשיאה מגיעה ל-bandwidth בערך P , אבל בממוצע משדרת A , נהיה חייבים לשריין עבורה P כדי לאפשר לה לשדר בזמן עומס, אבל בפועל נקבל ניצולת נמוכה של המשאבים שהוקצו עבורה, ביחס של $\frac{A}{P}$. אפליקציות קלות הן בדרך כלל ביחס $P:A$ של 3:1, אבל יש אפליקציות עם burst שמגיע לשיאים של למעלה מ-100. לסוגים כאלה של אפליקציות (עם קצבי שידור משתנים בקיצוניות), circuit switching לא מתאים.
- **התאמה לשמע** – את הסוג הזה של מיתוג בנו עבור רשתות טלפוניה ולכן יש אופטימיזציה ל-voice והגעה לפי סדר. זה נוח שיש מסלול קבוע לרשתות טלפוניה. באופן כללי, נרצה שהדרך שבה נעשה מיתוג ברשת יתמוך בהרבה סוגי אפליקציות.
- **זמן setup גבוה** – צריך לשמור משאבים עד לייצור מעגל, קבלת אישור וכדומה – מדובר ב-latency של תחילת התקשורת. קיימים מקרים בהם יש שאילתה קצרה, אבל בגלל זמן ה-setup הגבוה היא תלווה בתקורה גבוהה כשהשירות הנדרש (החלק העיקרי של התקשורת) הוא קצר מאוד.

הרצאה 2

מצגת 1

שקף 62-66

אפשר לחלק תקשורת ל-2 סוגים:

- (1) Broadcast – כמה מחשבים על bus משותף, כולם כותבים וקוראים מאותו ערוץ תקשורת. אין דרך לשלוח למישהו הודעה מבלי שזה יגיע לאחרים. Wi-Fi פועל באופן דומה. מה המגבלות בשיטה זו? הגבלות טווח (יש עניין של קרבה), בעייתיות כשיש כמה שרוצים ליזום תקשורת באותו זמן על פני אותו bus, בעיית פרטיות – כולם מקבלים את כל ההודעות.
- (2) Switched – באמצעות ניתוב ברשת. דרך לקבוע שהודעות יעברו בערוץ אחד ולא באחר. נתקלנו ב-circuit switching בשיעור הקודם לפי זמן או לפי תדר, היום נדבר על סוג נוסף – **Packet Switching**.

Packet Switching

שקף 67-68

פקטה – פיסת מידע שנשלחת ברשת ומכילה תדר שמסייע במיתוג – להביא את ההודעה ליעדה ולא לאחרים. מדובר בשינוי קונספטואלי מ-circuit שלא הכיל header מזהה.

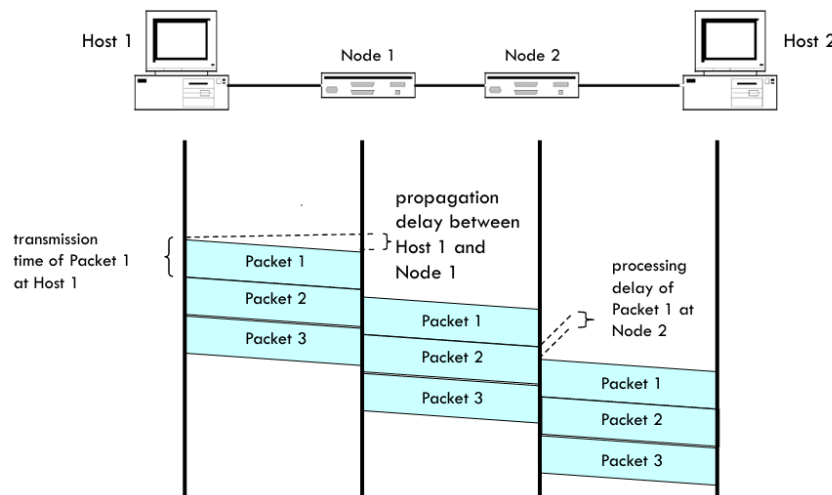
ב-Payload ימצא הדאטה שאנחנו מעבירים, וב-Header הוראות לרשת איך לטפל ב-Payload. כל פקטה תטופל בנפרד על ידי הרשת. זה נוח כי לא צריך מראש להקצות משאבים ומסלול.

שקף 69-72

באזור Host1 רוצה לשלוח הודעה. אין לנו איך למזער propagation delay כי מדובר בזמן שבו לביטים לוקח לעבור לצד השני (בפייבר יהיה מעט, בלווייני יהיה הרבה). כמו כן, קיים עדיין transmission delay על הזמן שלוקח לפקטה שלמה לעבור. השוני הוא שנוסף עכשיו גם סוג דילאי נוסף.

Processing Delay – הזמן שלוקח לצומת בדרך לעבד את חלקיה השונים של הפקטה.

בגלל צורת העטיפה של הפקטה, נדרש צומת שידע לעבד את כל אחת מההודעות (לקרוא header וכדומה).



שקף 73-75

באיור Host A מעוניין לשלוח הודעה למשל ל-Host D. ב-circuit לא היה מותר לנו להשתמש בשני מסלולים שונים, אבל כאן מותר לבחור ביותר ממסלול אחד. זה עלול ליצור בעיה שההודעות לא יגיעו ליעדן בסדר הנכון ונצטרך להתמודד עם זה.

בחירת פקטה

כמה ארוכה פקטה צריכה להיות? קטנה יחסית, כדי שאובדן של פקטה בודדת לא יהיה מאוד קריטי. צמתים ברשת גם צריכים לקבל פקטה שלמה כדי להתחיל לבצע עיבוד, ואם הפקטה תהיה ארוכה מאוד יהיה "זמן מת" שבו מחכים לביטים שבסוף הפקטה – אילו כל פקטה הייתה גדולה היה לנו יותר transmission delay. מצד שני, הפקטה לא יכולה להיות ממש קטנה כי לכל payload נוסף גם header ואז אם ה-payload קטן נקבל תקורה גבוהה על עיבוד headers.

שקף 79

בעבודה בפקטות לא שומרים על סלוטים קבועים ל-flow מסוים. אם אחד מה-Hosts שולח הרבה הם פשוט יתקבלו ב-burst בצד השני. יש בשיטה זו יכולת טובה יותר להתמודד עם bursts כי המצב דינאמי ולא קבוע מראש.

מה קורה כשמישהו שולח הרבה מידע והוא מגיע לצומת הבא?

אופציה 1: הצומת מספיק מהיר כדי לנתב את כל המידע לצומת הבא.
אופציה 2: יש עומס בנקודה מסוימת, אין ברירה וצריך להעביר את התקשורת דרך אותו צומת. מותר לזרוק חלק מהפקטות ובתקווה השולח יבין שחלק מהמידע חסר וישלח אותו מחדש אם המידע שאבד היה חשוב.
דרך טובה יותר היא הוספת buffer. אם יש עומס רגעי ההודעות ייכתבו לתוך buffer וכשה-burst נגמר העודף ישתחרר. חשוב לציין ש-buffer לא פותר את הבעיה לחלוטין, כי לכל buffer שנבחר תמיד יהיה burst שיגרם לו להתמלא ונאלץ לזרוק הודעות. כמו כן, buffer יוצר בעייתיות נוספת כאשר נשלח הודעות שנשמרו בתוכו, למרות שהתזמון שלהן כבר לא רלוונטי יותר ל-flow הנוכחי. זה נקרא **queuing delay**, כי ההודעה נתקעה מאחורי תור ארוך.

שקף 84-86

אפשר לחלק frames ל-slots. נניח שיש flow של תקשורת עם שיא P וממוצע A . אם יש רק flow אחד נצטרך להקצות (לתפוס) P סלוטים, כי ה-buffer צריך להיות מסוגל לספוג מידע בגודל P . הבעיה היא כשה-peak יותר גדול מהממוצע, וזה מה שיקבע עלויות בספיות.

לעומת זאת, אם מתסכלים על הרבה flows במקביל, אפשר להרוויח **מחוק המספרים הגדולים**. נדמיין מצב עם הרבה flows שמתקשרים דרך אותו צומת וקצב בין 0 ל- P עם אותו ממוצע. אם כולם בלתי תלויים, אז סביר

להניח שלא כולם יגיעו לשיא באותה נקודת זמן – זה מאפשר לנו לחסוך בגודל ה-buffer. בגדול, נקבל ערך שקרוב יותר לממוצע ככל שיש יותר flows, ואז הדיוק יהיה טוב יותר.

אם צריך לתכנן צומת ברשת, נקצה קצת יותר מהממוצע $N \times A$, וזה נעשה מדויק יותר ככל ש- N (מספר ה-flows) גדל.

השוואות נוספות ל-Circuit Switching

שקף 87-89

יתרונות של Circuit Switching

- הבטחה על bandwidth שנקבל
- ביצועים שניתן לחזות
- אבטסרקציה פשוטה להבנה למי שכותב את האפליקציה מעל הרשת
- אין בעיית תזמון להודעות (הודעות יגיעו לפי סדר שליחתן)
- Forwarding פשוט יותר עבור מי שמכין את ציוד התקשורת.
- כשנוצר מעגל הוא מקצה מקום – אין parsing ואין buffers.
- אין header, כלומר תקורה נמוכה יחסית על ביטים.

חסרונות של Circuit Switching

- בזבז של bandwidth – הקצינו מסלול עבור מישורו והוא לא תמיד ינוצל.
- תעבורה ב-burst תדרוש הקצאת מקום לפי ה-peak.
- לא תמיד נוכל לאפשר תקשורת "סבירה" לאחד ו"טובה" לאחר.
- צריך לשמור מצב, זה גורם לבעיה ולנפילות.
- כשצומת נופל – כל מי שעבר דרכו בבעיה.
- תקלות לוקחות זמן לזיהוי ולתיקון (ייצור מסלול מחדש).
- יש setup delay שיכול להיות גבוה (מפריע בעיקר כשההודעה קצרה).

Packet vs. Circuit

אותם צמתים ברשת לא שומרים מידע לטווח ארוך על מה שעובר דרכם – אם צומת נופל הכשל הוא לא לכל החיבור ב-Packet Switching.

מבחינת יעילות משאבים – מנצלים ב-Packet נתונים סטטיסטיים כדי לתת ביצוע טוב יותר בצורה חסכונית.

מבחינת deploy – בצמתים שאנחנו מחוברים אליהם לא צריך להבטיח דבר על האיכות מראש.

החיסרון של Packet – ניתקל במצבים שלא יכולנו להגיע אליהם לפני, כמו עומס על צומת ספציפי, ובעומס

הזה צריך לטפל. בעיות של חוסר סדר מקצה לקצה (הודעה לא תמיד תגיע בסדר שבו נשלחה). בגלל ההקצאה

הדינמית הכל תלוי באיך אחרים משתמשים ברשת, לעומת circuit שבה יש בידוד מאחרים.

מצגת 2 שקף 2

מטרת העל בקורס היא לבנות רשת גדולה ומורכבת. שיטת המיתוג היא עקרון בסיסי ברשת. צריך לבנות באופן מודולרי – כמה רכיבים. כל אחד ייתן שירותים לרכיבים אחרים. בזה שנתכנן רכיבים שונים למשימות שונות נסתיר מורכבות, וגם נוכל לשנות את המימוש הפנימי כל עוד נשמור על אותו ממשק.

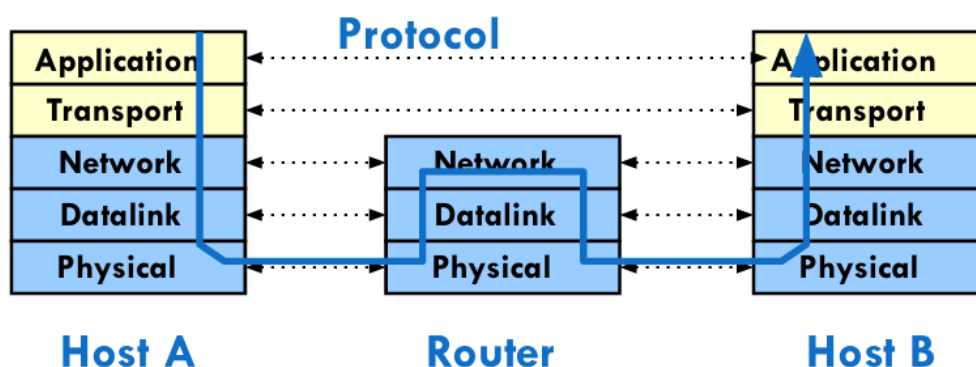
שקף 4

נחלק לשכבות – כל אחת תספק שירותים לשכבה שמעליה. נוחות בתכנון. הבעיה – יכול למנוע מאיתנו לבצע אופטימיזציות שהיינו יכולים לבצע אילו שברנו את מודל השכבות. אבל, במערכת מורכבת כמו האינטרנט – זה ה-tradeoff שצריך לבצע.

שקף 7

Layering

אילו משימות יש לנו? **מטרת העל** – העברת מידע שימושי בין משתמשי קצה. הכי בסיסי – העברת אלקטרוניק על ערוץ תקשורת, לפרש אותם בביטים, אחר כך כפקטות, ואותן נרצה להעביר לנקודות קרובות יחסית ברשת לוקאלית (LAN). בהמשך, העברת פקטות בין LANs ויודא שהמידע הגיע לצד השני (לטפל באובדן) ומטרת העל – ביצוע שימוש במידע.



שקף 8

את המשימות שמנינו נחלק לשכבות שונות, כל אחת אחראית על 1-2 משימות. לכל שכבה הולך להיות ממשק לשכבה שמעליה, ולא מסתכלים על איך השכבה הקודמת מומשה.

שקף 13

איך זה עובד בתקשורת מחשבים? 2 אפליקציות של Hosts שונים מנסות לתקשר. האפליקציה וה-transport מממשות רק בקצוות, השאר גם בציר שבדרך. כל שכבה מתקשרת עם השכבה המקבילה לה בצד השני. מבחינתם לא מעניין מה קורה באמצע. ב-Transport יש בקרת עומסים (לדעת אם הודעות הגיעו, לא הגיעו, להעלות קצב, להוריד..). כל שאר הדברים לא מעניינים אותה.

שקף 14-17

התקשורת מתבצעת באמצעות פרוטוקול:

יש לפרוטוקול צד סינטקטי – x ביטים ראשונים מתייחסים ל..

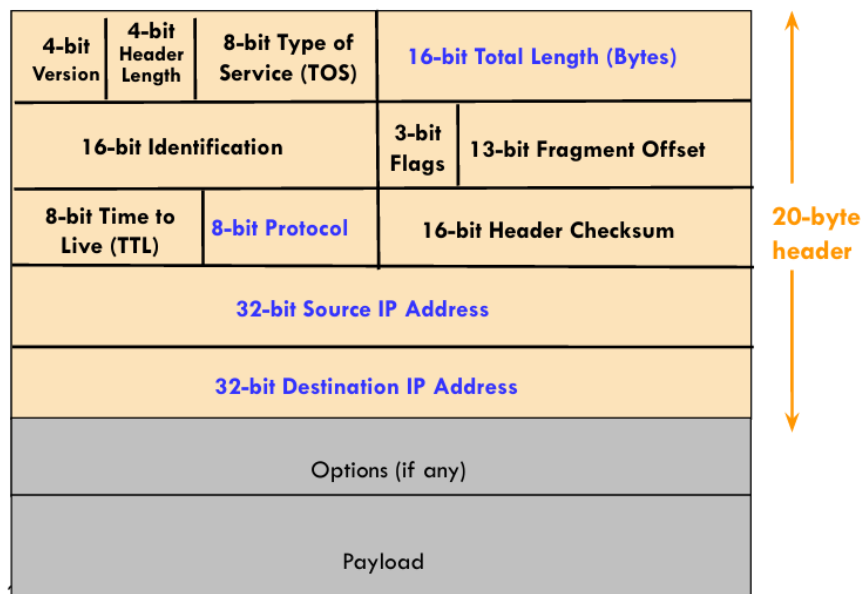
סמנטי- מה עושים עם זה? שכבת Transport אותה על בעיה, מה עושים איתה זו סמנטיקה. יש פרוטוקולים שונים להתמודדות, למשל הורדת קצב האיתות וכדומה.

שקף 18

אי אפשר לתקשר בדרך שאינה הפרוטוקול – יש מוסכמות.

שקף 19

מבנה פקטת IP (פרוטוקול של שכבת ה-Network):



החלק הכתום הוא ה-header. כשראוטר מקבל הודעת IP הראוטר יבניס אותה למעין struct. המימוש של IP יסתכל על כתובת היעד וינסה להבין לאן לנתב אותה.

החלק האפור הוא מידע אופציונלי שיכול להתווסף ל-transport header, כשמעבירים את החבילה הלאה התוכן שלה לא מעניין כלל.

המטרה של שכבת ה-network זה להעביר את ה-payload שלה, של שכבת ה-data link זה להעביר את שלה.. ובך הלאה.

שקף 21

בגלל שיש לנו ציוד תקשורת שמיוצר על ידי גופים שונים, ואפליקציות שנכתבות על ידי גופים שונים – חשוב לשמור על סטנדרט מסוים שיעזור לכולם לשמור על דיבור ב"אותה שפה". IETF – גוף תקינה שאחראי לייצר את הסטנדרטים האלו. הסטנדרטים מפורסמים בתוך RFCs (Request for Comments).

חזרה על הסתברות

התרגול יעסוק בחזרה על קונספטים חשובים מהסתברות שישמשו אותנו לאורך הקורס, ולא חומר תיאורטי ברשתות תקשורת. לאורך הקורס נשתמש בהסתברות לפתירת בעיות שונות. דוגמה פשוטה לכך היא הסתברות לשליחה פקטה או יעילות.

תכונות חשובות (תכונות נוספות במצגת)

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \Leftrightarrow \mathbb{P}(A \cap B) = \mathbb{P}(A|B) \cdot \mathbb{P}(B)$$

$$\mathbb{P}(A|B) = \mathbb{P}(A) \stackrel{\text{מאורעות זרים}}{\Rightarrow} \mathbb{P}(A \cap B) = \mathbb{P}(A) \cdot \mathbb{P}(B)$$

$$\mathbb{P}(A \cup B) \leq \mathbb{P}(A) + \mathbb{P}(B) \quad (\text{אם המאורעות זרים, אפשר להחליף את } \leq \text{ ב-} =)$$

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(B|A) \cdot \mathbb{P}(A)}{\mathbb{P}(B)}$$

כאשר $\bigcup_{i=1}^n B_i = \Omega$ נוכל להפעיל הסתברות שלמה ולקבל:

$$\mathbb{P}(A) = \sum_{i=1}^n \mathbb{P}(A|B_i) \cdot \mathbb{P}(B_i)$$

מניחים שפקטה "טובה" אם היא שורדת ברשת לפחות T שניות בהסתברות e^{-T} . פקטה "לא טובה" אם היא שורדת ברשת לפחות T שניות בהסתברות e^{-1000t} .

שאלות

1. נניח שפקטה בודדת נוצרה בזמן $T = 0$, מה ההסתברות שהיא עדיין קיימת ברשת בזמן $T = t$?

פתרון:

נגדיר מאורעות:

$A = \text{a packet survived for } t \text{ seconds}$; $B = \text{a good packet was created}$

$$\mathbb{P}(A) = \mathbb{P}(A|B) \cdot \mathbb{P}(B) + \mathbb{P}(A|\bar{B}) \cdot \mathbb{P}(\bar{B}) = e^{-t} \cdot p + e^{-1000t} \cdot (1 - p)$$

2. ידוע לנו שפקטה שרדה ברשת למשך $T = t$ שניות. מהי ההסתברות שזו הייתה פקטה טובה?

פתרון:

נפעיל את כלל בייס ונקבל:

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(A|B) \cdot \mathbb{P}(B)}{\mathbb{P}(A)} = \frac{e^{-t} \cdot p}{e^{-t} \cdot p + e^{-1000t} \cdot (1-p)}$$

[תוחלת ושונות](#)

$$\mathbb{E}(X) = \sum_i x_i \cdot \mathbb{P}(X = x_i)$$

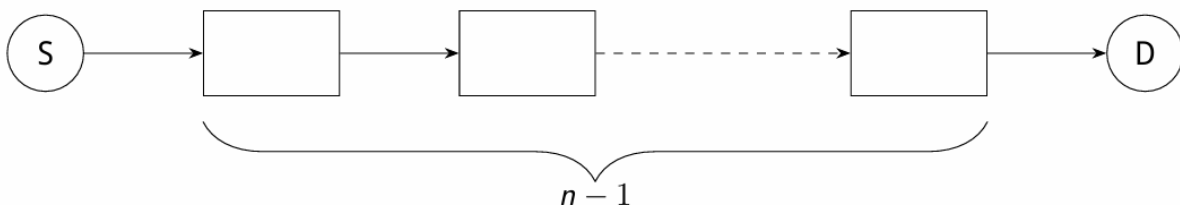
$$\mathbb{E}\left(\sum_i a_i \cdot x_i\right) = \sum_i a_i \cdot \mathbb{E}(x_i)$$

$$\text{Var}(X) = \mathbb{E}(X - \mathbb{E}(X))^2 = \mathbb{E}(X^2) - \mathbb{E}(X)^2$$

	Bernoulli	Binomial	Geometric	Poisson
Intuition	We make an experiment, and we could either fail or succeed	We make n independent Bernoulli experiments. We mark by X the sum of successes	We do independent Bernoulli experiments until we succeed. We denote by X the number of tries	Counting the number of events that occurred during some period of time
Probability mass function	$P(X=1) = p$, $P(X=0) = 1-p$	$P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}$	$P(X=k) = (1-p)^{k-1} p$	$P(X=k) = \frac{\lambda^k}{k!} e^{-\lambda}$
Notation	$X \sim \text{Ber}(p)$	$X \sim \text{Bin}(n, p)$	$X \sim \text{Geo}(p)$	$X \sim \text{Pois}(\lambda)$
Exp.	$\mathbb{E}(X) = p$	$\mathbb{E}(X) = np$	$\mathbb{E}(X) = \frac{1}{p}$	$\mathbb{E}(X) = \lambda$
Var.	$\text{Var}(x) = p(1-p)$	$\text{Var}(x) = np(1-p)$	$\text{Var}(x) = \frac{1-p}{p^2}$	$\text{Var}(x) = \lambda$

[תפאורה לשאלות](#)

- We want to send a message from node **S** to node **D** which are connected by a chain of **n** links.
- The probability of any link to fail is p (independently).
- At time $T = 0$, **S** sends a message to **D**



1. מה ההסתברות שהצומת D קיבלה את ההודעה?

פתרון:

מדובר בהסתברות שכל אחד מהצמתים הצליח להעביר את ההודעה. יש n צמתים, כל אחד יכול להיכשל בהסתברות p כלומר להצליח בהסתברות (המשלימה) של $1 - p$. סך הכל מאחר שהמאורעות בלתי תלויים אפשר לכפול אותם ונקבל שההסתברות שההודעה תגיע ליעד היא $(1 - p)^n$.

2. הניחו שאם ההודעה לא מגיעה ל-D, אז S תשלח אותה שוב ושוב עד להצלחה. מה תוחלת הפקטות שישלחו עד שפקטה תגיע ליעד?

פתרון:

נגדיר את X להיות "מספר הפקטות שיש לשלוח מ-S עד שפקטה מגיעה ל-D בהצלחה". נבחין ש- X מתפלג גיאומטרית ($X \sim \text{Geo}((1 - p)^n)$) (כי כל פקטה אינה תלויה באחרת) ולכן לפי תוחלת של מ"מ המתפלג גיאומטרית נקבל:

$$\mathbb{E}(X) = \frac{1}{(1 - p)^n}$$

3. עכשיו, נניח שכל צומת ממשיך לשלוח ללא הפסקה את ההודעה לצומת הבא עד שההודעה מתקבלת. הניחו שהצומת הראשון (S) שולח פעם אחת בלבד. מה ההסתברות שההודעה תגיע ל-D?

פתרון:

מאחר שכל צומת פרט ל-S שולחת את ההודעה עד שהיא מתקבלת, ההודעה תתקדם לצומת הבא בהסתברות 1 (תמיד). התלות היחידה היא בצומת S, אשר מצליחה בהסתברות $1 - p$, ולכן זו התשובה.

4. מה תוחלת מספר הפקטות שצומת בודד ישלח עד להצלחה?

פתרון:

נגדיר מ"מ X שמתפלג גיאומטרית $\text{Geo}(1 - p)$ ולכן לפי הנוסחה:

$$\mathbb{E}(X) = \frac{1}{1 - p}$$

משתנים רציפים

$$\mathbb{P}(a \leq x \leq b) = \int_a^b f(x)dx \quad ; \quad \int_{-\infty}^{\infty} f(x)dx = 1$$

$$\mathbb{E}(X) = \int_{-\infty}^{\infty} x \cdot f(x)dx \quad ; \quad \text{Var}(X) = \int_{-\infty}^{\infty} (x - \mathbb{E}(X))^2 f(x)dx$$

	Uniform	Exponential	Gaussian (Normal)
Notation	$X \sim Uni(a, b)$	$X \sim Exp(\lambda)$	$X \sim N(\mu, \sigma)$
probability distribution function	$f(x) = \begin{cases} \frac{1}{b-a} & x \in [a, b] \\ 0 & else \end{cases}$	$f(x) = \begin{cases} \lambda e^{-\lambda x} & x > 0 \\ 0 & else \end{cases}$	$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
Exp.	$\mathbb{E}(X) = \frac{a+b}{2}$	$\mathbb{E}(X) = \frac{1}{\lambda}$	$\mathbb{E}(X) = \mu$
Var.	$Var(X) = \frac{(b-a)^2}{12}$	$Var(X) = \frac{1}{\lambda^2}$	$Var(X) = \sigma$

[תכונת חוסר הזיכרון](#)

למשתנים מקריים גיאומטריים ואקספוננציאליים יש תכונת חוסר זיכרון:

$$\mathbb{P}(X > s + t \mid x > t) = \mathbb{P}(X > s)$$

כלומר לא משנה "כמה פעמים ניסינו".

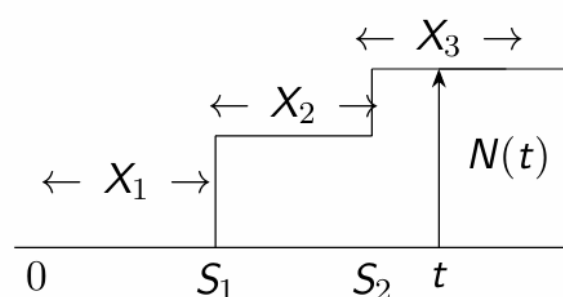
[תהליך רנדומלי סטוכסטי](#)

הטלת מטבע n פעמים זה תהליך סטוכסטי של אירועים ברנוליים. בזמן n אפשר להסתכל על זה ולהגיד שזה מתפלג כמו הסתברות בינומית. שימושי למשל עבור נתבים שמקבלים פקטה ולא ישר שולחים אותה הלאה אלא מחכים כמות זמן רנדומלית (מסיבות שנראה בהמשך). מבחינה ויזואלית:

Define

- $S_1 = X_1$
- $S_2 = S_1 + X_2$
- $\vdots$
- $S_i = S_{i-1} + X_i$

S_i is called an "arrival epoch" (the specific time at which an event occurs) $N(t) = n$ for $S_n \leq t < S_{n+1}$



תהליך ספירה

תהליך ספירה זה תהליך רנדומלי שסופר את מספר ה-epochs עד נקודת זמן t . הוא מוגדר לפי $\{N_t | t \geq 0\}$ כאשר:

$$\begin{aligned} N_0 &= 0 \\ N_t &\geq 0 \\ s \leq t &\implies N_s \leq N_t \text{ (monotonicity) and if } s < t \text{ then } N_t - N_s \text{ is the} \\ &\text{number of events that occurred in } (s, t] \end{aligned}$$

דוגמה: מספר הלקוחות שמגיעים לחנות.

תהליך פואסוני

תהליך פואסוני עם קצב λ הוא תהליך ספירה אבל בנוסף מקיים מספר תכונות:

- Independent increments: Number of events in any two disjoint intervals is independent
- Stationary increments: The number of events in any interval of length t is a Poisson random variable with parameter λt (depends on the interval's **length** and not on its timing). ie. $\mathbb{E}[N_t] = \lambda t$
- No bunching: The probability of > 1 arrivals in a tiny interval is negligible

האם מספר האנשים שמגיעים לתחנת אוטובוס הוא תהליך פואסוני? **לא**. כי בנקודת זמן אחת יכולים לעלות כמה אנשים לאוטובוס, ובשעות מאוחרות משמעותית פחות אנשים יעלו.

שאלה

בהינתן טווח T ששולחים בו פקטות בהתפלגות פואסונית עם פרמטר gT :

g זו ההסתברות שנשלח פקטה, T זה משך הזמן. מה ההסתברות שנשלח פקטה אחת?

פתרון:

תהליך פואסוני מתפלג באופן הבא:

$$X_p \sim \text{Poi}(gt) \quad \text{and} \quad \mathbb{P}_{t=T}(X = i) = \frac{(gT)^i}{i!} e^{-gT}$$

$$\mathbb{P}_{t=T}(X = 1) = gT \cdot e^{-gT}$$

מודל השכבות

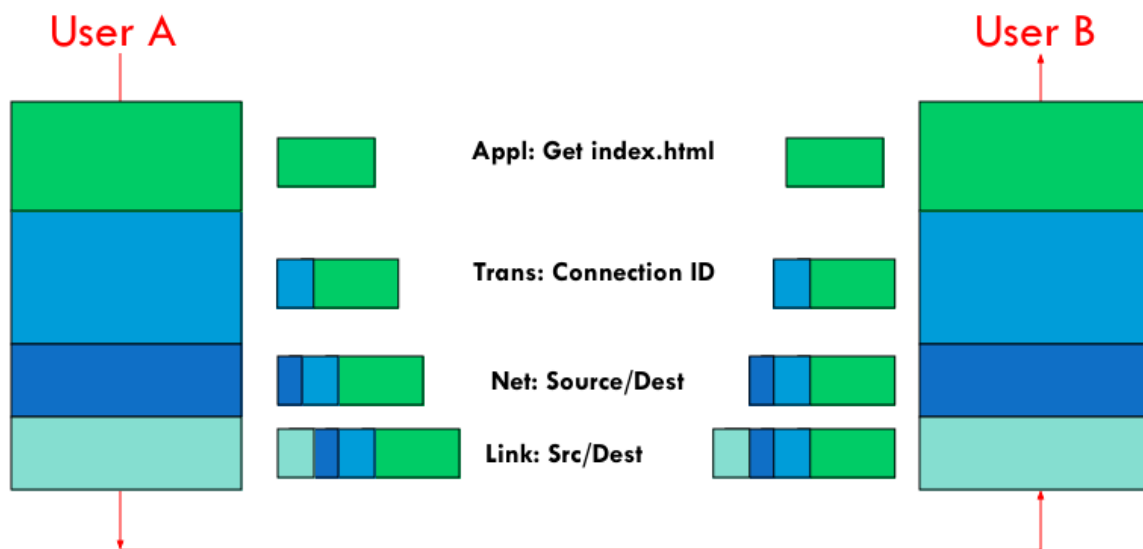
מצגת 2
שקף 13

דוגמה למימושים של שכבות שונות. למשל יש host שמכיל מימוש של https client והוא רוצה לדבר עם web server כדי לבקש עמוד web. מה שהדפדפן (האפליקציה) עושה זה לכתוב TCP. יוצר חיבור TCP בין הדפדפן לשרת web.

כל שני צמתים שבנים צריכים לתקשר באותה דרך, למשל Ethernet ל-Ethernet, אופטי לאופטי. כל שכבה מוסיפה header משלה.

שקף 23

כאן רואים שכל שכבה מוסיפה את ה-header שלה, נפוץ שסך כל ה-headers הוא 54 בתים. שרת ה-web למשל רואה רק את get index.html, אבל כרטיס הרשת קורא שכבות נוספות.



**Common case: 20 bytes TCP header + 20 bytes IP header
+ 14 bytes Ethernet header = 54 bytes overhead**

שקף 24

כל שכבה נותנת שירות מסוים ו-interface לשכבה שמעליה לקרוא לה. פרוטוקול זו הדרך שבה אנחנו מתקשרים בין מופעים קרובים זה לזה של אותה שכבה. כך אפשר לממש בדרכים שונות כל שכבה מבלי להשפיע על שכבות אחרות, כל עוד נצמידים לממשק זהה.

השכבה הפיזית/Physical Layer – אחראית על העברת ביטים על אות wire. פרוטוקולים אומרים איך לייצג את אותו ביט, ויש פרוטוקולים שונים לסוגים שונים של שכבות פיזיות.

שכבת הלינק/Data Link Layer – מטרתה להעביר הודעות שנקרא להן frames. אלו כתובות לוקאליות של מכשירי רשת (כתובות ברשת הלוקאלית) כלומר מוכרות רק בתוך הארגון ולא זמינות כלפי חוץ. הפרוטוקול אחראי על MAC (Media Access Control) – נרחיב על כך בהמשך.

שכבת הרשת/Inter Network Layer – תפקידה להעביר מידע בין רשתות על פני כמה bus-ים. כדי להעביר את ההודעה כל מכשיר רשת בדרך השתמש שוב בשכבת הלינק. מבחינת Network והלאה שולחים את אותו דבר, אבל ה-frame בכל פעם מעט שונה (payload זהה). ה-source הופך להיות הראוטר הנוכחי וה-destination הצומת הבא (ראוטר נוסף או היעד). למה אין כמה סוגי פרוטוקולי רשת? מדוע רק IPv4 (ובמשך שנים מנסים להחליף ל-IPv6)?

Transport Layer – בתקשורת בין תהליכים בתוך אותו מחשב, בין אפליקציות באותה מערכת. השכבה מוסיפה מזהה להודעה ב-scope של אותו מכשיר (שולח/מקבל), TCP ו-UDP, אבטחות מקצה לקצה, התאוששות מהודעות שהלכו לאיבוד, לדאוג שלא נשלח יותר ממה שהמקבל יודע לקבל או שהרשת מצליחה להעביר. אבסטרקציה ל-stream של מדיע. לא תמיד בתור המפתח קל להעביר 1000 בתים כאלו ואז 1000 בתים אחרים, נדרשת מעטפת (ממשק read/write). כותבים לתוך buffer ואז שכבת transport תחלק אותם לפקטות ותשלח, וכך גם בצד המקבל.

שכבת האפליקציה/Application Layer – כל שירות למשתמש הקצה, והממשק הוא תלוי אפליקציה. דוגמאות לשכבה זו הן כל קטע קוד שמשתמש בשכבת transport, כלומר כל אפליקציה שמשתמשת ברשת.

ראינו מימושים שונים לשכבות שונות. שכבת ה-Network אין לה מגוון פרוטוקולים, מדוע? כי היא אחראית על אינטרקציה בין רשתות, כלומר צריכים לדעת לדבר באותה שפה. מנסים כבר 20 שנה להחליף את IPv4 ל-IPv6 – זה קורה, אבל לאט מאוד. כל פרוטוקול אחר מערב הרבה פחות גורמים בלתי תלויים, למשל data link זה מערב רק את הצומת הנוכחי והבא בתור. מעבר בפרוטוקול Network קשה יותר – זה שהמחשב שלי ידבר בשפה IPv6 לא יעזור בלי שהראוטר יודע לנתב אותו למקום הנכון.

אתגר – שליחת הודעות מעל רשת מקומית. נתחיל משליחת הודעות מעל ערוץ אחד. אנחנו לא נדבר הרבה על שכבת האפליקציה ועל השכבה הפיזית. השיעור הזה נתמקד ב-Data Link.

שקף 33

כמה מחשבים שמחוברים לכבל אחד ארוך.

שקף 36

בשכבה 2 – ההודעות נקראות frame עם header של data link ו-payload של transport וכו'.

שקף 37-38

כל ראוטר מחובר בממשק לראוטר הבא בלינק. כרגע נתמקד בלינק בודד. רוצים לתאם שליחה על הלינק המשותף בין כל המחשבים שיש לנו ברשת. לצורך כך נדרש MAC Address – כתובת בשכבה 2 שנקבעת על ידי כרטיס הרשת. לכל כרטיס יש כתובת MAC שנצרכה עליו, והכתובת הזו רלוונטית רק לרשת המקומית.

מה נרצה להשיג? נרצה לדאוג שפריימים יישלחו כמו שצריך, ותחילה נרצה להימנע משגיאות. שגיאות עלולות להתרחש אם שני מחשבים שולחים מידע באותו הזמן, צריך לדעת להימנע ממקרים כאלה או לדעת לטפל בהם אם יקרו.

שקף 39

השכבה ממומשת בכרטיס הרשת שלנו. השכבות שמעל ממומשות ב-OS או באפליקציות שהיא מריצה ומשתמשת ב-CPU. את העיבוד של שכבה 2 (שכבת הלינק) נבצע בכרטיס הרשת שלנו. הוא יבצע הוספת header של הדאטה ושליחה של ה-frame. אחראי על תקשורת מול ה-OS.

שקף 40

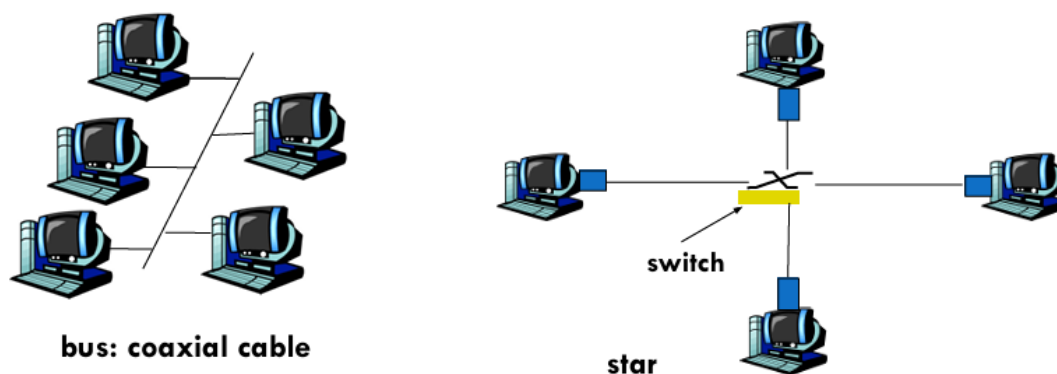
בצד המקבל קוראים את הפריימים אחד אחרי השני באמצעות parsing של הפריימים ובחינת כתובות. אם ממוען אלינו, יעבור הלאה. הכרטיס יכול לבחון גם corruption באמצעות checksum – סכימת בתים ובדיקה האם כמות הבתים שווה למה שמופיע בשדה checksum של ההודעה שהתקבלה. אם מספר הבתים לא תואם, ההודעה תיזרק.

שקף 41

נביח שיש ערוץ אחד והרבה שולחים. איך נתאם את השליחה ביניהם? שליחה מקבילית תהרוס את השידור. הקושי הוא שאין ערוץ נוסף לתאם את התקשורת, הערוץ היחיד הוא זה שמעבירים עליו את ההודעות. היינו רוצים פרוטוקול שיבטיח לנו כמה הודעות נצליח לשלוח בלי שתהיה התנגשות.

שקף 42

כרגע אנחנו מתעסקים ב-coaxial cable, אבל בהמשך נסתכל גם על switch שם יש מיתוג ולכן נקבל פחות התנגשויות: מתאפשר מקבול בזכות ה-switch שדואג שהודעות שונות לא תשלחנה על אותו הקו באותו הזמן.



שקף 43

ב-wireless בניגוד ל-wired הסיגנל הולך ודועך כתלות במרחק ומכשולים גיאוגרפיים.

שקף 44

למה אנחנו יכולים לקוות? נניח שיש לנו ערוץ עם capacity של R ביטים בשנייה. היינו רוצים שאם יש לנו רק שולח אחד אז הוא יכול לנצל את כל ה-capacity שלו ולשלוח בקצב R . אם יש יותר משולח אחד, למשל m שולחים – אז שכל אחד יקבל $\frac{R}{m}$. השאיפה היא שזה יקרה בלי גורם שיהיה אחראי על תיאום אלא באמצעות אלגוריתם לוקאלי שיגרום לכך שלא נקבל התנגשויות עם אחרים. כמובן, רצוי אלגוריתם פשוט – כי עלינו לממש אותו פיזית בחומרה של כרטיס הרשת.

שקף 48

בשיעור הבא נדבר על משפחת פרוטוקולים Random Access Protocols שמשתמשים בהטלת מטבע בפרוטוקול שמטרתו להתמודד עם התנגשויות.

OSI Model

מודל (OSI (Open-System Interconnect של 7 או 5 שכבות (תלוי בצורת חלוקה) שמתאר מה התפקיד של כל שכבה, והתעסקות בכל שכבה בנפרד. מטרת המודל היא שמחשב אחד יתקשר עם השני, תוך אפשרות להתעלם מהאחריות של ביט אחד בתהליך (מידה מסוימת של אבסטרקציה). נניח שהשכבה התחתונה עובדת, ומוסיפים מעליה.

#	Layer Name	Data Unit	Function
1	Physical	Bit	Transmit and receive raw bits over some physical medium (air, optics, etc.)
2	Data link	Frame	Transmission of data frames between two nodes
3	Network	Packet	Structure and manage multi-node network (Addressing, routing, etc.)
4	Transport	Segment	Transmission (reliable or not) of segments between points in the network (ack'ing, etc.)
5	Session	Data	Manage a continuous communication session
6	Presentation		Translation of data between a networking and an application (e.g., encryption/decryption)
7	Application		High level APIs between network applications (e.g., GET request)

(אנחנו בקורס נתעסק פחות בשכבה הפיזית קשורה יותר להנדסה ולפיזיקה).

נתחיל משכבה 2, **data link** שאומרת שיש כמה מחשבים שמחוברים ברשת אחת, אם מישהו מדבר – כולם יודעים. השפה שידברו כדי לאפשר לכמה שיותר מידע לעבור נקרא פרוטוקול. מעליה יש את שכבת ה-**Network**. הם לא מחוברים ביחד באופן ישיר, אבל יש להם כתובת אחד של השני והם מעבירים הודעות דרך נתבים בדרך. שכבת **transport** מסתכלת בראייה רחבה – לא תמיד ההודעות יגיעו כמו שצריך. בשכבת הדאטה לינק הנחנו שאם מישהו משדר ואין בעיות בפרוטוקול אז זה יגיע לצד השני. בפועל המצב אחר, הטיפול באבדות הוא באחריות שכבה זו. שכבות 5-7 מסתכלות על המידע כולו (לא בחלוקה לפקטה), ב-**session** יש לנו דו-שיח מול אתר כמו למשל פייסבוק. ברגע שהתחיל session אנחנו יכולים להיכנס לכל האפשרויות בתוך פייסבוק מבלי לאמת בכל פעם מחדש את התקשורת ביננו. כדי שזה יקרה, יש ענייני הצפנה בהם ניגע בהמשך. ב-**presentation** מתייחס להצפנה של הודעות כדי למנוע דליפת מידע בהאזנות, ובסוף שכבת **אפליקציה** שהיא הממשק בין הרשת לאפליקציות של המשתמשים.

דוגמאות של פרוטוקולים לשכבות שונות

שכבת האפליקציה: DNS, HTTP

שכבת Transport: TCP, UDP

שכבת Network: IP

שכבת Data Link: MAC

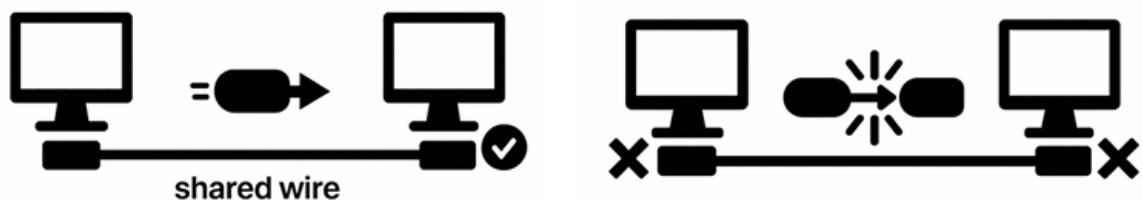
שכבה פיזית: Wi-Fi

Random Access Protocols

נניח שאנחנו בשכבה 2 (Data Link) וכולנו מחוברים לאותו Network. אני רוצה להתחיל לדבר עם אנשים, כמובן לא ארצה שידברו באותו זמן כמוני אחרת אף צד לא יבין את השני. יש כמה הנחות:

1. בתווך הזה אין רעש, כלומר כל עוד אני היחיד שמדבר – כולם שומעים.

2. אם שני אנשים מדברים באותו זמן – יש שיבוש בתנועת ההודעות.



הגדרות

1. Bandwidth – כמה ביטים אפשר לשלוח ביחידת זמן כלשהו. סימון: B . יחידות מידה: $\frac{bit}{sec}$.

2. Throughput – החלק היחסי של ה-bandwidth שנוצל לשליחת הודעות.

3. Goodput – החלק היחסי של ה-bandwidth שנוצל לשליחת הודעות שהתקבלו בהצלחה.

$$\eta = \frac{\text{total average effective bandwidth usage}}{\text{total bandwidth}} = \frac{\text{total average effective time}}{\text{total time}}$$

(נבחין כי $0 \leq \eta \leq 1$)

שאלה

אם יש לנו פקטה בגודל 30 bits , כאשר ה-bandwidth הוא $5 \frac{bit}{sec}$ נקבל שהזמן להעביר את הפקטה הוא:

$$T = \frac{|packet|}{B} = \frac{30}{5} \cdot \frac{bit}{bit/sec} = 6 \text{ sec}$$

לשים לב, יש חשיבות גם לחלוקת היחידות ולא רק לערכים – זה גם מנגנון לוודא שביצענו חלוקה נכונה (אכן קיבלנו שהיחידות הן שניות כאשר נשאלנו כמה זמן).

עבשיו נניח שאנחנו רוצים לשלוח 3 פקטות שגודל כל אחת 30 bits בתוך 2 שניות, מה רוחב הפס הנדרש?

$$B = \frac{\#packets \cdot |packet|}{T} = \frac{3 \cdot 30}{2} \cdot \frac{bit}{sec} = 45 \frac{bit}{sec}$$

ALOHA

General Model

ALOHA זה פרוטוקול שמנסה למצוא מצב ביניים שבו הודעות כן יעברו בין אחד לשני, אולי לא כל הזמן, אבל לפחות חלק מהזמן.

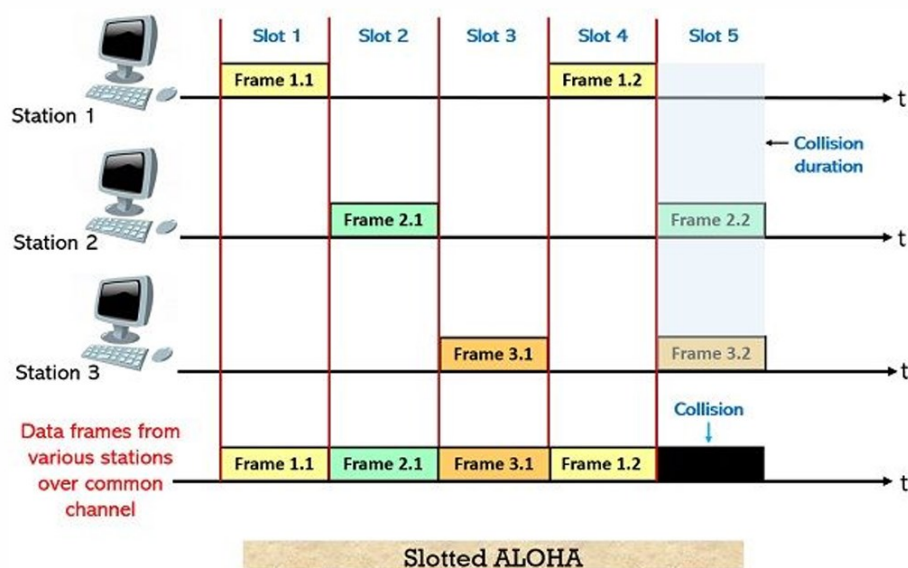
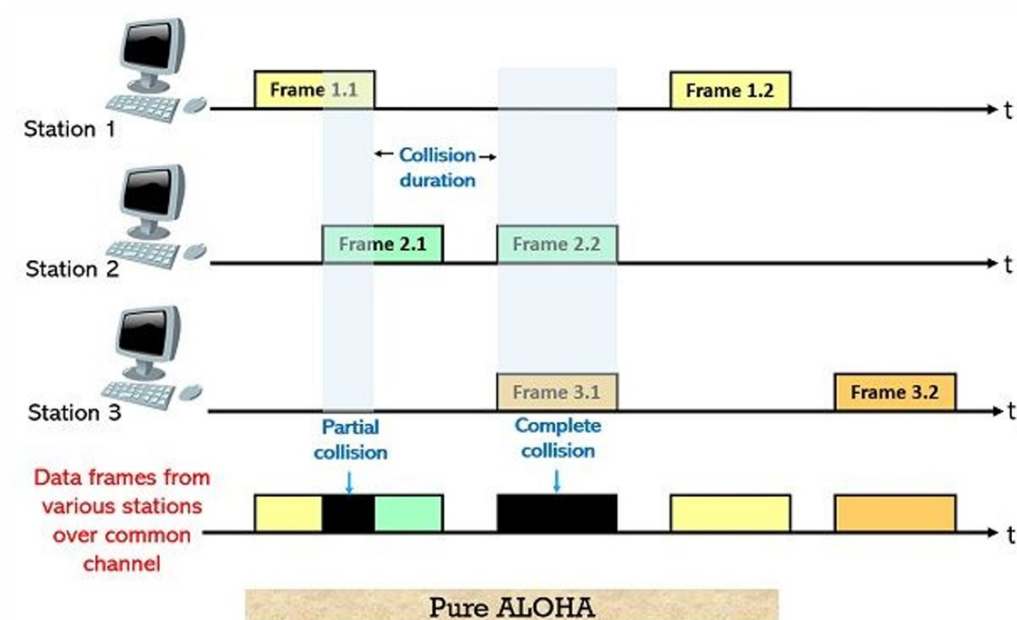
הנחות:

- יש N קודקודים במערכת, ולרוב ננתח את הפרוטוקול כאשר $N \rightarrow \infty$.
- B – רוחב הפס
- L – כמה ביטים יש בתוך frame
- $T = \frac{L}{B}$ – זמן, יחושב לפי גודל ה-frame חלקי רוחב הפס: $T = \frac{L}{B}$.
- אם אנחנו שולחים את ההודעה בצורה תקינה ההנחה היא שאנחנו מנצלים את כל רוחב הפס לאורך כל יחידת הזמן.
- לכל אחד יש slot זמן שבו הוא שולח הודעה, הודעות תשלחנה רק בתחילת slot. אם ההודעה הגיעה באמצע slot בזמן היא תישלח בתחילת ה-slot הבא.
- כאשר צומת מסיים לשלוח הודעה, קיימת דרך לדעת שהייתה התנגשות בהודעות (ולשלוח אותה שוב בעתיד).
- שידור לוקח זמן, אבל אנחנו נתעלם מההפרשים האלו – הם זניחים לצורך החישוב.

Types of ALOHA

יש שני סוגים שונים של ALOHA:

1. **Pure ALOHA Assumption** – כל צומת הוא בעל שרון משלו (סנכרון השעונים היא פעולה יקרה).
2. **Slotted ALOHA Assumption** – כל השעונים מסונכרנים.



ביתוח בינומי לעומת ביתוח פואסוני:

ההנחה היא שלכל מחשב ברשת יש משהו לשדר, והוא לא תמיד משדר את זה. מניחים שיש הסתברות p לשדר, ו- $(1 - p)$ לא לשדר. מדובר בניסוי בינומי ושואלים מה ההסתברות שמישהו אחד ישדר, או שכמה ישדרו יחד. אם נרצה לספור כמה פעמים ניסוי ברנולי הצליח, זה מתפלג כמו משתנה מקרי בינומי: $X_p \sim \text{Bin}(n, p)$

אפשר להסתכל על זה גם בצורה פואסונית – יש לנו n קודקודים, אבל n גדול מאוד ואז $X_p \sim \text{Poi}(\lambda T)$.

ניזכר שעבור N מאוד גדול ו- p קטן מתקיים $\text{Bin}(N, p) \approx \text{Poi}(\lambda)$ כאשר $\lambda = Np$. אם נרצה למדל את המערכת עם **מעט קודקודים** נתייחס להתפלגות **בינומית**, ואם נרצה למדל עם **המון** (∞) **קודקודים** נתייחס להתפלגות **פואסונית**.

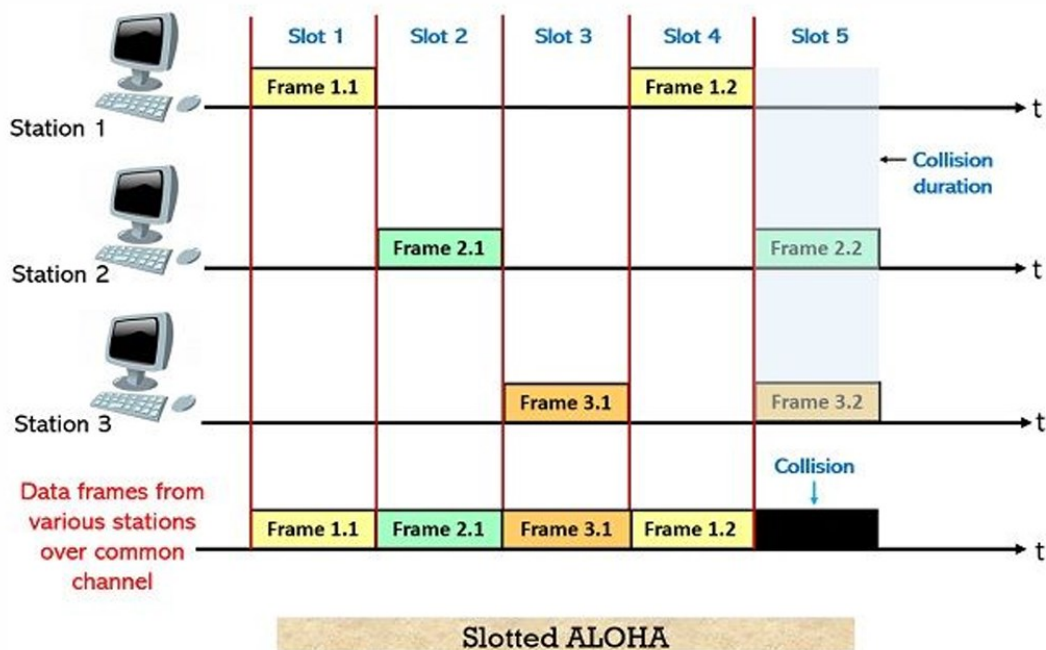
לכן יש 4 מודלים שאפשר לנתח:

Poisson	Binomial	
3	1	Slotted ALOHA
4	2	Pure ALOHA

בתרגול הזה נתמקד בביתוח הבינומי.

Slotted ALOHA – Binomial Approach

ב-slotted יש לכל תחנה slot משלה, ואין חפיפה ביניהם כלל.



בדוגמה הזו אנחנו רואים שלמשל תחנה 1 הצליחה (בסיכוי p) לשלוח הודעה ב-slot 1, תחנה 2 ב-2, 3 ב-3 וכן הלאה. ב-slot 5 אנחנו שמים לב שגם תחנה 2 וגם תחנה 3 הצליחו לשדר – ולכן יש התנגשות. כל עמודה היא ניסוי ברנולי שאנחנו מגדירים בהצלחה אם קיבלנו את הערך 1 (רק תחנה אחת הצליחה לשדר הודעה). ב-slot 5 קיבלנו את הערך 2 (כי 2 תחנות שונות הצליחו לשדר) ולכן הניסוי נכשל.

הנחות:

- כל מחשב מנסה לשדר משהו, כל הזמן.
- רק בהסתברות p_{send} הצומת יצליח לשדר הודעה.
- כל הודעה שהצליחה להתקבל השתמשה בכל ה-bandwidth.

שאלות לדוגמה:

1. נניח שכל ה-slots זהים, נסתכל על slot 1. כמה זמן ננצל בתוחלת על שליחה מוצלחת?

פתרון:

נגדיר את X_p בתור מספר השליחות ביחידת זמן אחת, ונרצה למצוא את $\mathbb{P}(X_p = 1)$. נזכור שאנחנו בניתוח הבינומי ולכן $X_p \sim \text{Bin}(n, p_{send})$.

○ מה ההסתברות ל-frame מוצלח? מהנוסחה:

$$P_{bin} = \binom{n}{x} p^x (1-p)^{n-x}$$

נציב $x = 1$ ונקבל:

$$\mathbb{P}_{suc} = \mathbb{P}(X_p = 1) = n \cdot p_{send} \cdot (1 - p_{send})^{n-1}$$

○ נסמן ב- T_{suc} את הזמן שנוצל על שליחה מוצלחת ב-slot יחיד. מה הוא $\mathbb{E}[T_{suc}]$?

$$\mathbb{E}[T_{suc}] = T \cdot \mathbb{P}_{suc} + 0 \cdot (1 - \mathbb{P}_{suc}) = T \cdot \mathbb{P}_{suc}$$

למה T כפול $\mathbb{P}_{suc}$? כי כשהוא מצליח הוא משדר על כל רוחב הפס במשך כל T השניות, ועוד

ההסתברות שהוא נכשל כפול הערך שמתקבל עבור כישלון – כלומר 0.

○ מה ה-Goodput של הרשת?

$$\eta = \frac{\mathbb{E}[T_{suc}]}{T} = \frac{T \cdot \mathbb{P}_{suc}}{T} = \mathbb{P}_{suc} = n \cdot p_{send} \cdot (1 - p_{send})^{n-1}$$

תוחלת של כמה שהצלחנו להעביר, חלקי זמן נתון.

2. מה היחס בין slot מוצלח ללא מוצלח? אפשר לשאול כמה slots בממוצע יש לנו עד שנקבל אחד מוצלח?

פתרון:

נגדיר X_p מ"מ שמייצג את מספר הנסיונות. מתקיים: $X_p \sim \text{Geo}(\mathbb{P}_{suc})$ וכן $\mathbb{E}[X_p] = \frac{1}{p_{suc}}$. כמו כן, ה-

goodput יחושב לפי:

$$\eta = \frac{T}{T \cdot \left(\frac{1}{\mathbb{P}_{suc}}\right)} = \mathbb{P}_{suc} = n \cdot p_{send} \cdot (1 - p_{send})^{n-1}$$

נרצה לדעת מתי הערך הזה יהיה מקסימלי, לכן נגזור לפי p_{send} ונשווה לאפס. בסיום פיתוח מתמטי (הופיע בצורה חלקית בסיכום התרגול) נקבל $p_{send} = \frac{1}{n}$. משתמשים ב- p^* כדי לסמן את ההסתברות

שמביאה את η לאופטימום, וב-slotted ALOHA זה יהיה $p^* = \frac{1}{n}$. אבל, אם נשאף $n \rightarrow \infty$ נקבל:

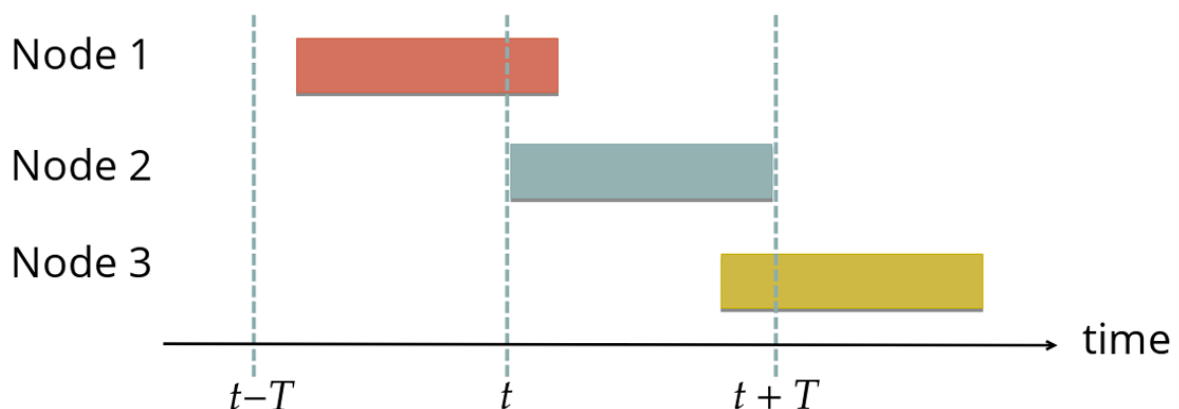
$$\lim_{n \rightarrow \infty} \eta(p^*) = \lim_{n \rightarrow \infty} (np^*(1 - p^{*n-1})) = \dots = \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^{n-1} = \frac{1}{e} \approx 0.37$$

מסקנה: אם יש לנו מערכת ב-slotted ALOHA ויש אינסוף צמתים – כולם משתמשים ב- p_{send} אופטימלי, אז שליש מהזמן נצליח לשדר משהו – לא מאוד אפקטיבי. בלי עוד הנחות זה התוצאה שנגיע אליה.

Pure ALOHA – Binomial Approach

לכל תחנה יש slot משלה, אבל יש חפיפה בין slots של תחנות שונות. זה אומר שיכולה להיווצר התנגשות כמו ב-slotted ALOHA, אבל כאן יכולה להתרחש גם התנגשות חלקית. לשני הסוגים נתייחס כאי הצלחה, גם אם הצלחנו להבין חלק מההודעה בעקבות התנגשות חלקית.

הקושי הנוסף כאן, הוא שלא מספיק לדאוג שלא ישדרו בתחילת frame של תחנה אחרת, באמצע או בסוף – גם אם מישהו התחיל לשדר לפני שה-frame של תחנה אחרת התחיל, אבל משך השידור "גלש" ל-frame אחר, עדיין נקבל התנגשות. ואז יש זמן ברוחב $2T$ שיכול להיות בעייתי:



מה ההסתברות לשליחה מוצלחת?

$$\mathbb{P}_{suc} = \mathbb{P}(\text{some node transmitted}) \cap \mathbb{P}(\text{no other node transmitted in } 2T)$$

ומאחר שמדובר במאורעות בלתי תלויים:

$$\mathbb{P}_{suc} = \mathbb{P}(\text{some node transmitted}) \cdot \mathbb{P}(\text{no other node transmitted in } 2T)$$

נציב ונקבל:

$$\mathbb{P}_{suc} = n \cdot p_{send} \cdot ((1 - p_{send})^2)^{n-1} = n \cdot p_{send} \cdot (1 - p_{send})^{2 \cdot (n-1)}$$

n – כי כל אחד מהצמתים היה יכול להיות זה שהצליח.

הריבוע – כי אנחנו צריכים שב-2 slots אף אחד לא ישדר.

$n - 1$ – כי אסרנו על כל שאר הצמתים לשדר.

מה ה-Goodput של המערכת הזאת?

נגדיר משתנה מקרי:

$$T_{suc} = \begin{cases} T & \text{successful slot} \\ 0 & \text{else} \end{cases}$$

$$\mathbb{E}[T_{suc}] = T \cdot \mathbb{P}_{suc}$$

$$\boxed{\eta} = \frac{\mathbb{E}[T_{suc}]}{T} = \frac{T \cdot \mathbb{P}_{suc}}{T} = \mathbb{P}_{suc} = \boxed{n \cdot p_{send} \cdot (1 - p_{send})^{2 \cdot (n-1)}}$$

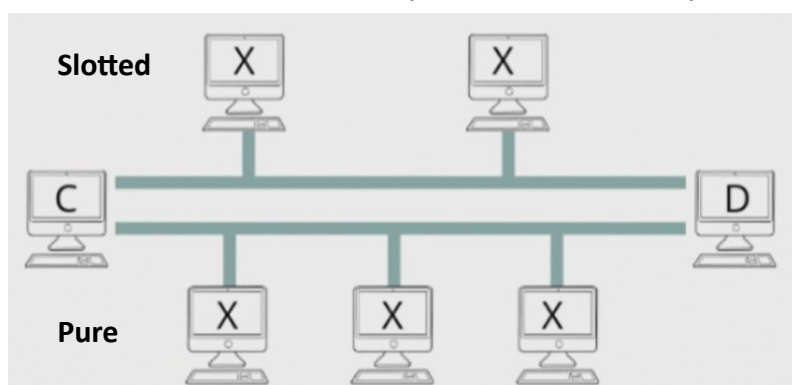
כמו מקודם, כדי למצוא את p_{send} אידיאלי נגזור ונשווה לאפס, נקבל $p^* = \frac{1}{2n-1}$ ובמקרה הזה $\eta = \frac{1}{2e}$ בדיוק

חצי מה-goodput של slotted ALOHA.

שאלות תרגול

[שאלה 1](#)

בהינתן מערכת ו-hosts שנקראים C,D שמחוברים דרך שתי מערכות נפרדות:



הרשת העליונה עם שני צמתים ועובדת לפי slotted ALOHA, התחתונה עם שלושה צמתים ועובדת לפי pure ALOHA. D אף פעם לא שולח הודעה. C מחליט להשתמש ברשת העליונה בהסתברות p , ושולח הודעה בהסתברות q . הצמתים X פשוט עושים רעש, ובהסתברות q שולחים משהו.

א. C מעוניין לשלוח ל-D. מה ההסתברות שההודעה תגיע?

פתרון:

נחלק למקרים לפי הרשת שבה ההודעה נשלחה, ומאחר שהמאורעות זרים, איחודם שווה לסכום ההסתברויות.

ברשת העליונה (slotted):

$$\mathbb{P}_{\text{slotted_suc}} = \mathbb{P}(\text{C send a message}) \cdot \mathbb{P}(\text{the others don't send}) = q \cdot (1 - q)^2$$

ברשת התחתונה (pure):

$$\mathbb{P}_{\text{pure_suc}} = \mathbb{P}(\text{C send a message}) \cdot \mathbb{P}(\text{no other transmissions for two slots}) =$$

$$= q \cdot \mathbb{P}(\text{no other transmissions for two slots}) = q \cdot ((1 - q)^2)^3 = q \cdot (1 - q)^6$$

סך הכל:

$$\begin{aligned} \mathbb{P}_{\text{total}} &= \mathbb{P}(\text{use slotted ALOHA}) \cdot \mathbb{P}_{\text{slotted_suc}} + \mathbb{P}(\text{use pure ALOHA}) \cdot \mathbb{P}_{\text{pure_suc}} = \\ &= q(p(1 - q)^2 + (1 - p)(1 - q)^6) \end{aligned}$$

ב. C מעוניין לשלוח בתחילת כל slot. מה ה-goodput של הרשת העליונה?

פתרון:

כאשר הרשת העליונה נבחרה, יש 3 צמתים שמתקשרים (C, X, X). לכן:

$$\mathbb{P}_{\text{suc}}^1 = \binom{3}{1} \cdot q \cdot (1 - q)^2$$

ייתכן גם ש-C יבחר לדבר ברשת התחתונה, ואז ברשת העליונה יש רק 2 צמתים שמתקשרים (X, X). לכן:

$$\mathbb{P}_{\text{suc}}^2 = \binom{2}{1} \cdot q \cdot (1 - q)$$

C בוחר את הרשת העליונה בהסתברות p , לכן סה"כ:

$$\eta = p \cdot \mathbb{P}_{\text{suc}}^1 + (1 - p) \cdot \mathbb{P}_{\text{suc}}^2$$

שאלה 2

ברשת יש $2n$ קודקודים.

ה- n הראשונים משתמשים ב-pure ALOHA עם סלוטים בגודל T .

ה- n האחרונים משתמשים ב-slotted ALOHA עם סלוטים בגודל $2T$.

כל קודקוד משדר בתחילת כל slot בהסתברות p , וההודעה המשודרת לוקחת את כל ה-slot.



א. נבחר slot רנדומלי. מה ההסתברות שצומת שמריץ slotted ALOHA הצליחה לשלוח הודעה בהצלחה ב-slot הזה?

פתרון

צריכים שהצומת הספציפי ישדר – הסתברות p

שאף צומת slotted ALOHA אחר לא ישדר – הסתברות $(1 - p)^{n-1}$

שאף צומת pure ALOHA לא ישדר ב-3 slots, כי נשים לב לנתון ש-slotted ALOHA עם slot באורך $2T$, ולכן ל-Pure יש T אחד לפני תחילת השידור שאסור לשדר בו, ועוד $2T$ לזמן השידור ממש – הסתברות

$$(1 - p)^{3n}$$

$$\text{לכן סה"כ: } p \cdot (1 - p)^{n-1} \cdot (1 - p)^{3n}$$

ב. נבחר slot רנדומלי. מה ההסתברות שצומת שמריץ pure ALOHA הצליחה לשלוח הודעה בהצלחה ב-slot הזה?

פתרון:

ייתכנו שני מקרים – או שההודעה חופפת עם slot יחיד של slotted ALOHA, או שהיא חופפת עם 2 slots (שוב, מאחר ש-slotted ברוחב $2T$).

• חפיפה עם slot יחיד:

הסתברות שהוא ישדר: p

כל slotted ALOHA אחר לא ישדר: $(1 - p)^n$

כל pure ALOHA אחר לא ישדר במשך 2 slots: $(1 - p)^{2(n-1)}$

$$\text{סה"כ מקרה ראשון: } p \cdot (1 - p)^n \cdot (1 - p)^{2(n-1)}$$

• חפיפה עם שני slots:

הסתברות שהוא ישדר: p

כל slotted ALOHA לא ישדר בטווח של שני slots: $(1 - p)^{2n}$

כל pure ALOHA אחר לא ישדר במשך 2 slots (אין שוני מהמקרה הראשון כי בכל מקרה ב-pure

מסתכלים על 2 slots): $(1 - p)^{2(n-1)}$

סה"כ מקרה שני: $p \cdot (1 - p)^n \cdot (1 - p)^{2(n-1)}$

ג. בסעיף זה נניח שכל n הצמתים שהריצו pure עבשיו רצים ב-slotted ALOHA עם slots באורך T (האחרים עדיין בגודל $2T$).

מה ה-Goodput של המערכת?

נסתכל על טווח של $2T$ ונבחן את כל המקרים שיוגדרו בהצלחה:

A: פקטה slotted מוצלחת.

B: פקטה pure מוצלחת בסלוט הראשון.

C: פקטה pure מוצלחת בסלוט השני.

D: 2 פקטות pure מוצלחות, אחת ב-slot הראשון והשנייה ב-slot השני.

נחשב אותם:

A: כדי שתהיה פקטה slotted מוצלחת, בדיוק אחד מתוך n הצמתים של "slotted 2T sized" יצליח לשדר:

$$\binom{n}{1} \cdot p \cdot (1 - p)^{n-1} = np(1 - p)^{n-1}$$

כל שאר ה-"pure T sized" לא ישלחו: $(1 - p)^{2n}$

האירועים ב"ת ולכן נכפיל את ההסתברויות: $np(1 - p)^{n-1} \cdot (1 - p)^{2n}$

D: כדי שתהיינה 2 פקטות pure מוצלחות, צריך שיתקיים:

בדיוק "pure T sized" אחד יצליח לשדר ב-slot הראשון:

$$\binom{n}{1} \cdot p \cdot (1 - p)^{n-1} = np(1 - p)^{n-1}$$

חישוב זהה עבור ה-slot השני.

כל ה-"slotted 2T sized" לא ישדרו בכלל, בהסתברות: $(1 - p)^n$.

סה"כ נקבל: $(np(1 - p)^{n-1})^2 \cdot (1 - p)^n$

B: כדי שתהיה פקטה pure מוצלחת בסלוט הראשון, צריך שבדיוק אחד "pure T sized" ישדר

בהסתברות של: $\binom{n}{1} \cdot p \cdot (1 - p)^{n-1} = np(1 - p)^{n-1}$

שכל ה-"slotted 2T sized" לא ישדרו, בהסתברות $(1 - p)^n$.

לא להצליח בסלוט השני, בהסתברות $1 - np(1 - p)^{n-1}$ (המשלים של להצליח).
סה"כ נקבל:

$$np(1 - p)^{n-1}(1 - p)^n(1 - np(1 - p)^{n-1})$$

C: המקרה שקול ל-B.

עכשיו לחישוב ה-Goodput:

במקרים שבהם ניצלנו רק חצי מהזמן $(2T)$, כלומר מקרים B ו-C, עלינו לכפול בחצי. נקבל:

$$\eta = A + \frac{1}{2}B + \frac{1}{2}C + D$$

הרצאה 4

הקדמה וחזרה

בשיעור שעבר התחלנו לדבר על שכבת ה-datalink והתפקיד שלה. עצרנו בבעיה שיש כמה צמתים שרוצים לתקשר והם חולקים לינק משותף – ערוץ תקשורת אחד עם כמה שולחים פוטנציאליים. בשיעור הזה נראה פרוטוקולים שמטרתם לסנכרן את הגישה למשאב המשותף, ולתת "זכות דיבור" למחשב אחד בכל פעם. אלו פרוטוקולים מבוזרים, אין ישות שאומרת למי יש זכות לשלוח והם גם לא מבצעים חלוקה א-פריורית (כמו חלוקה בזמן או בתדר). הפרוטוקולים האלה יאלצו לסנכרן בין המחשבים מבלי להשתמש בערוץ המשותף, ואת זה יבצעו באמצעות הסתברות, כלומר האלגוריתמים אינם דטרמיניסטיים – וזה מה שמבטיח שחלק מההודעות יעברו ליעדן.

מצגת 2

שקף 42-43

ברשתות wired יש שני סוגים של רשתות, bus משותף או switch (בו נתמקד בחלק השני של הרצאה זו). חלוקה נוספת ורחבה יותר היא בין wired ל-wireless. ברשת אלחוטית יש אתגרים נוספים, למשל שכולם שומעים את ההודעות.

שקף 44

אידיאלית, היינו רוצים פרוטוקול שיחלק בצורה הוגנת את השליחה בערוץ המשותף בין כל השולחים. לא נרצה לקבל סנכרון בין הצמתים, ונרצה להתחשב בכמות השולחים.

Random Access Protocols – ALOHA

שקף 70

Slotted ALOHA

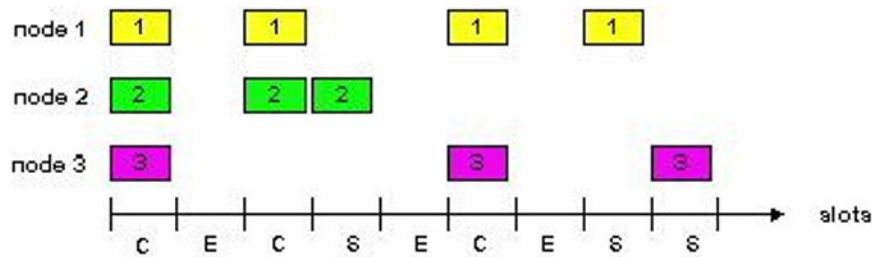
בפרוטוקול נניח שכל ההודעות באותו אורך, ונחלק את הזמן ל-slots שבכל slot אפשר לשלוח הודעה. נניח שהצמתים מסונכרנים, כלומר שהם יודעים מתי מתחיל כל slot באותו הזמן יחד. נניח שאם יש שניים שמשדרים הם מודעים להתנגשות.

איך הפרוטוקול עובד?

כאשר אני כצומת קיבלתי הודעה לשלוח מהשכבה הקודמת, אנסה לשלוח אותה ב-slot הבא, ואבחין אם הייתה התנגשות או לא. אם לא הייתה התנגשות ויש לי משהו אחר לשלוח, אעשה זאת. אחרת אם ההודעה התנגשה, אני צריך לשלוח את ההודעה מחדש ובנקודה הזאת אני יודע שיש שולח נוסף, לכן כדי להתנתק מהסנכרון ממנו אגריל מספר שיקבע אם אשלח ב-slot הבא או לא.

שקף 71

בדוגמה כאן יש 3 שולחים שמזהים את ההתנגשות, והגרלת המספרים הובילה לרצף שאנחנו רואים בדוגמה עד שכולם הצליחו לשלוח:



זה מדגים שהפרוטוקול לא פותר לגמרי התנגשויות, ואנחנו כן נבזבז slots. יש שני סוגים של בזבז – או שיותר מאחד שלח ואז הייתה התנגשות (ההודעות לא הגיעו), או שאף אחד לא שלח (לא נוצל slot בכלל). משני המצבים היינו רוצים להימנע בשאיפה. **היתרון** בפרוטוקול, ביחס לפונקציונליות האידיאלית נבחין שאם יש שולח בודד הוא תמיד יכול לשלוח. כמו כן, אין הרבה סנכרון בין הצמתים אבל הוא כן דורש סנכרון שעונים.

שקף 72

נרצה לנתח את הפרוטוקול. אנחנו שואפים למקסם את ה-**Goodput**, כלומר למקסם את מספר הסלטים שהשתמשנו בהם ללא התנגשות. לעומת זאת **Throughput** מייצג את % ה-slots שמישהו שלח בהם. כלומר, אם ב-slot מסוים שני צמתים שלחו הודעה זה נחשב גבוה מבחינת Throughput, אבל מבחינת Goodput לא.

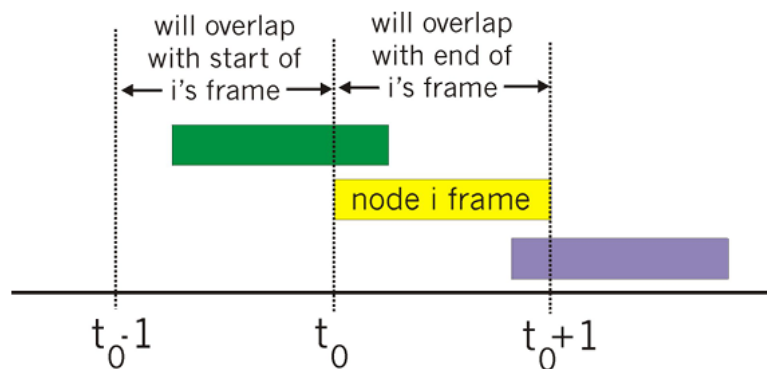
שקף 73

נניח שיש לנו N צמתים שתמיד יש להם מידע לשלוח. כל אחד מהם מנסה לשלוח בהסתברות p . מה ההסתברות שיש צומת שמצליח לשלוח? הסיכוי הוא שהוא כן שלח, ושכל $N - 1$ האחרים לא מצליחים לשלוח, כלומר: $\binom{N}{1} \cdot p \cdot (1 - p)^{N-1}$ (התפלגות בינומית, אפשר לבחור צומת 1 מתוך N כאלה, ולכן ה-choose). נרצה למצוא את ה- p האופטימלי במונחי N . באופטימום מגיעים לניצולת $\frac{1}{e}$ (פיתוח מלא בתרגול).

שקף 74

Pure ALOHA

השוני הוא שעכשיו הצמתים לא מסונכרנים על מתי מתחיל ונגמר כל slot, אבל הספירה עדיין באותו קצב בין צמתים שונים. נצפה שזה יגרום לניצולת לרדת, מדוע? בדוגמה הבאה נניח שהסימונים של ה-slots הם על פי הספירה שלי:



נניח ששלחתי ב- t_0 . מאחר שה-slots של הצמתים האחרים לא מסונכרנים עם שלי, אז **הירוק** התחיל לשלוח קצת אחרי $t_0 - 1$ ולכן השליחה שלי הפריעה להודעה שלו לקראת הסוף. אותו דבר קרה עם השולח **הסגול**. בעצם כשאין סנכרון בהתחלה וסוף של slots הסיכון להתנגשות הוא כפול.

שקף 75

נסתכל על מישהו בודד ונבחן מה הסיכוי שהוא הצליח לשדר בלי הפרעה. זה מורכב מהסיכוי שהוא בן הצליח לשדר, והחלק השני זה שאף צומת אחר לא שידר בין זמן $t_0 - 1$ לזמן $t_0 + 1$, **כפול זמן** לעומת Slotted ALOHA:

$$p \cdot (1 - p)^{N-1} \cdot (1 - p)^{N-1} = p \cdot (1 - p)^{2(N-1)}$$

(עבור צומת כלשהו נצטרך להכפיל ב- N).

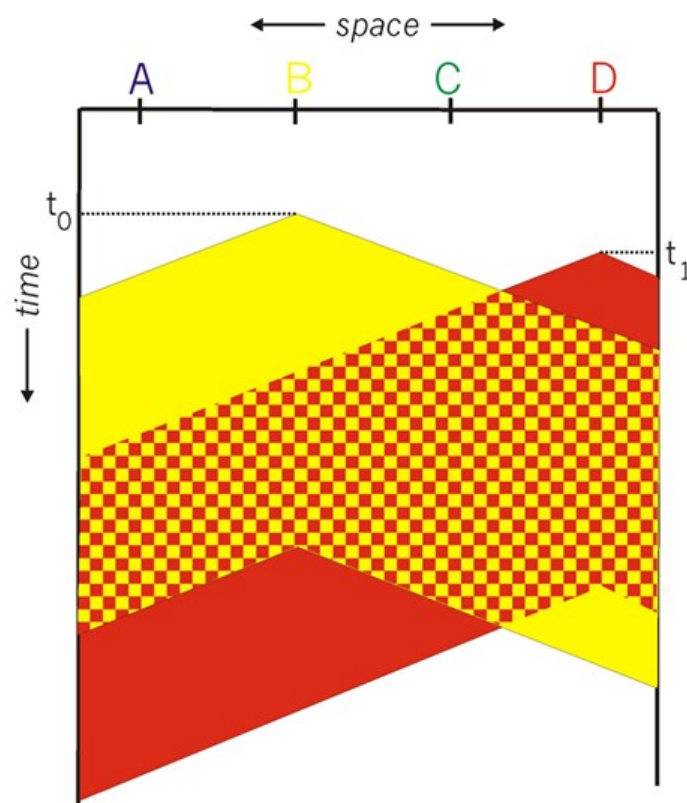
אחרי שמוצאים p אופטימלי מקבלים ניצולת אפילו פחות טובה משל Slotted, שהיא $\frac{1}{2e}$.

Random Access Protocol – CSMA

שקף 76-77

CSMA (Carrier Sense Multiple Access)

שאר הפרוטוקולים שנציג היום בשיעור שייכים למשפחה שנקראת CSMA. הרעיון בגישה הוא שלפני שנשלח הודעה, נסתכל האם הערוץ פנוי (אף אחד לא משדר כרגע). למה זה לא מונע התנגשויות לחלוטין? בתרשים הבא יש 4 שולחים וציר x הוא הזמן:



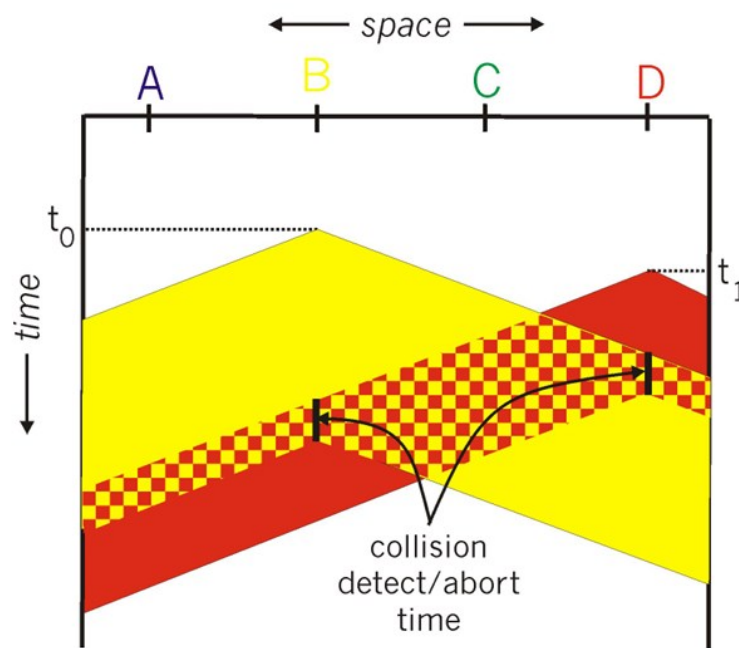
בזמן t_0 צומת B מנסה לשלוח. להודעה שלו יש propagation delay , אם נסתכל על הביט הראשון שהוא שלח, הוא הגיע ל-A ול-C בערך באותו זמן, אבל D רחוק יותר (פיזית) ולכן לקח לו זמן לקבל את הביט. B הסתכל בזמן t_0 על הערוץ והוא היה פנוי. בזמן t_1 D עושה את אותו הדבר, וגם הוא רואה שהערוץ פנוי (כי הביט הראשון של ההודעה של B עוד לא הגיע). קיבלנו התנגשות.

שקף 78

CSMA/CD (Collision Detection)

נציע שדרוג למודל שראינו מקודם, שנוסיף לו זיהוי התנגשויות. CSMA מנצל את היכולת להסתכל על הערוץ ולראות האם יש שולח. CSMA/CD עומד לקרוא מהערוץ ולראות האם יש שידור של מישהו אחר גם בזמן שאני שולח. ההבדל הוא ש-CSMA הבחין שהשידור שלו מתנגש עם מישהו אחר ובכל זאת המשיך (החלק המשובץ בגרף לעיל). CSMA/CD בא לנצל את המידע הזה.

הערה: בדרך כלל ב-CSMA/CD מדברים על wired LANs, כי מאוד קשה לעשות collision detection בצורה wireless (אולי אינני שומע אותו).



D משדר ובהתחלה הוא לא מזהה שהוא מתנגש עם מישהו אחר, כי המידע מ-B עדיין לא הגיע אליו. ברגע שהוא מזהה שיש שולח אחר D מפסיק לשלוח בעצמו, וכך עושה גם B. אם מסתכלים על כמה זמן בוזבז, זה משמעותית פחות מהמקרה הראשון, כי שני הצמתים ידעו להפסיק את השליחה שלהם.

שקף 80

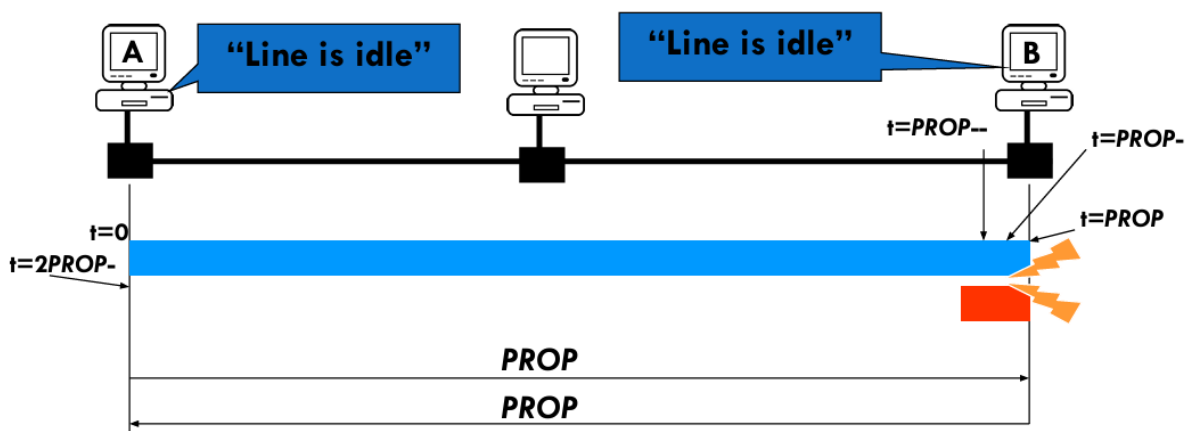
סיכום – carrier sense כלומר בודקים את הקו לפני תחילת השליחה, collision detection משמעותו זיהוי התנגשות כמה שיותר מוקדם, ועצירת השידור בעת הזיהוי.

אם הייתה התנגשות נחכה זמן אקראי ואז נחזור ל-carrier sense.

שקף 81

נראה עכשיו תכונה אינהרנטית של פרוטוקול CSMA/CD. ננסה לראות מה משך הזמן המקסימלי שיכול לעבור עד שאני אדע שהייתה התנגשות עם השידור שלי. מדוע זה משנה לי מה הזמן המקסימלי שצריך לעבור עד שאדע שהייתה התנגשות עם השידור שלי? כי זה **תוחם לי את הזמן המינימלי שאני צריך לשדר כדי לתפוס התנגשות בשידור**.

לשם בחינת התכונה הזאת, ניקח את הניתוח לקיצון – בשקף כל "-" מייצג ϵ . בזמן $t = PROP--$ ההודעה ממש כמעט מגיעה ל-B, ואז B מחליט שהוא רוצה לשדר. B מקשיב במשך ϵ שניות על הקו, רואה שהוא פנוי, ואז מתחיל לשדר בעצמו בזמן $t = PROP-$. המידע שלו יעבור על הקו ורואים התנגשות. A עומד לקבל את הביט הראשון ש-B הספיק לשדר בעוד $PROP$, סה"כ כעבור $2PROP-$.



כלומר A שידר, אבל זיהה שהשידור נכשל רק $2 \times \text{propagation time}$ מאוחר יותר. מה אפשר להסיק מזה?

שקף 82

הזמן לשלוח הודעה (TRANSP) צריך להיות לפחות פעמיים מה- propagation delay , אחרת נסיים לשלוח את ההודעה מבלי לדעת שהייתה בכלל התנגשות. לכן הדרישה: $TRANSP > 2 \times PROP$.

שקף 83-87

נניח שיש לנו slots בצורה לוקאלית, הזמן לשלוח הודעה הוא $2 \times PROP$ וההסתברות לשליחה היא p . ננסה להבין מה ה-Goodput של הפרוטוקול (כמה זמן שלחתי מידע, חלקי הזמן ששלחתי מידע ועוד זמן לא מנוצל):

$$\eta = \frac{\text{Time taken to send data}}{\text{Time taken to send data} + \text{overhead}}$$

באמצעות חישוב דומה למה שראינו קודם, הניצולת המקסימלית תהיה $\approx 40\%$, כלומר נצפה שעל כל slot מוצלח (שבו התקבלה הודעה) יהיו בממוצע 1.5 slots שלא הצליחו. נשתול בנוסחה ונקבל:

$$\eta_{\text{CSMA/CD}} = \frac{TRANSP}{TRANSP + E[\text{time wasted}]} = \frac{TRANSP}{TRANSP + A[2 \times PROP]}$$

$$= \frac{TRANSP}{TRANSP + [3 \times PROP]} = \frac{1}{1 + 3\alpha} \quad \left(\alpha = \frac{PROP}{TRANSP} \right)$$

time wasted – מספר ה-slots שבזבזו, כפול האורך של כל slot.

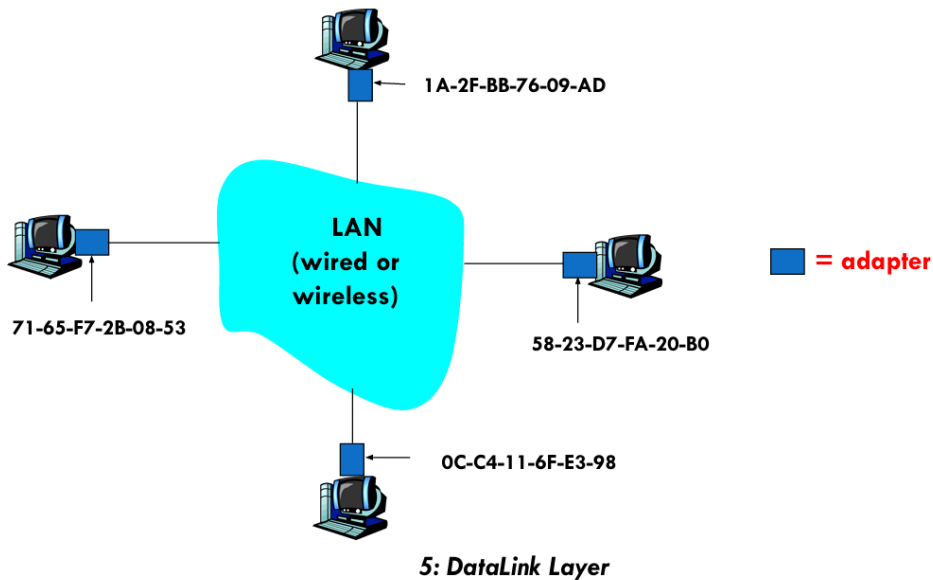
$A=1.5$ – כמות ה-slots שנכשלים, וזמן השליחה שמתבזבז זה פעמיים propagation time.

זה יותר טוב מ-slotted ALOHA כי עכשיו אנחנו תלויים ב-propagation time שהוא בתקווה זניח לעומת הזמן שלוקח לנו לשדר את ההודעות.

כתובות MAC

שקף 91-92

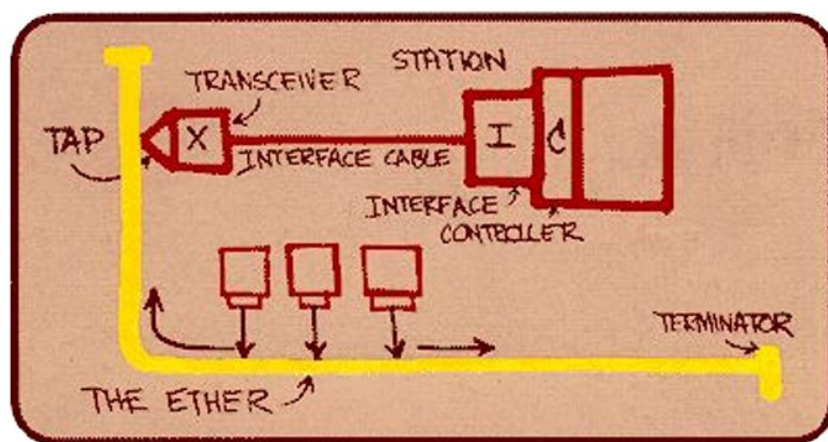
איך הפרוטוקולים ממומשים בפועל? דיברנו בשיעור שעבר על כתובות MAC, לכל כרטיס רשת יש כתובת MAC שהוטבעה בו על ידי היצרן – 48 ביטים שהם הזיהוי שלו. הביטים האלו מזהים אותו ללא תלות ברשת אליה התחבר.



שקף 93-94

כשאנחנו שולחים הודעה, כתובת ה-MAC מצורפת להודעות שלנו, היא חלק מה-header של ה-frame. כשיש מחשבים שמחוברים על bus משותף. כשאנחנו מקבלים הודעה אפשר לקרוא את ה-frame ולפי ה-destination MAC address אפשר לדעת האם זה מיועד אליי (ואז כרטיס הרשת יעלה את זה אליי שכבה אחת למעלה) או יזרוק אותה. בשיעור הבא נדבר על רשתות switched, ושם ה-switch ייעזר בכתובת כדי לנתב אותה ליעדה.

הטכנולוגיה הדומיננטית לחיבור רשתות wired היא Ethernet. הטכנולוגיה יחסית פשוטה, והיא עדיין תקפה למהירויות הגלישה של היום.



Ethernet CSMA/CD

איך הפרוטוקול עובד מקצה לקצה? הדבר היחיד שלא תארנו הוא איך עובד נושא האקראיות. אם זיהיתי התנגשות, איך אדע לעצור?

בניגוד ל-Slotted ALOHA שם היה פרמטר p שהגדיר מה הסיכוי להמשיך, כאן הסיכוי שלנו להמשיך לשלוח או לא תלוי בהיסטוריה של כמה פעמים הייתה התנגשות. כשעשינו ניתוח ל-Slotted ALOHA ראינו שה- p^* האופטימלי תלוי ב- N מספר השולחים, וקשה לקבוע אותו בזמן קידוד. המנגנון שהכניסו ב-CSMA/CD הוא exponential backoff שמטרתו להתאים את ההסתברות לשליחה לפי מספר השולחים. ככל שהיו יותר התנגשויות עם ההודעה שלי, אעלה בממוצע את הזמן שאחכה עד השליחה הבאה. בהתנגשות ה- m אסתכל על כל המספרים בטווח $\{0, \dots, 2^{m-1}\}$ ומתוכם אבחר באקראי מספר k שהוא יגדיר כמה אני עומד לחכות.

המטרה: להגיע למצב שככל שהתנגשו בי יותר אחכה יותר, ואז נצליח לפזר טוב יותר בזמן את שליחת ההודעות.

Random Access Protocols

חזרה והקדמה

רוב הפרוטוקולים בשכבה 2 (דאטה לינק) עובדים באמצעות Random Access Protocols. מדמים מערכת עם המון שחקנים שיכולים לצאת ולהיכנס במקומות שונים בזמן, ויש קושי להגדיר גוף אחד שיהיה אחראי על התזמון של כולם. בפרוטוקולים האלה אם אחד נופל, אין השפעה רחבה על כלל המערכת. אנחנו עובדים כרגע על תווך אחד – אם שניים משדרים על אותו התווך במקביל, השידור כושל.

הבהרות:

1. Bandwidth אינו שקול ל-Propagation Speed, מדובר בכמה זמן לוקח לשידור לעבור בתווך (בתרגול נתייחס לזמן הזה כאפסי, לא תמיד זו תהיה ההנחה).
2. גם ב-Pure ALOHA יש slots. תמיד אפשר לשלוח רק בתחילת slot, פשוט ה-frame של קודקוד אחד לא מסונכרן בהכרח עם ה-frame של קודקוד אחר, ולכן frames לא "מתלבשים" בצורה מושלמת מעל ה-slots.
3. במודל הגנרי של ALOHA ולצורך התרגול הזה כאשר מוסיפים collision detection, הצומת ידע על collisions רק בסיום השידור.

ALOHA – Poisson Approach

בתרגול הקודם דיברנו על הגישה הבינומית, היום נדבר על הגישה הפואסונית (עבור Pure ועבור Slotted):

$$X_p \sim \text{Poi}(gT)$$

עם קצב שליחה g וזמן T כלשהו, למשל זמן של slot כפול קצב השליחה ואז נדע כמה אמורים לשלוח באותו slot – אם זה יהיה בדיוק 1 אז הצלחנו לשדר בהצלחה ואחרת לא.

תזכורת – תהליך Poisson

תהליך ספירה – בהינתן שתי מאורעות שונים טווח הזמן שלהם לא חופף. מה שיעניין אותנו זה אורך הטווח (בניגוד למשל למידול כמה אנשים עולים על אוטובוס בשעה מסוימת שהוא אינו תהליך פואסוני).

תכונה חשובה: סכום של משתנים מקריים פואסוניים הוא משתנה מקרי פואסוני.

Notation	Explanation	Units
g	Rate of <i>total</i> messages (both new and failed messages) that are up for transmission	$\frac{1}{\text{sec}}$
$G = gT$	Average number of transmitted (successful and failed) messages in a time interval T	None
P_{suc}	Probability of a (single) successful transmission (i.e., no collisions occurred)	None

Slotted ALOHA – Poisson Approach

בעזרת נוסחה של משתנה מקרי פואסוני נקבל:

$$X_p \sim \text{Poi}(gT) \quad \text{then} \quad \mathbb{P}_{t=T}(X = i) = \frac{(gT)^i}{i!} e^{-gT}$$

T זה טווח הזמן, המשתנה המקרי X שווה לכמה הודעות נשלחו באותו טווח זמן (אם שווה 1, הצלחנו). לכן:

$$\mathbb{P}_{suc} = \mathbb{P}_{t=T}(k = 1) = gT \cdot e^{-gT} \stackrel{gT := G}{=} G \cdot e^{-G}$$

וה-**Goodput** יהיה פשוט $\mathbb{P}_{suc} = G \cdot e^{-G}$, כמו במודל הבינומי (לפי קצב שליחה והסתברות הצלחה). תזכורת מהחישוב במקרה הבינומי:

$$\eta = \frac{\mathbb{E}[T_{suc}]}{T} = \frac{T \cdot \mathbb{P}_{suc}}{T} = \mathbb{P}_{suc}$$

דרך נוספת לבצע את הניתוח הזה הוא **מפרספקטיבה של פקטה**. מהי הסתברות ההצלחה אם ההודעה נשלחה?

$$\mathbb{P}_0 = \mathbb{P}_{t=T}(k = 0) = e^{-gT} = e^{-G}$$

מהו ה-**Goodput**? כמו מקודם. יש קצב שליחה כלשהו G . אם נסתכל מנקודת מבט של פקטה, הסיכוי שהיא קיימת הוא G (לפי קצב השליחה) לכן נקבל:

$$\eta = G \cdot \mathbb{P}_0 = G \cdot e^{-G}$$

הערה: חישבנו כאן כאילו יש N קודקודים, אבל אם מסתכלים על פקטה שנשלחה מקודקוד מסוים היינו צריכים להתחשב רק ב- $N - 1$ קודקודים. מאחר שמסתכלים ב-ALOHA על המקרה שבו $N \rightarrow \infty$ ההבדל זניח.

מהו ה-**goodput המקסימלי**? אחרי גזירה מקבלים שהוא יתקבל עבור $G^* = 1$. נקבל $\eta_{G=1} = \frac{1}{e} \approx 0.37$. בדיוק כמו בתרגול הקודם עם משתנה מקרי בינומי, זה הגיוני כי מדובר במקרה מאוד דומה – פשוט במקום להסתכל על כל קודקוד כמשתנה מקרי שונה אנחנו מסתכלים על המערכת כולה כמכלול.

Pure ALOHA – Poisson Approach

החישוב יהיה מאוד דומה למקרה של Slotted:

$$\mathbb{P}_{suc} = \mathbb{P}_{k=0}(t = T) \cdot \mathbb{P}_{k=1}(t = T) = e^{-gT} \cdot gT e^{-gT} = gT \cdot e^{-2gT} = G \cdot e^{-2G}$$

כי אנחנו דורשים שבמשך זמן T שקדם לנקודת השליחה שלנו לא יישלח כלום ($k = 0$) ושבנקודת הזמן שלנו תשלח בדיוק הודעה אחת ($k = 1$).

מה יהיה ה-Goodput?

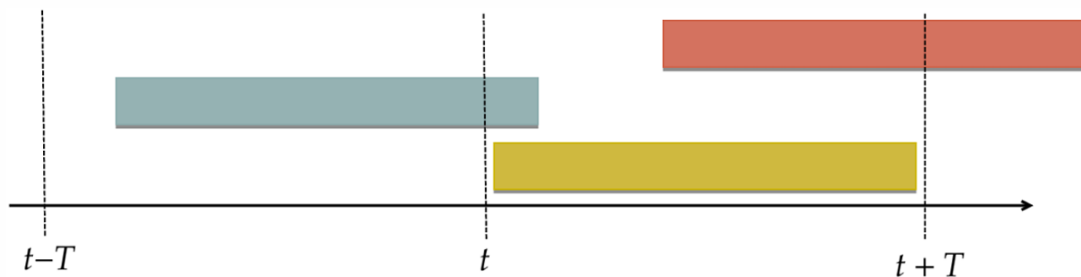
$$\eta = \mathbb{P}_{suc} = G \cdot e^{-2G}$$

כמו מקודם, אפשר לחשב מנקודת מבט של פקטה ולהגיע לתוצאות דומות. אנחנו יודעים שהפקטה נשלחה, אז נדרוש שבסלול הקודם ובסלול הנוכחי אף אחד אחר לא נשלח. נקבל:

$$\mathbb{P}_{suc} = \mathbb{P}_{k=0}(t = T) \cdot \mathbb{P}_{k=0}(t = T) = e^{-gT} \cdot e^{-gT} = e^{-2gT} = e^{-2G}$$

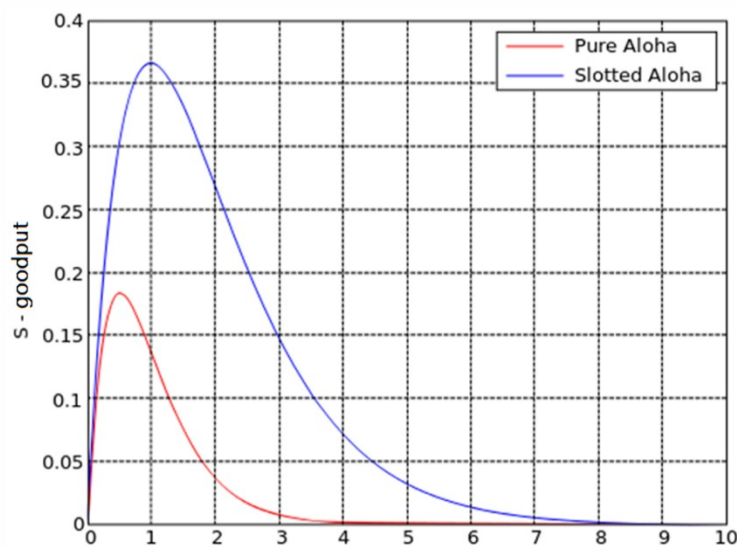
ואז ה-Goodput יהיה זהה גם כן, רק נצטרך להכפיל בהסתברות שהפקטה נשלחה:

$$\eta = G \cdot e^{-2G}$$



שוב, נרצה למקסם את ה- G עבור ה-Goodput. נקבל $G^* = \frac{1}{2}$. לכן $\eta_{G=\frac{1}{2}} = \frac{1}{2e}$ כמו בגרסה הבינומית.

השוואה בין Pure לבין Slotted



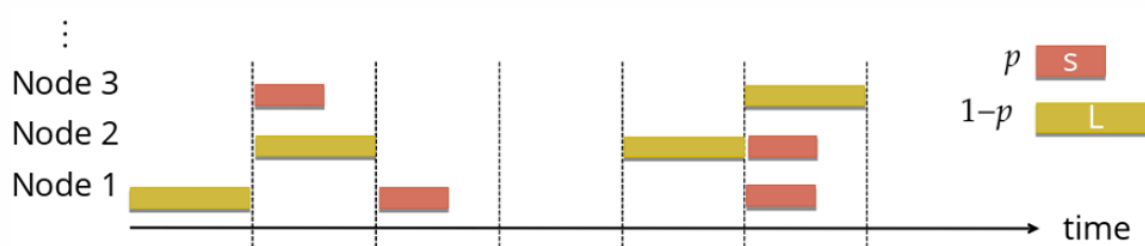
אם ה- G שנחבר לשים (ציר x) יהיה גדול נקבל יותר התנגשויות ולכן ה-goodput צונח. באופן דומה עבור G קטן מדי, יותר מדי קודקודים "חוששים" לשלוח, וגם בצד הזה נראה דעיכה מאוד גדולה ב-goodput.

שאלות תרגול

שאלה 1

נתונה מערכת Slotted ALOHA שבה הודעות נשלחות כתהליך פואסוני עם קצב שליחה g . ישנן שתי סוגי הודעות: פקטות עם S ביטים שמגיעות בהסתברות p , ופקטות עם L ביטים בהסתברות $1 - p$.

נתון גם $L > S$. קצב שליחה הוא $1 \frac{\text{bit}}{\text{sec}}$ וגודל slot הוא L שניות ($T = L$).



מה ההסתברות לשליחה מוצלחת?

ניזכר כי מתקיים: $X_p \sim \text{Poi}(gT)$ then $\mathbb{P}_{t=T}(X = i) = \frac{(gT)^i}{i!} e^{-gT}$

$$\mathbb{P}_{suc,L} = (1 - p) \cdot \mathbb{P}_{t=L}(k = 1) = (1 - p)gL e^{-gL}$$

כלומר ההסתברות לשלוח הודעה באורך L , כפול ההסתברות שבטווח זמן L רק אחד ישלח. עבור S :

$$\mathbb{P}_{suc,S} = p \cdot \mathbb{P}_{t=L}(k = 1) = pgL e^{-gL}$$

מהו ה-Goodput?

נגדיר משתנה מקרי שיועיל לנו לחישוב ה-goodput. הוא יעבוד בראי של "כמה ביטים של הודעות מוצלחות הצלחתי לשדר":

$$T_{suc} = \begin{cases} L & \text{successful L packet} \\ S & \text{successful S packet} \\ 0 & \text{else} \end{cases}$$

$$\mathbb{E}[T_{suc}] = L \cdot \mathbb{P}_{suc,L} + S \cdot \mathbb{P}_{suc,S}$$

$$\eta = \frac{\mathbb{E}[T_{suc}]}{L} = \frac{L}{L} \cdot \mathbb{P}_{suc,L} + \frac{S}{L} \cdot \mathbb{P}_{suc,S} = gL e^{-gL} \left(\frac{S}{L} p + \frac{L}{L} (1 - p) \right)$$

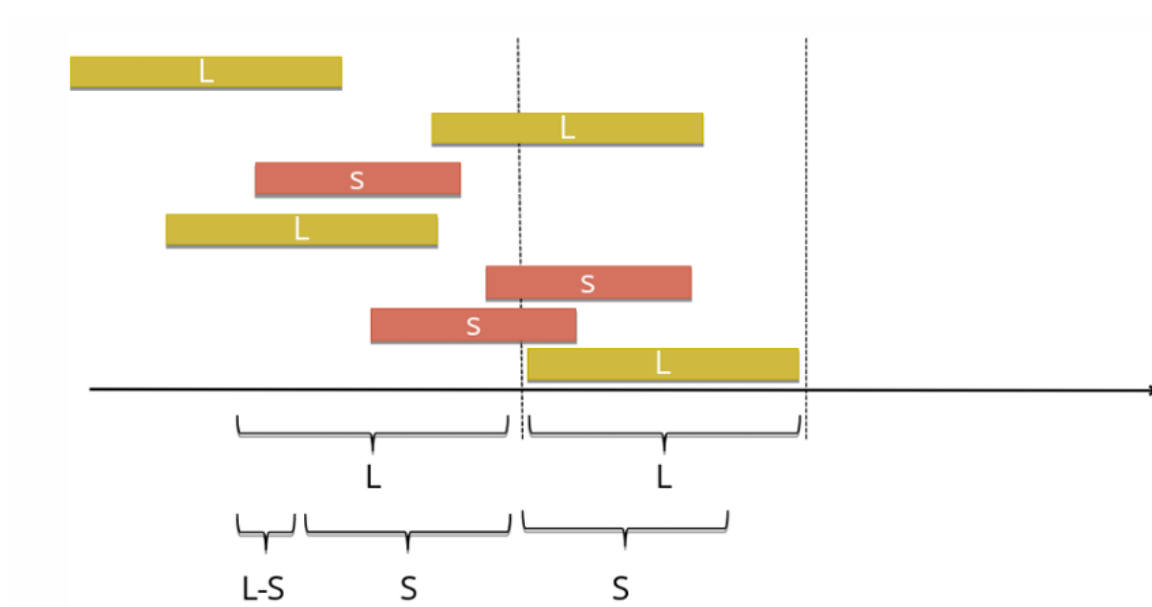
$$\boxed{\eta = gL e^{-gL} \left(\frac{S}{L} p + 1 - p \right)}$$

שאלה 2

נסתכל על נתוני השאלה הקודמת, אבל הפעם בהנחה שהמערכת היא Pure ALOHA.

מה ההסתברות לשליחה מוצלחת?

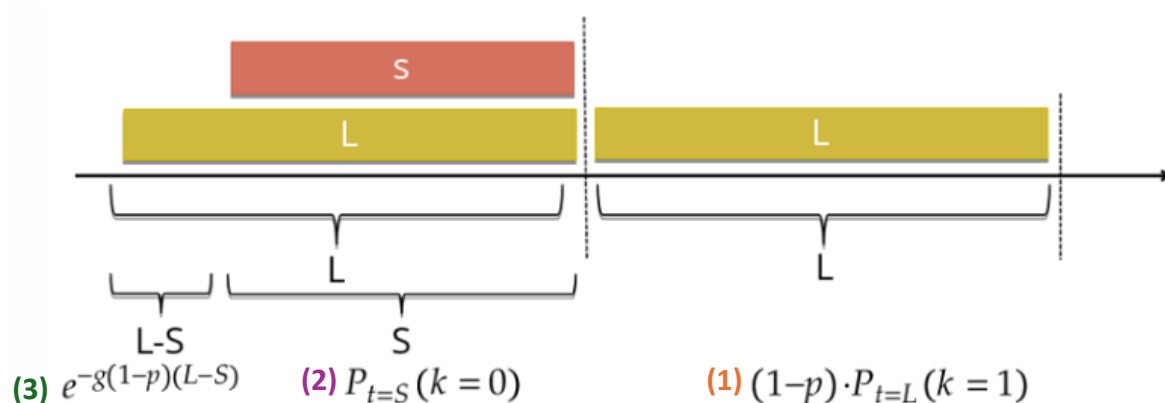
בגלל שזה Pure, נצטרך לעשות חישוב איפה יכולות להיות התנגשויות:



הודעה מסוג L יכולה להפריע בטווח של $(-L, +L)$. הודעה מסוג S יכולה להפריע בטווח $(-S, +L)$. נגדיר את זמני ההגעה לפי סוג:

$$g_S = p \cdot g \quad ; \quad g_L = (1 - p) \cdot g$$

סיכויי ההצלחה עבור הודעה מסוג L מתחלק ל-3 מקרים:

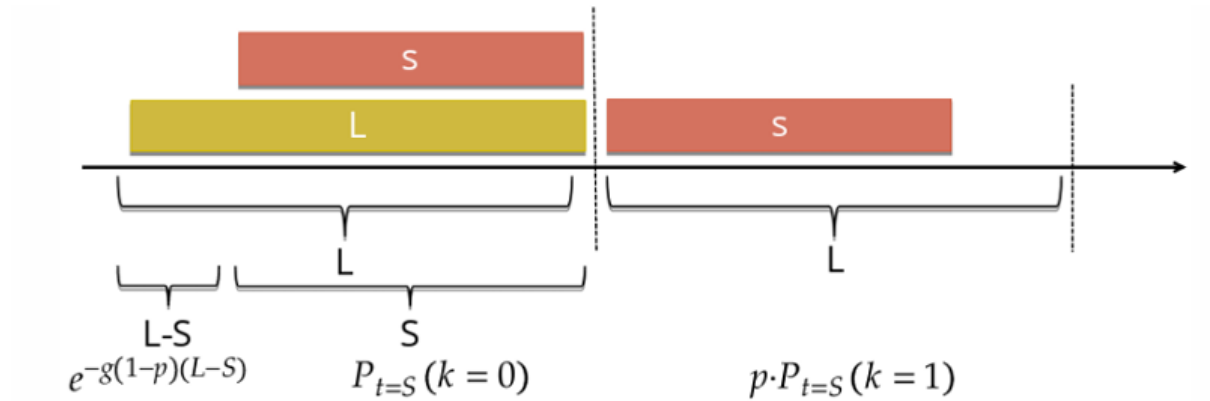


קודם כל (1), נדרוש שבזמן L שב-slot שבו התחלנו לשדר – אף אחד אחר מלבדנו לא ישדר ($k=1$). כמו כן (2) בין $(-S, 0)$ אנחנו נדרוש שאף סוג לא יישלח (לא S ולא L). לבסוף (3) נרשה כאן שליחה רק של הודעה מסוג S (לא יתנגש), אבל לא נרשה L .

שלושת המאורעות האלו זרים, לכן נכפיל אותם כדי לדרוש את שלושתם ונקבל:

$$\mathbb{P}_{suc,L} = (1-p)e^{-g(2L-p(L-S))}$$

סיכויי ההצלחה עבור הודעה מסוג S מתחלק ל-3 מקרים, נחשב באופן דומה:



עם הסברים דומים לחישוב סיכויי ההצלחה עבור הודעה מסוג L . נכפיל ונקבל:

$$\mathbb{P}_{suc,S} = pe^{-g(L+S-p(L-S))}$$

שוב נגדיר משתנה מקרי:

$$T_{suc} = \begin{cases} L & \text{successful L packet} \\ S & \text{successful S packet} \\ 0 & \text{else} \end{cases}$$

$$\mathbb{E}[T_{suc}] = L \cdot \mathbb{P}_{suc,L} + S \cdot \mathbb{P}_{suc,S}$$

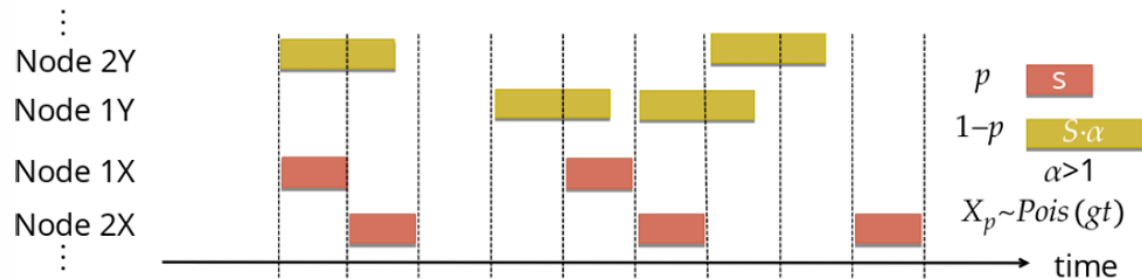
$$\boxed{\eta} = \frac{\mathbb{E}[T_{suc}]}{L} = \frac{L}{L} \cdot \mathbb{P}_{suc,L} + \frac{S}{L} \cdot \mathbb{P}_{suc,S} = (1-p)e^{-g(2L-p(L-S))} + \frac{S}{L}pe^{L+S-p(L-S)}$$

שאלה 3

במערכת Slotted ALOHA עם גודל slot של S יש שני קווים:

X שמכיל $N \cdot p$ קודקודים ששולחים הודעות בגודל S ומשתמשים ב- $\frac{1}{4}$ מרוחב הפס.

Y שמכיל $N \cdot (1 - p)$ קודקודים ששולחים הודעות בגודל $S \cdot \alpha$ עבור $\alpha > 1$ ומשתמשים ב- $\frac{3}{4}$ מרוחב הפס.



X ו- Y לא מפריעים אחד לשני, אפשר לדמיין את זה כשני מחשבים שמחוברים על ידי 4 bus. או ששולחים הודעות על bus יחיד, או על 3 bus ביחד (כדי להמחיש את הרעיון). אם שולחים הודעה על הפס היחיד זה לא יתנגש בהודעות שנשלחו על 3 ה-bus האחרים.

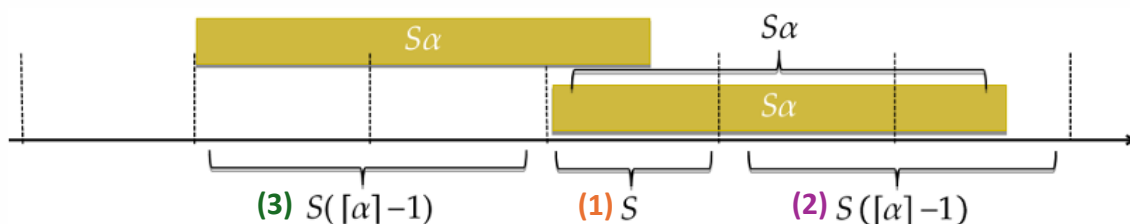
נגדיר את זמני ההגעה לפי סוג:

$$g_{S_\alpha} = (1 - p) \cdot g \Rightarrow S_p^\alpha \sim \text{Poi}((1 - p)g)$$

$$g_S = p \cdot g \Rightarrow S_p \sim \text{Poi}(pg)$$

מה ההסתברות לשליחה מוצלחת?

עבור הודעה מסוג S_α :



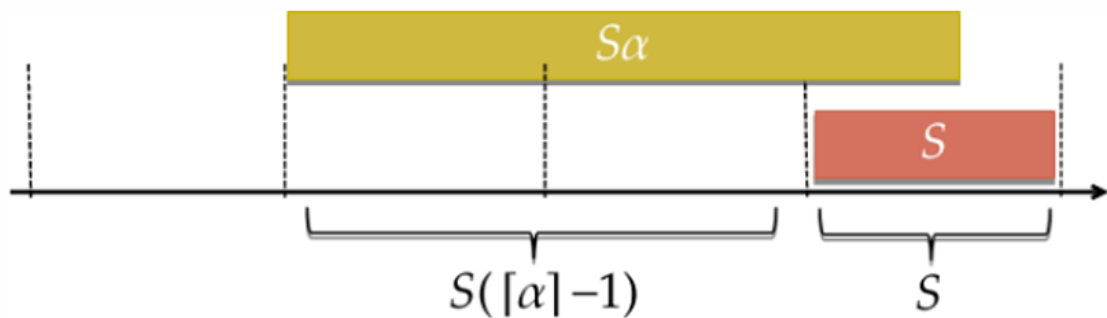
מתי עלולה להיות בעיה עם הודעה מהסוג? בדרך כלל ב-Slotted אנחנו צריכים להסתכל רק על ה-slot שלנו, אבל מאחר שההודעה שאנחנו שולחים היא יותר מאורך של slot, גם ה-slots הקודמים עלולים להפריע. מה שיעניין אותנו זה הערך $[\alpha]$, כי בפועל אם זה מתנגש בחלק מה-slot זה גם יהרוס את השידור. נחלק ל-3 מקרים.

במקרה (1) אנחנו מסתכלים על התחלה באורך S שבה נרצה שידור יחיד, ואז ב-(2) מסתכלים על כל השאר פחות ה-slot הראשון. המקרה ה-(3) דומה כי גם בו לא נרשה אף שידור:

$$\mathbb{P}_{suc, S_\alpha} = \mathbb{P}_{k=0}^{S_\alpha}(t = S([\alpha] - 1)) \cdot \mathbb{P}_{k=1}^{S_\alpha}(t = S) \cdot \mathbb{P}_{k=0}^{S_\alpha}(t = S([\alpha] - 1))$$

$$\mathbb{P}_{suc, S_\alpha} = g(1-p)Se^{-g(1-p)S(2[\alpha]-1)}$$

עבור הודעה מסוג S :



כאן לא נוצרת אותה בעיה כמו מקודם מאחר שפקטור S_α לא מתערבות בשידור של פקטור מסוג S , ומאחר שזה Slotted ALOHA לא יכולה להישלח עוד הודעה מסוג S במקביל:

$$\mathbb{P}_{suc, S} = \mathbb{P}_{k=1}^S(t = S) = gpSe^{-gps}$$

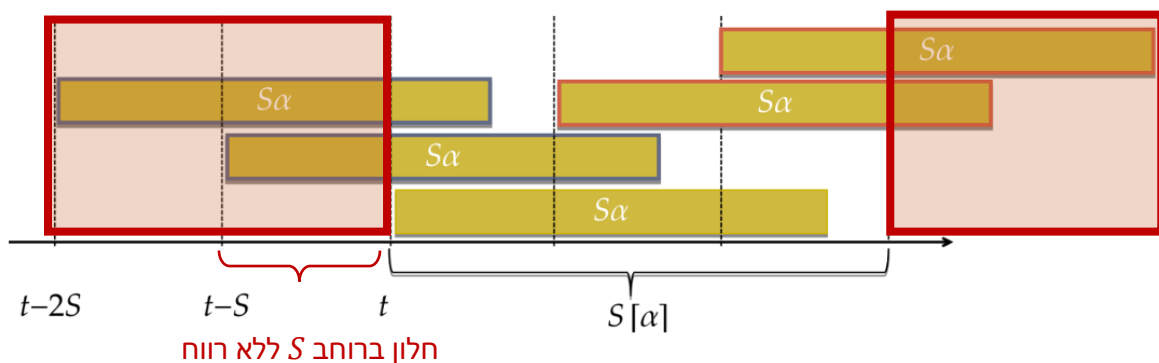
מהו ה-Goodput?

נשים לב שהודעות מסוגים שונים לא מפריעות אלו לאלו, אז נתייחס לכל מקרה לגופו.

עבור הודעות S_α :

נסתכל על טווח $S_{[\alpha]}$. ונרצה לבדוק כמה ביטים נרוויח בו.

עבור הודעה שנשלחה בהצלחה בזמן t נרוויח $S_{[\alpha]}$. אם מישהו שלח לפני זה, גם נרוויח ביטים – אבל רק בחלון שבו אנחנו מודדים (כלומר לא מה שמסומן באדום בתמונה):



עבור המקרה שבו שלחנו בדיוק בזמן t נרוויח $S \cdot \alpha$ ביטים.

⋮

עבור המקרה שבו שלחנו בזמן $t - S$ נרוויח $S(\alpha - 1)$ ביטים. חיסרנו S בגלל **החלון ברוחב S**

ללא רוח, כי הרווח התקבל לפני זמן המדידה.

⋮

עבור המקרה שבו שלחנו בזמן $t + S$ נרוויח $S(\lceil \alpha \rceil - 1)$ ביטים, שוב מאחר שהתחלנו למדוד

S שניות לאחר זמן t מתוך החלון עליו אנחנו מסתכלים שהוא בגודל $S \cdot \lceil \alpha \rceil$ אז נותרו רק $S \cdot \lceil \alpha \rceil - S$ ביטים.

מאחר שהזמן הוא רציף, יש הרבה מקרים בין לבין.

נגדיר משתנה מקרי:

$$T_{suc, S\alpha} = \begin{cases} S\alpha & \text{successful packet that starts at } t \\ S(\alpha - 1) & \text{successful packet that starts at } t - S \\ S(\alpha - 2) & \text{successful packet that starts at } t - 2S \\ \vdots & \vdots \\ S(\lceil \alpha \rceil - 1) & \text{successful packet that starts at } t + S \\ S(\lceil \alpha \rceil - 2) & \text{successful packet that starts at } t + 2S \\ \vdots & \vdots \\ S \cdot 1 & \text{successful packet that starts at } t + S(\lceil \alpha \rceil - 1) \\ 0 & \text{else} \end{cases}$$

נחשב את התוחלת ונקבל:

$$\begin{aligned} \mathbb{E}[T_{suc, S\alpha}] &= P_{suc, S\alpha} \left(S\alpha + \sum_{k=1}^{\lceil \alpha \rceil - 1} S(\alpha - k) + \sum_{k=1}^{\lceil \alpha \rceil} S(\lceil \alpha \rceil - k) \right) \\ &= P_{suc, S\alpha} S \left(\alpha + \sum_{k=1}^{\lceil \alpha \rceil - 1} \alpha - \sum_{k=1}^{\lceil \alpha \rceil - 1} k + \sum_{k=1}^{\lceil \alpha \rceil - 1} k \right) \\ &= P_{suc, S\alpha} S (\alpha + (\lceil \alpha \rceil - 1)\alpha) \end{aligned}$$

$$\mathbb{E}[T_{suc, S\alpha}] = P_{suc, S\alpha} S\alpha \lceil \alpha \rceil$$

את **הירוק** פיצלנו ל-2, **ובתכלת** הפכנו את סדר הסכימה. קיבלנו את כמות הביטים שנקבל. נעשה את אותו

דבר להודעות מסוג S , ושם (למזלנו) הסיפור הרבה יותר פשוט:

$$T_{suc,S} = \begin{cases} S & \text{successful S packet} \\ 0 & \text{else} \end{cases} \implies \mathbb{E}[T_{suc,S}] = S \cdot P_{suc,S}$$

מה ה-Goodput?

$$\begin{aligned} \eta &= \frac{3}{4}\eta_{S_\alpha} + \frac{1}{4}\eta_S = \frac{3}{4} \frac{\mathbb{E}[T_{suc,S_\alpha}]}{S[\alpha]} + \frac{1}{4} \frac{\mathbb{E}[T_{suc,S}]}{S} \\ &= \frac{3}{4} \frac{S_\alpha[\alpha] \cdot P_{suc,S_\alpha}}{S[\alpha]} + \frac{1}{4} \frac{S \cdot P_{suc,S}}{S} \\ &= \frac{3}{4} \alpha P_{suc,S_\alpha} + \frac{1}{4} P_{suc,S} \end{aligned}$$

נבחין שרוחב הפס בשאלה היה מחולק ל $\frac{1}{4}$ ול $\frac{3}{4}$, ולכן גם יש צורך להבדיל ב-Goodput שעובר בכל אחד מהם

לפי הקבוע המתאים.

הרצאה 5

תזכורת והקדמה

נמשיך בנושא CSMA – איך לאפשר לכמה מחשבים (נקודות שידור) לתקשר כאשר הם חולקים domain משותף. היום נדבר על CSMA\CA שזה וריאנט נוסף של CSMA לרשתות wireless, לאחר מכן נעבור לרשתות גדולות יותר, switches ואיך אפשר לחבר אותם יחד לרשת גדולה ומורכבת.

Wi-Fi CSMA/CA ברשתות

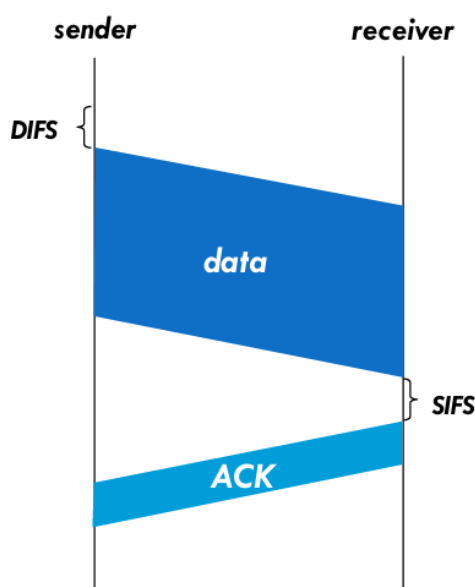
מצגת 2

שקף 99-100

תזכורת – CSMA זו שיטה בה מסתכלים על הערוץ לפני השידור, "מקשיבים" האם מישהו אחר מנסה לשדר. דיברנו על CSMA/CD שבו יש צפיית התנגשויות. אם התנגשנו עם מישהו אחר נפסיק את השידור.

ברשתות Wi-Fi המנגנון לא עובד כל כך טוב, בעיקר כי קשה לזהות שמישהו אחר משדר וגם כי קשה להקשיב בזמן שאנחנו משדרים בעצמנו (בעיית חומרה טכנית). הציגו וריאנט אחרת שנקרא CSMA/CA (Collision Avoidance) שמטרתו לנסות במקום לזהות התנגשויות להימנע מהן מלכתחילה, כלומר "להקדים תרופה למכה" (לא מובטח שיעבוד).

המנגנון: נחכה לערוץ שיתפנה זמן מסוים לפני שמתחילים את השידור (DIFS) ואם הוא לא פנוי אז יש מנגנון backoff שמערב אקראיות בדומה ל-CSMA/CD, והרעיון הוא שנקבל ACK מהמקבל אחרי SIFS (Short interface space) שזה זמן מאוד קצר שהמקבל מחכה לראות שהערוץ פנוי לפני שהוא שולח את ה-acknowledgement שלו (הסיבה היא שה-ACK בעצמו זו הודעה, אז הוא מחכה שהערוץ יתפנה כדי לא לדרוס מישהו אחר. אילוסטרציה:

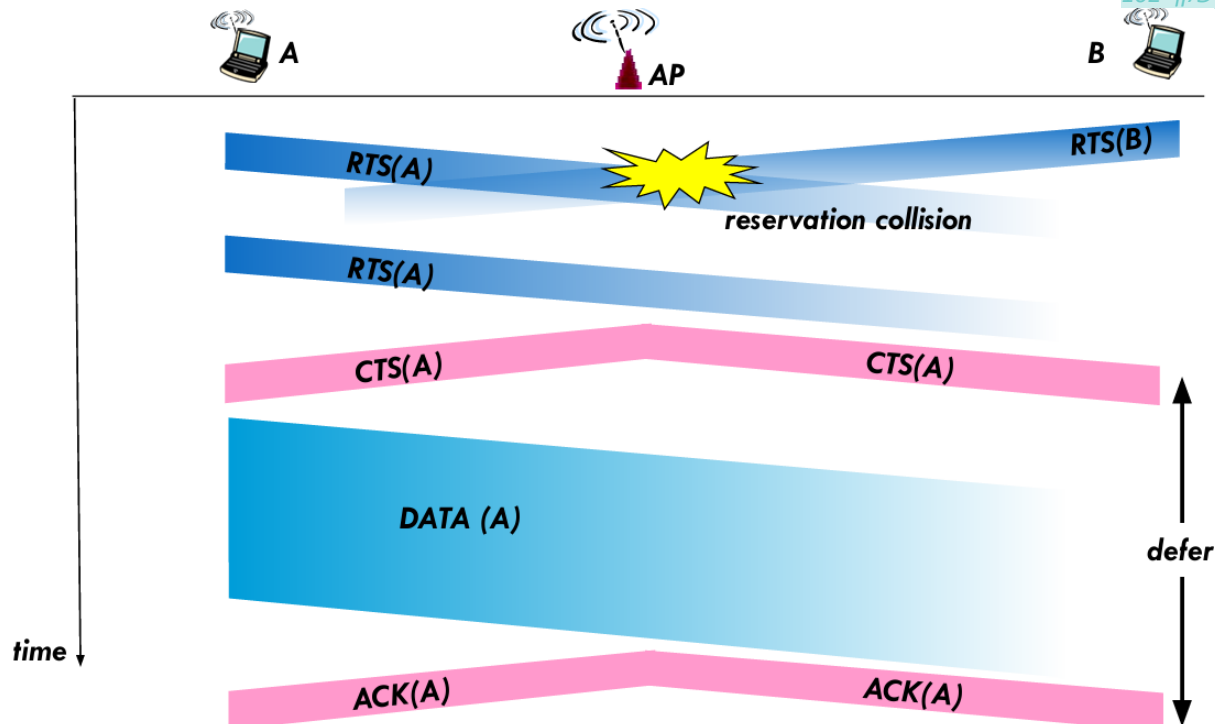


נדגיש שוב, $DIFS > ACK$, והסיבה לכך היא שאפקטיבית זה נותן עדיפות ל-ACK על פני הודעות data (אם שניים מתחילים לחכות בערך באותו זמן, מי שרוצה לשלוח ACK יקבל עדיפות ובתקווה יגרום לשולח להתחיל את המנגנון מחדש במקום להתנגש איתו).

שקף 101

המנגנון הזה עובד אבל הוא לא מבטיח שלא יהיו התנגשויות – הסיכוי להתנגשויות יחסית גבוה. כדי למזער את הסיכוי להתנגשות הרבה פרוטוקולים ברשתות wireless יש הרבה מכשירים שמחוברים ל-base station אחד, ומה שעושים הרבה פעמים זה להשתמש ב-CSMA/CA כדי לשמור רוחב פס (ברגע שאני רוצה לשדר אני אשלח request to send לתחנת הבסיס בתקווה שתפיץ לכל מי שיחלוק איתי תווך broadcast שזכות הדיבור עומדת להיות שלי). הסיכוי להתנגשות בהודעות הראשונות קיים, אבל אם קיבלתי מה-base station הודעה שמאשרת clear to send אני יכול להתחיל לשדר ואף אחד לא ישדר במקביל אליי.

שקף 102



זה מתאים למודל שבו התקשורת עוברת דרך נקודה מרכזית (כמו למשל Wi-Fi router או סלולר).

שקף 103

סיכום – אפשר לסווג את הסוגים של פרוטוקולים של גישה לערוץ תקשורת משותף ל-2 סוגים:

1. Centralized Coordination – כשמחלקים את הערוץ לפי תדר או זמן (Circuit Switching).
 2. Random Access – פרוטוקולים לא דטרמיניסטיים שמשתמשים בהטלת מטבע כדי להוציא את המשתתפים ממצב של סנכרון ולהימנע ממצב שבו כמה משתתפים משדרים יחד (ALOHA, CSMA).
- (Variants).

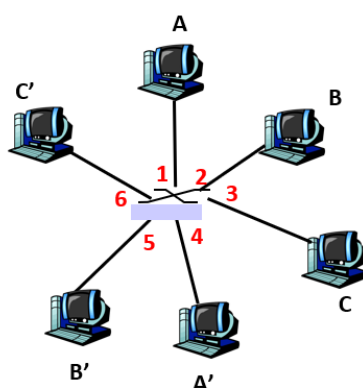
נרצה לחבר broadcast domains בדרך יעילה – הדרך לעשות זאת היא באמצעות Switch. מדובר במכשיר עם ports שמתחברים אליו בצורה פיזית ויש לו טבלה שאומרת לו בעת קבלת הודעה לאיזו רגל/port להוציא אותה, ולכל רגל יש תור הודעות משלה. זה מאפשר לו לשלוח במקביל על כל ה-ports במקביל ו"לשבור" את ההתנגשויות. ה-Switch ממתג את התקשורת כי הוא יודע לאן היא צריכה להגיע.

כל אחת מהרגליים של ה-switch מחוברת ל-broadcast domain ורץ CSMA/CD. הסוויץ' שקוף מבחינת התקשורת, מחשב הקצה לא שולח הודעה לסוויץ', אלא הוא שם כדי להעביר את ההודעות. הסוויץ' שולח הודעות גם בעצמו, אבל הן הודעות ללא תוכן אלא מטרתן לסנכרן הודעות עם סוויצ'ים אחרים (יפורט בהמשך).

הסוויץ' לומד את הטבלה שהזכרנו מקודם לבד (שאומרת לסוויץ' לאן להוציא כל הודעה), כלומר יבין לבד איפה נמצא כל מכשיר קצה ולאן להעביר את ההודעות (אין צורך בקונפיגורציה כדי "להכשיר אותו" למשימה).

כעת נפרט את התכונות של הסוויץ':

שכבת broadcast domains



בתמונה יש 6 מחשבים שמחוברים כל אחד לרגל אחרת של הסוויץ'. זה מאפשר ל-A לדבר עם A' ול-B לדבר עם B' במקביל בלי שיפריעו זה לזה, בשונה מ-hub שפשוט מחבר את כולם דרך bus משותף (ההודעות היו מגיעות לכולם). השימוש ב-CSMA/CD היה ממזער את זה, אבל עדיין היו בעיות.

טבלת Switch

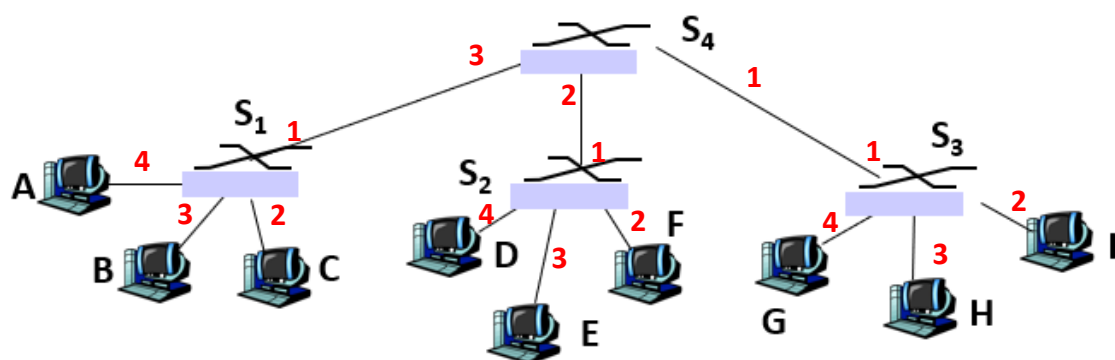
הטבלה פשוטה – כתוב בה MAC Address ואיזו רגל משויכת אליה, ו-timestamp שנותן אפשרות לנקות את הטבלה מ-entries לא רלוונטיים. הזכרנו שסוויץ' חדש לא דורש קונפיגורציה כדי להתחיל לפעול, אז אם כך איך בונים את הטבלה? הסוויץ' מסתכל על מקור ההודעות שמגיעות אליו ומבין שאותו מקור נמצא מאחורי הרגל שדרכה הגיעה ההודעה, ומתעד גם TTL (Time to Leave) שכשהוא מגיע ל-0 ימחק את ההודעה.

אם כתובת היעד נמצא בטבלה: ייתכן מצב בו המקור והיעד נמצאים באותו domain, מצב כזה ייתכן (למשל על ידי חיבור ב-hub) ובמקרה זה לא נעשה כלום – הסוויץ לא צריך לטפל בהודעה זו, ויזרוק אותה (המקור והיעד מחוברים לאותה רגל). אחרת, יוציא את ההודעה מהרגל הנכונה.

אם כתובת היעד לא נמצאת בטבלה: ההודעה תישלח לכל ה-interfaces (כל הרגליים) חוץ מרגל של השולח (כי במקרה זה הוא ישמע את ההודעה בדיוק כמו שהסוויץ שמע). כשהיעד יגיב (וזוה נפוץ) הסוויץ ישמור את הכתובת ויבין להבא מאחורי איזה interface הנקודה עומדת, ויודע כבר להחזיר לכתובת המקורית כי התקבלה ממנה הודעה בעבר.

שרשרת סוויצ'ים

בארכיטקטורה הזאת אפשר לשרשר סוויצ'ים. למשל בסוויץ S_4 מאחורי כל רגל שלו מחוברים 3 מחשבים, למשל A, B, C, יהיו כולם מאחורי אותה רגל בטבלה.



התכונה של ה-self learning נשמרת גם בשרשרת הסוויצ'ים. נמחיש באמצעות דוגמה כאשר A שולח הודעה ל-I ואז I מגיב חזרה ל-A, ונניח שכל הסוויצ'ים לא מכילים אף מידע בטבלאות.

A שולח הודעה שמגיעה דרך ה-interface היחיד שיש לו – הסוויץ הראשון של S_1 . כך S_1 למד איפה A נמצא, אבל לא יודע איפה I נמצא ולכן שולח אותה לכל הרגליים חוץ מרגל 4 (שממנו ההודעה הגיעה). המחשבים ברגליים 3 ו-2 מקבלים את ההודעה, מסתכלים על ה-MAC Address ורואים שזה ממוען ל-I (לא אליהם) ולכן ברטיס הרשת זורק את ההודעה.

מה שמעניין קורה דרך רגל 1, שממנה ההודעה מגיעה ל- S_4 . כבר מהודעה זו S_4 למד ש-A נמצא מאחורי רגל מספר 3. הוא לא יודע איפה I נמצא ולכן עושה שוב fludding להודעה לכל הרגליים חוץ מלזו שדרכה הגיעה ההודעה (ל-1 ול-2). מכך גם S_3 וגם S_2 יודעים איפה A נמצא.

ההודעה מגיעה ל- S_3 שלא יודע איפה I נמצא ולכן מבצע fludding גם הוא, H ו-I יזרקו את ההודעה כי היא לא ממוענת אליהם – אבל I יקבל אותה ויגיב.

בתגובה, S_3 לומד ש-I נמצא מאחורי רגל 2, והיעד של ההודעה היא A שאותו הוא כבר מכיר – יכוון לרגל מספר 1, זה יגיע ל- S_4 שילמד איפה I נמצא ויכוון ל-A ברגל 3, יעבור משם ל- S_1 שילמד איפה I נמצא ויכוון לרגל 4 (כי כבר מופיע לו A בטבלה).

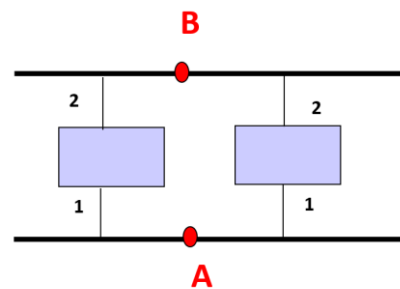
שקף 11

רשת ארגונית

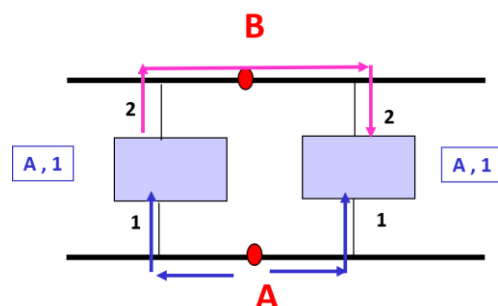
אנחנו נראה מחשבים שמחוברים דרך סוויצ'ים, וסוויצ'ים שמחוברים לסוויצ'ים אחרים. לרוב יהיה סוויץ' ראשי שמחובר לראוטר שמוציא את הרשת לאינטרנט (לא הגדרנו עדיין ראוטר, נגיע לשם), ויכולים להיות שרתים. בניגוד לסוויצ'ים ראוטר אינו שקוף (יש לו כתובת MAC), והוא "שובר" את הרשת שלנו, לכן כשנדבר על רשת בשיעור הזה נדבר עד הראוטר ולא החוצה ממנו.

שקף 12

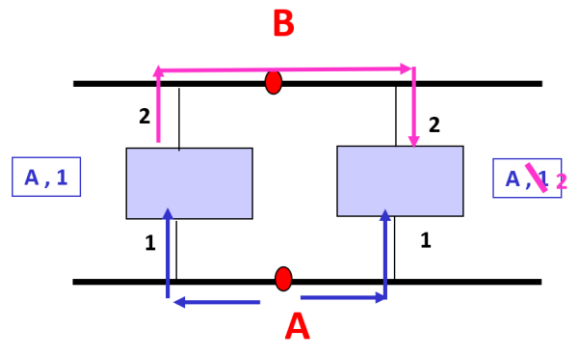
האם יכולות להיות לולאות? כן. נסתכל על רשת די פשוטה:



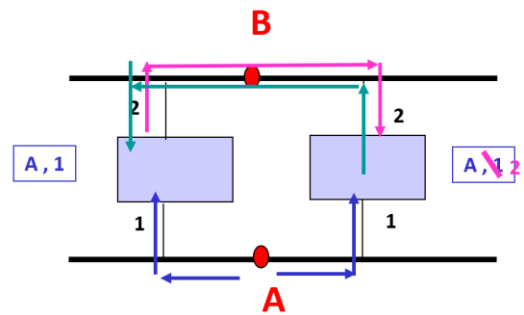
המלבנים הם סוויצ'ים ולצידם מספרי ה-ports, A ו-B הן נקודות קצה. מה הבעיה שיכולה להיווצר? A שולח הודעה (בכחול) שמגיע ל-2 הסוויצ'ים. הסוויץ' הראשון ינסה להוציא אותה דרך הרגל השנייה (בוורוד):



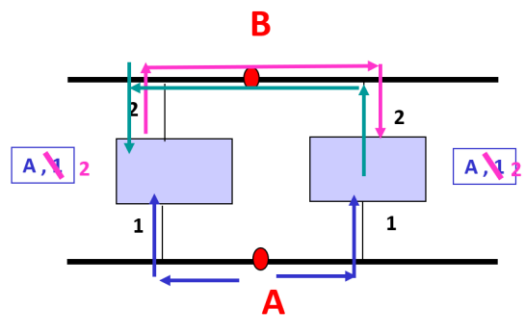
במלבנים אנחנו רואים את טבלת הסוויץ'. כל סוויץ' למד ש-A נמצא מאחורי רגל מספר 1 (ה-TTL קיים אבל לא רלוונטי לצורך המחשת הדוגמה). הבעיה היא שעבשיו הסוויץ' הימני חושב שהוא קיבל את ההודעה מ-A וש-A כנראה עבר מקום אז הוא מעדכן את המיקום שלו בטבלה:



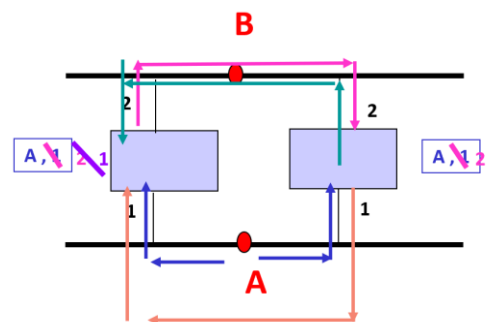
עכשיו יש לו הודעה שהוא קיבל מקודם (הבחולה), והוא רוצה לשלוח אותה עכשיו (בירוק):



עכשיו היא מגיעה לסוויץ' השמאלי, וגם הוא חושב ש-A עבר מקום אז מעדכן שהוא נמצא מאחורי רגל 2:



הסוויץ' הימני קיבל הודעה (בוורוד), אז הוא צריך להוציא אותה מהרגל השנייה (בכתום) ושוב נעדכן כתובות:

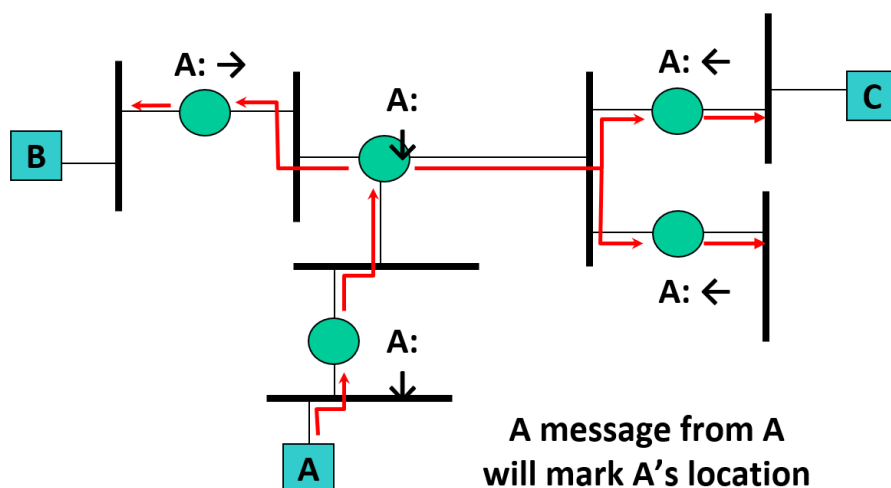


וכך קיבלנו הודעה שמסתובבת במעגל – בזבז משאבי רשת.

STP (Spanning Tree Protocol)

שקף 13-18

היינו רוצים רשת שהיא עץ בו לא יכולות להיווצר מעגלים. העיגולים הם סוויצ'ים, הקווים המודגשים הם broadcast domains והריבועים מכשירי קצה:



שקף 19

המטרה היא לבנות עץ. איך נבנה את הטופולוגיה של הרשת כך שתהיה בצורה כזו?

אופציה ראשונה היא קיום Network Administrator יעיל מאוד עם צוות IT שזהר לא לחבר כבלים במקומות שעלולים ליצור מעגל – קשה לביצוע, וגם אין בעץ redundancy (ברגע ש-port מתנתק הרשת מתחלקת לשני חצאים). לא היינו רוצים הגבלה פיזית על החיבורים, מעגלים יכולים לאפשר כמה דרכים להגיע לאותו יעד. נרצה לשפר את הגישה הזו.

נתאר פרוטוקול שהסוויצ'ים יריצו בינם לבין עצמם שמטרתו לקבוע איזה interfaces לבטל בכל סוויץ' – לא באמצעות ניתוק פיזי, אלא רק בצורה לוגית נוודא שכל סוויץ' יבנה חלק מהפורטים ונוודא שלא יעביר תקשורת דרכם. תצורת הרשת תהיה גרף, אבל מבחינה לוגית יתקיים עץ פורש (נוכל להגיע מכל מקום לכל מקום).

שקף 20,22

פרוטוקול עצים פורשים – Spanning Trees Protocol (STP)

הפרוטוקול הזה הוא לב רשתות Local Area. בפרוטוקול הזה הסוויצ'ים עצמם שולחים הודעות שמטרתן היא לקנפג את העץ. הסוויץ' ישלח הודעה שיעדה הוא סוויצ'ים אחרים שמחוברים אליו. להודעה קוראים BPDU (Bridge Protocol Data Unit) ויש סטנדרט שמגדיר איך היא נראית ואיך תטופל (IEEE 802.1D).

מטרות הפרוטוקול: לקבוע אילו סוויצ'ים לבטל – כל סוויצ' ידע דרך איזה interfaces הוא לא יעביר דרכן הודעות למשתמשי קצה. כדי לעשות זאת, צריך לענות על 3 שאלות לפי הסדר הבא:

1. מי שורש העץ?
2. איך ליצור spanning tree של סוויצ'ים?
3. איזה סוויץ' יחבר את ה-broadcast domain? (כל האחרים שמחברים את הדומיין יבטלו את ה-interface)

שקף 23

הפרוטוקול הזה הוא חלק ממשפחה גדולה של פרוטוקולים שנקראים Self-Stabilizing Protocols – פרוטוקול מבוזר שרץ על ידי מכשירים לא מתואמים ומטרתו להוביל את הרשת למצב יחיד (למשל, לוודא שכל הסוויצ'ים ישתמשו באותו העץ).

כעת נענה על השאלות:

שקף 24

1. בחירת שורש לעץ

מניחים שלכל סוויץ' יש מזהה ייחודי. כל סוויץ' ישלח את ה-BPDUs דרך כל ה-interfaces שלו, ואחת השדות יציין מה ה-id של הסוויץ' שלדעתו הוא שורש העץ. במילים אחרות, נסכים למשל שה-id הכי נמוך ברשת הוא ה-root, ואני אשלח שאני השורש. אם אני מקבל מאחד השכנים שלי כתובת נמוכה יותר, מעתה אני אעדכן שזה ה-root לדעתי. ה-id של השורש עומד לחלחל לכל הרשת עד שתתקבל הסכמה.

שקף 25-30

2. חישוב עץ פורש של סוויצ'ים

אנחנו עומדים להשתמש ב**אלגוריתם בלמן-פורד**. המטרה היא שכל סוויץ' ימצא את ה-interface שמחובר למסלול הקצר ביותר לשורש. חוץ מלשלוח מה ה-id של השורש, נשלח גם את המרחק שלנו ממנו. כלומר אם אני למשל חושב שאני השורש אשלח את ה-id שלי ומרחק 0. אחרת אם קיבלתי ממישהו אחר id יותר נמוך, אעדכן את כתובת ה-id של השורש להיות זו שראיתי עכשיו, ובמרחק אוסיף +1. כל סוויץ' ייקח את המינימום מכל השכנים שלו – וזה יהיה המרחק מהשורש. ה-interface שממנו קיבלתי את המרחק המינימלי יהיה ה-**parent pointer** שלי, המצביע לכיוון השורש. *דוגמה רצה בשקפים 27-30. הלינקים המודגשים ב-bold יהיו חלק מהעץ הפורש של הסוויצ'ים (לפחות עד עדכון נוסף). לינקים שאינם מודגשים יכולים להתבטל (לא פיזית, רק לוגית).*

שקף 31

האלגוריתם עובד וכולם מתכנסים לאותה תוצאה אפילו כשהוא עובד בצורה א-סינכרונית אצל כולם. לא משנה מה סדר ההודעות, כל סוויץ' ידע לזהות מה הלינק שלו לשורש.

שקף 32

3. בחירת switch שיחובר ל-broadcast domain

לכל סוויץ' יש פוינטר אחד לשורש, אבל אם יש broadcast domain שמחובר לכמה סוויצ'ים עדיין יש לנו מעגל (הודעה שתשלח על אותו broadcast domain תצא החוצה דרך שני מסלולים נפרדים שמגיעים לשורש מאותו מחשב). המטרה היא בכל broadcast domain להשאיר רק port אחד בסוויץ' אחד שיחבר אותו הלאה לרשת. איך מבצעים זאת?

כששולחים BPDUs אנחנו שולחים אותה לכל הרגליים של הסוויץ' – כלומר אם יש שני סוויצ'ים שמחוברים לאותו broadcast domain הם ישמעו את ההודעות אחד של השני. לכן, נסיף להודעה גם את ה-id של הסוויץ' השולח, וכל סוויץ' יבחר אם הוא צריך לנתק את הרגל שלו לאותו broadcast domain לפי ההודעות שקיבל. אם הוא שמע הודעה מסוויץ' שמחובר במסלול קצר יותר – לשם יחבר את הרגל שלו. אם שניהם באותו אורך נשבור שוויון לפי id ייחודי (נתעדף id נמוך). ייתכן מצב שבו 2 ports מאותו סוויץ' מחוברים לאותו broadcast domain (ואז יש שוויון גם ב-id) – במקרה זה נעדיף את ה-port עם ה-id הנמוך יותר.

Designated port – המונח שנבחר עבור ה-port שדרכו הסוויץ' בחר להתחבר ל-broadcast domain.

לכל broadcast domain צריך להיות designated port אחד, ולכל סוויץ' צריך להיות root port אחד (מצביע אחד לשורש).

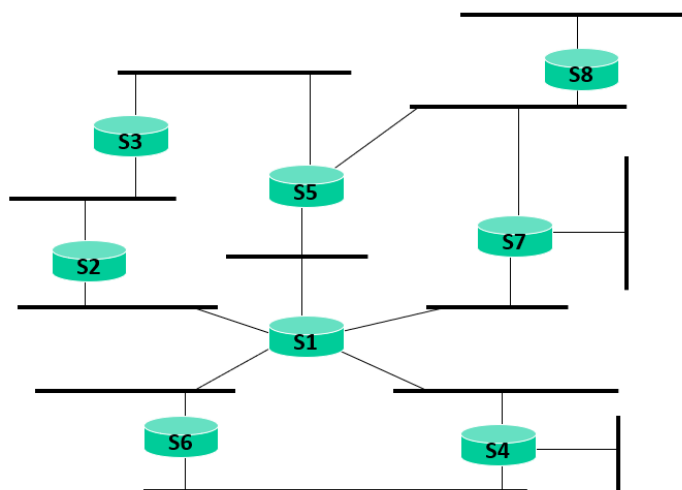
אם יש סוויץ' ללא designated port (לא מחבר אף broadcast domain) אז אפשר לנתק את כל הסוויץ' (לכבות את ה-root שמצביע לאבא).

שקף 34

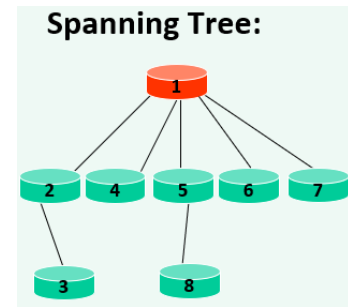
סיכום – לכל סוויץ' חוץ מהסוויץ' שורש יש Root port (ה-port שמצביע לכיוון שורש העץ שנבחר, זה שהוא עם המסלול הקצר ביותר לשורש) ולכל broadcast domain יש designated port אחד (ה-port שמעביר את ההודעות מאותו broadcast domain לכיוון השורש).

שקף 35-36

דוגמה: S_1 הוא הסוויץ' עם ה-id הנמוך ביותר, לכן הוא ייבחר להיות השורש:

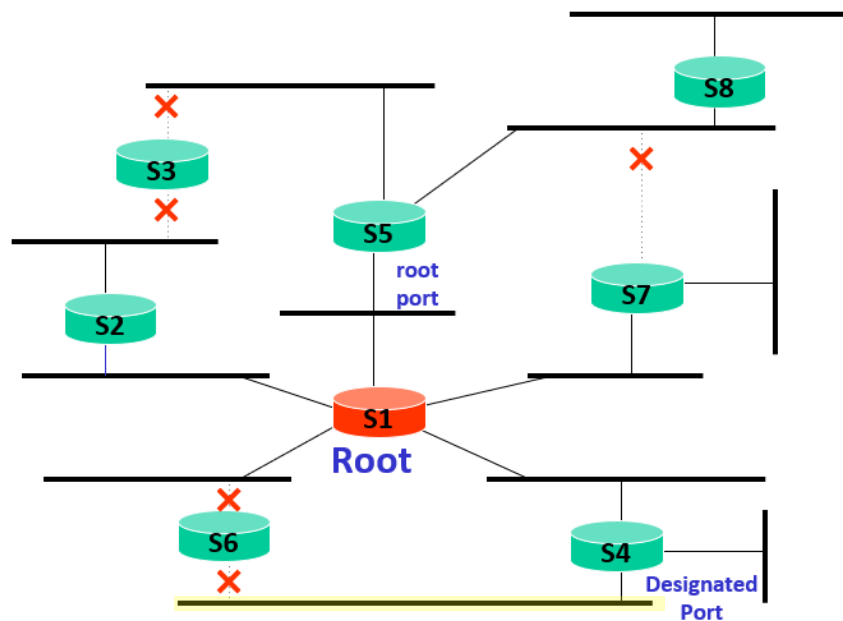


המטרה היא לקבל מבחינה לוגית עץ שנראה כך:



שקף 38

למשל ב-broadcast domain התחתון (מסומן בצהוב) אנחנו רואים שהוא מחובר על ידי 2 סוויצ'ים (S_4, S_6) שמחוברים ישירות לשורש (S_4 ו- S_6 שניהם במרחק 1 מהשורש), אבל אחד מהם עם id נמוך יותר. לכן מי שייבחר להיות designated port הוא S_4 , וסוויץ' S_6 ידע את זה ולכן יכבה את ה-port שלו שמחבר אותו ל-broadcast domain. נוצר מצב של- S_6 אין יותר designated port, כלומר הוא לא מחבר אף אחד ולכן כל הפורטים שלו, כולל ה-root port, יכולים להיסגר.



כך למשל אם נסתכל גם על S_3 , גם הוא ביטל את ה-designated port שלו, כי קיים מסלול קצר יותר לשורש שעובר דרך S_5 . למה את S_7 לא כיבינו? כי הוא עדיין עם designated port (ל-broadcast שנמצא מימינו בתרשים). יש פרסום מאחת הרגליים שלו ולכן הוא לא מתייתר.

אם לסוויץ' אין designated port זה אומר שהוא לא יעשה תעבורה למכשירי קצה, אז אפשר להפסיק תעבורה מהסוויץ' לגמרי (ורק ישתתף בתעבורת הודעות בעץ). לא מנתקים לגמרי כי אם מצב הרשת ישתנה בעתיד, אולי יהיה לו שימוש.

CSMA/CD (Carrier Sense Multiple Access/Collision Detection)

בעבר ראינו 2 וריאציות של פרוטוקול ALOHA. נרצה לשפר את ה-goodput שקיבלנו עבור הווריאציות האלו.

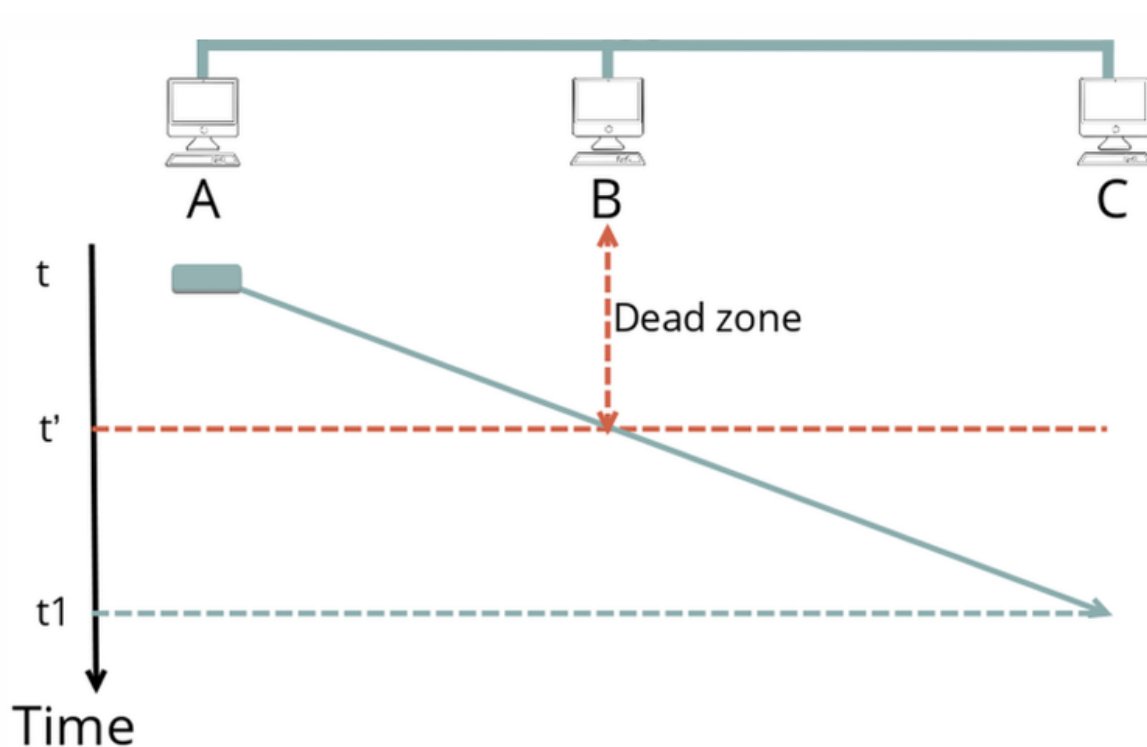
הצגת הפרוטוקול

נציג פרוטוקול חדש שנקרא CSMA. השוני במסגרת הפרוטוקול הזה לעומת הקודמים, הוא שעכשיו אנחנו מניחים שלצמתיים יש אפשרות להקשיב לפני שהם מתחילים לשדר. אם הצומת שומע מישהו שמשדר באפשרותו לפעול בהתאם.

בהינתן ההנחה הנוספת, איך נוכל להימנע?

- **להקשיב:** אפשר לבדוק האם התווך פנוי טרם השידור.
- **לתזמן:** אפשר לקבוע תזמון מתי כל צומת תורשה להשתמש בתווך (למשל זוגי ואי-זוגי).
- **לחלק תדרים:** אפשר לחלק את התווך לכמה תדרים, לא נתעסק בשיטה זו.

במציאות כשמישהו משדר לוקח זמן עד שהצד השני יקבל את ההודעה. נניח ש-A משדר בזמן t , B גם רוצה לשדר. B יקשיב, אבל בזמן t הוא לא ישמע שום דבר כי לוקח להודעה מעט זמן לעבור. ההודעה מגיעה ל-B בזמן t' ורק בזמן t_1 מגיעה ל-C:



יש טווח שבו B מקשיב אבל לא יידע ש-A שידר (Dead zone).

אם כן, לא מספיק רק להקשיב לתווך ולשדר כשנראה שהכל פנוי. נגדיר משתנה T_{prop} שיסמן את הזמן הארוך יותר בין שני הקצוות (בתרשים למעלה, $T_{prop} = t_1 - t$). גם אם קודקוד מקשיב לפני שהוא משדר, במשך T_{prop} השניות הראשונות עלולה להיות בעיה.

גישות שונות לשליחת הודעות

נניח שעברו T_{prop} שניות וצומת רוצה לשדר אבל יש רעש, מתי הוא ינסה לשדר?

Approach	Free?	Occupied?	Intuition
1-persistent	Send!	Wait until the channel becomes idle and then send immediately	High chance of collisions in loaded channels
non-persistent	Send!	Backoff and try again	Lower utilization on low load
p-persistent	Send with prob. p	Wait until the channel becomes idle and then send with probability p	A compromise between the other approaches

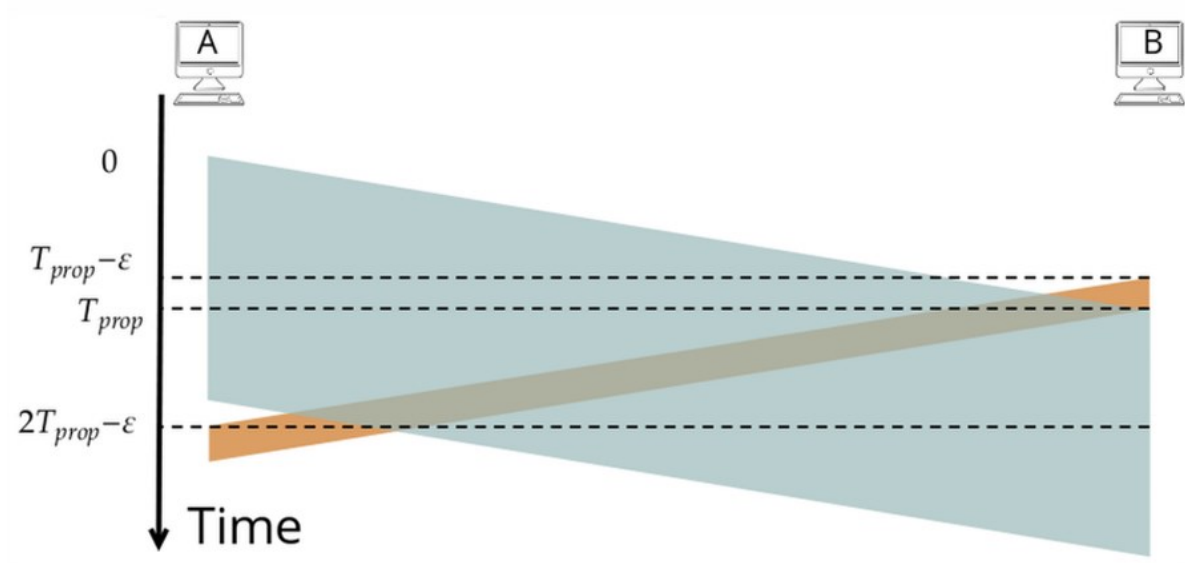
1-persistent: הצומת מקשיב – ברגע שהוא שומע שאין דיבורים ברשת הוא מיד ישדר. מתי זה יעיל? כשאין הרבה הודעות שרוצים לשלוח.

Non-persistent: אם יש כמה קודקודים שרוצים לדבר תוך כדי אבל אין לנו דרך לדעת כמה ידברו במקביל. יש מנגנון שיאפשר לנו לחכות יותר זמן אם יש עומס, ולחכות פחות זמן כאשר אין עומס. מנגד, זה אומר שאם חיכינו הרבה זמן ואז נגמר העומס אז לא יהיה ניצול טוב של התווך.

p-persistent: יש לצומת הודעה לשלוח. ברגע הוא שומע שהתווך תפוס אז הוא ממתין, אבל ברגע שהתווך פנוי לא מיד נשדר, אלא נשדר בהסתברות p . יפורט עוד בהמשך.

Collision Detection

נוסיף למנגנון שהצגנו גם CD. צומת מחכה ומזהה שהתווך פנוי. ראינו למעלה שזה לא מבטיח שלא תהיה התנגשות. ייקח לנו T_{prop} שניות כדי לזהות שהשידור שלנו התנגש עם שידור של מישהו אחר. לא מספיק שאנחנו נשים לב שיש התנגשות, אלא נצטרך להודיע לכולם על ההתנגשות. ננסה לשדר שוב אבל רק אחרי שבוודא שהתווך פנוי.



הסבר לתרשים:

מחשב A מתחיל לשלוח בזמן 0 והשידור שלו מגיע למחשב B בזמן T_{prop} . נניח שבמקרה הגרוע ביותר B הקשיב, ראה שהתווך פנוי והחליט לשדר בזמן $T_{prop} - \epsilon$. ייקח עוד T_{prop} שניות עד ש-A יידע ש-B שידר הודעה ויבין שהייתה התנגשות. סה"כ לקח $2T_{prop}$ שניות מרגע השידור הראשון עד שהמערכת הצליחה לזהות שקרתה התנגשות.

מאחר ש-B גם מאזין, ברגע שהוא קיבל הודעה מ-A הוא יפסיק לשדר כי הוא יודע שאין סיבה להמשיך (ההודעות לא יעברו בהצלחה). מנקודת המבט של A הוא שומע רעש אבל ברגע שהרעש נגמר הוא יכול לנסות להתחיל לשדר שוב (לא באופן מיידי, תלוי באיזה אלגוריתם להשתמש). ב-B לעומת זאת ייקח כמה זמן עד שמבחינתו התווך יהיה שוב ריק.

סיכום התרשים:

מניחים שהזמן שלוקח להודעה לעבור מ-A ל-B הוא T_{prop} שניות. A מתחיל לשדר בזמן 0, B מתחיל לשדר בזמן $T_{prop} - \epsilon$. A יזהה את ההתנגשות רק בזמן $2T_{prop} - \epsilon$, לכן כל הזיהוי נמשך לפחות $2T_{prop}$ שניות. **מסקנה** – כדי שפרוטוקול CSMA/CD יעבוד צריך שה-frames יהיו באורך המקיים $T_{trans} > 2T_{prop}$, כלומר שזמן השידור של ההודעות יהיה גדול יותר מהזמן שלוקח להודעה להתקבל בצד השני (ולחזור).

CSMA/CD – Jam Signal

אם אני מדבר ותוך כדי שידור שמעתי שהגיע עוד שידור לא מוצלח, אני אדע שהייתה התנגשות. הבעיה היא שמישהו שלישי (שלא שידר, רק הקשיב) יקבל רצף של ביטים שהוא לא יידע אם השילוב שלהם תקין או שהוא שילוב של שתי הודעות.

לשם כך יש **Jam Signal** – דרך להודיע לצמתים אחרים שהתרחשה התנגשות, למשל בערוץ אחר. יישלח על ידי מזהה ההתנגשות. ברגע שהצומת שהתחיל את השידור שגרם להתנגשות יקבל את ה-Jam Signal הוא גם יפסיק לשדר, וגם יידע את כולם שהמידע שהם קיבלו הוא לא תקין.

Waiting (Backoff) Time

אחרי שהייתה התנגשות והבנו שקרה משהו לא תקין, משתמשים בגישה שנקראת "Exponential Backoff". נניח ששידורתי משהו וקיבלתי Jam Signal, ניסיתי שוב וקיבלתי אותו סיגנל שוב. עכשיו כבר אתחיל לחכות יותר זמן, ובכך אני מעלה את הסבירות שאחרים יצליחו לשדר ואז כשאני אשוב לשדר אני לא אפריע להם ולא לי.

ה-backoff תלוי בפרמטר c (לרוב $c = 2$). אחרי הניסיון ה- k להעביר הודעה, נבחר j באופן אחיד מתוך הטווח $\{0, 1, \dots, c^k - 1\}$ ונמתין $j \cdot T$ יחידות זמן. כאשר הצומת מצליח לשלוח את ההודעה הוא יאפס את מספר הניסיונות שלו ל-0. ה-counter של צמתים אחרים לא יושפע מזה.

הערות:

1. למה בוחרים מתוך טווח $\{0, 1, \dots, c^k - 1\}$ ולא פשוט מחכים $c^k - 1$? כי אם שני צמתים A ו-B מנסים לתקשר באותם הזמנים עם צומת C, לבחור תמיד את אותו ערך רק ימשיך את גלגל ההתנגשויות. בממוצע מחכים יותר זמן, אבל מורידים את הסיכוי להתנגשות.
2. יש גם גישות אחרות לטיפול ב- k אחרי שליחת הודעה מוצלחת, אבל זו הנפוצה.

p-persist

נניח שכרגע זיהיתי שהתווך ריק. במקום לשדר מיד, אני אטיל מטבע ואשדר בהסתברות p . אם זכיתי בהגרלה אשדר, ואחרת אני אנסה שוב בסלוט הבא. אם יש הרבה קודקודים במערכת שמחכים לשדר אחרי שהיה שידור ארוך, יש עדיין סיכוי שרק אחד מהם ינסה לדבר (לעומת 1-persistent שם אחרי שידור ארוך כולם ינסו לשדר יחד ואף אחד לא יצליח).

כאן לא יהיה את המושג של Exponential-Backoff.

סיכום הנחות והגדרות

הנחות הרשת:

T_{prop} – משך הזמן שלוקח להודעה להגיע מקודקוד אחד לקודקוד אחר המרוחק ביותר.

N קודקודים במערכת.

T_{trans} זמן שידור.

הנחות פרוטוקול:

מניחים שהסלוט הוא באורך $2T_{prop}$ שניות.

תמיד יש לצמתים הודעה לשלוח.

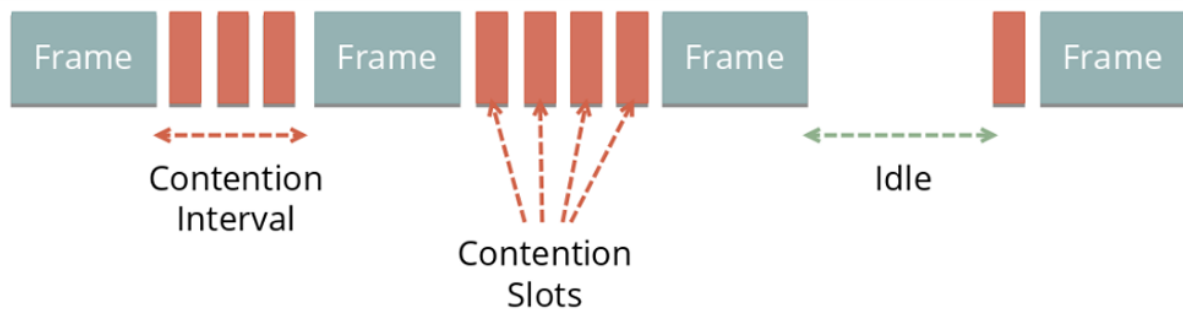
הסתברות שליחה p .

CSMA/CD Analysis

בפרוטוקול p-persistent אנחנו נקשיב ונחכה עד לסלוט הבא אם כרגע התווך לא פנוי, כאשר יתפנה נשדר בהסתברות p , ובמהלך השידור אנחנו נקשיב האם מתקבל Jam-Signal (או להודעה של מישהו אחר אם הוגדר כך). ברגע שהבנו שהייתה התנגשות נפסיק לשדר, ובסלוט הבא נשדר שוב בהסתברות p .

מהם מצבי המערכת האפשריים?

- Idle – אף אחד לא משדר.
- Transmitting – יש צומת שמצליח לשדר ללא הפרעות.
- Contention – יש שני קודקודים שמשדרים בהפרש קטן מ- T_{prop} , וידעו על ההתנגשות אחרי לכל היותר $2T_{prop}$ יחידות זמן.



חישובי goodput:

חישוב הסתברות הצלחה יהיה דומה לשל Slotted ALOHA:

$$\mathbb{P}_{suc}(p) = \mathbb{P}(X_p = 1) = np(1-p)^{n-1}$$

$$\max_{p \in [0,1]} \mathbb{P}_{suc}(p) = \left(1 - \frac{1}{n}\right)^{n-1} := S \text{ ואז נקבל: } p^* = \frac{1}{n}$$

בהינתן הנתון שגודל סלוט הוא $2T_{prop}$ נקבל תוצאה טובה יותר לעומת Slotted ALOHA כי כאן אנחנו מקבלים דרך להפסיק שידור ברגע שזיהינו התנגשות.

מה ההסתברות שהיו k סלטים בהם לא היה שידור מוצלח/התנגשות שלאחריהם הגיע שידור מוצלח?
 זה מתפלג כמו משתנה מקרי גיאומטרי עם פרמטר S . לכן התוחלת תהיה $\frac{1}{S}$ כלומר $1 - \frac{1}{S}$ סלטים מבוזבזים.
 כל זמן שלא שידרנו הפסדנו את האפשרות לשדר במשך $2T_{prop}$ שניות (גודל סלוט). נחשב ונקבל:

$$\frac{\text{זמן שנאבד על שידור לא מוצלח}}{\overbrace{2T_{prop}}} = \underbrace{\left(\frac{1}{S} - 1\right)}_{\substack{\text{כמות סלטים} \\ \text{לא מוצלחים בממוצע}}}$$

ולכן נוכל לחשב את ה-goodput בצורה הבאה:

$$\eta = \frac{\text{Transmission Time}}{\text{Transmission Time} + \text{Wasted Time}} = \frac{T_{trans}}{T_{trans} + 2T_{prop} \left(\frac{1}{S} - 1\right)}$$

כאשר נשאיף $n \rightarrow \infty$ נקבל את ה-goodput המקסימלי: $\left(\frac{1}{S} = e\right)$

$$\frac{T_{trans}}{T_{trans} + 2T_{prop}(e - 1)} = \frac{1}{1 + \frac{2T_{prop}}{T_{trans}}(e - 1)}$$

אנחנו רוצים להתקרב ל-1=goodput. איך נשיג זאת?

- להשאיף את T_{trans} לאינסוף (כלומר יש לי הודעה ארוכה מאוד לשלוח, בהתחלה בזמן T_{prop} תהיה בעיה – אבל לאחר מכן ברגע שנעבור אותו אני יודע שאף אחד לא יתנגש בי).
- להשאיף את T_{prop} ל-0 (כלומר אנחנו מאוד קרובים אחד לשני ושנינו רוצים לשדר, ייתכן שבדיוק באותו זמן שנינו נרצה לשדר ואז תהיה בעיה. אם יש הפרש כלשהו והוא קטן מאוד אז נשמע שמישהו אחר שידר ולא נתנגש, כלומר הזמן שיש בו סכנת התנגשות מזערי).

מה קורה ב-Slotted ALOHA ללא CSMA/CD?

לפעמים יעזור ולפעמים לא, נראה דוגמאות.

CSMA/CD – Questions

נתונים

יש לנו מערכת CSMA/CD עם רוחב פס 10Mbps, propagation speed של $2 \cdot 10^8 \left[\frac{m}{s}\right]$. נסיף רשת דומה במרחק 20 ק"מ מזו שלנו עם תווך כמו זה שהגדרנו למערכת. בתוך אותה רשת הקודקודים יחסית קרובים אחד לשני.



שאלה 1

האם בהינתן הודעה בגודל 64Bytes נוכל לנצל את התכונה של CSMA/CD?

הזמן שלוקח לביט בודד לעבור בין שתי הרשתות (נוסחת זמן-מהירות-מרחק):

$$T_{prop} = \frac{\text{distance}}{\text{propagation speed}} = \frac{20Km}{2 \cdot 10^8 \frac{m}{s}} = 10^{-4}s = 100\mu s$$

הזמן שלוקח להעביר 64B הוא:

$$T_{trans} = \frac{\text{bits}}{\text{bits per seconds}} = \frac{8 \cdot 64}{10 \frac{Mb}{s}} = \frac{8 \cdot 64}{10^7 \frac{b}{s}} = 51.2\mu s$$

נבחין שמתקיים:

$$T_{trans} < 2T_{prop}$$

כלומר לא מתקיים התנאי, ולא נוכל להשתמש בהודעה הזאת ברשת ולהשתמש ב-CSMA/CD בצורה שתתן לנו את ההטבות המגיעות עם הפרוטוקול. יכול להיות מקרה שבו מישהו משדר משהו באורך 64B ורק בסוף השידור גילה שהייתה בעיה, אבל היינו רוצים לעצור את השידור מיד.

שאלה 2

מה השינוי הקטן ביותר שאנחנו יכולים לעשות כדי לפתור את הבעיה הקודמת?

אנחנו צריכים שהפקטה תהיה מספיק גדולה כדי שייקח לי יותר זמן מ- $2T_{prop} = 200\mu s$ כדי לשדר אותה. נחזור לנוסחה הקודמת אבל הפעם עם נעלם. אם נגדיל את גודל ההודעה פי 4 לגודל 256B נקבל $T_{trans} = 204.8\mu s$.

שאלה 3

מה ישתנה בשאלה הקודמת אם רוחב הפס היה 100MBs?

רוחב הפס גדל פי 10, אפשר לשדר פי 10 יותר בזמן נתון, כלומר T_{trans} קטן פי 10. אם בעבר התקיים $T_{trans} > 2T_{prop}$ נרצה לשמר את אי-השוויון הזה, אז נגדיל את גודל הפקטה פי 10, כלומר להיות 2560B.



בשאלה זו יש $2 T_{prop}$: אחד שאומר כמה זמן לוקח להגיע מ-A ל-C, והשני שאומר כמה זמן לוקח להגיע מ-A ל-B. מהירות השליחה היא $v = 6 \cdot 10^7 \left[\frac{m}{s} \right]$, ורוחב הפס הוא $B = 3 \left[\frac{mb}{s} \right]$.

מה גודל הפקטה המינימלית (x) ש-A יכול לשלוח ל-C?

אנחנו רוצים לנצל את CSMA/CD ולכן נרצה שאי השוויון יתקיים.

$$T_{prop} = \frac{\text{distance}}{v_{prop}} = \frac{10km}{6 \cdot 10^7 \frac{m}{s}} = \frac{1}{6} 10^{-3} s$$

$$T_{trans} = \frac{x}{B} \geq 2T_{prop} \iff \frac{x}{3 \cdot 10^6} \geq \frac{1}{3} 10^{-3} \iff x \geq 10^3 \text{ bit}$$

מה גודל הפקטה המינימלית (x) ש-A יכול לשלוח ל-B?

מדובר בשאלה מעט מכשילה, גם במקרה הזה התשובה צריכה להיות 10^3 bit . הסיבה לכך היא שלמרות ש-B יקבל את ההודעה קודם, C עדיין יכול לשלוח הודעה ל-B שתפריע, או ש-A ישלח הודעה לכל המערכת (בשכבה 2 הודעות נשלחות לכולם), כלומר B יקבל את ההודעה ויבין אותה, אבל בזמן הזה C אולי יתחיל לשדר גם כן (לא יודע ש-A התחיל לשדר) ולכן יכול להפריע גם כן.

יש לנו מערכת שמריצה Slotted ALOHA ומקיימת את אי השוויון $T_{trans} > 2T_{prop}$. גודל סלוט הוא T_{trans} .

האם להוסיף CSMA ישפר את ה-goodput?

לא. הסיבה לכך היא שהבדיקה שהתווכ פנוי מתבצעת תמיד בתחילת slot. נניח ש-A שידר. גם אם B שידר וגם אם לא, להודעה של A בכל מקרה ייקח T_{trans} יחידות זמן להגיע ל-B, זה לא היה עוזר לו להקשיב כי בכל מקרה אם שניהם החליטו לשלוח הודעה הסלוט בוזבז, לכן הקשבה לתווכ לא תסייע כאן.

האם להוסיף CSMA/CD ישפר את ה-goodput?

לא. נניח ש-A התחיל לשדר ואחד מהם הבין שיש הודעה, שלח Jam Signal ואז A הפסיק לשדר. כך או כך, הסלוט מבוזבז והשידור שלו היה מתבזבז.

אילו המערכת הייתה Pure ALOHA, האם עכשיו הוספה של CSMA תשפר את ה-goodput?

כן. מאחר שהסלוטים לא מסונכרנים, הקשבה לתווך עשויה למנוע למשל מ-B להתחיל שידור באמצע שידור של A.

אילו המערכת הייתה Pure ALOHA, האם עכשיו הוספה של CSMA\CD תשפר את ה-goodput?

גם כן. אם שניהם שידרו כי למשל B ראה שהתווך פנוי (ההודעה של A עוד לא הגיעה), אם הוא יספיק לשלוח Jam Signal לפני ש-A מתחיל slot הוא יוכל לנצל את הסלוט הבא.

מסקנות

ב-CSMA/CD אנחנו מחכים שהתווך יהיה פנוי, מזהים התנגשויות ומודיעים עליהם בעזרת jam signal, נחכה זמן רנדומלי לפני ניסיון שידור חוזר.

נשתמש ב-CSMA/CD כאשר זמן ה-E2EP (End-to-end propagation) משמעותית קצר יותר מזמן ה-transmission.

Errors Detection and Recovery



אנחנו שולחים פקטה עם ביטים, לפעמים יש שיבושים בדרך. עם coding טוב נוכל לזהות שהיה שיבוש, ולפעמים אפילו לשחזר חלק מהנתונים. השיטה תהיה דומה לספרת ביקורת בתעודת זהות. ניקח את המידע שאנחנו רוצים לשלוח, נבין מזה מספר בקרה ונוסיף אותו לסוף הפקטה. קיבלנו פקטה חדשה שמכילה את המידע בתוספת coding correction. ככל שנוסיף יותר ביטים לבקרה נקבל יותר תקורה.

Repetition code

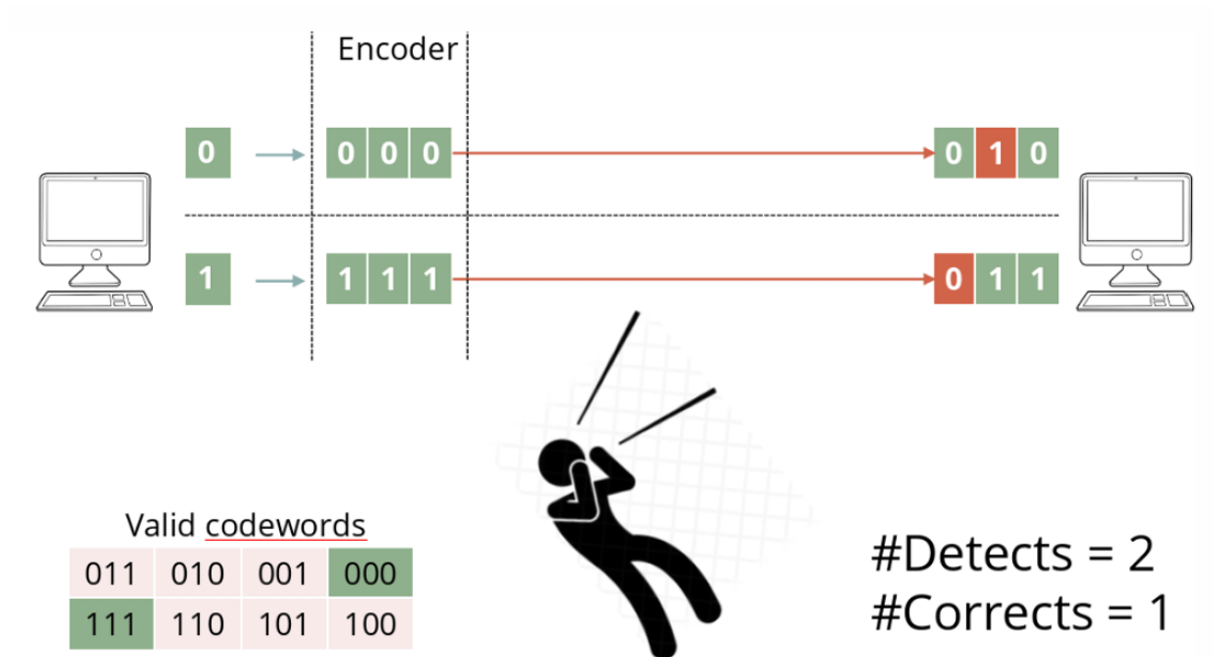
כל הודעה שנרצה לשלוח, במקום לשלוח פעם אחת נשלח אותה 3 פעמים. אם היו n ביטים בהתחלה עכשיו יהיה $3n$ ביטים. חישוב התקורה יהיה באופן כללי:

$$\frac{\text{Actual} - \text{Original}}{\text{Original}} = \frac{\text{Actual}}{\text{Original}} - 1 = \frac{\# \text{ Redundant Bits}}{\# \text{ Original Bits}}$$

ועבור המקרה של 3-repetition:

$$\frac{3N}{N} - 1 = 2$$

איך מתבצע התיקון? לפי מה היה הרוב:



אילו 2 ביטים היו לא נכונים, היינו משחזרים בצורה לא נכונה.

1D Parity

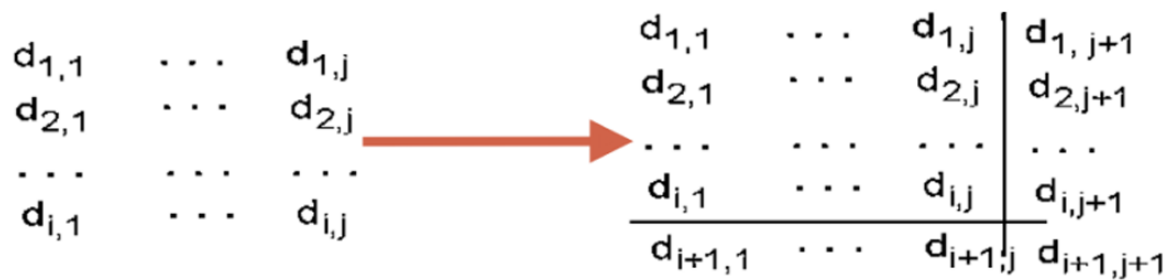
עדיין, מדובר על המון תוספת בשיטה של 3-repetition. מה שנבחר לעשות במקום היא אריתמטיקת ביטים על ידי ביצוע checksum. נסכום הכל ואז נוסיף את הסכום להתחלה/לסוף. השיטה היא בעצם לעשות xor משמאל לימין. אנחנו לא נוכל לדעת אילו מבין הביטים התחלף (אפילו bit ה-parity עלול להיות שגוי). אם היה ביט שגוי אני אדע שההודעה שגויה, אבל אילו היו 2 שגויים אני יכול לקבל checksum נכון למרות שההודעה שגויה. לא נוכל לתקן, אבל התקורה תהיה רק $\frac{1}{N}$.

הערה: ה-1 ב-1D הוא לא כי הוספנו רק ספרה אחת, אפשר להחליט להוסיף גם 5. נבין מה הסיבה שזה נקרא כך כשנגיע ל-2D.

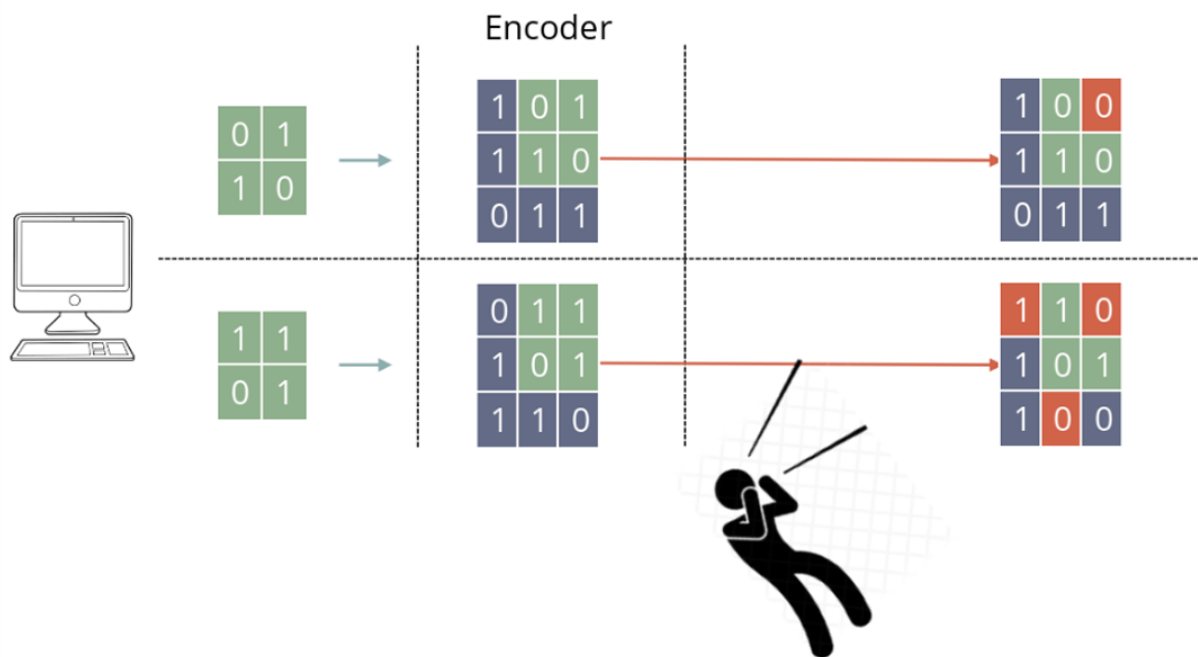
נרצה להיות מסוגלים לשחזר ולא רק לזהות שגיאה:

2D-Parity

לא נסתכל על וקטור של הודעות, אלא על מטריצה. נניח שההודעה שלנו היא 9 ביטים, נחלק לריבוע של 3×3 (אפשר גם מלבן), נסכום כל אחד מהם ונוסיף bit parity לכל שורה ולכל עמודה, בתוספת ביט בקרה לשורה ולעמודה:



עבשיו נוכל לזהות שהייתה בעיה כמו מקודם:



כשמזהים בעיה למשל בשורה הראשונה נוכל להצליב מול העמודה כדי להבין איפה הייתה הבעיה, ואז נוכל להחליף את הביט ולשחזר את הפקטה המקורית. כשמגדילים את כמות הביטים, עדיין יתכן מצב שלא נצליח לשחזר.

בכל שנוסיף יותר ממדים נוכל לשחזר יותר מידע, הנוסחה לחישוב כמה ביטים אפשר לשחזר היא $\frac{\text{dimension}}{2}$.

שאלה

מה תהיה התקורה של 2D-Parity עבור פקטה באורך 100 ביטים עם $i, j = 10$?

$$\frac{i \cdot j + i + j + 1}{i \cdot j} - 1 = \frac{100 + 10 + 10 + 1}{100} - 1 = \frac{121}{100} - 1 = 0.21$$

סיכום

Overhead	Corrects	Detects	Code
2	1	2	Repetition (3 times)
$\frac{1}{N}$	0	1	1D Parity
$\frac{i + j + 1}{N}$	1	3	2D Parity

כעת נביר עוד שיטה, אך לא נרחיב עליה הרבה:

Cyclic Redundancy Checks (CRC)

Ethernet משתמש ב-CRC-32. מוסיפים r ביטים, והסיכוי להיכשל בזיהוי בעיה היא $\approx 2^{-r}$. עבור r מאוד קטן יש סיכוי גבוה יותר שפקטות שגויות יחדרו את המנגנון.

ניקח פקטה ונתייחס אליה כאוסף מספרים. נתייחס אליהם במקדמים של החזקות בפולינום. יש למערכת פולינום גנרטור באורך המספר שבחרנו, ואת הפקטה שנרצה לשלוח נחלק בגנרטור ונקבל שארית. השארית תהיה ערך ה-CRC שאותו נשים בסוף ההודעה. אופן שימוש:

- בשולח נחשב את ה-CRC ונוסיף לפקטה
- במקבל ניקח את הפקטה שקיבלנו ונחלק בפולינום הגנרטור, נבדוק שארית. אם השארית 0, הכל תקין בהסתברות גבוהה. אם יש שארית נדע שהייתה שגיאה.

דוגמה ל-CRC-3

נבחר גנרטור פשוט, $G(x) = x^3 + x + 1 \Leftrightarrow G = 1011$.

הצד השולח:

המידע לשליחה הוא $D = 1101$. נסיף לו $r = 3$ ביטים בסוף: $D \cdot x^3 = 1101000$.

נחלק את 1101000 ב- $G = 1011$ בעזרת xor long division ונקבל שארית $R = 001$.

ההודעה שנשלח היא השרשור: $D||R = 1101\ 001$.

הצד המקבל:

מחלק את ההודעה שהתקבלה $D||R$ בגנרטור G . לאחר חישוב השארית היא 000 ולכן לא זיהה טעות.

אם אחד הביטים היה משתנה בהסתברות 2^{-r} נצליח לזהות את הבעיה.

לא נתבקש לדעת לבצע חישוב כזה, אבל כן לדעת שהוא קיים.

הרצאה 6

תזכורת והקדמה

בשיעור הקודם המשכנו לדבר על שכבת הדאטה לינק והתחלנו לדבר על spanning tree protocol, עליו נמשיך לפרט היום. בשלב זה בהרצאה חזרנו על שקפים 38-24 במצגת 3.

מצגת 3

שקף 39-40

דרישות הפרוטוקול

לכל סוויץ' צריך להיות id ייחודי משלו (מסופק על ידי היצרן), לכל port בסוויץ' צריך להיות id גם כן, וזה גם קורה מהמפעל – שני אלו מספקים לנו יחד את הצירוף הייחודי של סוויץ' + port הדרוש כדי לתעדף דברים באלגוריתם.

הפרוטוקול פשוט – כל סוויץ' שולח בכל אינטרוול את ה-id שלו, מחיר לשורש, ואת ה-id שלו. את ההודעה נשלח לכל השכבים. ברגע ששכן יקבל הודעה על סוויץ' עם id יותר נמוך הוא יפיץ אותו בתור השורש שלו, ויעדכן את המחיר. ההודעה הזו נקראת BPDU. דוגמה רצה של הפרוטוקול בשקף 42.

שקף 41

לכל סוויץ' תמיד root אחד, ולא תמיד יש לו designated port. כל ה-ports האחרים שהם לא designated ולא root לא מעבירים אף הודעה.

רשתות IP – IP Networks

מצגת 4

שקף 2-4

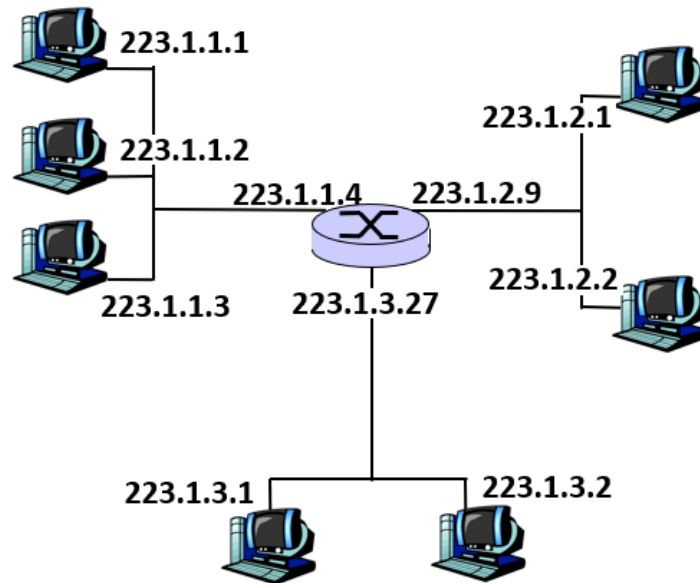
עכשיו אנחנו עולים שכבה לשכבת ה-Network. המטרה עכשיו היא לחבר LANs. מוטיבציה – למה צריך עוד שכבה? למה שלא יהיה סוויץ' אחד גדול שיחבר את כל המחשבים בעולם? מעבר לסיבוכיות בניית העץ היא מורכבת. עד עכשיו הסתמכנו על כתובות MAC ללא סדר, היצרנית מטביעה כתובת MAC על כרטיס הרשת, וכשהמחשב עובר בין רשתות הוא שומר על אותה כתובת. מה אם נעבוד ברשת ענקית? נרצה סדר בכתובות, ולא שכל סוויץ' יצטרך לזכור טבלה ענקית של כתובות (ואם לא יזכור לאן – יעשה broadcast לכל מקום, מאוד לא יעיל).

שקף 5

נדרשת כתובת לוגית היררכית, וזה מה שקורה בשכבת ה-Network. את השיעורים הקרובים נקדיש לשכבה זו. נתמקד בפרוטוקולי ניתוב, נדבר על פרוטוקול IP ועל פרוטוקולים משיקים כמו ICMP שמשמש לדיבוג הרשת. לאורך ההסבר ניגע גם בשכבת הטרנספורט בגלל חלקה הבלתי נפרד משכבת ה-Network.

שקף 6

רוב הדוגמאות שניתן יהיו מ-IPv4 שבו הכתובות הן 4 בתים (4 מספרים שכל אחד מהם בטווח 0-255), כאשר הכתיב המקובל הוא בצורה 223.1.1.2 למשל.



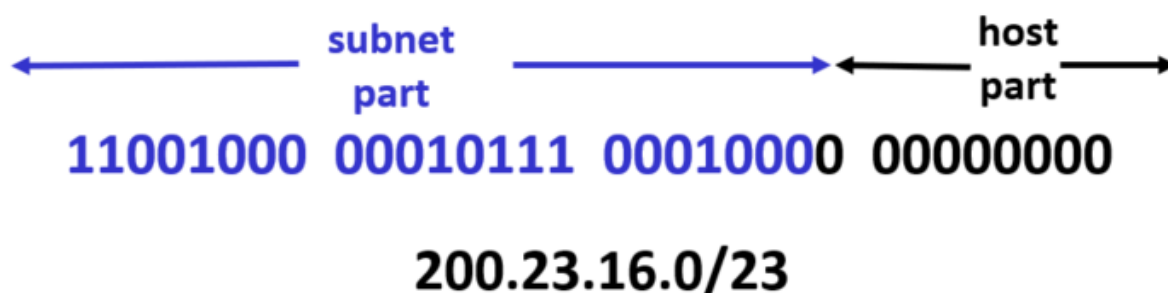
כאן יש לנו רשת עם מחשבים שמחוברים לראוטר שתפקידו לנתב פקטות (סוויץ' שימש למיתוג פקטות בשכבת הדאטה לינק, ראוטר משמש לנתב פקטות בשכבת ה-Network). לכל ראוטר יש כמה רגליים ולכל אחד מהן כתובת IP משלו שנמצאת ב-LAN אחר. סוויץ' היה שקוף בשכבת הדאטה לינק, לא הייתה לו כתובת MAC. הראוטר לעומת זאת אינו שקוף, ולכל אחת מהרגליים שלו כתובת IP. כשאנחנו שולחים הודעה בין מחשבים אשר עוברת דרך ראוטר, בעת ההגעה לראוטר ה-source address משתנה מהמחשב (למשל 223.1.2.1 באיור) להיות הכתובת של הרגל בראוטר ממנה יצאה ההודעה (למשל 223.1.1.4 באיור).

סיכום ההבדלים:

Switch	Router	
מיתוג פקטות בשכבת Data Link	ניתוב פקטות בשכבת ה-Network	שימוש
כתובות MAC	כתובות IP	דרך פעולה
שקוף מבחינת הרשת	לא שקוף, לכל רגל כתובת IP	שקיפות

Subnet – תת רשת, כל חלק של הרשת עד הראוטר נקרא sub-net. כל הכתובות באותו sub-net חולקות prefix משותף שמסייע לראוטר לבצע אגרגציה וניתוב.

את כתובת ה-IP אפשר לחלק לחלק של ה-subnet (ביטים עליונים) ולחלק של ה-host part (ביטים תחתונים).

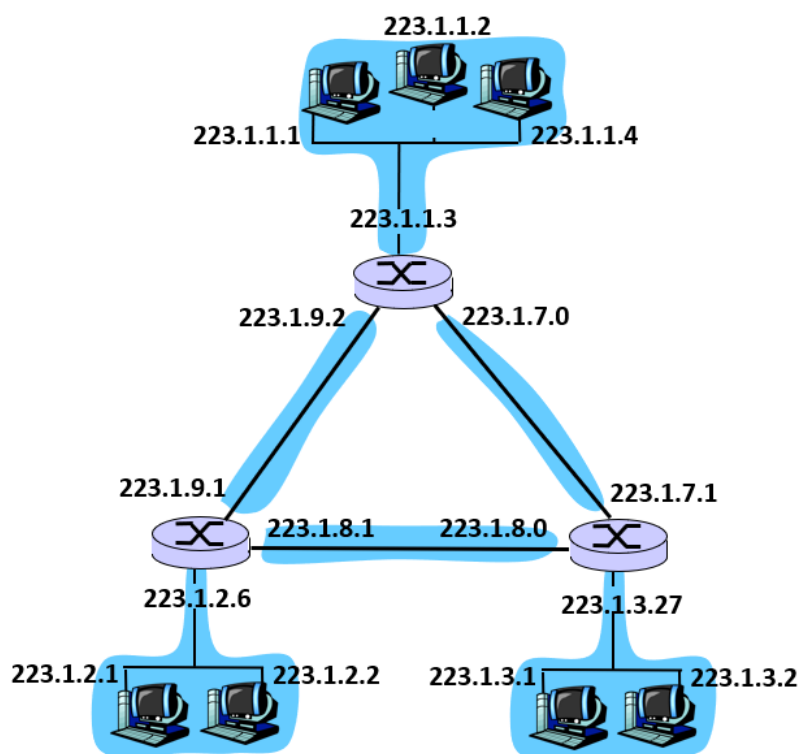


CIDR (Classless InterDomain Routing) זו דרך פופולארית (אך לא היחידה) לתאר כתובות של רשתות. מדובר בכיתוב נפוץ לתאר כתובות רשת. הכיתוב אומר כמה ביטים בכתובת מזהים את ה-subnet. בדוגמה לעיל יש 4 מקטעים של 8 ביטים, וה-23 /אומר ש-23 הביטים העליונים מזהים את הרשת, וה-9 התחתונים את ה-hosts בתוך הרשת, כלומר ברשת הזו יש 2^9 כתובות host. ככל שמה שמאחורי ה-/ גדול יותר כך הרשת קטנה יותר.

Subnet Mask

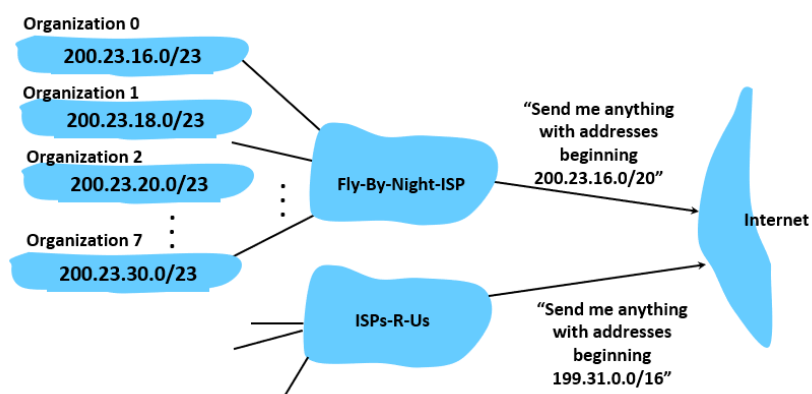
אנחנו רוצים להבין אילו subnets יש ברשת. איך נדע כמה subnets יש באיור? מוציאים את הראוטר מהטופולוגיה, ומסתכלים על ה"איים" שנוצרים. כל אי כזה הוא sub-net.

בדוגמה הספציפית הזו ה-subnet mask של תת-הרשת התחתונה הוא /24 כלומר ה-byte התחתון בתת-הרשת מציין את ה-host וכל השלושה האחרים את הרשת. למשל בתת-הרשת התחתונה כל ה-hosts יתחילו מ-223.1.3 ואז הבייט האחרון (הספרה האחרונה) שלהם ישתנה. גם הרגל המתאימה של הראוטר תקבל כתובת מתאימה (223.1.3 ואז מספר אחרון).



נוציא את כל הראוטרים ואז נשארים עם 3 subnets. אבל כל לינק בין הראוטרים הוא subnet בפני עצמו, כי הם לא שייכים לאף רשת אחרת. סה"כ 6 subnets.

התחלנו את השיעור בלדבר על למה האינטרנט הוא לא LAN אחד גדול, והסיבה לכך היא בעיית אגרגציה – לא נרצה בטבלה של הסוויץ' רשומה לכל כתובת אינטרנט בעולם. זה כן מתאפשר באמצעות כתובות IP. בעוד שכתובת MAC מוטבעת במכשיר (גלובלי, אבל שובר היררכיה), כתובות IP מתקבלות באופן דינמי מספקית האינטרנט.



באן הספקית העליונה קיבלה הקצאה, והרשת שלה היא **200.23.16.0/20**, כלומר היא יכולה $2^4 = 16$ רשתות קטנות יותר (זו רשת גדולה, כזכור לנו, ככל שה-/ עם מספר קטן יותר ככה הרשת גדולה יותר). הספקית למטה קיבלה **16/**, יש לה רשת גדולה אפילו יותר.

הרשת העליונה Fly-By-Night-ISP מחברת ארגונים, לכל אחד היא הקצתה רשת, וכל רשת היא **23/**. כמה כאלו היא יכולה לספק? 20 ביטים מוקצים לרשת הפנימית, כלומר יש לה סך הכל 12 ביטים להוסטים, אז $2^{12} = 4096$ IP addresses. כל רשת של **23/** עם 9 ביטים להוסטים, כלומר $2^9 = 512$ IP addresses. נחלק ונקבל שבמות הרשתות שהספקית יכולה לספק היא $4096 \div 512 = 8$.

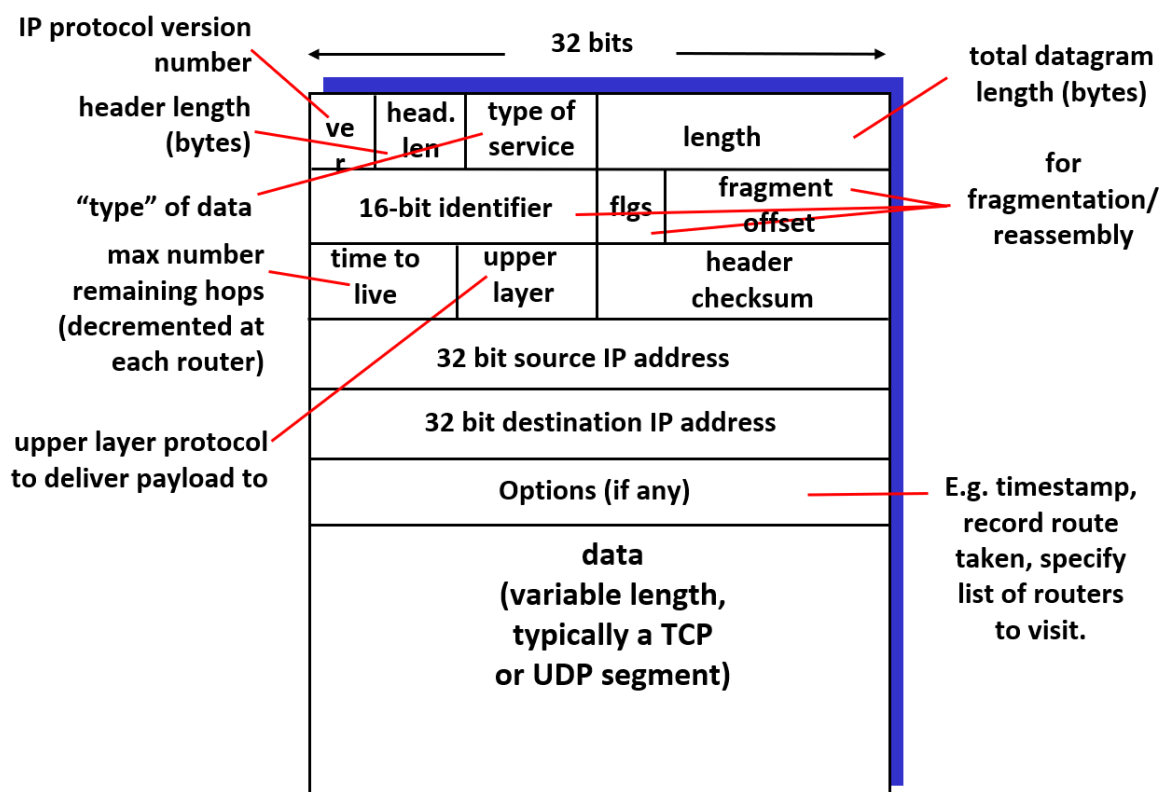
הספקית יכולה להודיע לספקית שלה לשלוח לה הודעות שממוענות לכתובת שמתחילה ב-200.23.16/20, כי זו הרשת שלה. כל ראوتر הבא בתור לא צריך לזכור את הרשתות הפנימיות (0-7 organizations), אלא רשומה אחת בטבלת הניתוב שלו שמכילה רק את ה-prefix הנדרש וכך יחליט לאן ההודעה נשלחת (הלאה).

נניח שארגון אחד שקנה את הכתובות שלו דרך Fly-By-Night-ISP רצה לעבור לספקית השנייה, שלה יש בלוק 16/200.23.18.0/23 או 199.31.0/16 שמתחילים או ב-16/199.31.0 או 23/200.23.18.0. עכשיו הספקית ISPs-R-Us מפנה דרכה ניתובים שמתחילים או ב-16/199.31.0 או 23/200.23.18.0. החוק הוא להעביר לפי הכתובת הספציפית ביותר שאפשר למצוא, ולכן הודעה כזו לא תגיע לספקית העליונה אלא לתחתונה. מבחינת הספקית העליונה שום דבר לא השתנה, היא עדיין מפרסמת את אותו הניתוב. כן נפגעה פה האגרגציה.

שקף 13

איך מקבלים בלוק כתובות באינטרנט?

יש ארגון שנקרא ICANN שאחראי על חלוקת כתובות IP לארגונים, חלוקת resources באינטרנט. מהרגע ש-ISP קיבל כתובת IP זה הפך להיות הרכוש שלו, הוא יכול להשתמש למכור חתיכות מהנתח וכו'.



כל האריזה הזו היא הדאטה שנמצא בתוך data link frame. את ה-source וה-destination אנחנו לא רואים, הם נמצאים מעל האריזה שאנחנו רואים מולנו. מצד שני, גם ל-IP יש דאטה, וה-data של IP ימוקם איפה שהשכבה הבאה מתחילה (כלומר ה-header של שכבת transport נמצא בתוך ה-data של IP, וה-data של שכבת transport יהיה בשכבת האפליקציה). בקיצור – השכבה התחתונה עוטפת את המידע שהגיע אליה מהשכבה שמעליה.

הרכיבים שאנחנו רואים בדיאגרמה:

יש מספר גרסה של הפרוטוקול (ver), אורך של ההודעה (length), שדות שמשמשים לפרגמנטיה (אם הפקטה ארוכה – IP תומך בפיצול וחיבור של הודעות). השדות הכי חשובים הם שדה **source IP address** ושדה **destination IP address**. מדובר בשדה של 32-ביט ב-IPv4, והם מזהים host שולח ו-host מקבל, ואינם משתנים במהלך חיי הפקטה. אם שלחתי הודעה וכתבתי בה את ה-source IP שלי וגם source MAC שלי, בכל פעם שנעבור router ה-source MAC משתנה לכתובת ה-MAC של רגל הראוטר שהעבירה את ההודעה הלאה, אבל **source IP נשאר זהה** (בכל שלב יאתרו מה ה-longest prefix match של כתובת היעד). הדגשה: ה-MAC address רק משמש אותנו לצעד הקרוב הבא.

אחד היתרונות ש-IP מאפשר לעשות זה לכתוב מה הפרוטוקול של השכבה הבאה (מעליו), האם זה פרוטוקול TCP או UDP של שכבת ה-Transport, או פרוטוקול ICMP שמשמש לאבחון בעיות ברשת. כשההודעה שלנו מגיעה למכונה ביעד, מכונת הקצה מסתכלת על ה-Header IP ומבינה מי הפרוטוקול של השכבה הבאה שאמור לטפל בהודעה (וכך המימוש המתאים יטפל בהודעה – TCP או UDP לפי מה שצוין).

בהודעות של שכבת הלינק אין הרבה משמעות לציון הפרוטוקול של השכבה הבאה, כי מעל שכבת הדאטה לינק יש את שכבת ה-Network, שבה הפרוטוקול הוא תמיד IP, אין עוד אופציות.

(הצצה קדימה: גם בשכבת ה-Transport ניתקל במנגנונים דומים – ה-port יזהה את האפליקציה)

נרצה לפתור כמה בעיות שאין להן כרגע מענה:

1. איך מקבלים כתובת IP? **DHCP**
2. איך אני משיג את כתובת ה-IP של היעד שלי? **DNS**
3. ביצוע המרה מכתובת IP לכתובת MAC וחזרה. **ARP**

פרוטוקול DHCP

אחד מהפרוטוקולים הבסיסיים שמשמשים hosts, ומטרתו להקצות ל-host כתובת IP. כדי לשלוח הודעה צריך לכתוב את ה-IP source שלי, מאיפה קיבלתי אותו בכלל? כאן DHCP נכנס לתמונה.

בשונה מכתובת MAC, אני צריך לקבל כתובת IP בתוך הרשת, אחרת אשבור את המבנה ההיררכי. יש שתי דברים לעשות את זה, אחת מהן היא באמצעות כתובת IP קבועה שה-administrator קינפג לתוך המערכת. כל מערכת הפעלה נותנת גישה לקבוע את כתובת ה-IP של ה-host שאפשר לקנפג עם הרשאות מתאימות. אם אותו מחשב ילך לרשת אחרת וינסה לשלוח הודעות עם כתובת שלא נמצאת ברשת הוא יהיה בבעיה, ויותר חמור אם הכתובת הזו כבר שייכת למישהו אחר ברשת.

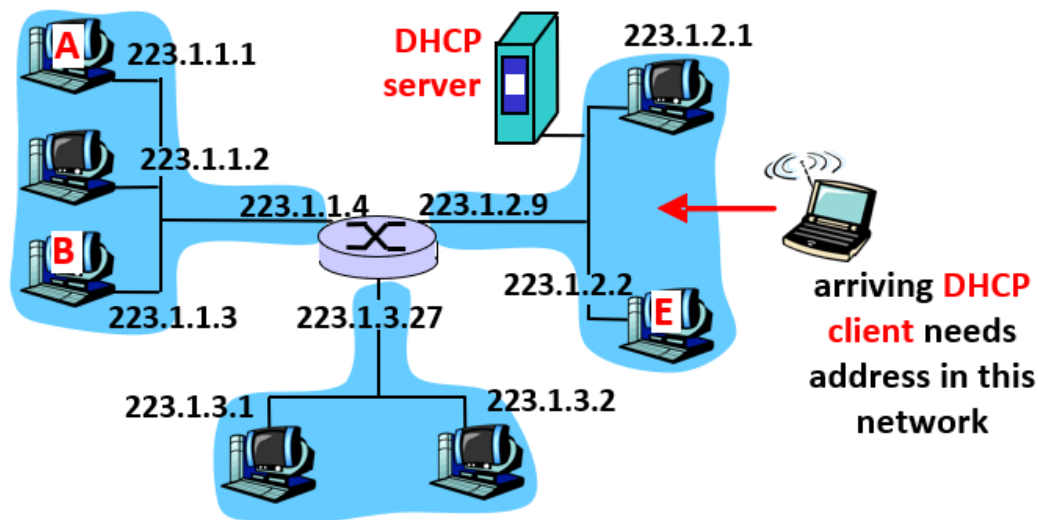
הדרך השנייה לעשות את זה היא הקצאת הכתובת בצורה דינמית בעת התחברות לרשת באמצעות פרוטוקול **DHCP (Dynamic Host Configuration Protocol)**. לרוב ברשת יש שרת DHCP שמטרתו להקצות כתובות ברשת למחשבים חדשים שמתחברים.

לפרוטוקול כמה שלבים שנציג בהמשך. הפרוטוקול הוא מעין שיחה בין ה-host לשרת ה-DHCP, אבל בנקודת הזמן הזו ל-host עדיין אין כתובת IP והוא גם לא יודע מה כתובת ה-IP של שרת ה-DHCP לכן יש בעיית תקשורת שצריך לפתור. במונחי DHCP, השרת ייתן lease לכתובת IP (סוג של "שכירות") למחשב החדש שהתחבר. נקרא lease כי הוא תחום בזמן (כמו חוזה שכירות), וכשפרק הזמן פוקע או שהמחשב מבקש

להאריך את ה-lease שלו ולשמור את אותה כתובת IP, או שכתובת ה-IP חוזרת לבנק הכתובות שהשרת מתחזק. המטרה של הפקיעה היא שאם מחשב כבה נרצה לשחזר את מה שהקצינו לו בצורה אוטומטית.

שקף 22

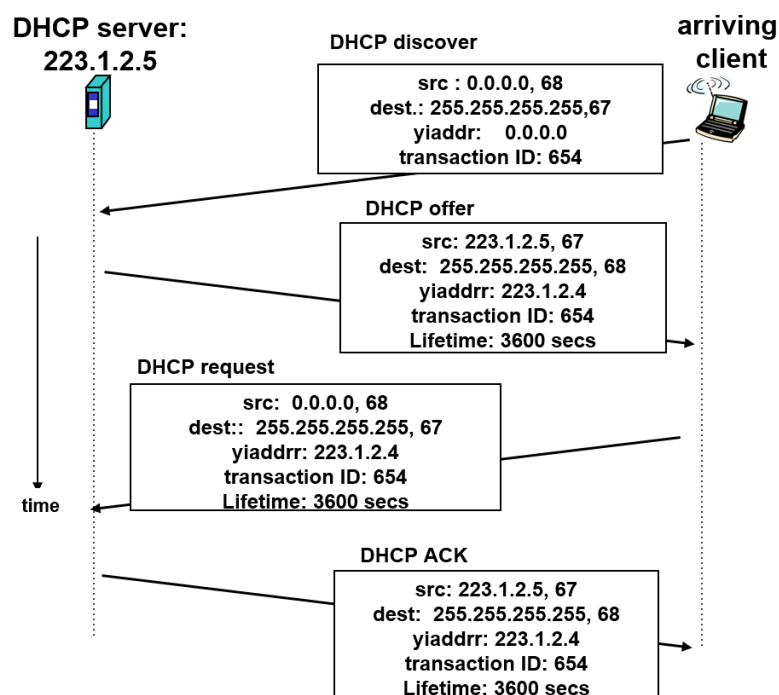
תרשים שמתאר את הרשת:



כשלקוח מגיע לרשת הוא מנסה להתחבר, ואז שרת ה-DHCP צריך להקצות לו כתובת. נראה בקרוב שהוא עושה קצת יותר מזה – נותן לו מידע שימושי על איך להשתמש ברשת (מה ה-subnet mask, מי ה-router).

שקף 23

תרחיש שימוש בפרוטוקול DHCP



1. לקוח מגיע והוא צריך לתקשר עם שרת ה-DHCP שלרוב רץ על הראוטר כחלק מהשירות הבסיסי שהראוטר מספק. אותו לקוח שמגיע אין לו כתובת IP והוא גם לא יודע את כתובת ה-IP של שרת ה-DHCP. הוא שולח הודעה שנקראת **DHCP Discover** שה-IP source שלה הוא כולו אפסים בביטים (0.0.0.0), וה-IP destination הוא הכל 1 בביטים (255.255.255.255). השימוש בכתובת שכולה 1-ים משמעותה broadcast ב-subnet. כחלק מהודעת ה-discover נקבע גם transaction ID כדי שיוכל לדעת לשייך את התגובה של השרת אליו.

2. בתגובה הלקוח יקבל **DHCP Offer**, שמשודר ב-broadcast משרת ה-DHCP ומתויג על ידי אותו transaction ID, בתוספת lifetime שמציין לכמה זמן הכתובת ניתנה לקליינט אם הוא ייקח אותה.

3. לאחר מכן הלקוח מבקש את הכתובת ב-**DHCP Request**. למה יש צורך בזה אם הרגע הציעו את הכתובת? כי יכול להיות שברשת יש כמה שרתי DHCP שענו ללקוח במקביל, ולכן הוא צריך אקטיבית לבקש כתובת מסוימת.

4. לבסוף שרת ה-DHCP שולח חזרה **DHCP ACK** שמאשר שהכתובת שלו, ומאותו רגע ללקוח יש כתובת IP ברשת.

בתמונה – 67 ו-68 זה הפורטים של שרת DHCP, נגיע לפורטים בהמשך כשנדבר על שכבת ה-Transport (לשייך הודעה לאפליקציה שנקראת DHCP).

Spanning Tree

[Interconnecting Nodes](#)

עד עכשיו, אמרנו שכל המחשבים מדברים על אותו תווך, ואנחנו מאפשרים את התקשורת בעזרת פרוטוקולים כמו ALOHA ומבצעים הנחות כמו איך הגעת ההודעות מתפלגת. אם שני מחשבים מדברים באותו הזמן על אותו התווך, ההודעות לא עוברות הלאה. אנחנו עדיין נמצאים ברמה 2, אבל נרצה לשפר את המודל שהצגנו (שכן במציאות מחשבים לא מתקשרים על תווך בודד).

Broadcast domain – סט כל הצמתים שמקבלים את ה-frames (של שכבה 2) אלו של אלו.

נאמר שרשת מכילה אחד או יותר broadcast domain. אם מחשב אחד שולח הודעה, כל המחשבים שומעים את אותה הודעה (במקרה של הודעה מוצלחת).

Collision Domain – עכשיו הוא יהיה שונה מה-broadcast domain. יהיה אזור שבו מחשבים יוכלו לשמוע זה את זה, אבל גם אם ידברו באותו זמן זה לא דווקא יגרום להתנגשות.

[מבנה הרשת](#)

בדרך כלל לרשת יש טופולוגיה מאוד מורכבת.

הרשת מורכבת מציוד רשת, ונהוג לחלק אותו לפי מודל השכבות OSI שראינו בקורס:

- שכבה פיזית (שכבה 1 – ביטים): Hub (כניסה אחת וכמה יציאות) או Repeater (מעביר את ההודעה שהוא קיבל מבחוץ)
- שכבת לינק (שכבה 2 – MAC): Switch (להעביר הודעות בין מחשבים שונים, מכיר כתובת MAC)
- שכבת הרשת (שכבה 3 – IP): Router (נפגוש בשיעור הבא)

לכל המכשירים האלו בדרך כלל יש כמה יציאות ומקבלים רשת מסועפת.

Hub – מכשיר בסיסי, כמו מפצל. מקבל אינפוט אחד ומעתיק אותו לכמה outputs. יכול להיות שיבצע error correction, אבל אולי גם לא. כל מה שמחובר ל-Hub הוא Collision domain כי אם אחד משדר אז השני בהכרח שומע גם כן. אם שניהם ישדרו באותו זמן – הם יפריעו אחד לשני.



Switch – יש רכיב זיכרון שזוכר מידע ומבצע את ניתוב הפקטות בצורה חכמה יותר למקומות הנכונים. אם כמה הודעות מגיעות במקביל הוא יודע לטפל בזה ללא התנגשות.



אם הוא מקבל 2 פקטות באותו הזמן, לסוויץ' יש באפר שמאפשר לו לשדר את השנייה לאחר מכן. אם מחשב אחד שולח פקטה למחשב אחר אז בהתחלה משדרים לכולם, אבל יש שלב שבו לומדים את המיקומים אחד של השני, ואז ההודעה מנותבת ישירות ליעדה (לא ב-broadcast לכולם).

כדי ללמוד, צריך לשלוח נתונים שמסייעים לסוויץ' להבין איפה המחשבים נמצאים. כשצומת שולח frame (בשכבה 2) הוא מציין מה ה-MAC Address של היעד. כאשר switch מקבל פקטה (פריים) לאחד מה-ports שלו הוא ישמור את הצמד (SRC MAC, IN PORT) בטבלה.

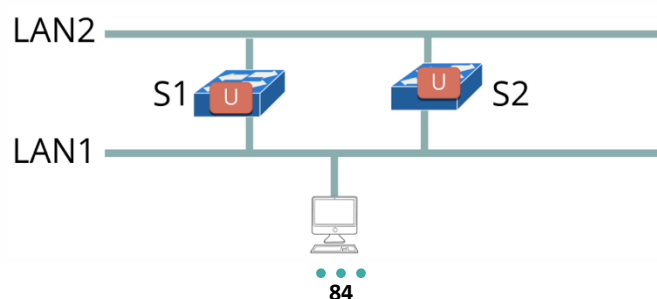
אם קיבלנו entry מהצורה (DST MAC, PORT k) אז נשדר את הפקטה דרך port k למעט במקרה שבו k הוא גם ה-port ממנו הגיעה ההודעה. למה? כי זה אומר שמחשב המקור והיעד שניהם נמצאים באותו broadcast domain, ולכן ההודעה כבר הגיעה ללא התערבות ה-switch. **אחרת**, נשלח את הפקטה לכל ה-ports חוץ מ-l-port k.

עם הזמן, נמלא את הטבלה – אבל היא אינה סטטית. ל-entries בטבלה יש timeout ואם כתובת MAC לא הייתה בשימוש במשך x שניות, השורה תימחק מהטבלה. הסיבות לכך: אנחנו לא רוצים לקנפג את הסוויץ' בצורה ידנית, מעדיפים שהכל יבוצע אוטומטית. סיבה נוספת – יכול להיות שדברים התקלקלו אז עדיף לנתק אזור שלא יכולות להגיע אליו הודעות.

בתרגול בשקפים 11-22 מופיעה דוגמה. הרעיון הוא לנצל את למידת הכתובות בכל הזדמנות – כאשר מגיע פריים ממקור מסוים, כל סוויץ' שהפריים עובר דרכו זוכר גם את כתובת המקור, למרות שהבעיה הנוכחית היא כביכול מציאת כתובת היעד. כל סוויץ' זוכר מאיזה port שלו הוא קיבל הודעה למשל ממחשב A, ולשם יעביר את ההודעה שממוענת ל-A תוך כדי שהוא זוכר עכשיו מי הוא זה שהשיב ל-A ומכניס לטבלה את המיקום שלו גם כן.

Forwarding Loops

כל הרעיון שתיארנו למעלה עובד כל עוד אין לולאות:



במקרה הזה הסוויצ'ים יתבלבלו ולא ידעו לאיזה port לבחון הודעה שממוענת למחשב למטה, כי הם קיבלו ממנו הודעה בשני ה-ports. יכול להיות שהוא ידרוס את ה-entry בטבלה או שהוא ישדר כל פעם לכל הרגליים (מעלה התנגשויות וכו').

האם נרצה להיפטר מלולאות לגמרי? ממש לא, ליתירות חשיבות:

- יכול להיות שבעתיד תהיה בעיה, ונרצה למצוא דרך חלופית (הלולאה תהפוך למסלול ללא מעגל כי היה ניתוק, והתקשורת תוכל להמשיך להתקיים).
- הסתגלות והתפתחות – עם הזמן נתיבים מסוימים יתגלו כטובים יותר ואחרים יהיו פחות בשימוש, לאחר מכן דברים עשויים להשתנות. מתאפשר בין היתר בגלל הטבלה בסוויצ'ים שמוחקת רשומות אחת לכמה זמן. כך עד לקבלת יציבות.

איך נתמודד עם לולאות?

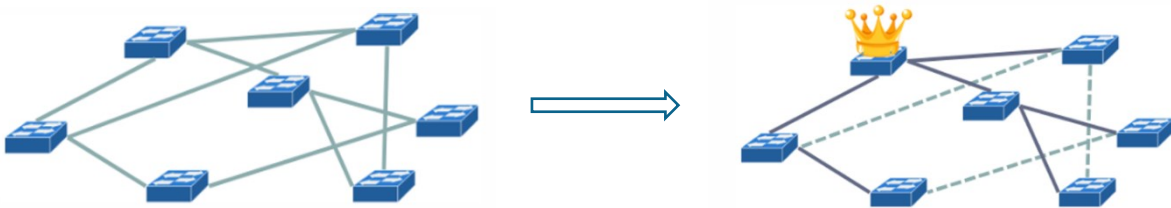
- הוספת מונה לפקטות שירד עם כל ביקור ב-switch
- לבנות מסלול אחד שיהיה בשימוש.

Dealing with Forwarding Loops

נרצה לבנות מבנה טופולוגי ברשת שבו אין מעגלים, תוך הכרה בעובדה שתתכן יותר מדריך אחת בין מקור ליעד. אם בהמשך תהיה תקלה, המסלולים ישתנו בהתאם – ולכן חשוב לא לנתק שום דבר.

Spanning Tree Protocol

את הפרוטוקול הזה יריץ כל סוויץ'. המטרה:



זה קצת שונה מפרוטוקולים שראינו עד עכשיו – עד עכשיו כל מחשב עשה בדיוק את אותו דבר כמו מחשבים אחרים. כאן כולם עדיין ישתמשו באותו פרוטוקול אבל יש שינוי קל: לכל סוויץ' יהיה id ייחודי (בערך) משלו, והמחשב עם ה-id הנמוך ביותר הוא ה-root.

כשאנחנו מייצרים את העץ אנחנו מתעלמים מהמחשבים שמחוברים ומתייחסים רק ל-LAN שמחוברים לסוויצ'ים. המטרה היא שכל ה-LANs יהיו מחוברים אחד לשני בעזרת הסוויצ'ים. יש פרוטוקול אינטרקטיבי שלוקח לו כמה איטרציות להתייצב, אבל לאחר מכן יהיה שורש אחד שכולם מכירים, ולכל LAN תהיה דרך אחת בלבד להגיע ל-root. כל סוויץ' שלא מחובר ל-LANs יכבה את עצמו (לא לחלוטין, אם יש תקלה כלשהי הם יתעוררו אולי מחדש).

אופן ביצוע הפרוטוקול

אנחנו נניח שה-id הוא ייחודי. כל סוויץ' ישלח פריים שנקרא BPDU (מכונה גם "Hello") שכולל:

- ה-id הכי נמוך בכל הרשת. מה אני חושב שהשורש, בהתחלה אחשוב שזה אני באופן דיפולטיבי.
- מרחק מהשורש. עלות המסלול מהסוויץ' השולח אל השורש.
- ה-id של השולח.

כל סוויץ' מבצע את אותו פרוטוקול במקביל לשאר: (DTR – Distance to Root, RID – RootID)

1. שולחים פריימים HELLO דרך כל ה-ports
2. אם קיבלתי פריימים HELLO: אעדכן במידת הצורך את RID ואת DTR
3. אם ציפיתי לקבל פריימים HELLO והרבה זמן לא קיבלתי (אניח שהיה ניתוק ולכן): אחשב RID מחדש

מי שייבחר להיות השורש הוא זה עם ה-RID הנמוך ביותר. לפי פריימים HELLO אדע לאיזה port לתעדף. אם קיבלתי שתי פקטות מפורטים שונים: אחד אומר ש-RID=1 והשני אומר RID=3, אני אתעדף את המסלול שמוביל אותי ל-1. מה קורה כשיש שוויון? אסתכל על האחד שה-DTR קצר יותר. אם גם הוא אותו דבר, אשבור שוויון בדרך אחרת (למשל אתעדף את ה-port number הנמוך יותר). כאן כבר אין עניין של יעילות, רק קבלת החלטה.

סוגי Ports כתוצאה מהפרוטוקול

Root Port (RP) / Parent Port (PP) – בכל סוויץ' נבחר Root Port (RP) כלומר ה-port שמוביל לשורש העץ. ה-RP חשוב לבניית ה-spanning tree, לא לצורך שידור הודעות (לשם כך יש את הפרוטוקול שראינו בתחילת התרגול היום). יהיה רק אחד לכל סוויץ'.

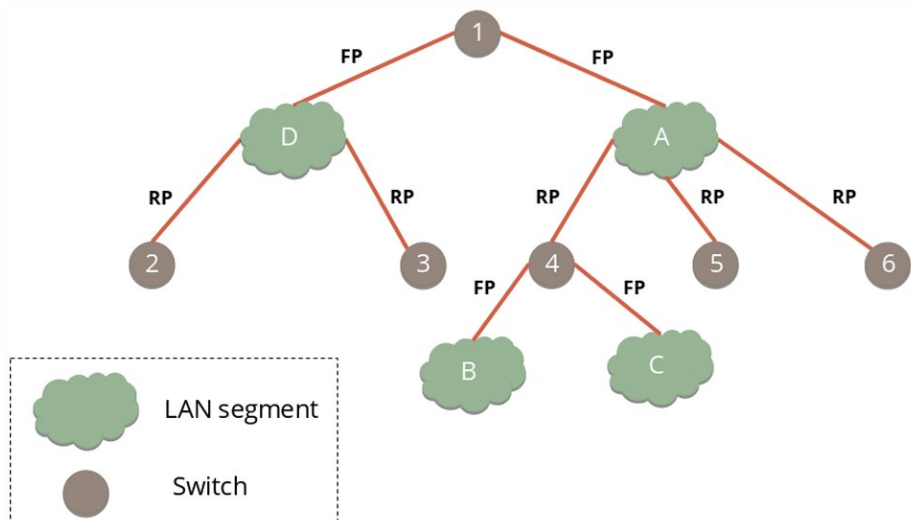
זה ה-port מנקודת המבט של ה-switch: "מאיזה port שלי אני אגיע הכי מהר ל-root?"

Forwarding Port (FP) / Designated Port – ה-port שבו ה-LAN משתמש כדי להעביר הודעות לסוויץ' (לכל LAN יהיה אחד כזה, אבל בסוויץ' אחד יכולים להיות כמה FPs ל-LANs שונים).

זה ה-port מנקודת המבט של ה-LAN: "מ-port של איזה switch אני אגיע הכי מהר ל-root?"

Blocked Port (BP) – אם יש port שאינו RP ואינו FP, הוא נחסם והופך להיות BP. ה-port לא נכבה לגמרי, ועדיין הסוויץ' יאזין להודעות HELLO שמתקבלות דרך הפורט הזה.

RP ו-FPs יהיו חלק מהעץ הפורש. אם שלחתי מסוויץ' אחד HELLO לסוויץ' אחר, הוא לא יעביר את אותה הודעה בדיוק, אלא יכין הודעה חדשה בהתאם למידע שקיבל ב-HELLO (כלומר HELLO הוא בין שני LANs בלבד).



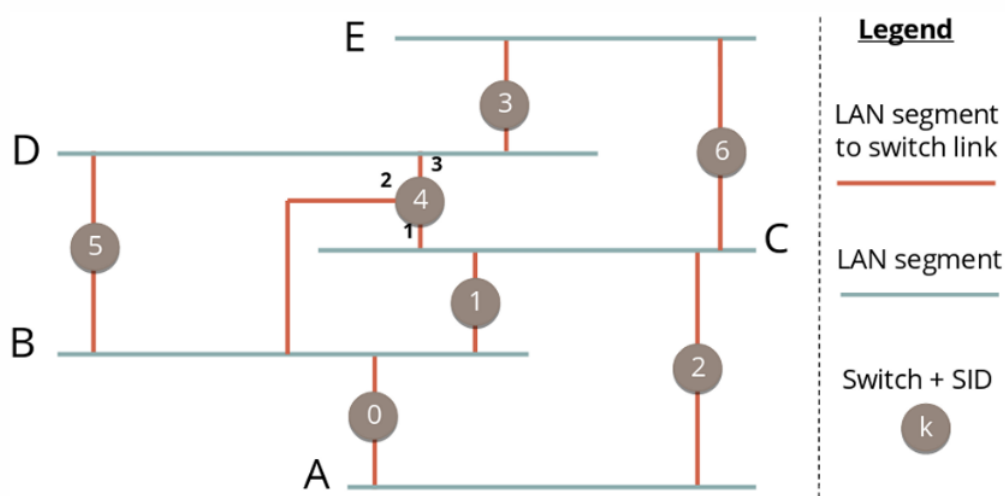
יכול להיות שחיבורים מסוימים עם Bandwidths שונים. במקרה זה נתעדף BW יותר גדול ולשם כך נשתמש באלגוריתם ממושקל (קיים אבל לא נתעסק בזה בקורס).

כשפותרים בעיות עם Spanning Tree Protocol, למרות שבפועל זה מתבצע במקביל בכל הסוויצ'ים לנו נוח דווקא לעבוד בצורה איטרטיבית ולהתחיל מסוויץ' 0.

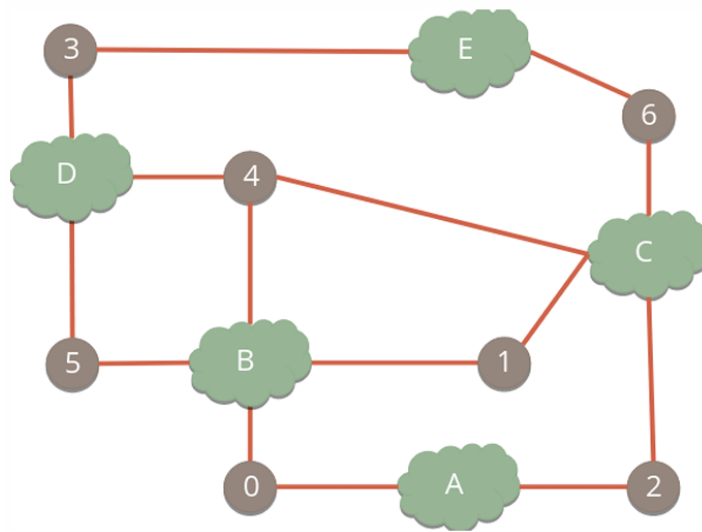
שאלות בנושא Spanning Tree Protocol

[פתרון במבט על הרשת כולה](#)

בתרגול עבדנו עם הדוגמה הזו:



שיכולה להיות מומרת גם לתרשים הבא:



נוח לעבוד עם טבלה שבאיטרציה הראשונה נראית כך:

SID	RID	DTR
0	0	0
1	1	0
2	2	0
3	3	0
4	4	0
5	5	0
6	6	0

נתחיל עם 0 ששולח HELLO frame, ואז סוויץ' 2 וסוויץ'ים 1,4,5 שומעים את ההודעה ומעדכנים:

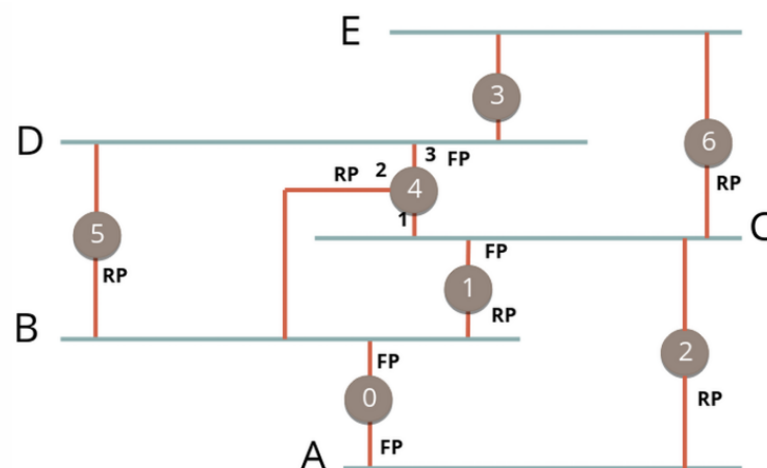
SID	RID	DTR
0	0	0
1	0	1
2	0	1
3	3	0
4	0	1
5	0	1
6	6	0

בשלב זה אפשר לעדכן מנקודת המבט של כל LAN עליו לבחור איזה switch קרוב יותר כדי להגיע אל ה-root, ולבחור את ה-FP שלו אל הסוויץ' הקרוב יותר.

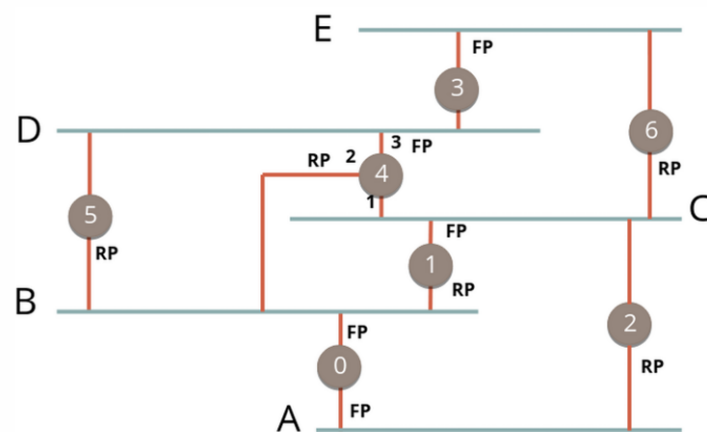
עבשיו נבדוק את D. D מקבל הודעות מסוויצ'ים 3,4,5. 4 ו-5 כבר מכירים את ה-root אבל 3 לא. 3 עבשיו יכיר את ה-root הנכון:

SID	RID	DTR
0	0	0
1	0	1
2	0	1
3	0	2
4	0	1
5	0	1
6	0	2

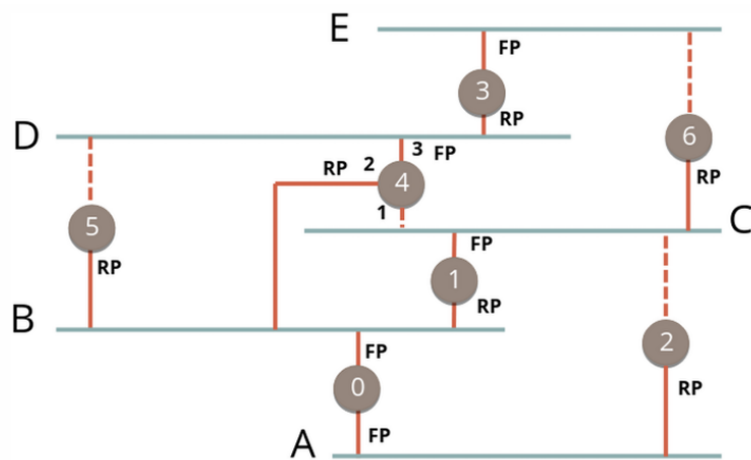
איך D יבחר את ה-FP שלו? ההתלבטות היא בין סוויץ' 4 ל-5 כי שניהם במרחק 1 מהשורש (לעומת 3 שהוא במרחק 2 ולכן לא נשקול לגביו). נשבור שוויון ובחר את 4 (הנמוך יותר).



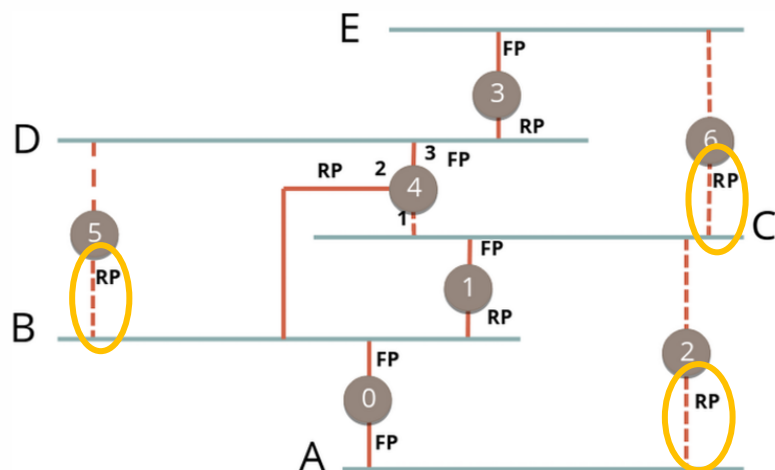
עבשיו נותר רק E.LAN E יקבל הודעות מ-3 ומ-6, שניהם במרחק 2 מהשורש ולכן יבחר את ה-port של סוויץ' 3 להיות ה-FP שלו.



סיימנו לסמן את כל ה-FP וה-RP. עבשיו אפשר להתחיל לבנות את העץ. קודם כל, נהפוך כל port שלא בשימוש (כלומר הוא לא FP ולא RP) ל-BP:

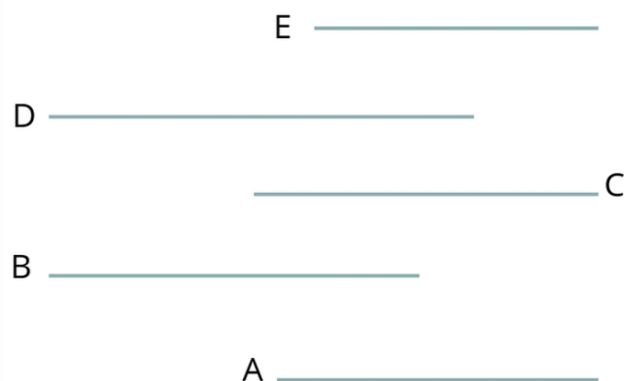


עבשיו, לא משנה לנו שה-switches יהיו מחוברים לרשת, מה שמשנה לנו זה LANs יהיו מחוברים לרשת. כל port שלא מחבר בין 2 LANs יתייתר, ונוכל לבטל גם אותו (מודגשים באיור):

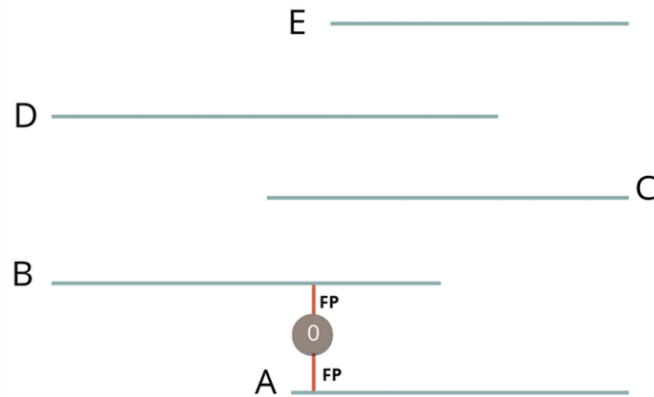


פתרון שמתחיל מנקודת מבט על ה-LANs

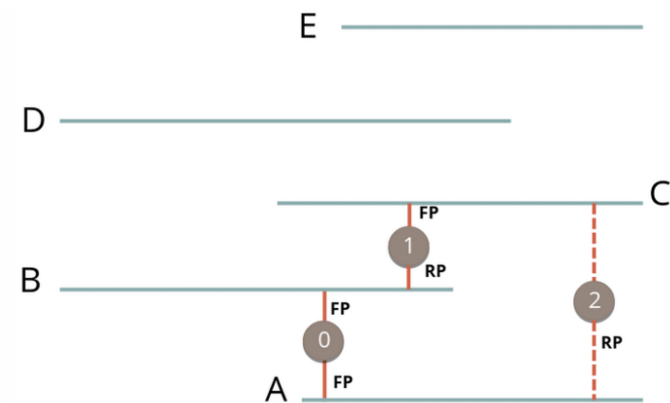
השיטה הזו פחות משקפת את מה שמתרחשת במציאות, אבל אפשר להחליט איזו שיטה יותר נוחה. מה שחשוב לנו זה לחבר את כל ה-LANs בצורה של עץ.



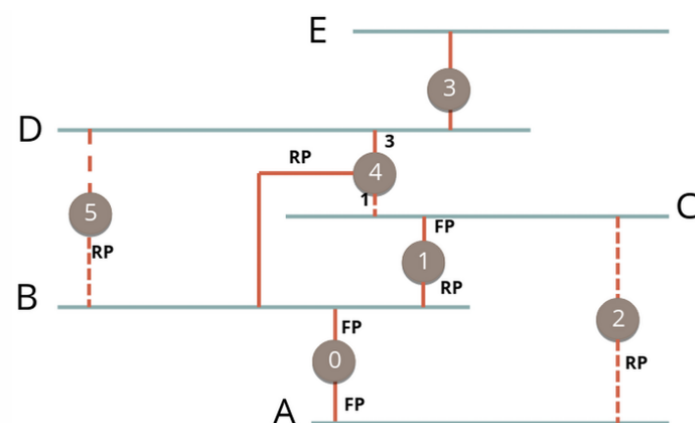
נתחיל מלחבר את השורש:



עכשיו אנחנו צריכים לחבר את C. C יכול להגיע ל-B או ל-A בכמה דרכים, אבל כדי להגיע בצורה המהירה ביותר ל-0 נוכל להשתמש ב-1,2,4. נשבור שוויון ונבחר ב-1. C ו-A כבר מחוברים אז 2 מיותר.

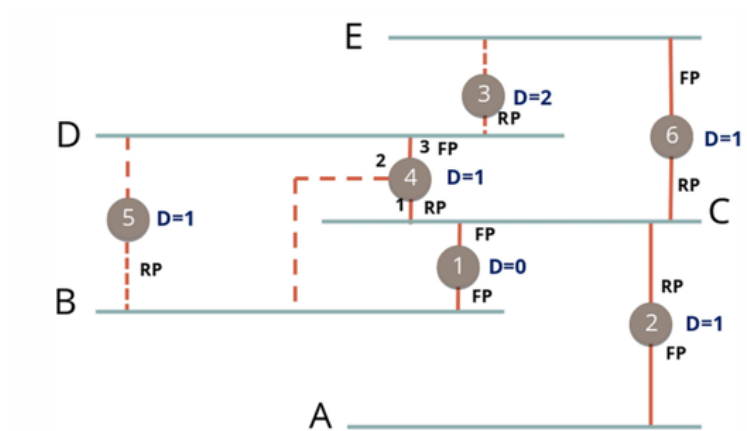


עכשיו נחבר את D. D יכול להתחבר ל-root switch? דרך 3,4,5. הדרך המהירה ביותר היא דרך ה-port השמאלי של 4 ונקבל:



את E כבר חיברנו דרך 3 כי המסלול דרך 6 switch באורך זהה, אבל ה-ID של switch 3 נמוך יותר.

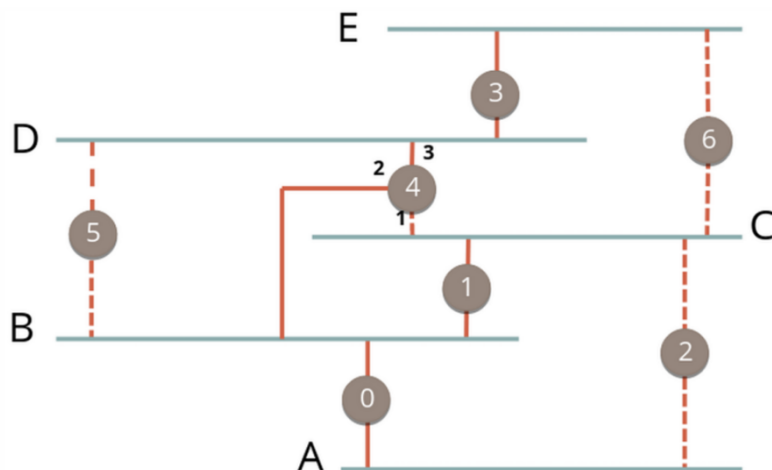
מה קורה אם switch 0 נופל?



אפשר להתחיל מהתחלה, אבל אפשר גם להסתכל מה הושפע:

- עכשיו switch 1 יהפוך להיות השורש (הבא בתור עם ID נמוך ביותר).
- A היה מחובר דרך switch 0, לכן עכשיו יהיה חייב לעבור דרך switch 2.
- ננתק את port 2 של switch 4 כי עכשיו יהיה שוויון באורך המסלולים דרך port 1 או port 2, ולכן port 1 ייבחר להיות RP.
- כדי להגיע מ-E ל-switch 1 יותר מהר לעבור דרך switch 6 ולכן נכבה את switch 3.

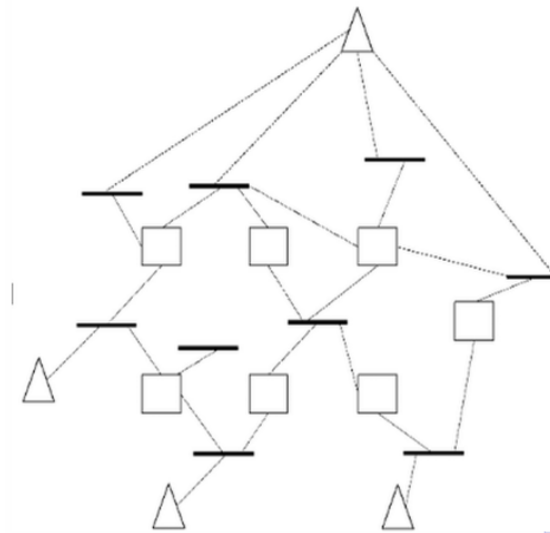
מה קורה אם switch 1 נופל?



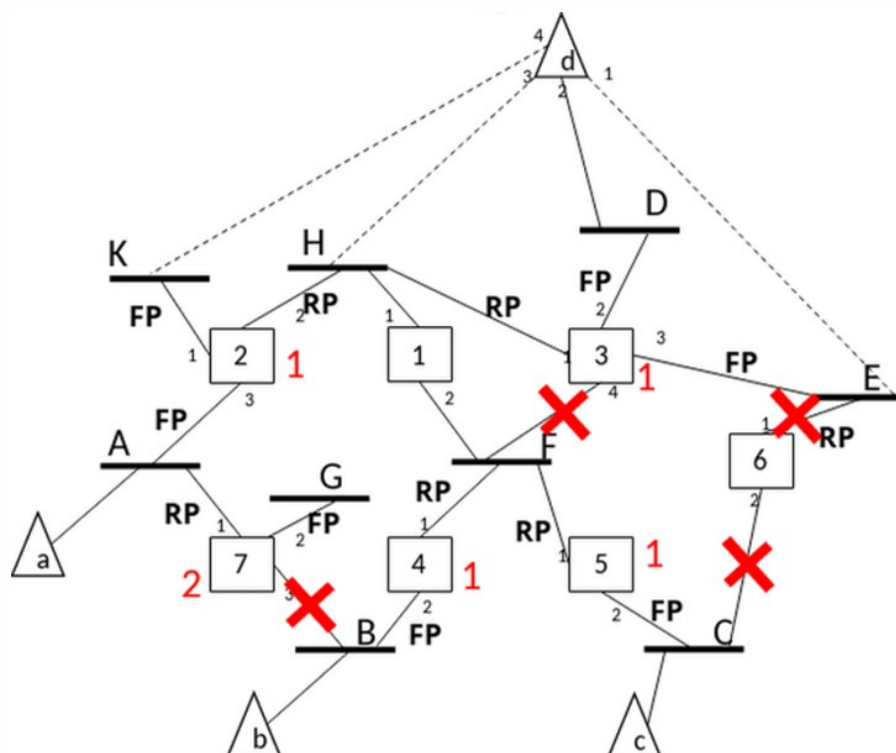
שאלה ממבחן

נתונה רשת הבנויה באופן הבא:

כל קו בתשים הוא LAN, המשולשים הם מחשבי קצה, והריבועים הם switches.



לכל אחד מהקודקודים כתובת MAC, והם מחוברים ל-LANs או חלקם דרך כמה LANs. אם למחשב d (כמו למשל המשולש העליון) יש כמה פורטים, נסמן את כתובת ה-MAC של הפורט כ: $MAC(d,3)$. הריצו STP כאשר הקווים המקווקווים מנותקים.



עכשיו הריצו את STP שוב, אבל כשהקווים מחוברים.

התשובה היא שלא יהיה הבדל לעומת ההרצה הקודמת, כי המחשב לא switch, הוא לא יעביר הודעות הלאה – אם יקבל הודעה שממוענת אליו יתייחס אליה, אחרת יתעלם.

מלאו את הטבלה עבור switch 1 (בהנחה שהטבלאות בסוויצ'ים כבר מלאות):

MAC Address	MAC(a)	MAC(b)	MAC(c)	MAC(d,1)	MAC(d,2)	MAC(d,3)	MAC(d,4)
Port							

נצטרך להחליט דרך איזה port נגיע לכל אחד מכתובות ה-MAC. נשתמש בעץ שיצרנו:

MAC Address	MAC(a)	MAC(b)	MAC(c)	MAC(d,1)	MAC(d,2)	MAC(d,3)	MAC(d,4)
Port	1	2	2	1	1	1	1

באותו אופן, מילאנו את הטבלה עבור switch 2:

MAC Address	MAC(a)	MAC(b)	MAC(c)	MAC(d,1)	MAC(d,2)	MAC(d,3)	MAC(d,4)
Port	3	2	2	2	2	2	1

הרצאה 7

חזרה

מצגת 4

שקף 22-23

בשיעור הקודם דיברנו על פרוטוקול DHCP. ראינו איך הפרוטוקול עובד. השיעור הזה נדבר על פרוטוקולים בשכבת האפליקציה - אפליקציה שמשתמשת ב-network. ב-DHCP שולחים הודעות IP אבל מטרת הפרוטוקול לקנפג את שכבת ה-IP של ה-Host.

ב-DHCP יש בעיה שבה הרשת מנסה להתחבר, אבל כדי לדבר עם שרת DHCP הוא צריך להשתמש בפרוטוקול IP כשהוא לא יודע שום דבר על הרשת שבה הוא נמצא.

שקף 24

עם DHCP אנחנו יכולים לתמוך בלקוחות חדשים לרשת בלי לקנפג שום דבר מראש. נשים לב שללא התכונה הזאת האינטרנט לא היה עובד כמו שאנחנו מכירים היום (מישהו שהגיע היום עם לפטופ חדש לאוניברסיטה היה צריך לעשות קונפיגורציה רק כדי להתחבר לרשת). יש יתרון בפעילות הדינאמית. DHCP לא משמש רק עבור קבלת כתובת IP (למרות שזה תפקידו המרכזי), אלא נותן עוד כמה פרמטרים שעוזרים למחשב חדש להשתמש ברשת אליה הוא מתחבר. הפרמטרים הם:

1. כתובת IP של ה-gateway (הרגל של הראוטר שמחבר אותנו לאינטרנט החיצוני). מדוע היא קריטית? אם נרצה ליצור קשר מחוץ לרשת שלנו נוכל לעשות זאת.
2. כתובת IP של שרת ה-DNS ברשת (בו אנחנו נשתמש, הוא יכול להיות מרוחק מה-subnet).
3. את ה-Network Mask (מה נמצא בתוך הרשת ומה מחוצה לה). כך יוכל לדעת עם מי הוא יכול לדבר ישירות, ולמי הוא צריך לעבור דרך הראוטר.

שקף 25

דוגמה עם הנפשה. אמרנו ש-DHCP זה פרוטוקול בשכבת האפליקציה, כלומר הודעת DHCP כותבת את כל השכבות שמתחתיה, שרצה מעל UDP (בשכבת Transport), שרץ מעל IP (בהתחלה שימוש מנוון עד לקבל כתובת), שרץ מעל Ethernet שרץ מעל השכבה הפיזית.

ההודעה הזו יכולה להיות למשל DHCP Request. בהתחלה כלום לא ידוע, הסוויץ' שולח לכולם עד שזה מגיע ל-DHCP server, והמענה יכולל פקטה שמכילה את כל ה-Network stack. כשהודעה מגיעה השרת עושה parse לשכבות השונות.

שקף 26

אפליקציית השרת כותבת הודעת ACK שמכילה את כתובת ה-IP. במקרה של DHCP יש לנו פרוטוקול שהוא מעין Meta-Protocol, הוא גם משתמש ברשת וגם מקנפג אותה.

איך מגלים את כתובת ה-IP של היעד שלי?

שקף 27-31

חשוב להבחין בין שמות לכתובות. אנחנו כאנשים רגילים לדבר בשמות כי נוח יותר לזכור אותן ויש להם משמעות לוגית עבורנו. כשהמחשב שלנו ירצה לשלוח הודעה ליעד הוא נזקק לכתובת IP.

חשיבות השמות

כמו כן, שמות נותנים לנו רמה של אבסטרקציה - יכול להיות שירות עם שם מסוים, אבל השרת שלו משתנה. למשל CNN.com מרימים שרת חדש, לא הגיוני שכל לקוח בעולם יצטרך לדעת למה להחליף את כתובת ה-IP החדשה. מאוד נוח לנו לפנות לאותו שם אבל לקבל כתובת IP חדשה. יתרון נוסף הוא שאפשר לעשות מיפוי של כמה כתובות IP לשם אחד (שירות ללקוחות בצורה גלובלית). היינו מעדיפים להתחבר לשרת מקומי כדי לקצר תהליכים - אותו שם יכול להיות ממופה לכתובות IP שונות כתלות במבקש. אפשר גם להתאים תוכן לגיאוגרפיה - לקוחות במדינה מסוימת רוצים תוכן שמתאים להם גיאוגרפית.

יתרונות נוספים הם load balancing - חלק מהלקוחות הולכים לשרת א' ואחרים ל-ב' כדי למנוע עומס על אותו שרת.

אפשר גם למפות כמה שמות לאותה כתובת IP - שירותי אחסון: כמה אתרים שרצים מעל אותו שרת פיזי (מכונות בענן), גם aliases (יש מקרים בהם לא חייב לשים www, הוא לפעמים ממופה לאותו מקום).

DNS - Domain Name System

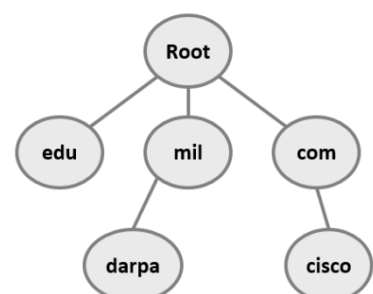
היום נבין איך הופכים שם לכתובת IP שהמחשב יכול להתחבר אליה ולבקש ממנה תוכן או לבצע כל התקשרות שיבחר.

שקף 33

DNS

שירות מבוסס, היררכי שממפה שמות לכתובות IP. הפרוטוקול שלו הוא די פשוט: יש לקוח ששולח בקשה "מה הכתובת שמתאימה לשם ____?", ושרת שעונה תשובה.

הוא מנצל מבנה היררכי: top level domain כמו .com, לעומת שלבים נמוכים יותר בהיררכיה כמו google.com וגם באותו ארגון אפשר ליצור היררכיה mail.google.com, למשל יצירת domain לכל שירות שהחברה מספקת. גוגל לא צריכים לעדכן את .com. מה הם עושים עם ה-domain שלהם.



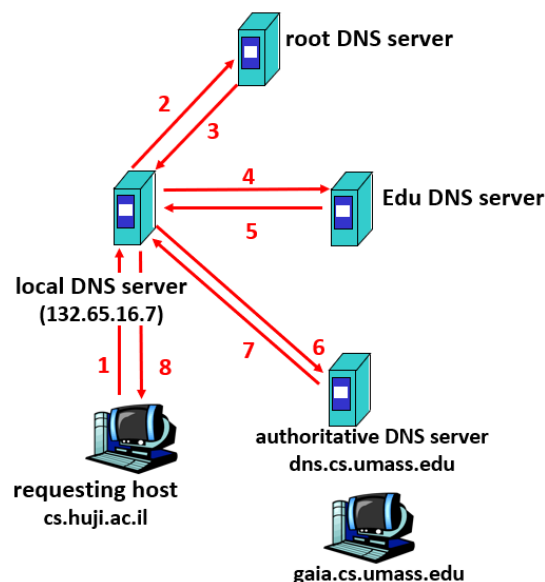
אנחנו רוצים לקבל data מבוזר כדי שהעומס יתחלק על הרבה שרתים. המבנה הוא היררכי: בראש ההיררכיה יש שרתי root (ללא שם) ומתחתם ה-top-level domains, ומתחתיו domains ששייכים לארגונים מסוימים ולבסוף את השרתים של אותו ארגון.

כל אחד אחראי למי שמתחתיו (ל-com. אין השפעה על הענף של edu). אין גבול לרמת ההיררכיה, תמיד אפשר לייצר עוד היררכיה לשמות.

שקף 37 דוגמה

נניח שאנחנו נמצאים באוניברסיטה (requesting host) ואנחנו רוצים לפנות לשירות שנמצא באוניברסיטה בארה"ב umass. אנחנו יודעים את השם אבל לא את כתובת ה-IP, וגם ידועה לנו כתובת ה-DNS Server הלוקאלית שלנו שאמור לסייע לנו להמיר שמות לכתובות IP.

נשלח את השאלה לשרת ה-DNS הלוקאלי שלנו (1), ה-resolver מתחיל תהליך רקורסיבי של נסות להבין מה כתובתה-IP ששייכת לשם הזה. DNS היא מערכת היררכית כאמור, ובכל שלב בתהליך אותו resolver הולך למישהו בהיררכיה ומקבל את המידע על ההבא בתור אם אותו אחד לא יודע את התשובה הסופית. (2) ה-resolver יודע רק את הכתובת של שרתי ה-root (בקונפיגורציה מראש), לכן שואל את ה-root, ואז הוא מחזיר לו "אני לא יודע, אבל תשאל את השרת של edu" - שלב (3), נשאל אותו (4), ואז הוא אומר "שאלת על umass.edu, תפנה אליו" (5), לכן נפנה אליו (6) ואז כבר ממנו נקבל ב-(7) את התשובה הסופית שתחזור ב-(8) ללקוח.



מבחינת הלקוח (requesting host) הוא רואה רק את (1) ו-(8), כל מה שקרה בין לבין מוסתר ממנו, קופסה שחורה.

שקף 38

בפועל, אחד המנגנונים המהותיים של DNS הוא מנגנון caching, כלומר שרת DNS זוכר את התשובות שהוא כבר ראה. התהליך שראינו עכשיו מתבצע מעל האינטרנט ועל גבי מרחק - כל זה צריך לקרות לפני שהלקוח בכלל רוצה לעשות משהו. אם בפרוטוקול היינו צריכים כל הזמן לבצע באופן רקורסיבי את הבקשות האלו, זה היה יכול לקחת הרבה זמן. אחד הדברים שעושים זה להשתמש במטמון בו יימצאו תשובות שכבר קיבלנו.

יש שרתים שבמעט בטוח יימצאו ב-resolver, כמו למשל top-level domains נפוצים. סביר להניח שברשת כמו האוניברסיטה ה-local DNS resolver כבר מכיר את com. ולכן לא ישאל על זה (כך גם על שירותים נפוצים כמו גוגל) - עליהם הלקוח יקבל תשובה ישירה, אבל ללקוח אין דרך לדעת מה ה-DNS resolver עשה. עם כל תשובה כזאת ה-DNS server מקבל גם TTL (time to leave) שאומר לכמה זמן התשובה רלוונטית - כלומר "אל תשאל אותי על זה שוב בפרק הזמן הזה".

כשה-TTL נגמר אנחנו נשאל שוב.

שקף 39-41 DNS Root

איך שרתי ה-root בנויים? ה-resolver מכרים את הכתובות של שרתי ה-root וככה הם יכולים לפנות אליהם ישירות בלי לבצע עוד שאילתה. יש 13 שרתי root שמופעלים על ידי גופים שונים, ומכילים עותקים של אותו מידע (זהים בתוכן) ואפשר לפנות לכל אחד מהם. חלק מהשרתים הם replicated בעצמם - כלומר גם שרת root מסוים יש לו מספר עותקים (למשל ל-Verisign). איך זה עובד? יש טריק - אין להם את האופציה להשתמש בטריק עם השמות (למפות את השם של ה-root server לכמה כתובות IP כי אין לו באמת שם). הדרך היא להשתמש ב-anycast - טריק ניתוב. יש כמה שרתים עם אותה כתובת IP, ואז (כשנדבר על ניתוב באינטרנט) נראה איך מתקבלת הודעת DNS לכל אחד מהשרתים - מוריד את העומס.

שקף 42 DNS Resource Records

פרוטוקול DNS מכיל רק שאלה ותשובה, אין חיבור ארוך טווח או קשר עם שרת שצריך לשמור. השאילתה היא לשאול על שם או record עבור שם - מה ה-IPv4 של שם מסוים? או IPv6? מה שרת המייל הארגוני ששייך לכתובת הזו? כך נדע למי לשלוח אימייל. התשובה נקראת resource record בפורמט הבא:

$$RR = (name, value, type, TTL)$$

name – the name you were looking for.

value – the requested IP address.

type – the type of question (IP, NS).

TTL – time to leave, time in which this answer is relevant.

בדוגמה למעלה, כל אחד מהשרתים החזיר RR מסוג NS (למרות שביקשנו כתובת IP).

אותה דוגמה כמו למעלה, אבל הפעם נדבר גם על סוג הודעת ה-RR.

הלקוח שואל את ה-DNS resolver מי זה www.google.com. נניח שה-resolver לא יודע כלום על העולם פרט לכתובת ה-IP של ה-root server.

ה-Root server לא יודע, אבל מחזיר לו תשובה "לך ל-Name server הבא בתור" (תשובה מסוג NS) כלומר לך לבדוק את www.google.com. ה-resolver למד שהוא יכול לשאול את שרת ה-NS הבא שהוא גם קיבל את הכתובת שלו (חוסך עוד שאילתה). אחר כך פנייה ל-www.google.com. שבתגובה מחזיר עוד תשובת NS עם השם וה-IP הבא.

השרת האחרון בשרשרת נקרא ה-authoritative name server, במקרה הזה השרת של google שאחראי על כל מה שנמצא ישירות מתחת ל-www.google.com, ממנו כבר יקבל את התשובה, אותה הוא יעביר ללקוח.

הלקוח מקבל רק את הסוף, אבל ה-resolver זוכר את כל ה-RR שהוא קיבל בדרך.

אנשים יכולים לרשום שמות, איך? אנחנו יכולים לקבל בלוק כתובות מה-ISP שלנו. נניח שלחברה שלנו קוראים FooBar. אנחנו רוצים לקבל בעצם שליטה על שרת ה-DNS של FooBar.com - כדי להגיד ל-www.google.com. שיש לנו אחריות עליו. לקנות דומיין אומר לשלם למישהו מעליך בהיררכיה כדי לקבוע את שרת ה-Name Server שאתה מעוניין בו.

בדוגמה הזו יכולנו לרשום שרת DNS, שרת web, שרת מייל (MX).

בעיות אבטחה

DNS הוא שירות מרכזי באינטרנט, ואנחנו די תלויים בו. הזהות שלנו תלויה בשירותי DNS - אם למשל ישנו את רשומת ה-DNS ב-gmail כל המיילים שממוענים אליי יגיעו למקום אחר. אם לא תהיה כאן אמינות לקוחות ינסו להתחבר לשירות מסוים ובלי לדעת יתנו את הפרטים לתוקף.

אם DNS נופל ונהיה חייב להגיע למקומות רק באמצעות כתובות IP - המון שירותים יפלו וגם אלו שזמינים מניחים שימוש ב-domain names כדי לבזר עומסים. לכן תלויים בשרתים גם לזמינות וגם לבטיחות.

אם מבצעים Denial of Service (DoS) שרתי DNS יכולים ליפול - מה קורה אם תשובות לא מתקבלות מספיק מהר כי מישהו בצורה זדונית העמיס על שרת DNS מסוים? מישהו יכול לכתוב תוכנה שקוראת בקשות ועונה תשובות. אם היא לא מספיקה לענות מספיק מהר נוצר תור גם ב-DNS וגם בכרטיס הרשת, והודעות שאין להן מקום בתור יימחקו, ואז שירות הופך ללא זמין.

היו שני אירועים גדולים כאלה שניסו לפגוע בזמינות של שרתי ה-root. התקפות ענק שהתרחשו ב-2002 וב-2015, אבל הנזק שנגרם בעקבותן לא היה משמעותי. למה? כמעט אף פעם לא ניגשים לשרתי ה-root, הם גם replicated וגם רוב מה שהם עונים כבר נמצא ב-cache של ה-resolvers. אמנם יש יחסית מעט מהם, אבל אף אחד כמעט לא צריך לדבר איתם.

המערכת של DNS נחשבת די resilient בפני התקפות מהסוג.

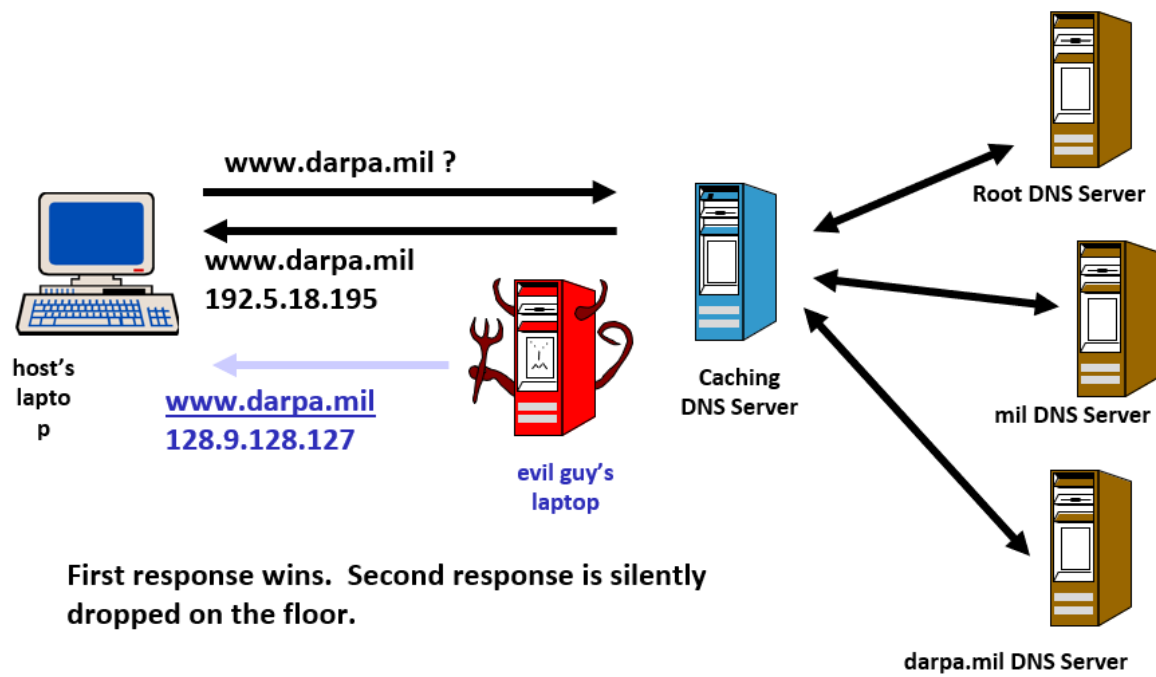
נניח שאנחנו גולשים באינטרנט ומנסים לגשת לאתר למשל google.com. כשאנחנו מנסים להגיע לאתר אנחנו צריכים להסתמך על DNS שייתן לנו את הכתובת של השירות. איך אנחנו יודעים שהגענו לכתובת הנכונה? לרשת אליה התחברתי יש local resolver שאנחנו לא בטוח יודעים מי אחראי עליו. יש כאן מנגנון אמון מובלע בעייתי – אותו local resolver יכול בצורה תיאורטית לענות להם איך שירצה.

עוד סיטואציה אפשרית – נניח שאנחנו נמצאים ברשת WiFi ומנסים לשאול שאלה את ה-resolver הלוקאלי ברשת. יכול להיות שאחרים מאזינים לבקשה, למה הם לא יכולים לענות לי כאילו הם ה-resolver?

בשני המקרים העלינו בעיה – איך אנחנו יודעים שהמידע שאנחנו מקבלים הוא מהימן, מהמקור שציפינו לקבל ממנו תשובה?

ברוב הפעמים – לא ניתן לדעת. אפשר להשתמש ב-TLS (לא נרחיב בקורס). בפרוטוקולים כמו HTTP אפשר להשתמש ב-TLS (ולקבל HTTPS) ואז אנחנו יודעים שהמידע הוא מהמקור שרצינו. זה לא אומר שהמיפוי של DNS היה הנכון, עדיין יכולים לרמות כאן, אבל זה אימות על התוכן – מי שהתחזה לא יוכל לעשות יותר מדי חוץ מלהעביר את הבקשה שלי כמו שהיא למי שכן רציתי, ולהעביר את התשובה שלו בחזרה אליי. התוקף ילמד את גודל הבקשות אבל לא יוכלו לרמות בתוכן.

המחשה של התהליך:

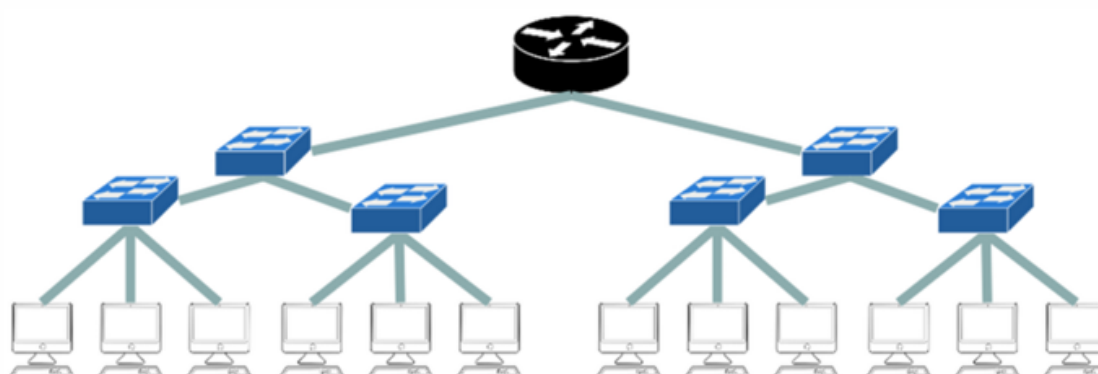


תרגול 6

הקדמה

היום נגיע לשכבה 3 – שכבת ה-Network. בתחילת הקורס דיברנו על broadcast domain שבו כשמחשב אחד מדבר כולם שומעים, כל עוד אין שידורים אחרים באותו זמן. בתרגול הקודם דיברנו על סוויצ'ים שמנתבים את ההודעות למקומן הנכון.

היום נדבר על layer 3 שבו אנחנו מוסיפים נתב:



רק הודעה שרלוונטית לאגף שמאל תגיע לשם, אם זה יעדה. מתייחסים לכל קודקוד כבעל כתובת ייחודית שנקראת כתובת MAC. אם יש מחשב אחד עם כמה network interface cards (NIC) אז יהיו פשוט כמה כתובות MAC (למשל כתובת ה-MAC לכבל Ethernet במחשב שונה מכתובת ה-MAC לגישת WiFi בלפטופים).

המטרה היום היא להבין מה קורה כשמחברים מחשב ל-L2 Network. Layer 2 עדיין ממשיכה לתפקד, פשוט ב-Layer 3 נשלח הודעה מבלי לדעת מה קורה מתחת.

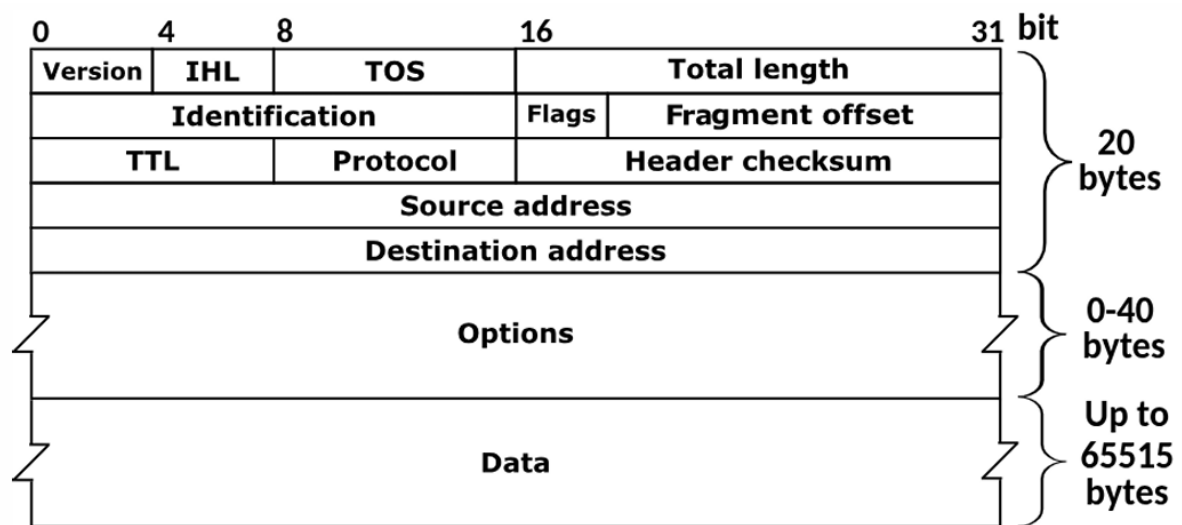
Internet Protocol (IP)

כתובת IP

כתובת IP נועדה כדי לזהות מחשבים ברשת. אנחנו נדבר על IPv4, אבל קיים גם IPv6 שפשוט מאפשר יותר כתובות. הכתובת מורכב מ-4 מספרים בין 0 ל-255 מופרדים בנקודה: 127.0.0.1.

כדי לבצע תקשורת על גבי הרשת אנחנו חייבים כתובת IP, וכתובת ה-IP מוגבלת בזמן, נועד לאפשר מחזור של כתובות בתום שימוש.

ההודעות שיישלחו על גבי הרשת נקרא **פקטות** (בניגוד ל"פריימים" בשכבה 2). מבנה פקטה:



כשהפקטות נבנו לא היה צורך בווידוא אמינות של כתובת ה-source, אבל כשהאינטרנט הפך נפוץ יותר הבינו שאין בטיחות בפרוטוקול – וכבר "מאוחר מדי" לשנות את הכל.

ה-Data זה הדבר האחרון שנשלח. הגודל המקסימלי של הדאטה הוא לפי שדה total length, ונשים לב שהערך המקסימלי שיכול להיכנס בשדה הזה הוא $2^{16} = 65,536$ bytes.

מהראי של Layer 3 זה נראה כך:



אם אנחנו נסתכל על המידע שעובר בראי של Layer 2 זה נראה כך:



אבל בעצם ה-data מחולק כך:



אם אנחנו ב-Layer 3 אנחנו מסתכלים על כל מה שהוא לא כתובת מקור ויעד כ-Data, לא רלוונטי מה קורה בפנים כדי לבצע את התפקיד של השכבה.

שאלה: בניח שיש לנו 2 hosts. איך A host יודע ש-B host לא נמצא איתו באותו LAN?

באמצעות **IP Subnets**. דרך לחלק כתובות IP לקבוצות. IP Subnets מוגדרים לפי IP Prefix וגודל, כמו למשל: 211.12.3.0/24. ה-24 אומר ש-24 הביטים הראשונים הם קבועים, וה-8 האחרים משתנים, כלומר הכתובות יהיו בטווח 211.12.3.0-211.12.3.255, כלומר יש ל-Subnet הזה 255 כתובות IP לחלק.

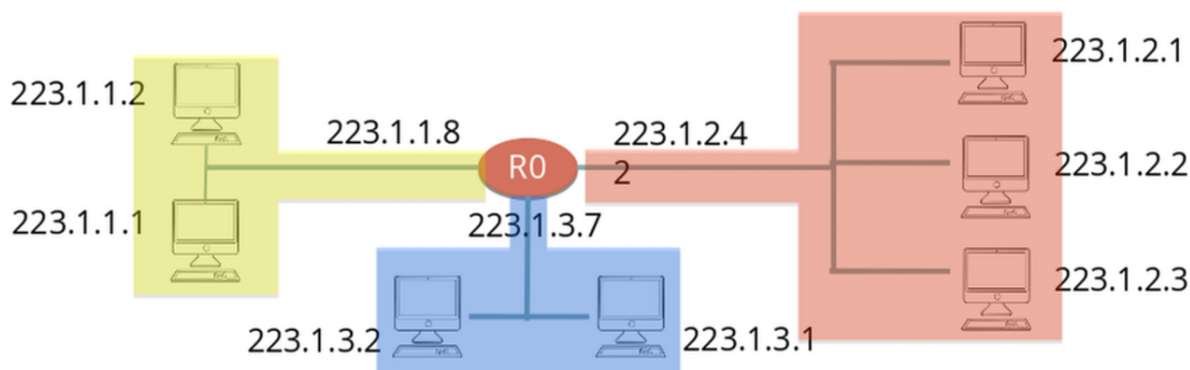
המספר 24/מתייחס למספר הביטים הקבועים – ככל שהוא יותר גדול כך יש פחות כתובות לחלק, ככל שהוא יותר קטן כך יש יותר כתובות לחלק. דרך נוספת לייצג את המקרה של 24/ באמצעות **Subnet Mask** היא 255.255.255.0, וזו צורה שמחשבים מעדיפים לצורך חשבון בינארי.

IPv6 משתמש ב-8 קבוצות של 4 ספרות hexadecimal, fun fact: פייסבוק רכשו את כל הכתובות מהצורה: ...:...:face:b00c:...:...:...

יש כמה מחשבים שמחוברים לאותו router, כל יציאה מ-router זה subnet משלו:

● In this example we have 3 subnets:

- 223.1.1.0/24
- 223.1.2.0/24
- 223.1.3.0/24



איך משיגים כתובת IP?

- (1) בעזרת Network Admin – פחות נפוץ כי לרוב הכתובות לא משנה עד כדי כך שנצטרך את אותה אחת בכל שימוש.
- (2) ISP – ספקית האינטרנט (בזק למשל). גם כאן אלא אם כן שולם מעבר הכתובות יהיו משתנות.
- (3) שירות שזמין בתוך הרשת שלך.

Dynamic Host Configuration Protocol – DHCP

יש רכיב ברשת שיכול לתת למכשירים כתובות IP, והוא גם מגלה בעצמו שכתובת ה-IP פנויה בסיום השימוש או מקצה שוב כתובת IP אם תם הזמן והמכשיר עדיין ברשת.

הפרוטוקול פשוט (כל 4 ההודעות הבאות הן **broadcasted**):

(1) ה-Host שולח "DHCP Discover" – "אני מחשב חדש שצריך IP", כולם שומעים את זה ולרוב מתעלמים. מזה פרט לשרת DHCP.

(2) ה-Server מחזיר "DHCP Offer" – "הנה כתובת ה-IP שאני מציע לך".

(3) ה-Host שולח "DHCP Request" – "הכתובת שהשרת נתן מתאימה לי, אני מבקש אותה".

(4) ה-Server שולח "DHCP ACK" – "אני מאשר לך שזו הכתובת, היא שמורה לך".

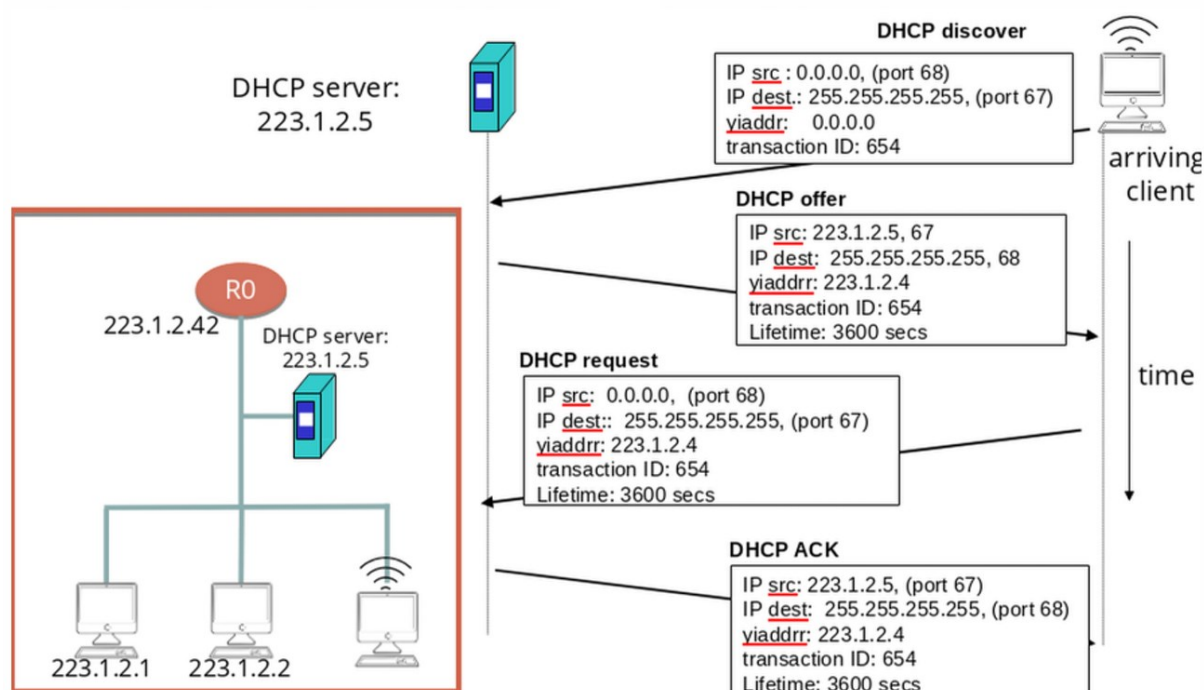
אבל אם ה-host מחשב חדש, איך הוא יידע איזה כתובת יעד לשים בפקטה של הודעת ה-DHCP Discover? ומה ישים בכתובת המקור? מוסכם:

Source IP = 0

Destination IP = Broadcast

YIADDR (your IP address) = כשהשרת ישיב בפעם הראשונה זה יכול להיות מה הכתובת המוצעת, וכשה-host משיב את הכתובת שהוא רוצה היא יכולה להופיע בשדה הזה.

כעת נבחן את הדיאגרמה (הוצג גם בהרצאה):



הסימון ל-port 67 אומר שמנסים לדבר עם שרת ה-DHCP. נשים לב שבכל הודעה מופיע גם transaction ID, וכך יודעים שהתשובה ממוענת אל ה-host שביקש כתובת.

שני השלבים הראשונים (1) DHCP Discover ו-(2) DHCP Offer הם אופציונליים. נניח שאנחנו מחוברים לרשת כבר הרבה זמן וכתובת ה-IP שלנו עומדת לפוג, נוכל ישר לשלוח לשרת ה-DHCP ישירות את שלב (3) DHCP Request כדי לקבל את אותה כתובת שוב.

הערות נוספות על שרת DHCP

- מספק כתובת IP של DNS Server.
- מספק כתובת IP של Default gateway לצורך שליחת פקטות מחוץ לרשת.
- מספק את ה-Subnet Mask של ה-LAN.
- לפעמים יש יותר משרת DHCP אחד ברשת, מי שמעניק את ה-IP למכשיר חדש הוא בדרך כלל זה שהיה הראשון לתת הצעה. זה מעיד על כך שיש race condition שמוביל לבעיות אבטחה (כמו DHCP Spoofing – כשמישהו מבקש כתובת IP אפשר גם להתחזות לשרת DHCP ולתת לך כתובת אחרת, זה נקרא Denial of Service, ויש דרכים נוספים לעשות את זה כמו למשל להציף בפקטות). בעיות האבטחה לא חמורות, בסך הכל התקשורת לא תתאפשר וננסה לחבר מחדש.

Address Resolution Protocol (ARP)

אחרי שקיבלנו כתובת IP מהשרת, מה הלאה? אנחנו יכולים להתחיל לתקשר עם hosts בתוך ומחוץ ל-LAN. אבל בתוך ה-LAN אם נרצה לתקשר עם אחד המכשירים נצטרך גם כתובת MAC, למה? כי בתוך הרשת מדברים ב-Layer 2 ולא 3.

איך משיגים כתובת MAC?

באמצעות פרוטוקול **Address Resolution Protocol (ARP)**. גם כאן יש שלבים למקרה בו Host 1 רוצה את כתובת ה-MAC של Host 2:

(1) Host 1 משדר ב-broadcast "ARP Request" עם כתובת ה-IP של Host 2.

(2) Host 2 שולח "ARP Response" עם כתובת ה-MAC שלו ישירות ל-MAC של Host 1.

לכל קודקוד תהיה טבלת ARP שממפה כתובות IP לכתובות MAC כדי לחסוך את ההודעות מהסוג כשאין צורך. בדומה לטבלאות אחרות גם כאן יהיה TTL. גם כאן יש בעיית race conditioning שמאפשרת ARP Spoofing, מישהו שהוא לא Host 2 יכול לענות תשובה אחרת שאינה כתובת ה-MAC של Host 2, וכאן זה יותר בעייתי מהמקרה הקודם של DHCP Spoofing, כי זיוף של ה-gateway יכול להוביל את התוכן של אחרים אל הגורם ה-malicious.

שאלות תרגול ARP ו-DHCP

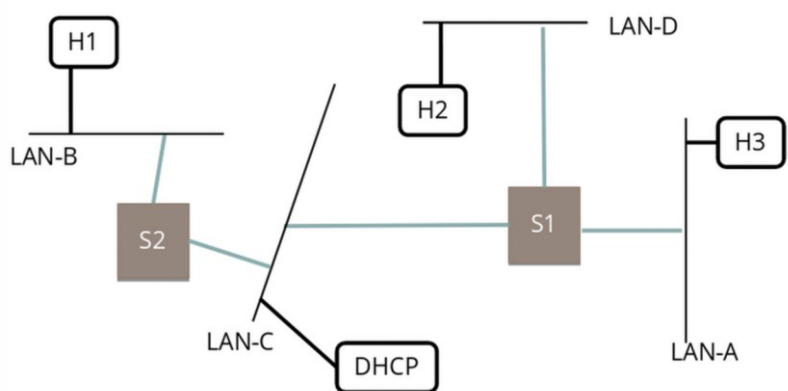
[שאלה 1] האם אתה יכול לוודא אם כתובת ה-IP הנוכחית שלי עדיין זמינה באמצעות ARP?

כן! אני יכול לשלוח הודעת ARP Request עם כתובת ה-IP שלי, אם לא קיבלנו מענה (אין כתובת MAC שמשויכת ל-IP הזה) סימן שהוא שוחרר.

[שאלה 2] הציעו דרך להשתמש ביותר מסרבר DHCP אחד בתוך LAN.

אפשר לחלק טווחים לכל אחד מהשרתים.

[שאלה 3] נניח שברשת הזו H1 חדש, והוא רוצה לתקשר עם H3. מה הוא עושה?



H1 צריך לבקש IP Address, ואז למצוא מה כתובת ה-MAC של H3. כשנשלם את כל הפרטים נוכל לשלוח את ההודעה, וזה המידע שגלוי לנו עד כה (נניח שיש לנו DST IP):

✗	✓	✓	✗	✓
SRC IP	DST IP	SRC MAC	DST MAC	DATA

אילו הודעות נשמעו ברחבי הרשת?

- (1) DHCP Discover – מדובר בהודעת broadcast שמטרתה להקצות עבור H1 כתובת IP.
- (2) DHCP Offer – שוב הודעת broadcast כי עדיין אין IP ל-H1.
- (3) DHCP Request – עדיין הודעת broadcast.
- (4) DHCP ACK – גם כן.

עכשיו אנחנו יודעים מה ה-IP לכן המצב הוא:

✓	✓	✓	✗	✓
SRC IP	DST IP	SRC MAC	DST MAC	DATA

(5) ARP Request – נשלח לכולם שאלה מה כתובת ה-MAC של המכשיר בעל IP (ואז את ה-IP של H3).

בדרך הסוויצ'ים ישמרו את כתובת ה-MAC של H1 בצירוף כתובת ה-IP שלו.

(6) ARP Response – ההודעה הזו כבר תגיע ישירות ל-H1 ולא יהיה broadcasted, הודות לכך שהסוויצ'ים

שמרו את כתובת ה-MAC של H1.

✓	✓	✓	✓	✓
SRC IP	DST IP	SRC MAC	DST MAC	DATA

ומה קורה אם הרשת שלנו מורכבת מכמה LANs?

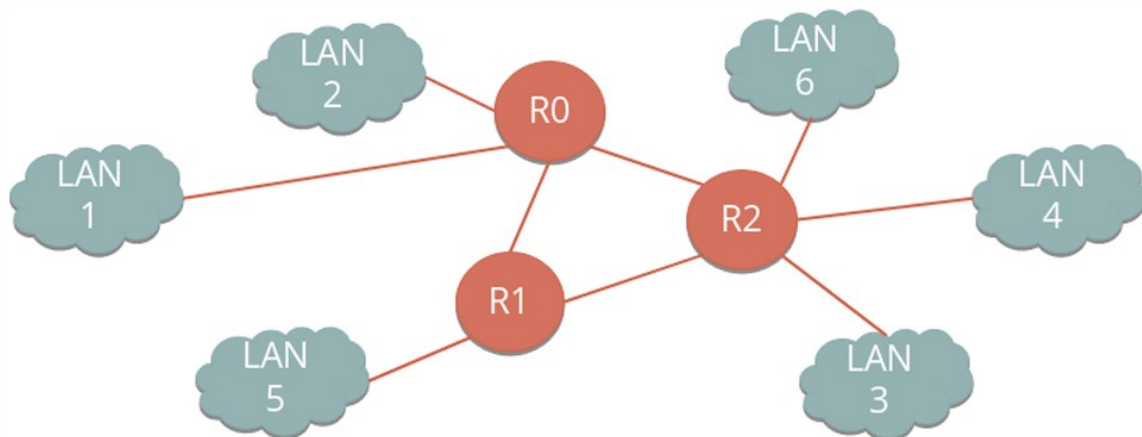
נצטרך להבין איך להעביר את ההודעות בין LANs, וכאן נכנס תפקיד הנתב. במקרה הזה אם H1 רוצה לדבר עם H3, מה שומעים ברשת? כמו מה שתיארנו, ההודעות לא ישתנו כי הכל נשאר בתוך אותו LAN. מה הרשת השנייה תשמע מתוך זה? **כלום**.

Routers and Broadcast Domains

Routers – Bounding Broadcast Domains

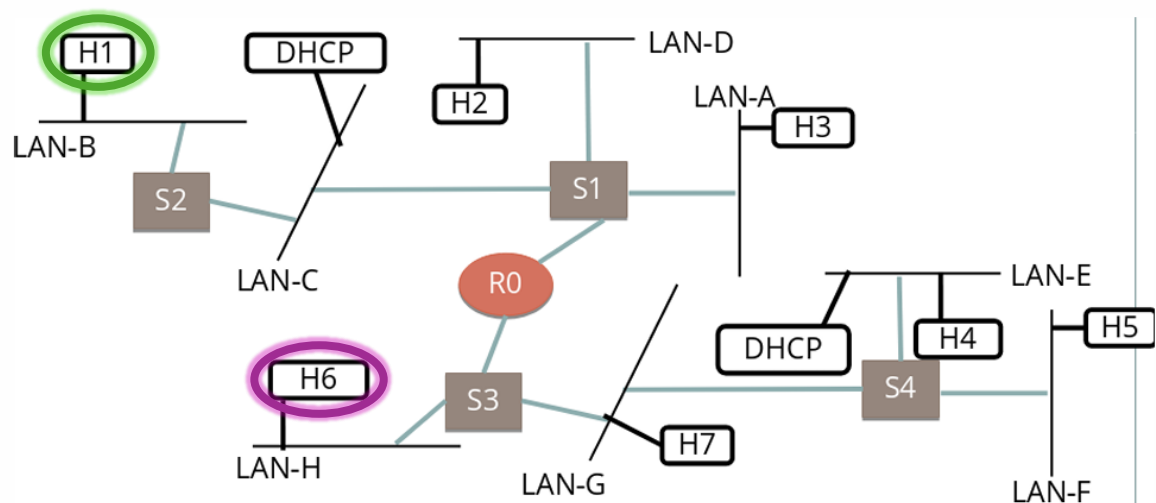
לכל ראوتر יש יציאות, ובכל יציאה יש broadcast domain. כשמגיעים לראوتر לא מעבירים את ההודעות הלאה אלא אם כן יש סיבה לכך.

[שאלה] כמה broadcast domains יש ברשת הבאה?

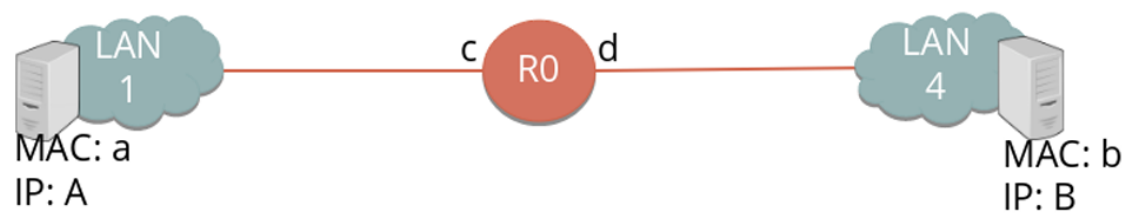


[תשובה] 9, אחד לכל LAN (ויש 6 LANs), ועוד 3 שמחברים בין הראוטרם.

[שאלה] מה קורה כש-H1 רוצה לדבר עם H6?



נתחיל במשהו פשוט יותר – Host A רוצה לדבר עם Host B בשכבה 3, דרך ראוטר. מה הוא צריך לעשות?



לראוטר יש 2 כתובות MAC (c,d). ה-Hosts לא באותו LAC, וכשהפקטה מ-A תעבור בראוטר, הראוטר ישנה את ה-header שלה. זה תקין לשנות metadata כל עוד לא נוגעים בדאטה. נפרט לפי השלבים:

Step	SRC MAC	DST MAC	SRC IP	DST IP
$A \rightarrow R_0$	a	c	A	B
$R_0 \rightarrow B$	d	b	A	B
$B \rightarrow R_0$	b	d	B	A
$R_0 \rightarrow A$	c	a	B	A

לשים לב לשינוי ב-DST MAC.

[שאלה] מה קורה אם אנחנו לא יודעים את כתובת ה-MAC של הראוטר (או של Host B)?

משתמשים ב-ARP כדי למצוא את הנדרש. ראוטרים לא מעבירים הלאה הודעות ARP בין LANs.

הערה: במבחן נפוצות שאלות בסגנון "מצטרף מחשב חדש לרשת בשם A, תאר את כל השלבים למסירת הודעה למחשב B". בסוף המצגת יש שאלה כזו לדוגמה עם פירוט של כל השלבים.

Multiple Routers

ראינו מה קורה לפקטה כשעוברים מרשת L2 אחת לאחרת – בעיקר שינוי של כתובת MAC של המקור ויבצע ARP Requests רלוונטיות.

[שאלה] מה קורה אם שרת ה-DHCP הן מחוץ לשרת?

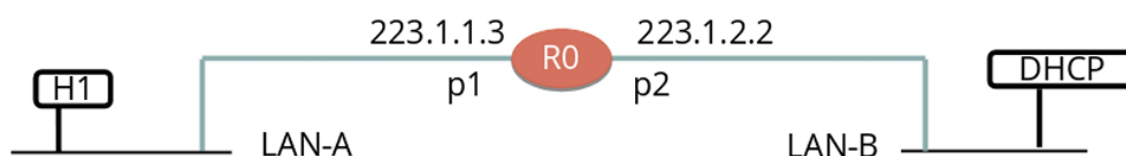
בקשות DHCP במצב כזה לא יישארו בתוך אותו ה-LAN, הראוטר יהיה מודע לזה ויעביר את הבקשות הלאה.

DHCP Relaying

נדרשת קונפיגורציה ל-R0 כדי להעביר בקשות DHCP לרשת אחרת. זה נקרא **DHCP Relaying**, ו-R0 נקרא **relay agent**.

עד עכשיו דיברנו על broadcast domains, אבל בשלב הזה כנראה שלאחר כמה זמן הסוויצ'ים ילמדו את הבקשות ותהיה דרך טובה בין R0 לשרת ה-DHCP שתקל על עומסים.

[שאלה] איך H1 יכול לקבל IP Address כאשר שרת ה-DHCP לא איתו באותה הרשת?



Message	SRC MAC	SRC IP	DST MAC	DST IP
DHCP discover	MAC(H1)	0.0.0.0	broadcast	255.255.255.255
disc. (GIADDR=223.1.1.3)	MAC(R0,2)	IP(R0,2)	MAC(DHCP)	IP(DHCP)
DHCP offer (223.1.1.x)	MAC(DHCP)	IP(DHCP)	MAC(R0,2)	IP(223.1.1.3)
DHCP offer (223.1.1.x)	MAC(R0,1)	IP(R0,1)	Broadcast	255.255.255.255

המבנה הזה מכיל גם מידע layer 2 וגם מידע layer 3, כי אנחנו צריכים תקשורת בתוך LAN וגם בין LANs. הערה: למי שמשתמש במצגת – שימו לב שהכתורות של SRC IP ו-DST MAC התחלפו בטעות, בטבלה כאן זה תוקן.

Domain Name System (DNS)

נניח שיש לנו כתובת URL – אנחנו מתקשרים באמצעות כתובות IP, ולא כתובות URL. איך נדע לאן להגיע?

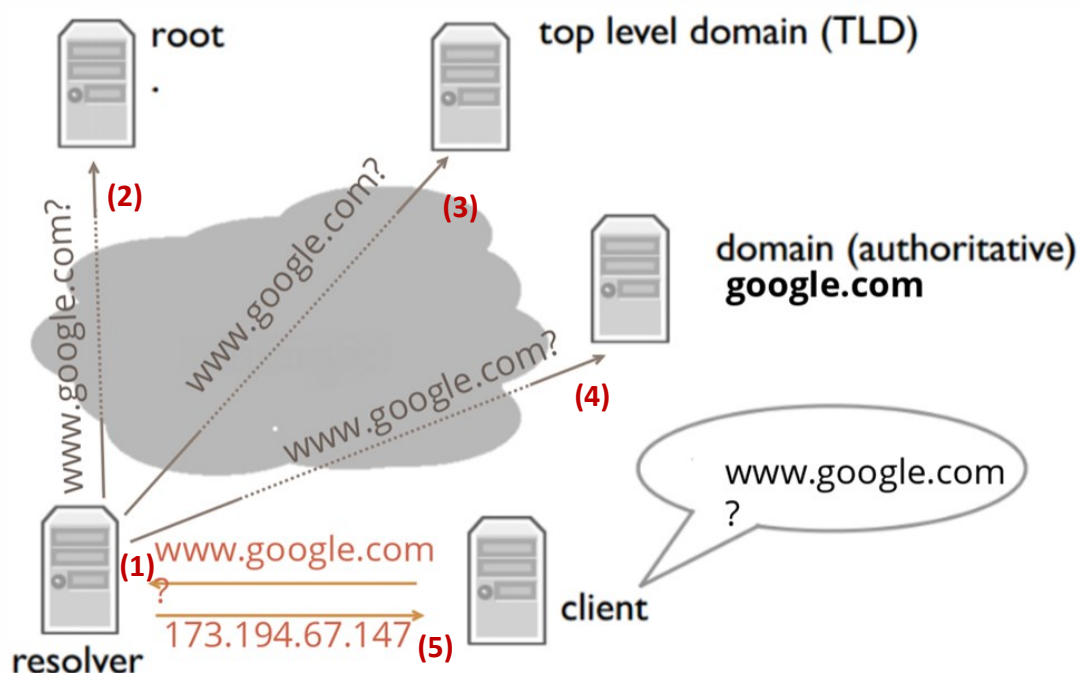
.DNS Protocol

DNS הוא מערכת של מסד נתונים מבוסס – בכל פעם נבצע בקשות רקורסיביות עד שנקבל בחזרה את הכתובת הנדרשת, כאשר אנחנו עוברים מלמעלה למטה. לבקשה יש "זמן חיים", נשמור בקשות אחרונות כדי לא לבצע בכל פעם שאילתה. גם כאן יש פרצות – נראה אותן בהמשך.

הסיומות המוכרות כמו .org או .com. הם ה-Top Level Domains (TLD) וכשנרצה עבור עצמנו כתובת שנגמרת בסיומת הזו, נצטרך לשלם לאותו ארגון.

כל מה שמגיע מתחת לזה בכתובת נקרא sub domains.

קודם כל, נצטרך לשאול את שרת ה-DNS הלוקאלי שלנו (מכונה "resolver") אם הוא מכיר את הכתובת, זה נוח אם שרת ה-DNS נמצא בתוך ה-LAN כי אז כל בקשת IP תהיה פנימית. אם הכתובת מוכרת – נקבל אותה והתהליך יסתיים. אחרת, יתחיל תהליך ה-resolving עד למציאת הכתובת בתהליך היררכי עד למציאת הכתובת.



(1) תן לי את הכתובת של www.google.com.

(2) Resolver שואל את ה-root (מעל אפילו .com) – הוא נניח לא יודע, אז הוא אומר "לא יודע, אבל אני מכיר את .com. שהוא ה-TLD".

(3) Resolver שואל את ה-TLD הרלוונטי (כאן .com) אם הוא מכיר – נניח שלא, אז הוא אומר "אני מכיר

את google.com, תשאל אותו"

(4) Google.com מכיר את www.google.com ומחזיר את הכתובת.

(5) מסירת כתובת ה-IP לקליינט.

הערה: לפעמים (לא בגוגל) כתובת ה-IP יכולה להשתנות אחרי זמן מסוים, ואם תהליך ה-resolving נמשך הרבה זמן לפעמים נקבל כתובת לא נכונה ונצטרך להתחיל את התהליך שוב.

סוגי שמות ב-DNS Record

(1) מיפוי של Host Name לכתובת IP

(2) מיפוי של Host Name ל-Name Server

Root Servers

יש 13 Root Servers, ממופים A עד M. כתובת ה-root נמצאת בbuilt in בתוך הדפדפן כדי לתת לנו מקום להתחיל ממנו.

כשאנחנו קונים domain אנחנו משלמים לאתר שאחראי על זה, והם מעדכנים את ה-DNS server שלהם כך שפעם הבאה שמישהו יקבל את ה-URL הזה, הוא יקבל את ה-IP שקינפגת.

בעיה בתהליך

מה קורה אם מישהו אחר ענה לנו ולא ה-resolver? במקום ללכת ל-IP הנכון נלך ל-IP זדוני ולא נוכל לדעת את זה. אפשר להצפין את ההודעה, אבל לא כולם יישמו אותה.

סטודנטית רכשה מחשב חדש ומחברת אותו לרשת הביתית שלה (שמחוברת לאינטרנט דרך הספקית). היא הולכת ל-www.youtube.com. נניח שזו הפעם הראשונה שהמחשב מתחבר לרשת. כל שאר הדברים שהיו ברשת מחוברים, אז נניח שטבלאות ARP מלאות ולא צריכות רענון ($TTL \rightarrow \infty$). מניחים ששרתי ה-DNS וה-DHCP נמצאים בתוך ה-LAN.

[שאלה] אילו הודעות עוברות ברשת?

	Step	Message	SRC MAC	SRC IP	DST MAC	DST IP
DHCP	1	DHCP discover	PC	0.0.0.0	broadcast	255.255.255.255
	2	DHCP offer	DHCP server	DHCP server	broadcast	255.255.255.255
	3	DHCP request	PC	0.0.0.0	broadcast	255.255.255.255
	4	DHCP Ack (IP,DNS ...)	DHCP server	DHCP server	broadcast	255.255.255.255
ARP	5	ARP request – DNS server	PC	—	broadcast	—
	6	ARP response – DNS server	DNS server	—	PC	—
DNS	7	DNS request – youtube.com	PC	PC	DNS server	DNS server
	8	DNS response – youtube.com	DNS server	DNS server	PC	PC
ARP	9	ARP request – default gw	PC	—	broadcast	—
	10	ARP response – default gw	Default gw	—	PC	—
HTTP	11	HTTP request	PC	PC	Default gw	IP(youtube.com)
	12	HTTP response	Default gw	IP(youtube.com)	PC	PC

שלבים 1-4: DHCP

קודם כל, המחשב החדש צריך לקבל כתובת IP, לכן השלבים של discover, offer, request, ACK.

שלבים 5-6: ARP

אנחנו מעוניינים ללכת ליוטיוב, ולשם כך צריך להגיע לשרת ה-DNS. נתון לנו שהוא בתוך ה-LAN שלנו, ולכן התקשורת תהיה מעל שכבה 2 אז אנחנו צריכים את כתובת ה-MAC של שרת ה-DNS, ולנו יש רק את ה-IP.

שלבים 7-8: DNS

שולחים בקשה ל-DNS server בבקשה לקבל את כתובת ה-IP המתאימה ל-URL שביקשנו, שרת ה-DNS יביא לנו את הכתובת (בין אם שמורה במטמון ובין אם נאלץ לחפש אותה באמצעות תהליך resolving).

שלבים 9-10: ARP

יש לנו את כתובת ה-IP של יוטיוב, אבל עדיין אנחנו צריכים לשלוח פקטה ל-gateway ולשם כך אנחנו צריכים את כתובת ה-MAC של ה-default gateway (קיבלנו רק IP שלו מה-DHCP).

שלבים 11-12: HTTP

בשלב זה המחשב יכין פקטת HTTP (עוד לא הגענו לשלב הזה בקורס, שייך לשכבה 5).

(בקובץ התרגול יש שאלות נוספות עם פתרון)

הרצאה 8

הקדמה

היום נלמד על DNS ועל ARP.

בעיות אבטחה – המשך

עמידת של DNS מפני Denial of Service

בהרצאה הקודמת דיברנו על בעיית האבטחה הראשונה והשנייה. נרחיב:

מצגת 4

שקף 48-50

בעיית בטיחות #2

למרות תקיפות בעבר השירות המשיך להתקיים בגלל מנגנון ה-caching שגם לכך שבפועל הנזק לא היה גדול. איך נדע שתשובת DNS שמתקבלת היא תשובה מהימנה מהשרת הנכון? אנחנו יכולים להיות ברשת שאנחנו לא סומכים עליה, וה-local DNS resolver הוא לא מישור שאנחנו מריצים ויכול לענות לנו תשובות לא נכונות. מחשב הקצה שולח שאילתה ל-local DNS resolver שלו, אשר מבצע את התהליך האיטרטיבי שתיארנו עד שהוא מגיע ל-authoritative name server ומקבל תשובה - כתובת IP. מה קורה אם זו לא הכתובת האמיתית של ה-domain שהוא ניסה להגיע אליו.

איך זה יכול לקרות?

יש "man in the middle" שרואה את השאילתות באמצע ויכול לקבוע לאן להפנות את מי שביצע את השאילתה. הוא יזייף גם את כתובת המקור כדי לדמות כאילו שרת ה-DNS הוא זה שענה על השאלה - IP Spoofing. בהמשך מגיעה תשובה אמיתית אבל היא מתייתרת. הפרוטוקול חייב להמשיך הלאה בתשובה הראשונה, לא הגיוני לחכות לראות אם מגיעות תשובות נוספות.

הבעיה הזו לוקאלית ללקוח - התוקף ענה ללקוח ספציפי כתובת IP מזויפת והלקוח הזה הוא הקורבן היחיד, לקוחות אחרים לא ייפגעו.

שקף 51

מה שמחמיר את הבעיה הוא מנגנון caching שמאפשר לנצל את הפרצה הזו בצורה רחבה יותר. כל DNS Resolver שומר תשובות שקיבל ב-cache לוקאלי כדי לחסוך את השאילתות לתקופה יחסית ארוכה.

שקף 52-53

בעיית אבטחה #3: DNS Cache Poisoning

אם הגורם הזדוני רוצה לעשות השפעה גדולה יותר, הוא יכול לתת תשובה לא נכונה ל-DNS resolver, ואז כל הלקוחות המקושרים אליו יקבלו תשובות לא נכונות.

נשים לב שהתוקף גם יכול לקבוע את ה-TTL, ולכן יכול לקבוע אותו להמון זמן (למשל יום, כשבדרך כלל תשובה לגיטימית תקפה לכ-10 דקות).

שקף 54-55

למה זה לא קורה כל הזמן אם הפרוטוקול כל כך בעייתי? בהודעת DNS יש שדה transaction ID שמזהה עבור לקוח מסוים את הבקשה של, מספר 16 ביטים רנדומלי. כשהתוקף רוצה לזייף תשובת DNS הוא צריך לכתוב את מה שהוא קיבל. הבעיה היא שבשהתוקף מחוץ לרשת הוא לא יודע את ה-ID, ולכן צריך לנחש אותו (הוא יענה נכונה בהסתברות $1/2^{16}$, ואם הוא טעה התשובה שלו תכנס ל-cache והוא לא יוכל לנסות שוב).

סוגים של DNS Resolvers

1. היו מימושים של resolvers שכל שאילתה שהם שלחו הגדילו את ה-ID transaction ב-1, על זה מאוד קל לעלות ולזייף transaction ID.

2. 16 ביט שהשולח מגריל בכל בקשה.

אם נשכנע את ה-resolver להוציא הרבה שאילתות, ואנחנו נענה הרבה תשובות, אז בסיכוי גבוה מתישהו נענה נכונה (פרדוקס יום ההולדת).

היום גם שיטה זו לא עובדת, כי שרתים זוכרים מה הם ענו לאחרונה, ולכן לא ישאלו שוב ושוב את אותה שאלה.

שקף 56-63

Kaminsky Attack

בשנת 2008 דן קמינסקי מצא התקפה שתקפה לכל מימושי ה-DNS resolvers. לתוקף יש יתרון משמעותי:

הוא קובע מה השם שה-resolver שואל עליו ומתי (אם יש לקוח מאחורי ISP או באמצעות דרכים אחרות כמו למשל אם שולחים למישהו אימייל עם reference למשהו שנמצא במקום אחר, כך מחייבים DNS resolution שתגרום לשאילתת DNS לצאת מהארגון. קמינסקי אומר "אני יכול להתחרות על הרבה שמות, ואנצח באחד מהם" בכל פעם הסיכוי הוא $1/2^{16}$, ואם אפסיד אי אפשר לנסות שוב על אותו שם, אבל נגיד אחד השמות עבד, למשל 183.google.com ועליו הוא יוכל להחזיר איזו תשובה שירצה.

לכאורה, למה שמישהו ירצה לגשת לכתובת הזו? אינהרנטית בפרוטוקול DNS אנחנו מרשים לשרת שעונה על דומיין מסוים לתת תשובות לדומיינים אחרים. אם הצלחנו להרעיל את 183.google.com נצליח להרעיל גם את www.google.com באותה התשובה. מה ההגיון מאחורי? הבעלים של כל ה-sub domains הם אותם בעלים, אז התשובה אמורה להתקבל מאותו המקום. ההנחה היא שהדומיין שאנחנו רוצים להרעיל לא נמצא ב-cache, כי אם הוא כבר נמצא אז אי אפשר להרעיל אותו.

למה ההתקפה עובדת כל כך טוב? הוא הבין שהוא יכול להתחרות על המון שמות ולא לשלם את ה"עונש" (זמן שהתשובה נמצאת ב-cache) עבור כישלון.

[ההשלכות](#)

התוצאה הייתה שרצו למצוא פתרון לפרצת ה-DNS Poisoning כמה שיותר מהר. אחת הבעיות שהתוקף ניצל זה שהיה לו שדה transaction ID באורך 16 ביט - הסיכוי לפגוע נמוך אבל לא זניח. שקלו להגדיל את האנטרופיה - לגרום לו להיות חייב לפגוע ביותר ביטים, למשל להשתמש בשדה של 64-ביט, אבל זה כבר גורר שינוי בפרוטוקול שלא ניתן ליישם (ישנה את כל תשתית ה-DNS). חייב פתרון שרק ה-Resolver מתקין והוא כבר יהיה חסין.

הפתרון הזה הוא בדמות ה-ports. נלמד ש-DNS רץ מעל פרוטוקול UDP (של שכבת transport), וזה מתבצע לפי port number שהוא גם 16 ביטים. לפני ההתקפה ה-port הנבחר היה קבוע, אבל עכשיו ה-UDP Source port יהיה רנדומי (למה ה-destination לא רנדומי? כי השרת מקשיב על פורט מסוים). כך קיבלו עוד 16 ביטים שהתוקף צריך "לפגוע בהם", עכשיו הסיכוי לפגוע הוא בערך $1/2^{32}$ כי יש כמה שלא בשימוש, אבל עדיין סיכוי נמוך מאוד.

[שקף 64](#)

[DNSSEC](#)

הפתרון שתיארנו בעת הוא הפתרון שמיושם עד היום, אך הוא לא ארוך טווח. בשנים האחרונות מנסים (ומצליחים) לגרום ליישום של פרוטוקול חדש DNSSEC שמשתמש בחתימות דיגיטליות שיעניקו את הבטיחות אליה שאפנו. מדובר בכלי קריפטוגרפי שנותן לאמת שהודעה מסוימת הגיעה ממשהו מסוים. מנצל את המבנה ההיררכי של ה-DNS כך שכל שכבה תחתון על הרשומות של השכבה שמתחתיה.

כך למשל verisign יחתמו אישור שהמיפוי נכון עבור com. לפני שעוברים לאימות החתימה הנמוכה יותר בהיררכיה.

הפרוטוקול נטמע, אבל באמצעות תמיכה אחורנית - לקוח שמנסה לעשות DNSSEC ונכשל ילך ל-DNS. הרבה מנסים ליישם וכיום בערך 93% יישמו, כשהחשובים ביניהם כבר יישמו - אבל מכאן תהליך היישום איטי יותר (זנב ארוך).

ARP (Address Resolution Protocol)

שקף 67-69

מטרה

נניח שקיבלנו את כתובת ה-IP של מישהו, אבל צריך שההודעה תגיע לכרטיס הרשת שלו כי התקשרות בתוך השכבה שלו היא בשכבה 2 שעובדת לפי כתובות MAC. מטרת הפרוטוקול היא לקבל כתובת MAC בהינתן כתובת IP של מכשיר.

לכל מחשב יש כתובת IP (דינמית) וכתובת MAC שמוטבעת בכרטיס הרשת. שני מחשבים רוצים לדבר זה עם זה, אז לכל מחשב ברשת יש טבלת ARP שממפה כתובות IP לכתובות MAC.

כל הסוויצ'ים ברשת יעשו ניתוב לפי כתובת ה-MAC, כדי לשלוח הודעה ברשת חייב לדעת את שתי הכתובות - גם IP וגם MAC. אם יש לנו בטבלה את הכתובת אז נשלוח את הרשומה המתאימה. אם היא לא בטבלה, יש תהליך ARP שעובד ב-broadcast כי אין לנו את כתובת ה-MAC של היעד, ומטרתו למצוא את כתובת ה-MAC.

שקף 70-71

Routing across LANs

יש לנו שני מחשבים A ו-B שרוצים לתקשר מעל האינטרנט ונמצאים ב-subnets שונים. נניח ש-A יודע את כתובת ה-IP של מחשב B, וביניהם יש router. כש-A שולח הודעה הוא שולח אותה עם ה-destination IP של B, אבל הנתב צריך לדעת להעביר אותה לנתב הבא בשרשרת. איך ההודעה של A תגיע ל-router שהוא על ה-gateway הנכון ברשת? הוא צריך להגיע לרגל הנכונה בראוטר, ולכן יעשה שאילתת ARP לקבל את כתובת ה-MAC שלה. ה-destination IP יהיה של B, אבל ה-destination MAC יהיה של הרגל הנכונה בראוטר. הראוטר יודע מאיפה הגיעה ההודעה לפי כתובת ה-MAC, ואז מחליט מאיפה היא צריכה לצאת לפי ה-destination IP (שולף מתוך הטבלה שמחזיקה כתובות IP). ה-source IP הוא הכתובת של A, אבל ה-source MAC יהפוך להיות הרגל של הראוטר ממנה ההודעה יוצאת, וה-destination MAC יהיה של B.

כל hop אנחנו מחליפים את כתובות ה-MAC, ורק כתובות ה-IP מלוות את ההודעה מקצה לקצה. ARP מאפשר לנו בכל פעם למצוא את כתובת ה-MAC של הקפיצה הבאה.

שקף 72-73

האם יש מספיק כתובות IP?

מרחב הכתובות של IPv4 בנוי מ-4 בתים כל כתובת. בשלב מסוים הבינו שזה לא מספיק. בעשורים האחרונים מנסים לעבור ל-IPv6 ששם יש כתובות של 8 בתים, שיפור משמעותי לעומת IPv4, ובמקביל תיקנו דברים נוספים. אחד מהם זה שינוי פורמט ה-header כך שיתאפשר parsing טוב יותר בחומרה - גודל קבוע ולא משתנה.

שקף 74

הבעיה היא שבגלל שכל הניתוב מתרחש לפי כתובות IP המעבר לא חלק - לא ברור מתי ראوتر (חומרה) צריכה לדבר ב-v6 במקום ב-v4, ובמקביל יכול להיות לקוח שמדבר רק ב-v4. יש זמן ביניים בו אנחנו צריכים לדעת לעבוד עם שני הפרוטוקולים במקביל - לקוח IPv6 בן מצפה לתקשר עם שרת אינטרנט בכתובת IPv4 (יש צורך לגשר על הניתוב). זה יצר סיבוך במעבר ופתרונות ביניים שנועדו לעזור לנו לאמץ את הפרוטוקול החדש בהדרגתיות.

שקף 77

הגרף מראה את האימוץ של IPv6 באינטרנט, 15 שנה אחרי תחילת היישום אנחנו בערך בנקודת האמצע (הגרף לא מופיע בשקפים שהועלו למודל).

Routing

נדבר בתרגול על Routing בין רשתות שונות (בשכבה 3). נראה שני פרוטוקולים: Distance Vector ו-Link State. בתרגול הקודם דיברנו על פרוטוקול IP בשכבה 3, ועל פרוטוקולים שעוזרים לנו לתקשר בתוך רשת מקומית: DHCP שמאפשר להצטרף לרשת מקומית (לקבל כתובת IP + מידע שהכרחי לתקשורת), על פרוטוקול ARP שמאפשר לקבל כתובות MAC בהינתן כתובת IP, ועל המרה של כתובות שקריאות לנו אל כתובות IP.

היום נדבר על מה קורה כשהראוטר מקבל הודעה שהוא צריך להעביר הלאה. נתמקד בתקשורת בין רשתות מקומיות (עדיין לא על האינטרנט ככלל).

מה זה Routing ואיך מבצעים אותו?

נצטרך לבחור את המסלול שדרכו אנחנו מעבירים את ההודעה – זה **Routing**. המושג של העברת הודעה מרשת אחת לאחרת הוא **Forwarding**.

את ה-Routing נבצע על ידי להסתכל רק על מבנה הראוטרים (ללא הרשתות המקושרות לכל אחד מהם). למבנה הזה נתייחס כגרף שהקודקודים בו הם ראוטרים והצלעות הם הקשרים ביניהם. אפשר להוסיף משקולות על הלינקים שיכולים לסייע לנו לבטא עד כמה אנחנו רוצים להשתמש בלינק.

הפרוטוקולים שנראה מבוססים על אלגוריתמים בלמן-פורד ודייקסטרה.

Routing Table – כשנמצא את המסלולים הכי טובים נשמור אותם בתוך טבלה שנקראת **Routing Table**. כל ראוטר מחזיק טבלה כזו, ובה הוא ישתמש כדי לבחור את המסלול דרכו יעביר את ההודעה.

אלגוריתמי Routing

נבדיל בין שני סוגי אלגוריתמים:

1. האלגוריתם הראשון ישתמש **בגישה מבוזרת** בו המידע לא נמצא במקום אחד, אלא לכל ראוטר יש מידע מקומי שקשור אליו, ומידע נוסף יתקבל מהשכנים שלו.
2. האלגוריתם השני ישתמש **בגישה גלובאלית** בו כל קודקוד מכיר את כל הרשת ויודע לקבל החלטות על סמך כל המידע שהוא נדרש לו.

Distance Vector Protocol

זה הפרוטוקול שמשתמש בגישה המבוזרת ומבוסס על אלגוריתם בלמן פורד:

$$D_s(y) = \min_{v \in \Gamma(s)} \{c(s, v) + D_v(y)\}$$

המרחק מקודקוד s לקודקוד y כלשהו הוא לקחת את המינימום מבין לעבור על כל השכנים של s ולחבר את המרחק אליו עם המרחק שלו ליעד.

כל קודקוד מחשב וקטור מרחקים (Distance Vector) ממנו אל הקודקודים האחרים ברשת, וקודקודים שאנחנו לא מכירים את המרחק אליהם נסמן באינסוף. נקבל את וקטורי המרחקים מהשכנים וכך נוכל לעדכן את המרחק הלוקאלי אצלנו.

שלבי האלגוריתם

האלגוריתם מורכב מאתחול ומעדכון.

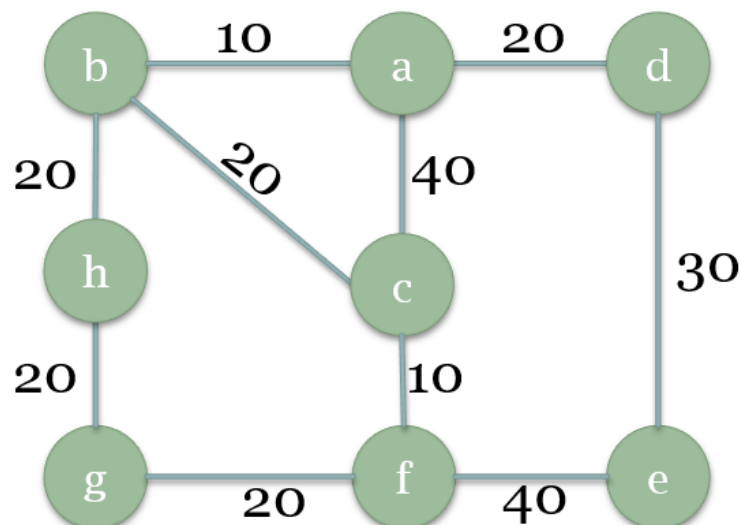
אתחול:

- בהתחלה לכל שכן מידי נשים בווקטור את המרחק שלנו אליו, וכל קודקוד אחר נקבע ל- ∞ .
- הגדרת וקטורי המרחקים של השכנים: בהתחלה נגדיר לכל שכן שהמרחק ממנו לכל קודקוד אחר הוא אינסוף.
- נשלח את ה-DV שלנו לכל השכנים.

עדכון:

- עבור כל קודקוד נעדכן את המרחק לפי הנוסחה שתיארנו.
- שליחת וקטור המרחקים שוב אם הוא השתנה.

דוגמת הרצה



כל קודקוד מריץ את הפרוטוקול בפני עצמו. אנחנו נתמקד ב-Routing Table של קודקוד a (השאר מופיעות במצגת).

אתחול:

Node a	a	b	c	d	e	f	g	h
a	0	10	40	20	∞	∞	∞	∞
b	∞	∞	∞	∞	∞	∞	∞	∞
c	∞	∞	∞	∞	∞	∞	∞	∞
d	∞	∞	∞	∞	∞	∞	∞	∞

השורה הראשונה היא עבורו, שאר השורות יכולו את וקטורי המרחקים של שכניו.

בשלב הבא נקבל עדכונים מהשכנים ומעדכן את המרחקים שרשומים אם צריך:

Node a	a	b	c	d	e	f	g	h
a	0	10	30	20	50	50	∞	30
b	10	∞	20	∞	∞	∞	∞	20
c	40	20	∞	∞	∞	10	∞	∞
d	20	∞	∞	∞	30	∞	∞	∞

מ-a ל-b המסלול הזול ביותר נשאר 10, אבל את המסלול אל c אפשר לעדכן כי נוכל ללכת קודם ל-b ומשם ל-c במחיר 30. כך מעדכנים גם את השאר. עבור e – השכן היחיד שבשלב זה ידוע לנו על מסלול ממנו ל-e הוא d ולכן המסלול מ-a ל-e יהיה $a \rightarrow d \rightarrow e$ בעלות $50 = 20 + 30$. ל-g ברגע אין לנו מסלול, נשאיר אותו ∞ .

הערה: בנוסף למחיר יישמר גם next hop כדי לדעת איך להתקדם לראוטר הבא.

בסוף הטבלה שתתקבל היא:

Node a	a	b	c	d	e	f	g	h
a	0	10	30	20	50	40	50	30
b	10	0	20	30	60	30	40	20
c	30	20	0	50	50	10	30	40
d	20	30	50	0	30	60	70	50

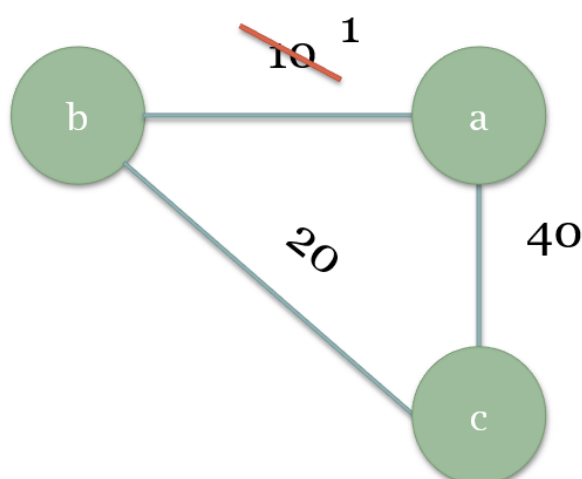
את האלגוריתם מריצים עד שאין שינוי באף טבלה. ברגע שכל ראוטר קיבל את ה-routing table הסופי, הוא צריך "לתרגם" אותה ל-forwarding table שנראה כך:

Dest.	Via	Cost
b	b	10
c	b	30
d	d	20
e	d	50
f	b	40
g	b	50
h	b	30

שינוי משקל קשתות

מה זה אומר שמשקל של לינק השתנה? יכול להיות בעקבות עומס, משהו נשחק, התנתק... בסוף הרשת אינה קבועה ויכולים להיות בה שינויים. אנחנו יכולים להחליט שבלינק מסוים יעבור יותר/פחות מידע. אם יש שינוי של משקל השכנים של אותו שינוי מודעים לשינוי, מעדכנים את וקטורי המרחקים ומעבירים את העדכון הלאה עד שמגיעים להתייצבות מחדש. נראה עכשיו מה קורה עם שני סוגי שינויים, נסתכל על תת רשת של הדוגמה למעלה עם הקודקודים a,b,c.

שינוי משקל חיובי – הורדת משקל צלע:



מה קורה כאשר נשנה את משקל הקשת בין a ל-b מ-10 ל-1?

Node a	a	b	c
a	0	10	30
b	10	0	20
c	30	20	0

Node b	a	b	c
b	10	0	20
a	0	10	30
c	30	20	0

Node c	a	b	c
c	30	20	0
a	0	10	30
b	10	0	20

אלו הטבלאות בהתחלה. מי שמודע לשינוי בהתחלה הם רק a,b. c יגלה על השינוי רק אחרי העדכון מ-a ו-b:

Node a	a	b	c
a	0	1	21
b	10	0	20
c	30	20	0

Node b	a	b	c
b	1	0	20
a	0	10	30
c	30	20	0

Node c	a	b	c
c	30	20	0
a	0	10	30
b	10	0	20

קודקוד a מזהה שיש לו דרך טובה יותר ל-c אחרי העדכון (כלומר דרך b). c בשלב זה לא מודע לשינוי כי עוד לא התרחש העדכון. אחרי העדכון c יעדכן את ה-DV של a ושל b בטבלה שלו ויעדכן את מרחקי המסלולים המיטביים בהתאם.

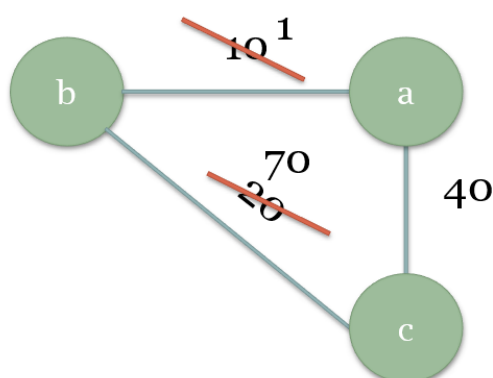
Node a	a	b	c
a	0	1	21
b	1	0	20
c	30	20	0

Node b	a	b	c
b	1	0	20
a	0	1	21
c	30	20	0

Node c	a	b	c
c	21	20	0
a	0	1	21
b	1	0	20

כאן a ו-b שולחים את וקטורי המרחקים שלהם לכל השאר ולכן מתעדכן אצל c. בהתאם, c יוכל לחשב מחדש את המרחקים (אצל a ו-b יצא שאין שינוי בשלב הזה כי העדכון של וקטורי המרחקים לא השפיע בדוגמה הזו). כשמתרחש שינוי **חיובי** (הורדה במשקל צלע) האלגוריתם מתכנס למצב תקין.

שינוי משקל שלילי – העלאת משקל צלע:



נמשיך עם אותן טבלאות ממקודם. מי ששמים לב לשינוי ראשונים הם b, c. מבחינת b הוא רואה שיש מסלול מ-a ל-c שעולה 21, לכן הוא יכול להעביר ל-a בעלות של 1 וסה"כ המסלול יעלה 22. מבחינת c המסלול ל-b יעלה 70 או 40 ועוד 1, לכן יעדיף 41 דרך a. במסלול ל-b הוא רואה 70 או 70 ועוד 1, לכן יבחר במסלול הישיר ל-a בעלות 40.

Node a	a	b	c
a	0	1	21
b	1	0	20
c	21	20	0

Node b	a	b	c
b	1	0	22
a	0	1	21
c	21	20	0

Node c	a	b	c
c	40	41	0
a	0	1	21
b	1	0	20

אחר כך יהיה עדכון של השכנים. נוצר כאן מצב ש-a לא מודע ללולאה, וממשיך בכל פעם לעדכן +1 את המרחק שלו ל-c. לכן בכל איטרציה b צריך לעדכן שהמחיר שלו ל-c עולה ב-1.

יש לנו 2 בעיות:

1. **Routing Loop**: יש לולאה – אם יש הודעה מ-b שמיועדת ל-c הוא יעביר אותה דרך a, ואז b יעביר אותה דרך a וכן הלאה. הודעות מ-b לא יגיעו ל-c.
2. **Count to Infinity**: העדכון של הטבלאות לוקח הרבה זמן (בהמשך נראה דוגמה שבה אפילו אינסוף).

לגבי בעיית count to infinity:

נפסיק רק כאשר הצלע מ-a ל-b ועוד המרחק מ-b ל-c הופל להיות יקר יותר מהמרחק הישיר a ל-c. בדוגמה שלנו $19=40-20-1$ איטרציות. 19 איטרציות נשמע יחסית מעט, אבל אם כל איטרציה אורכת שניה מדובר בהשהיה משמעותית בהגעת פקטות. נזכור שב-19 שניות של CPU אפשר לבצע כמות גדולה של פעולות שיתבזזו.

מה קורה לפקטות מ-c ל-b? c שולח את הפקטות דרך a ומשם ל-b, כאן לא תהיה בעיה.

פתרון אפשרי – Poisoned Reverse:

הבעיה נוצרה מזה ש-a העביר ל-b, ו-b העביר ל-a מבלי לדעת שיש ביניהם לולאה. הרעיון הוא שאם a צריך להעביר הודעה ל-c והוא עובר דרך b בשביל זה, אז כש-a ישלח את וקטור המרחקים שלו ל-b הוא יגיד שהמרחק שלו ל-c הוא אינסוף – כך יבטיח ש-b לא ינסה להחזיר את ההודעה דרכו.

הכללה: אם המסלול שלי אל קודקוד c עובר דרך קודקוד a, ו-a הוא התחנה הבאה במסלול (next-hop), אני אדווח ל-a שהמסלול שלי ליעד c הוא ∞ (שורה תחתונה – אל תיצור מסלול שעובר דרכי).

חזרה לדוגמה – עם יישום Poisoned Reverse:

נתחיל רגע לפני השינוי מ-20 ל-70. a יודע שכדי להגיע ל-c הוא צריך לעבור דרך b, לכן כשהוא מדווח על המרחקים שלו ל-b הוא יגיד לו שהמרחק שלו (של a) מ-c הוא ∞ . גם c עובר ל-a דרך b, אז כדי ש-b לא יבנה מסלול חזרה דרך a הוא ידווח לו שהמרחק שלו מ-a זה ∞ .

הערה: מומלץ להחזיק תמיד מה ה-next hop, כשנשלח את המרחקים שלנו למישהו שהוא ה-next hop נצטרך "לשקר" עם ∞ , אלא אם כן זה מסלול ישיר (למשל a ל-b ישירות ל-b, ה-next hop יהיה b אבל כאן נשלח את המרחק האמיתי).

אחרי העדכון של המשקל (בתוספת poisoned reverse) נקבל:

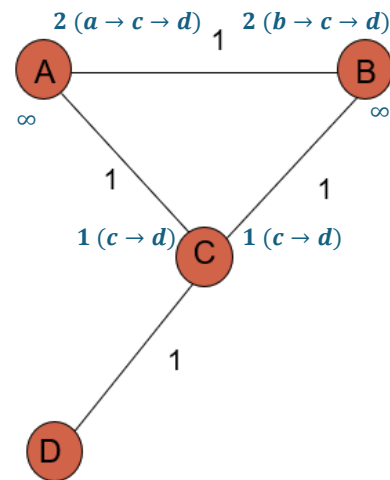
Node c	a	b	c
c	40	41	0
a	0	1	21
b	1	0	20

Node b	a	b	c
b	1	0	70
a	0	1	∞
c	∞	20	0

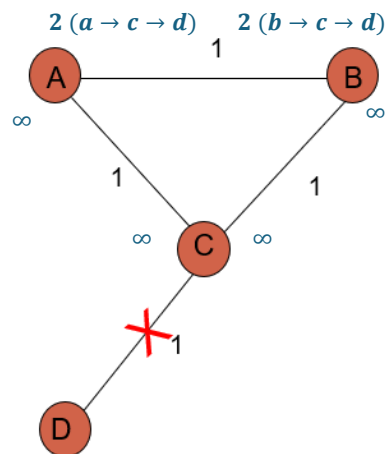
אצל a לא חל שינוי.

האם poisoned reverse תמיד פותר את הבעיה? **לא**, זה יכול להיכשל בלולאות גדולות מ-2. דוגמה:

לפני שנפל לינק, המצב הוא כזה:



בכחול סימנתי את המרחקים שכל אחד מדווח לשכנים שלו לגבי המרחק מקודקוד D (המספר שצמוד לקודקוד זה מה שהקודקוד הזה דיווח לשכן). עכשיו הלינק ל-D נפל, ו-C היחיד שמודע לשינוי לכן יעדכן:



עכשיו A רואה שהמסלול שהיה לו לא קיים, אז יבחר במסלול דרך B שמבחינתו עולה 2 ואז ידווח ל-C שהעלות היא 3, ול-B דרכו הוא עובר ידווח ∞ . נניח ש-C קיבל את העדכון, כלומר הוא חושב שיש מסלול ל-D שעובר דרך A בעלות סה"כ 4. הוא לא יספר את זה ל-A (ידווח לו ∞) אבל ל-B הוא ידווח שיש לו מסלול ל-D בעלות 4. כך מתגלגלים הלאה שוב ושוב.

פתרון אפשרי הוא להגביל את כמות התחנות שפקטה יכולה לעבור, אבל לא נכנס לנושא הזה מעבר.

Link State Protocol

זה הפרוטוקול שמשתמש בגישה הגלובלית ומבוסס על אלגוריתם דייקסטרה: לכל ראוטר יש את המידע על כל הרשת. גם כאן נחזיק **routing table**.

שלבי האלגוריתם

אתחול:

$$\text{Set my } D(v) = \begin{cases} c(s, v), & v \in \Gamma(s) \\ \infty & \text{else} \end{cases}$$

כלומר נגדיר את המרחק לשכנים שלנו לפי המרחק שהוגדר, ולכל קודקוד אחר שאין לנו מסלול ישיר אליו נגדיר ∞ . נשמור את ה-next-hop לכל קודקוד אחר.

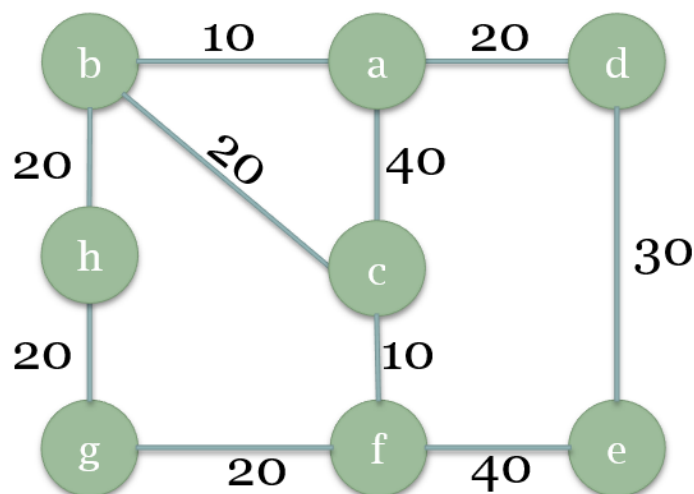
נשמור קבוצת קודקודים N' שאנחנו יודעים שהמסלול שמצאנו אליהם הוא הקצר ביותר (סיימנו לערוך אותם).
 $N' = \{s\}$ באתחול

לולאה:

כל עוד $N' \neq N$:

- נמצא $w \notin N'$ שמקיים שהמרחק אליו הוא המינימלי מבין אלו שלא טיפלנו בהם עדיין.
- נוסיף אותו ל- N' : $N' = \{w\} \cup N'$
- נעבור על כל השכנים של w . עבור שכן v אם המרחק שכתוב לי אליו גדול יותר מלהגיע אל w ואז מ- w ל- v , כלומר אם $D(v) > D(w) + c(w, v)$ אז נעדכן את המרחק ל- v בהתאם, ושה-next-hop ל- v הוא w .

דוגמת הרצה



נעבור על האלגוריתם מנקודת מבטו של הקודקוד c. השכנים הישירים שלו הם a,b,f. נעדכן את המרחקים אליהם לפי הצלעות, next-hop יהיו היעדים עצמם, ועבור כל השאר ∞ .

Step	N'	a	b	d	e	f	g	h
0	c	40,a	20,b	∞	∞	10,f	∞	∞

עבשיו נוסיף את הקודקוד שלא נמצא ב- N' והמרחק שלנו אליו הוא המינימלי: f. השכנים של f הם e,g אז נעדכן את המרחקים אליהם בהתאם (המרחק ל-f ועוד המרחק מ-f לשכן). בשני המקרים זה כמובן קטן יותר מ- ∞ ולכן העדכון יתבטא בכל השכנים:

Step	N'	a	b	d	e	f	g	h
0	c	40,a	20,b	∞	∞	10,f	∞	∞
1	c f	40,a	20,b	∞	50,f	10,f	30,f	∞

כעת מוסיפים את מי שהמרחק אליו הוא מינימלי ועוד לא ב- N' כלומר את b. מעדכנים את המרחק לשכניו:

Step	N'	a	b	d	e	f	g	h
0	c	40,a	20,b	∞	∞	10,f	∞	∞
1	c f	40,a	20,b	∞	50,f	10,f	30,f	∞
2	c f b	30,b	20,b	∞	50,f	10,f	30,f	40,b

ובן הלאה עד שמקבלים את הטבלה המלאה מנקודת המבט של c:

Step	N'	a	b	d	e	f	g	h
0	c	40,a	20,b	∞	∞	10,f	∞	∞
1	c f	40,a	20,b	∞	50,f	10,f	30,f	∞
2	c f b	30,b	20,b	∞	50,f	10,f	30,f	40,b
3	c f b a	30,b	20,b	50,b	50,f	10,f	30,f	40,b
4	c f b a g	30,b	20,b	50,b	50,f	10,f	30,f	40,b
5	c f b a g h	30,b	20,b	50,b	50,f	10,f	30,f	40,b
6	c f b a g h d	30,b	20,b	50,b	50,f	10,f	30,f	40,b
7	c f b a g h d e	30,b	20,b	50,b	50,f	10,f	30,f	40,b

גם פרוטוקול DV וגם פרוטוקול LS נמצאים בשימוש בחיים האמיתיים. Routing Information Protocol (RIP) עושה שימוש ב-DV, Open Shortest Paths First (OSPF) עושה שימוש ב-LS (לא נרחיב עליהם). DV יותר טוב לרשתות קטנות כי לוקח לו זמן להתכנס (ברשתות גדולות זה יכול להיות בעייתי). לרשתות גדולות נעדיף LS.

Interconnecting IP subnets

ראינו איך שני מחשבים שנמצאים ב-LAN שונים מתקשרים. היום נדבר איך הנתב יודע לאן הוא צריך להעביר את ההודעה.

שקף 3

אנחנו רוצים לחבר את הרשתות באמצעות נתב. שני מאפיינים שיש לכל רשת תקשורת זו הטופולוגיה של הרשת (מי הרכיבים ואיך הם מחוברים) וה-routing שהוא אופן התעבורה.

שקף 4-5

טופולוגיה

יש תכונות שיכולות להיות חשובות בנושא:

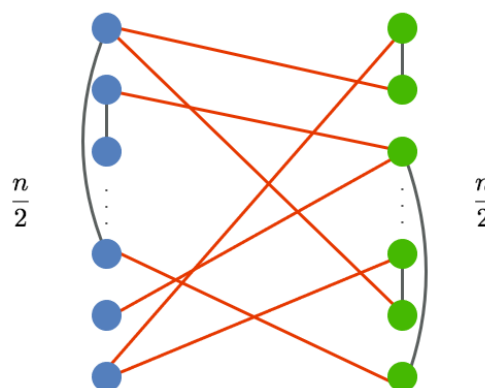
1. קוטר הגרף (המרחק המקסימלי של המסלול המינימלי בין זוגות הקודקודים).
2. דרגת הקודקודים.
3. Bisection Bandwidth – מסתכלים על כל החלוקות של הגרף לשני חצאים של קבוצת הקודקודים, ובוחרים את זו עם רוחב הפס המינימלי.

מה הופך טופולוגיה לטובה?

שקף 6-8

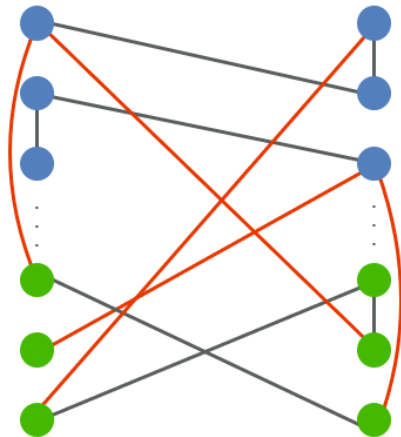
דוגמה

נניח שזו הרשת:



אנחנו מסתכלים על חלוקה לשתי קבוצות זהות. שתי קבוצות - קודקודים כחולים וקודקודים ירוקים. כל תעבור מהכחול לירוק צריכה לעבור דרך הקשתות האדומות.

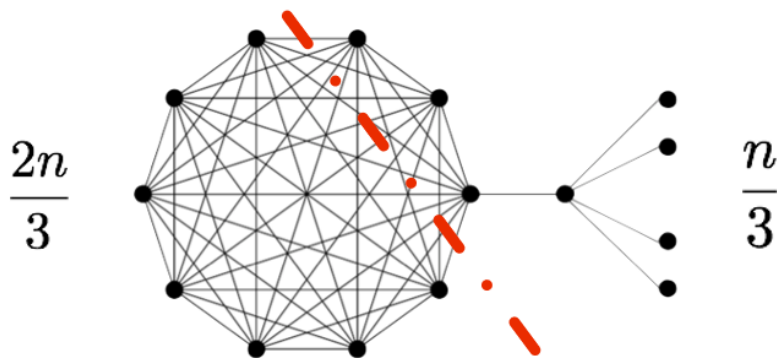
יכולנו להסתכל על חלוקה אחרת:



ואז היינו מקבלים קשתות אחרות בחתך. האם זו מטריקה אידיאלית? לא.

שקף 9

בהינתן דוגמה מורכבת יותר:



כאן $\frac{2}{3}$ מהקודקודים מחוברים כולם אלו לאלו, ובצד השני $\frac{1}{3}$ מהקודקודים מחוברים כמתואר. ביניהם יש לינק אחד. האינטואיציה אומרת שזו לא טופולוגיה טובה, כי חייב לעבור דרך הצלע במרכז, למרות שה- intersection bandwidth כל חתך שנסתכל עליו מספר הצלעות בחתך יהיה לא קטן, אז לא תמיד זה ישקף את המצב האידיאלי.

למה לא להסתכל על כל חלוקה ל-2 קבוצות? נניח שנחלק ל-1 ול- $n-1$, ברור שלא יהיו הרבה קשתות שעוברות בחתך אז צריך משהו שמתייחס לגודל הקבוצה. מכאן נגיע למושג expansion:

$$\min_{S \subset V, 0 < |S| \leq \frac{n}{2}} \frac{\text{EdgesBetween}(S, V \setminus S)}{|S|}$$

לוקחים את המינימום על פני כל החלוקות האפשריות. בדוגמה מקודם, במונה יהיה לנו 1 (כי יש רק קשת אחת בין הקבוצות) ובמכנה $\frac{n}{3}$. זה נקרא edge expansion של גרף.

בהסתברות גבוהה ה-edge expansion יהיה קרוב ל- $d/2$.

שקף 12

כשמדברים על טופולוגיה מבחינים בין בניות מפורשות לבין בניות לא מפורשות. לא מפורשות זה כמו שראינו עד עכשיו - קודקודים עם לינקים ואפשר ליצור טבלת שכנויות. בבנייה מפורשת לקודקודים יש שמות והחיבור הוא לפי השמות.

שקף 13-14

Linear Arrays and Rings

נתבונן בשתי דוגמאות:



בדוגמה הלינארית: קוטר 3, דרגה 1 או 2, ו-bw bisection הוא 1.

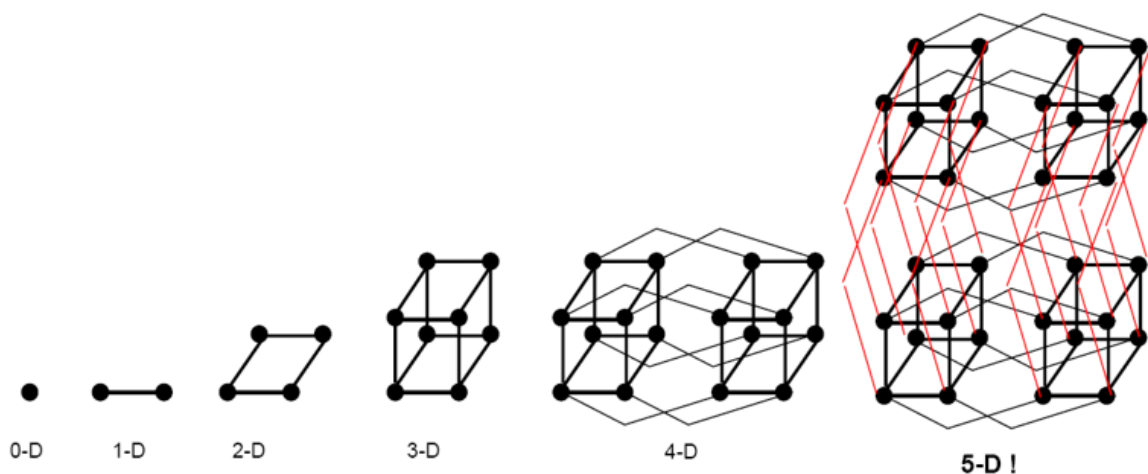
בדוגמה הטבעית: הקוטר הוא 2, דרגה 2, ו-bw bisection הוא 2.

אפשר גם בצורה מפורשת להגדיר איך הגרפים האלו בנויים.

שקף 15

Hypercubes

הכללה של הקובייה שאנחנו מכירים:



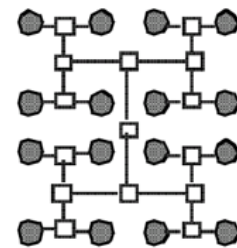
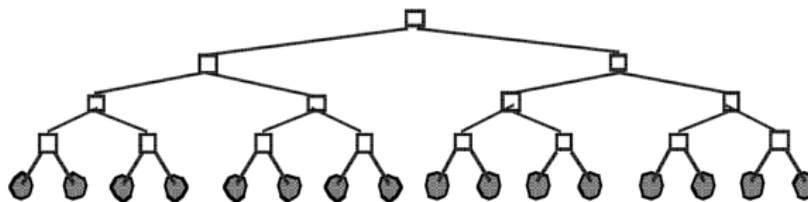
יש 2^n קודקודים לפי הממד. החיבורים ביניהם נקבעים לכל קודקוד את הייצוג הבינארי שלו ב- n ביטים. שני קודקודים מחוברים ביניהם אם המרחק בין הענפים הוא 1. למשל $(0,0,0)$ יהיה מחובר ל- $(0,0,1)$. כך מוגדר גרף.

קוטר הגרף הוא הממד n , כי בכל צעד שנעשה נשנה רק קואורדינטה אחת. דרגת הקודקודים גם היא n . ה-bisection bw יתקבל בעזרת חסם: נסתכל על חלוקה של כל הקודקודים ל-2 קבוצות, אלא שהקואורדינטה הראשונה שלהם היא 0 ואלה שהקואורדינטה הראשונה שלהם היא 1. לאלה שמתחילים ב-0, כמה לינקים יש לאלה שמתחילים ב-1? רק לעוד מישו אחד. הגודל של כל קבוצה זה 2^{n-1} ולכן זה מספר הקשתות שיש בחתך (כלומר שעוברות בין הקבוצות).

שקף 16

טופולוגיית עץ

הדרגה קבועה, הקוטר הוא $\log N$ ו-bisection bw הוא $O(1)$ כי אפשר לחלק בשורש. התעבורה תהיה לעלות עד האב הקדמון המשותף ואז ירידה חזרה ליעד.



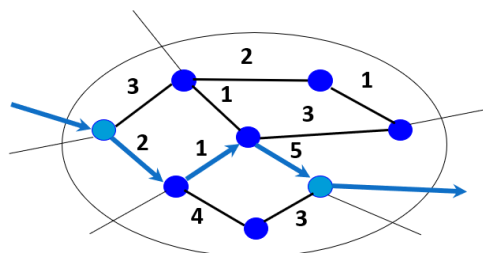
שקף 17-18

נתמקד כרגע ב-routing בתוך אותה רשת.

Forwarding

כשנתב מקבל פקטה הוא צריך להחליט מהר מאוד לאן הוא מנתב אותה, כלומר לאיזה interface להעביר. יש אלגוריתם ניתוב שהפלט שלו היא טבלת הניתוב. ניתוב עובד בדרך כלל לפי מסלולים בעל משקל מינימלי.

דוגמה:



הקודקודים הם נתבים והמספרים הם משקלים נתונים על הצלעות (נדבר בהמשך מאיפה הם מגיעים). אנחנו רוצים מנגנון שמקבל טופולוגיה ומקבל מסלולים קצרים ביותר. לכל יעד נחשב מי הנתב הבא ("next hop") שאליו נעביר את הפקטה, לא צריך ללמוד את המסלול כולו אלא רק מה הצעד הבא שלי.

Link-State Routing (LS) & Distance Vector Routing (DV)

שקף 20

יש שתי סכימות עיקריות לניתוב פנים-ארגוני.

Link-State Routing

כל נתב מספר לכל אחד מהאחרים את הטופולוגיה המקומית שלו (למי הוא מחובר, מה משקל כל קשת). כל נתב מקבל את המידע הזה וכך מקבל תמונה מלאה של הרשת באמצעותה הוא יוכל לחשב את המסלולים הכי קצרים (למשל באמצעות דייקסטרה).

Distance Vector Routing

כאן החישובים נעשים מכל קודקוד לכל קודקוד באמצעות בלמן-פורד.

שקף 21

ב-LS כל אחד יכול לחשב באופן עצמאי את המסלולים הקצרים ממנו לכל אחד מהיעדים (מחזיק מידע גלובלי על הרשת). ב-DV כל קודקוד יודע רק למי הוא מחובר ומה המשקל אליהם. את העדכונים הוא עושה על סמך מידע שהוא מקבל ומפיץ.

שקף 22-23

דרך פעילות אלגוריתם Link-State

כל נתב שולח לכל האחרים כך שהם יקבלו את התמונה של הרשת כולם, ואז יריץ דייקסטרה כדי לחשב את המרחק ממנו לקודקודים האחרים. תזכורת דייקסטרה:

אלגוריתם:

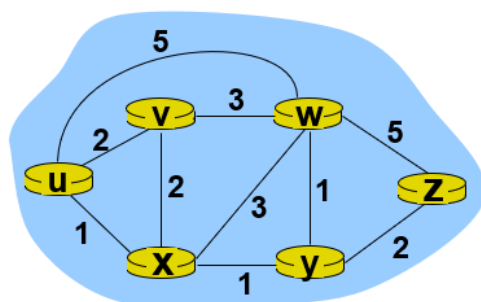
נוטציות:

```
1 Initialization:
2   $V' = \{u\}$ 
3  for all nodes  $v$ 
4    if  $v$  adjacent to  $u$ 
5      then  $D(v) = c(u,v)$ 
6    else  $D(v) = \infty$ 
7
8  Loop
9    find  $w$  not in  $V'$  such that  $D(w)$  is a minimum
10   add  $w$  to  $V'$ 
11   update  $D(v)$  for all  $v$  adjacent to  $w$  and not in  $V'$  :
12      $D(v) = \min( D(v), D(w) + c(w,v) )$ 
13   /* new cost to  $v$  is either old cost to  $v$  or known
14     shortest path cost to  $w$  plus cost from  $w$  to  $v$  */
15 until all nodes in  $V'$ 
```

Notation:

- $c(x,y)$: link cost from node x to y ;
= ∞ if not direct neighbors
- $D(v)$: current value of cost of path from source to dest. v
- $p(v)$: predecessor node along path from source to v
- V' : set of nodes whose least cost path definitively known

Step	V'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	{u}	2,u	5,u	1,u	∞	∞
1	{u,x}	2,u	4,x		2,x	∞
2	{u,x,y}	2,u	3,y			4,y
3	{u,x,y,v}		3,y			4,y
4	{u,x,y,v,w}					4,y
5	{u,x,y,v,w,z}					



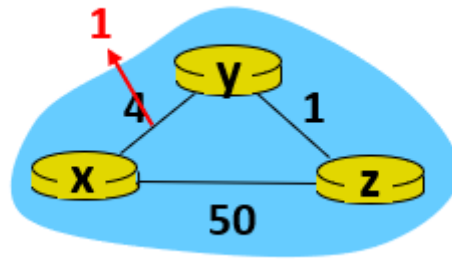
מה שצריך לדעת מאלגוריתם הניתוב - מכל יעד דרך איזה שכן אנחנו צריכים להגיע אליו. למשל להעברת הודעה מ-u ל-z: עבור z הקודקוד הכי קרוב הוא y, ל-y הכי קרוב x, ומ-x אפשר להגיע ישירות דרך u. החיסרון בשיטות האלה הוא שבכל פעם שיש שינויים ברשת אנחנו צריכים להריץ את האלגוריתם בכל פעם מחדש, אין באן אדפטיביות.

האלטרנטיבה – Distance Vector Routing. נגדיר $d_x(y)$ להיות המחיר של המסלול הזול ביותר מ-x ל-y. כל קודקוד מקבל את המרחקים מהשכנים שלו, ומעדכן את המרחק לכל אחד מהיעדים לפי:

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

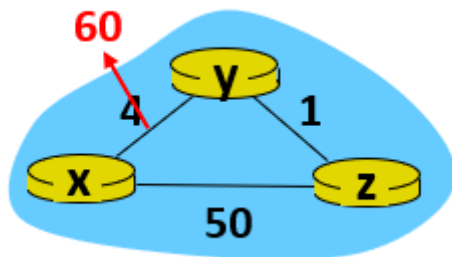
השכנים מעדכנים גם את הווקטור שלהם, כך בתהליך אסינכרוני (כל כמה זמן או בעת עדכון) עד להתכנסות, ומובטח לנו שזה יתכנס למינימום.

באופן כללי כשיש שינוי במשקל של קשת הנתב יעדכן את הטבלה שלו, ואם יש שינוי הוא יגיע לשכנים, שיעדכו את שכניהם וכן הלאה עד להתכנסות. בדוגמה נניח שהמשקל של הצלע בין x ל-y משתנה מ-4 ל-1. y מזהה שהיה שינוי ולכן מעדכן את המרחק שלו ל-x. בשלב הבא המרחק יתעדכן גם אצל z. לתכונה הזו קוראים "good news travel fast" - כי בשמשקל של צלע ירד העדכון מגיע מהר.



שקף 32

מה יקרה כשנגדיל את המשקל ל-60? y יזהה שהיה שינוי, וצריך לחשב את המרחק החדש שלו ל- x . לפי האלגוריתם שראינו הוא יסתכל על הווקטורים שיש לו מהשכנים ואז יבדיל בין 2 אפשרויות: ללכת ישירות ל- x או דרך z . המרחק בין z ל- x כרגע רשום אצלו כ-5 (1+4). לכן האלגוריתם יודע שיש מסלול באורך 5 מ- z ל- x . כעת משקל הקשת עלה, אז מבחינת y יש מסלול באורך 5 בין z ל- x (ילך במחיר 1 ל- z ומשם במחיר 5 ל- x), אבל לא נלקח בחשבון שנוצר פה מעגל ניתוב. כך שוב ושוב המרחק יתעדכן בטבלה ויעלה ב-1 בכל פעם עד שבשלב מסוים המסלול יעלה 51 עד ש- z יעדיף את המסלול הישיר שלו ל- x ונקבל התכנסות: כולם ידעו מה המרחקים הנכונים.



הבעיה: זה לוקח המון איטרציות, הבעיה נקראת count to infinity (המספרים היו יכולים להיות גדולים מאוד והיה לוקח המון זמן להתכנס).

שקף 42

הבעיה נבעה מזה ש- y חשב שהוא יכול להגיע ל- x דרך z , ו- z חשב שהוא יכול להגיע ל- x דרך y . איך מתמודדים? Poisoned reverse: כשאני מעדכן את המרחק, אגיד שהמרחק דרכי הוא אינסוף ואז לעולם לא יעדיפו לקחת את המסלול הקודם.

האם זה יפתור את הבעיה לחלוטין? לא, מה יקרה בלולאות ארוכות יותר?

Criteria	LS	DV
Message Complexity	With n nodes, E links, $O(nE)$ messages sent. Each node sends a broadcast message, flooded to all links.	Exchange between neighbors only. Convergence time varies.
Speed of Convergence	$O(n^2)$ runtime algorithm. May have oscillations.	Varies. May be routing loops, count to infinity problem.
Robustness <i>What happens if router malfunctions?</i>	Node can advertise incorrect link cost. Each node computes only its own table.	Node can advertise incorrect path cost. Each node's table used by others.

הזכרנו מספר פרוטוקולים שממומשים במציאות כמו למשל (Routing Information Protocol) **RIP**. פירטנו על (Open Shortest Path First) **OSPF**, פרוטוקול בו משתמשים בפקטות שמפיצים בהן מידע על טופולוגיה, ומבצעים שימוש בדייקסטר למציאת מסלולים קצרים ביותר - כל ההודעות נשלחות מעל שבבת IP.

The Story of ARPAnet Routing

עד עכשיו היו לנו משקלים ומהם נקבע הניתוב. מאיפה מגיעים המשקלים?

נתחיל מלדבר על ARPAnet, בניתוב המקורי העדכון של המשקלים נעשה בזמן מת לפי העומס. הסתכלנו על מספר הפקטות בתור (queue) של נתב ולפי זה קבעו את העומס על הקשת. ככל שהעומס היה גדול יותר, נתנו משקל גבוה יותר.

לסכימה הזו כמה בעיות - היא לא יציבה ומשנה לעתים קרובות את המסלולים, כל פעם שמסלול אחד עמוס עוברים למסלול השני, ואז שוב בחזרה וכן הלאה. בסוף נוכל לעבור למסלולים שיש בהם יותר קשתות ואז נבזבז capacity של הרשת וגם מעמיסים על הנתבים בדרך. זה לא בדיוק פותר את הבעיה שניסינו לפתור, אידיאלית היינו רוצים מראש לחלק את התעבורה בין המסלולים.

בהמשך ביצעו תיקונים - במקום להסתכל על העומס בנקודת זמן ואז להגדיר משקל, הסתכלו על עומס לאורך זמן.

שקף 53-55

איך נגרום לתעבורה להיות מנותבת בדרך שאנחנו בוחרים?

היום בעיות ניתוב מטופלות בנפרד - איך לקבוע משקלים של קשתות מתבצע בקבועי זמן יחסית ארוכים (לא בזמן אמת). כל כמה זמן מנהל הרשת קובע את המשקלים. בשכבה 4 יש מנגנונים שנדבר עליהם כמו congestion control שבזמן אמת קובעים את קצב השליחה של הפקטות.

אלגוריתמי הניתוב לא מודעים לעומס בזמן אמת, אלא נותנים משקלים סטטיים. מצד שני בשכבה 4 אלגוריתמים המטפלים בעומסים לא יכולים להשפיע על המסלולים באופן ישיר (מבחינתם - המסלולים סטטיים), הם עובדים על המסלולים שהוחלט עליהם.

שקף 56-57

Measure, Model and Control

מנהל הרשת מסתכל על מדידות של טופולוגיה ותעבורה ברשת (יש קודקודים ששולחים יותר, שולחים פחות, כתלות בזמן). משתמשים במודל "what if" - מה יקרה אם נשנה את הקונפיגורציה? אנחנו רוצים למצוא קונפיגורציה שתשפר את הביצועים לעומת המצב הנוכחי. אם מצאנו כזו - נעדכן.

מסתמכים על מדידות של טופולוגיה ותעבורה ברשת, יש מודל ואלגוריתם שאומר איך לבחור את המשקלים.

שקף 58-59

Theory: Flow Optimization

אלגוריתם Max-Flow

יש לנו גרף ולכל קשת יש קיבולת. אנחנו רוצים למצוא את הזרימה המקסימלית בין s ל- t תחת אילוצים כמו לא לחרוג מקיבולת הצלעות ושימור הזרימה. ראינו באלגוריתמים משפט שאומר $\text{max-flow} = \text{min-cut}$.

שקף 60

Maximum-Multicommodity-Flow

ברשתות הבעיה שאנחנו מסתכלים עליה היא רחבה יותר - אין לנו קודקוד מקור אחד וקודקוד יעד אחד, כולם פוטנציאלית שולחים לכולם. לשם כך נציג את Multicommodity Flow. הקלט הוא גרף לא מכון ומטריצת ביקושים שבה הכניסה ה- i, j היא כמה הקודקוד ה- i רוצה לשלוח לקודקוד ה- j .

שקף 61

יש כמה מטרות שאנחנו עשויים לרצות לאפטם:

1. **Maximum-Multicommodity-Flow**: הכללה טבעית של max-flow שמטרתה למקסם את סך

התעבורה שנשלחת ברשת תחת המגבלות הטבעיות של שימור זרימה ואי חריגה מקיבול מקסימלי.

לבעיה הזו אין מקבילה נקייה כמו "max-flow=min-cut".

2. **Minimize Congestion**: למזער את העומס המקסימלי ברשת - כלומר את העומס על הקשת הכי עמוסה. לכל קשת נגדיר את העומס לפי השטף שעובר בה חלקי הקיבולת. נרצה לאזן את העומסים כך שלא תהיה אחת עמוסה במיוחד. בניגוד לבעיה הקודמת, כאן שולחים את כל הביקושים ואפשר לחרוג מה-capacity. ההיגיון הוא שאם נחרוג מה-capacity זה אומר שחסר לנו capacity ברשת ואז אנחנו עוברים לבעיה אחרת.

3. **נוספים**: יש פונקציות אחרות שנותנות fairness מסוים (לא יהיה אחד ששולח המון ואחד שלא שולח בכלל).

כל מנהל רשת מחליט מה הוא רוצה לאפטם בהתאם לצורכי הרשת.

שקף 62

Multicommodity Flow Meets the Internet

ב-Multicommodity Flow כשאנחנו מחשבים את הזרימה (או פתרון לבעיה), בדומה ל-max flow מחשבים באילו מסלולים אנחנו שולחים, ואיך נחלק את התעבורה בין המסלולים. השאלה היא האם זה עובד ככה גם כשמתעסקים בניתוב ברמת ה-IP, אבל קודם לכן – איך מנהל הרשת אמור להחליט מה המשקלים?

שקף 63-64

החלטה על משקלים בזמן אמת

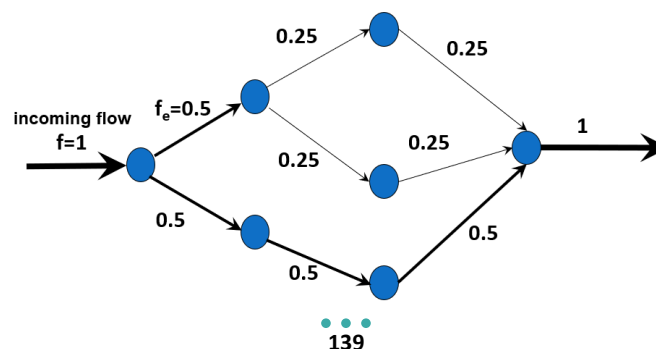
ראינו שיש פונקציות שמנהל הרשת רוצה לאפטם. איך בוחרים משקלים? אחרי שהוא בחר אותם, אלגוריתם הניתוב קובע איך התעבורה תעבור בהתאם למשקלים שנבחרו. מקודם ראינו שלשנות את המשקלים בזמן אמת זה לא רעיון כל כך טוב (וגם היוריסטיקות לא בהכרח אופטימליות).

שקף 67

ECMP

עוד מנגנון שמשתמשים בו הוא ECMP (Equal Cost Multipath). יכול להיות שיש יותר ממסלול אחד הכי קצר. ECMP מפצל בין שכנים שדרכם עובר המסלול הכי קצר (חצי מהתעבורה תעבור במסלול אחד וחצי במסלול השני).

דוגמה - נניח שכל המסלולים במשקל 1. התעבורה לא תפוצל לשלישים. כשהקודקוד הראשון יקבל את התעבורה יש שני שכנים שדרכם הוא יכול להעביר, אבל בהמשך אנחנו רואים שההסתברות לשליחה תלויה בכמות הפיצולים שיש לקודקודים בהמשך. חלק יישארו עם הסתברות $\frac{1}{2}$ אבל אחרים עם $\frac{1}{4}$:



יש כאן בעיית אופטימיזציה - מקבלים גרף ומטריצת ביקושים, ופונקציה שרוצה למשל למקסם את התעבורה או למזער את העומס על הצלע העמוסה ביותר. אנחנו רוצים למצוא משקלים כך שאם נשתמש ב-ECMP נקבל פתרון אופטימלי.

האם זה אפשרי תחת ההגבלות של ECMP וניתוב שתלוי במשקלים להגיע לפתרון אופטימלי?

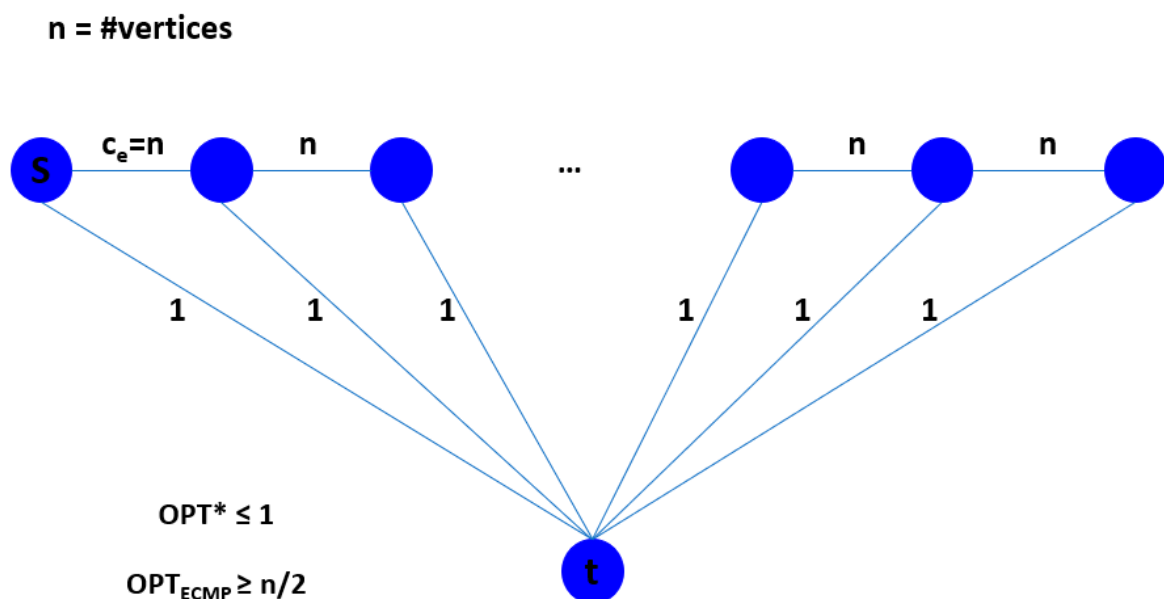
מוגבלים בשני דברים:

1. אנחנו צריכים לקבוע משקלים, והניתוב נעשה רק על מסלולים קצרים ביותר (וצריך למשקל הכל).
2. ECMP מפצל בין השכנים שדרכם עובר המסלול הכי קצר באופן שווה (כאשר בפתרון אופטימלי אולי היה צורך לפצל בצורה אחרת).

אם אי אפשר לקרב, האם אפשר לקבל קירוב?

72 שיקר

כאן נראה דוגמה שמראה שזה לא אפשרי לקבל את הפתרון האופטימלי:



נניח שאנחנו רוצים למזער את העומס על הצלע הכי עמסה. כרגע בגרף רק s רוצה לשלוח ל- t גודל של n יחידות כאשר ה- $capacity$ ביניהם הוא 1, ושל כל קשת למעלה היא n . מה הפתרון? לשלוח יחידה אחת ל- t , להעביר $n-1$ ימינה. הקודקוד הבא ישלח 1 ל- t ויעביר $n-2$ ימינה.. כך הלאה עד שהעברנו את כל n היחידות. כך נגיע לעומס של 1.

עכשיו נניח שאנחנו רוצים פתרון שמבוסס על משקלים ב-ECMP. יש כמה אפשרויות:

1. נניח ש- s רוצה שוב לשלוח תעבורה, אם המשקל של המסלול ל- t יותר קטן מהמרחק דרך הקודקוד שמימין ל- s הוא ישלח הכל בקשת הישירה ל- t . במקרה זה שלחנו n יחידות ל- t על קשת עם קיבולת 1, לכן העומס הוא n .

2. יש לנו מסלולים נוספים באותו אורך. במקרה הזה s יפצל את התעבורה בין שני הקודקודים באופן שווה, כלומר $n/2$ ישירות ל- t , ועוד $n/2$ לקודקוד שמימינו ומשם ל- t .

3. המסלול דרך הנתב שנמצא מימין הוא קצר יותר מאשר המסלול הישיר ל- t , ואז נעביר את כל n היחידות לנתב מימין. גם כאן נשארו עם אותו עומס כמו אפשרות 1.

לא משנה מה יהיו המשקלים, העומס המקסימלי לפי ECMP יהיה לכל הפחות $n/2$. לא רק שלא הגענו לאופטימום, גם לא קירבנו.

שקף 73-74

נשאלת השאלה האם אנחנו יכולים בכל זאת למצוא משקלים שיביאו אותי לתוצאה הכי טובה שאפשר בהינתן זה שאנחנו מוגבלים ל-ECMP, משקלים ומסלולים קצרים ביותר. גם כאן, מדובר בבעיה NP-קשה, וגם קירוב לה הוא NP-קשה.

שקף 75

אם כך מה עושים? משתמשים בהיוריסטיקות לא אופטימליות אבל מספיק טובות ברשתות אמיתיות.

בשיעור הבא נדבר על איך ECMP מפצל באופן שווה את המידע שעובר בין השכנים שמהם יש מסלול קצר ביותר.

הקדמה – Traffic Engineering

אנחנו נדבר על מקרה קצה ספציפי שבו יש כמה דרכים שוות להגיע. איך נרצה לפעול? איך המערכת תראה? בתרגול הקודם דיברנו על המרחקים שהיו נתונים לנו, לא הרשת בחרה אותם. בתרגול הזה אנחנו **נקבע** את המרחקים, ונחליף את המרחק כתלות בבעיה שנרצה לאפטם (מינימום/מקסימום) תחת מגבלות כמו קיבולת. בתרגול הבא כשנדבר על TCP נראה מה ההשלכות של העברת הרבה מידע על לינק בלי קיבולת מספיקה. בתרגול הזה אפשר לחשוב על המושג traffic כתעבורה בכבישים – יש את הדרך הכי מהירה להגיע ליעד, אבל לא תמיד נרצה שכולם יעברו בה כדי למנוע עומסים.

Modeling Network as a Linear Problem

Intro to Optimizations & Linear Programming

כדי לאפטם את הבעיה, נתמקד בבעיות לינאריות. נרצה למצוא את המקסימום/מינימום, ואם נצליח למדל את הבעיה שלנו כלינארית – בתקווה נוכל לפתור בזמן פולינומי.

תכנון לינארי הוא הבאת ביטוי לאופטימום תחת אילוצים, למשל:

$$\begin{aligned} \min_{x_1, x_2, x_3} \quad & 7x_1 + x_2 + 5x_3 \\ \text{subject to} \quad & x_1 - x_2 + 3x_3 \geq 10 \\ & 5x_1 + 2x_2 - x_3 \geq 6 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

בנוסחה הכללית:

For a given set of coefficients $\{c_j\}_{j=1}^n$ and constraints $\{a_{ij}\}_{i,j=1}^{m,n}, \{b_i\}_{i=1}^m$ (where m signifies the number of inequalities we optimize)

$$\begin{aligned} \max_x \quad & \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_j \quad i = 1, \dots, m \\ & x_j \geq 0 \quad j = 1, \dots, n \end{aligned}$$

האילוץ האחרון בדרך כלל יופיע כשאנחנו מדברים על דברים במציאות כמו למשל כמות של אוכל שצריך לתת, ולא נרצה לתת "1000- קלוריות" כדי לאפטם את הביטוי, זה חסר משמעות.

הבעיה הדואלית

לכל בעיית LP יש ייצוג נוסף – זו הבעיה הדואלית והיא נותנת חסם עליון לבעיית ה-LP המקורית. למשל במקום לדבר על המקסימום של $\sum_{j=1}^n c_j x_j$ נדבר על המינימום של $\sum_{j=1}^n b_j y_j$.

Max Flow

בעיה שבה יש לנו מקור, יעד ועל כל צלע קיבול מקסימלי. הבעיה היא לנתב את התעבורה מהמקור כך שיגיע כמה שיותר ליעד.

אילוצים:

- שימור חומר: אם לקודקוד מסוים נכנס מידע בהיקף א, זה גם מה שיוצא.
- קיבולת: כל צלע לא תעביר מידע מעבר לקיבולת שלה.
- הזרם תמיד אי-שלילי.

Multi-Commodity Flow

ברשת עליה אנחנו נדבר הבעיה מורכבת יותר ממקור אחד ויעד אחד. אנחנו נעשה מיקסום או מזעור על כמה קודקודים – כל קודקוד יכול להיחשב כמקור או כיעד.

נשמור אילוצים בתוך Demand Matrix – כמה זרימה נרצה להזרים בין כל שני קודקודים. בעזרת זה אפשר לבנות את האילוצים של משוואה לינארית: לכל קודקוד (לא כולל המקור והיעד) מה שנכנס הוא מה שיוצא.

האילוצים:

$$\forall i, \forall v \in V \setminus \{s_i, t_i\} : \sum_{u:(u,v) \in E} f_i(u, v) = \sum_{w:(v,w) \in E} f_i(v, w)$$

$$\forall (u, v) \in E : \sum_{u:(u,v) \in E} f_i(u, v)$$

$$\forall i, \forall (u, v) \in E : f_i(u, v) \geq 0$$

נשים לב שהזרם חייב להיות אי-שלילי, וחל שימור חומר לכל הקודקודים חוץ מהמקור והיעד.

אפשר להזרים הרבה מידע, אבל אפשר גם לפצל מסלולים כך שלא יהיה עומס על אותו מסלול. אבל מה המטרה לאופטימום?

מטרות:

1. Maximum-Multicommodity Flow: בקצרה Max-MCF. יש לנו את מטריצת האילוצים, אם אנחנו מצליחים להעביר הכל – מעולה. אחרת, נרצה להעביר כמה שיותר, למקסם את סך התעבורה. כאן הקיבולים על הצלעות הן אילוץ שלא נחרוג ממנו, אבל ה-Demand Matrix הוא לא אילוץ, רק נשאף למקסם אותו.

2. Minimize Congestion: בקצרה MinCong-MCF. נעביר את כל הדאטה שמופיע במטריצת האילוצים ועל זה לא נתפשר, אבל כן נוכל להתפשר על הקיבול (נחרוג ממנו, אולי יהיה איבוד מידע אבל נספוג את זה). נרצה להביא למינימום את העומס על הצלע עם העומס המקסימלי. עומס מוגדר להיות $\frac{f}{c}$ כלומר

$$\max_e \frac{f_e}{c_e} \text{ כמות הזרימה דרך הצלע חלקי הקיבול שלה, סה"כ נרצה להביא למינימום את:}$$

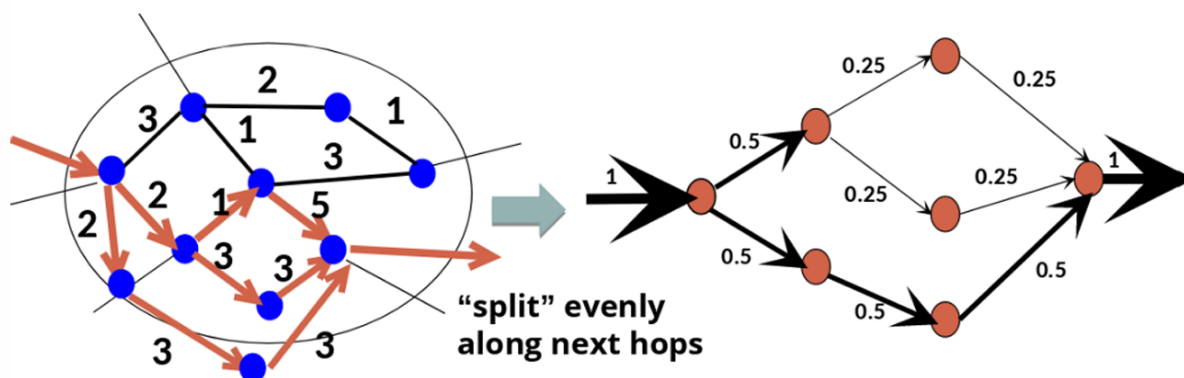
3. תנאים נוספים: למשל הוגנות, לא נתמקד בדברים כאלה בקורס.

עד עכשיו דיברנו על מצב שבו בונים רשת ומחפשים איזה מסלול הכי טוב להעביר בו את הדאטה. איך routing עובד במקרים שבהם נפצל את המידע? זה עומד לסייע לנו לאפטם את האופן שבו אנחנו מעבירים את המידע. עד עכשיו לא סיפרנו איך קובעים את המרחקים (המשקולות בצלעות), עכשיו נתמקד בזה.

המטרה: לנתב את התקשורת בצורה שתאפטם את אחת השיטות (Max-MCF \ MinCong-MCF).

Equal Cost Multi-Path (ECMP)

בנוסף לשיטות שראינו קודם של DV/LS, נדבר על שיטת ECMP. כאן אם יש שתי דרכים להגיע ליעד ושתיהן באותו מחיר, נפצל את התקשורת ל-2 ונעביר אותה בשתי הדרכים:



נשים לב איך בכל פעם שיש פיצול במסלול גם השטף מתפצל ל-2, ובסוף השטף הכולל נשאר 1 (שימור חומר). בעזרת ECMP אנחנו נחליט כמנהלי הרשת איזה מרחקים לשים על הקשתות כדי שיבוצעו הפיצולים שמתאימים לנו. את הפיצולים נרצה לבצע לפי קיבולת או demand matrix.

Optimizing ECMP

אנחנו מחליטים מה יהיו המרחקים ואז ECMP עובד לפי הרעיון שראינו, על מנת לאפסם מטרם מסוימת.

[שאלה] נניח שהחלטנו איך הזרימה תראה, האם אנחנו תמיד יכולים לשים משקלים שיביאו את ECMP לאופטימום?

[תשובה] לא.

[שאלה] האם אפשר לפחות לקרב?

[תשובה] לא. נראה את המקרה שגם עבדנו עליו בהרצאה 9.

למצוא את המשקלים האופטימליים תחת אילוצי ECMP זו בעיה NP-קשה. גם למצוא קירוב למשקלים זו בעיה NP-קשה. הפתרון – שימוש בהיוריסטיקות. אם אין פתרון שיכול להוכיח שהפתרון הוא אופטימלי, אפשר לבצע הקלות שמבוססות על הנחות. אנחנו נתמקד בהיוריסטיקות של ECMP.

Understanding ECMP

מה זה אומר שאנחנו מפצלים מסלולים? איך נדע איפה לפצל? דרך אחת היא להעביר את המידע בכל המסלולים שמצאנו כשווי מחיר בשיטת "Round-Robin" (RR), כלומר בכל פעם לפי סדר נבחר במסלול אחר, כשבחרנו במסלול הוא ירד לסוף התור ונבחר בו שוב רק אחרי שבחרנו בכל האחרים. אנחנו לא נפצל **פקטות** לחצי אלא את השטף.

שיטה נוספת – נניח שיש 3 קודקודים שמחוברים ל-A: x, y, z . על A הוטלה המשימה להעביר מהם את המידע לקודקוד C. דרך לפצל את המידע היא לחלק את המסלולים לפי המקור: כל מה שנשלח דרך x יעבור במסלול אחד, מה שנשלח דרך y במסלול אחר, וכן הלאה.

[בעיה] כשמפצלים את המידע בכמה מסלולים, איך נוודא שהפקטות מגיעות ליעד בסדר הנכון?

[פתרון] לפעמים אפשר לסבול את זה, ולפעמים לא. אפשר להגיד שה-flow מתפצל וכל מה שמאותו מקור נשמר באותו מסלול, ומניחים שיש הרבה זרימה בדרך, אז גם אם לא כל הזרימה מתחלקת שווה בשווה לפחות פיצלנו עומסים. עבשיו נרחיב.

ECMP Hashing

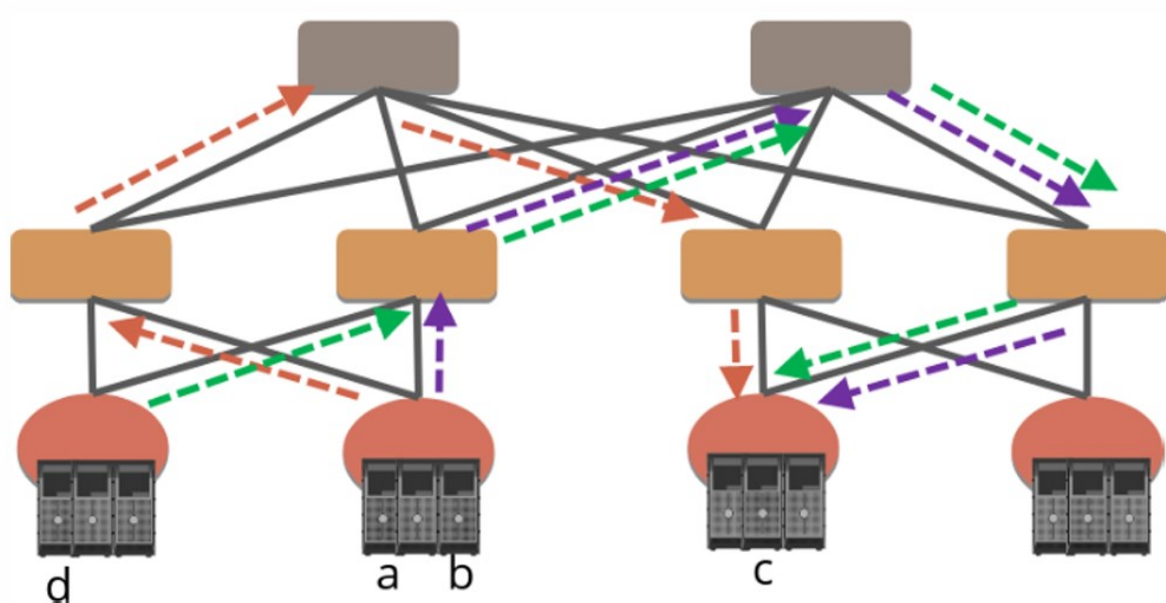
ניקח את הנתונים הבאים:

1. Source IP Address
2. Source IP Port Number
3. Destination IP Address
4. Destination IP Port Number
5. Protocol in Use

את כל החמישייה הזו נמפה באמצעות פונקציית hash שתפזר באופן יחסית שווה את ה-flow בין המסלולים.

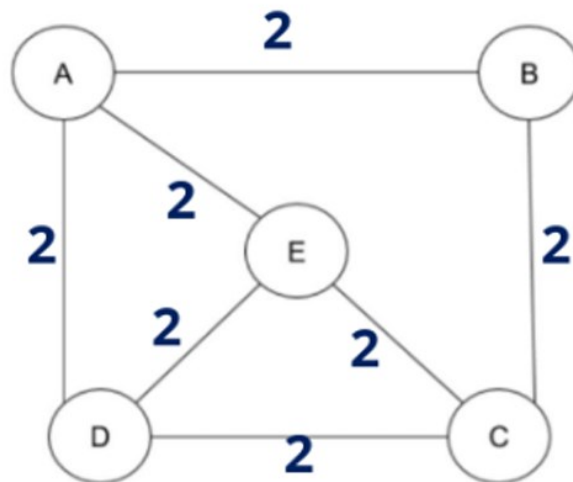
איפה יכולה להיווצר בעיה בכל זאת?

- All link weights are 1
 - Demands: $a \rightarrow c$ = green (short) , $b \rightarrow c$ = purple , $d \rightarrow c$ = green (long)



a,b,d רוצים לשלוח ל-c. נשים לב שהמידע מ-a ומ-b מפוצל דרך מסלולים שונים. הבעיה היא שאם a מעביר ל-c למשל מידע קל (הודעת טקסט למשל) בעוד ש-b ו-d מעבירים סרטון או שיחת Zoom – יצא במקרה ששני הזרמים הכבדים יותר של מידע נפלו על אותו המסלול. ECMP Hashing לא יודע מה כמות הזרימה ולכן לא יודע לטפל במקרים כאלה.

פתרון אפשרי הוא להוסיף ל-ECMP Hashing גם את עומס השטף, אבל לא נכנס לזה.



הקיבולת על כל צלע היא 2 (לפעמים זה אילוך ולפעמים לא, תלוי בבעיית האופטימיזציה). המשקלים לצורך OSPF (שמשתמש ב-LS) יהיו חיוביים/אינסוף. הזרימה יכולה להיות בכל כיוון, ואם יש זרימה בשני כיוונים (למשל a ל-b וגם b ל-a) אז הזרימה הכוללת תהיה סכום הזרימות בשני הכיוונים.

נוטציות בעיות מקסימום

$\text{Max-MCF}_{\text{OSPF/ECMP}}$ – מיקסום השטף הכולל תחת מגבלת קיבול הצלעות. אנחנו נשלח את הקיבול של הצלעות אותם נשלח לאלגוריתם ECMP שהוא יעשה את החלוקה השווה שהצגנו מקודם.

$\text{Max-MCF}_{\text{OPT}}$ – אותו דבר כמו מקודם, אך הפעם לא נקבע ל-ECMP לחלק בחצי אלא שיחלק לפי איך שהוא רואה לנכון (עדיין יבחר את ההכי קצר, אבל כשיש שוויון יחלק בדרך שיבחר).

נוטציות בעיות מינימום

$\text{MinCong-MCF}_{\text{OSPF/ECMP}}$ – כאן ה-demand matrix הוא אילוך שלא נחרג ממנו, אבל אפשר לחרוג מהקיבול על הצלעות, תוך כדי שנאפטם שהחריגה תהיה מינימלית, כלומר נביא למינימום את החריגה על הצלע שחרגה הכי הרבה. כאן נכריח חלוקה שווה בין המסלולים.

$\text{MinCong-MCF}_{\text{OPT}}$ – אותו דבר כמו מקודם, אך הפעם אפשר לחלק את הזרימה בין מסלולים קצרים ביותר איך שנרצה.

שאלה 1

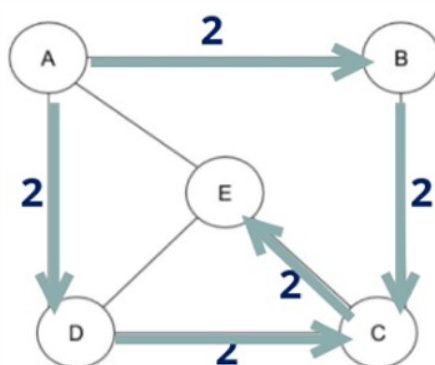
עם הגרף מקודם, ידוע לנו שמטריצת האילוצים מכילה:

$$\text{Demand-Matrix} = \begin{bmatrix} & 5 \\ & 3 \end{bmatrix}$$

השורות והעמודות מסומנות לפי סדר A-E, ונתון לנו $(A,C, 5)$, $(C,E, 3)$.

[סעיף א'] היעד לאיפטום הוא $\text{Max-MCF}_{\text{OPT}}$, כלומר flow מקסימלי בלי לחרוג מהקיבול.

[פתרון]



השטף המקסימלי ברשת הזו הוא 6. A שולח 4 יחידות, C שולח 2 יחידות.

[סעיף ב'] הוכיחו שהפתרון שקיבלתם הוא אופטימלי.

[הוכחה] נניח בשלילה שיש פתרון יותר טוב $s' > 6$. נבחין כי כל השטף חייב לעבור דרך C, כלומר דרך C

עובר יותר מ-6 יחידות, בסתירה לאילוץ של קיבול הצלעות היוצאות מ-C. סתירה.

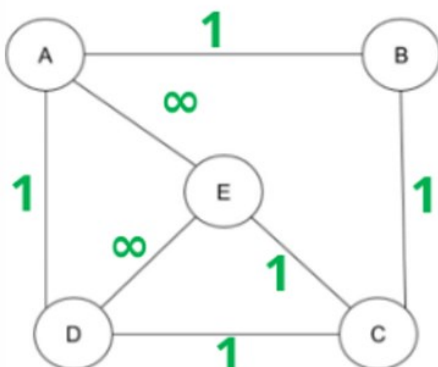
שאלה 2

אזתה רשת כמו קודם ועם אזתה קיבולת, אבל המטרה היא הפעם $\text{Max-MCF}_{\text{OSPF/ECMP}}$.

[סעיף א'] האם יש השמה ריאלית למשקלי הצלעות תחת המטרה שנקבעה? כלומר, איך נבחר משקלים

לצלעות שתביא לנו את השטף האופטימלי שראינו בשאלה הקודמת?

[פתרון]



הערת הבהרה: המספרים בירוק אלו **המרחקים/המשקולים** שבחרנו, לא הקיבול. על הקיבול אין לנו דרך להשפיע, השאלה היא במנהלי הרשת אילו מרחקים נגדיר על הצלעות כדי להוביל לשטף שנרצה תחת האילוצים שבחרנו.

הדרך הכי קלה היא לבחור 1 לצלע שאנחנו רוצים להשתמש בה ו- ∞ לצלע שלא נרצה להשתמש בה. כשהרשת תעבוד לפי LS היא תבחר בהכרח בצלעות עם המשקל הנמוך יותר, ורק שם יעבור השטף. בשאלה הזו ראינו שאם אנחנו רוצים לאפטם $\text{Max-MCF}_{\text{OSPF/ECMP}}$ ואנחנו רוצים להשתמש באלגוריתם ECMP אז נגיע לפתרון האופטימלי (זה לא תמיד אפשרי).

שאלה 3

אותה רשת, אבל הפעם האילוצים הם $(B,D, 5)$, $(C,E, 3)$ – במקום האילוצים הקודמים.

[שאלה] מה הפתרון האופטימלי עבור $\text{MinCong-MCF}_{\text{OPT}}$? כלומר מותר לחרוג מהקיבול, אבל נרצה שהחריגה המקסימלית תהיה קטנה ככל האפשר.

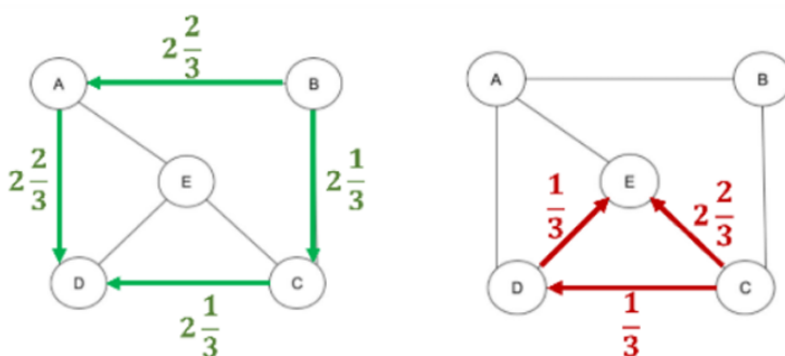
[פתרון] כאן קצת פחות טריוויאלי לראות את זה, השיטה היא להסתכל על חתך. דרך החתך של BC יוצא דאטה כדי להגיע לחתך השמאלי שמכיל את כל השאר. סך הדרישה שלנו היא $8=5+3$ שצריך לצאת מהחתך, ולחתך יש 3 יציאות כל אחת בקיבול 2 כלומר סה"כ 6. מכאן אנחנו מבינים שהעומס המקסימלי הנמוך ביותר יהיה $\frac{8}{6} = \frac{4}{3}$. זה עומס שחורג מהקיבול, אבל אנחנו מרשים את זה במטרה שבחרנו.

על כן, ה-flow המקסימלי על כל צלע:

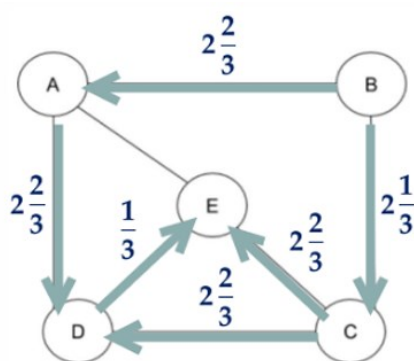
$$\frac{f}{c} = \frac{4}{3} \stackrel{c=2}{\Rightarrow} \frac{f}{2} = \frac{4}{3} \Rightarrow f = 2\frac{2}{3}$$

וממנו לא נחרוג.

בירוק נסמן את האילוץ $(B,D, 5)$ ובאדום את האילוץ $(C,E, 3)$, פתרון אפשרי:



ולכן סה"כ הזרימה ברשת:

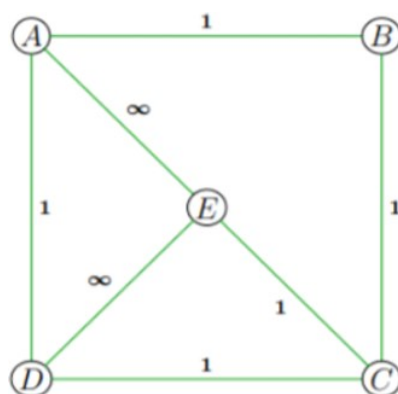


שאלה 4

[שאלה] אילו משקלים אפשר לשים כדי לגרום לעומס המקסימלי בכל צלע להיות 1.5 תחת האילוץ $(B,D, 5), (C,E, 3)$?

[פתרון] המקסימום צריך להיות 3 לפי $f = 3 \Rightarrow \frac{f}{c} = \frac{f}{2} = 1.5$. העומסים על הצלעות והמשקלים שנבחר יהיו כך:

$$(A, B), (A, D), (B, C), (C, D) : \frac{2.5}{2}, (C, E) : \frac{3}{2}$$



חזרה

התחלנו לדבר בשיעור הקודם על ECMP, המנגנון העיקרי לביצוע traffic engineering כשהרעיון הוא בעצם לבחור את המשקולות שיש לנו ברשת כך שהתעבורה תעבור כמו שנרצה. זה מנגנון שמסתכל בנקודה מסוימת האם יש שני מסלולים עם אותו משקל ואם כן אז לפצל את התעבורה בהם שווה בשווה.

אנחנו רוצים לראות האם אנחנו יכולים לתמרן את המשקולות ברשת כדי להשיג מטרות. דיברנו על בעיות אופטימיזציה כגון להיות מסוגלים לקבל throughput מקסימלי, למזער את העומס על הקשת הכי עמוסה וכדומה. חזרנו על הדוגמה מסוף ההרצאה הקודמת.

מצגת 5
שקף 76

A closer look at ECMP

ECMP הוא מנגנון שמפעילים על ראوتر באמצעותו הוא בוחר מה יהיה ה-next hop, אילו קשתות מובילות למסלולים שווים באורך ובמשקל שלהם למינימום המגיע ליעד, ומפצל ביניהם את התעבורה. אם כל פקטה שנכנסת נשלח פשוט ב-round robin בין המסלולים, נפגע ביעילות התקשורת - יהיה יותר מדי packet re-ordering אם באחד המסלולים ההודעות פשוט מגיעות מהר יותר, ואז הפקטות מגיעות ליעד לא באותו הסדר. נרצה למפות בצורה קונסיסטנטית את התקשורת בין שני מכשירים אל אותו המסלול, לאורך כל הרשת. על המיפוי הזה להיות כמובן זול כדי לא לעבוד הרבה באותו הקטע, ושהפיצול יהיה יחסית מאוזן.

הפתרון - פונקציית Hash

הקלט יהיה source ו-destination, והיא תחזיר מספר בטווח של כמות הלינקים עם אותה עלות, אפקטיבית, תבחר על איזה לינק לשלוח את ההודעה. כך כל ההודעות מאותו source לאותו destination יעברו באותו הלינק. הפונקציה דטרמיניסטית, לכן ההודעות יגיעו בסדר הנכון לראوتر הבא, שבתורו יפעיל פונקציית hash משלו וכן הלאה עד היעד.

הקלטים נשארים קבועים כי כתובות ה-IP נשמרות מקצה לקצה ולכן יש קונסיסטנטיות. הפיזור תלוי בזוגות source, destination שיש לנו. אם יש ECMP אבל תקשורת רק בין זוג מכשירים אחד ברשת הם ינצלו רק מסלול אחד, ואם יש שני זוגות עדיין סביר שיפלו על אותו הלינק. אם יש 100 או 1,000 זוגות שמדברים הפיזור יהיה אחיד אבל הוא לא ממושקל בשום צורה. אם אחד ה-flows יהיה ממש כבד הוא עדיין יקבל רק מסלול אחד כמו כל flow קטן אחר. כך נקבל מצב שעדיין יש לנו מסלול עמוס, כלומר ECMP לא יודע להתחשב בכמות התעבורה שעוברת.

Transport Layer

שקף 3

נרצה בדרך כלל לאזן בין flows כדי ליהמנע מהתנגשויות, להעביר traffic maximum throughput. engineering הייתה הדרך שלנו לעשות את זה בשכבת הרשת על ידי משקול סטטי של הרשת על ידי מנהל הרשת, פתירת בעיית אופטימיזציה בצורה offline שמטרתה לקבל פיזור אופטימלי.

בשכבת ה-transport אחד הדברים המרכזיים זה congestion control שמטרתה להעביר כמה שיותר מידע תוך הימנעות מיצירת עומסים. זו בעיה דינמית באופייה, כי יש flows שנוספים וכאלה שעוזבים - אין לנו דרך לדעת את זה בזמן אפס.

נדבר על שני פרוטוקולים בשכבת ה-Transport: פרוטוקול UDP ופרוטוקול TCP שעובדים מקצה לקצה.

שקף 5

Transport Services and Protocols

חלוקת המידע לסגמנטים ושליחתו על גבי הרשת. שכבת ה-transport תגדיר את הסגמנטים ואז כל סגמנט יעטף ב-header המתאים של כל אחת מהשכבות. המקבל יקרא סגמנטים, ובשכבת ה-transport הוא ירכיב את הסגמנטים ויחבר מהם רצף של בתים אותם יעביר לשכבת האפליקציה. השכבה הזו גם מחברת בין אפליקציות, סוג של Multiplexing אם שתי אפליקציות רוצות לדבר. שכבת ה-transport תעביר את התקשורת לאפליקציה הנכונה בצד המקבל.

שקף 7

ההבדלים בין UDP לבין TCP

ההבדלים בין UDP ל-TCP משמעותיים. נצטרך לבחור מה פרוטוקול ה-transport כשבונים אפליקציה. זו הבחירה היחידה שלנו, כי אין ברירות אלא פרוטוקול IP ואין בחירה לגבי פרוטוקולים בשכבת ה-data link. יש שתי אפשרויות:

TCP נותן congestion control (לא להעמיס יותר מדי על הרשת) ו-flow control (ידאג לזה שלא נעמיס יותר מדי על המקבל, הוא לא יקבל יותר ממה שהוא יכול לשמור). מתבצע בעזרת הקמת connection.

האלטרנטיבה היא להשתמש ב-UDP שם אין משמעות ל-connection ולא צריך להקים אותו, זה שירות שנותן את המינימום הנדרש כדי לבנות מעל IP גשר שיאפשר תקשורת בין אפליקציות, אבל מעבר לזה אין יותר מדי.

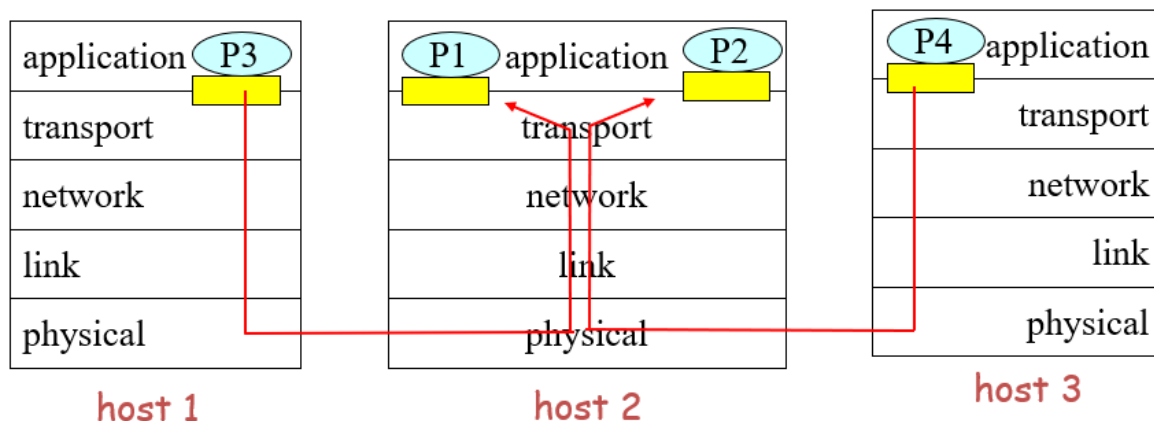
יש גם תנאים שאף אחד מהפרוטוקולים לא מבטיח: שכבת ה-transport לא יכולה להבטיח latency מסוים ("תעביר את ההודעה רק אם היא תגיע תוך 10ms"). מי שכותב את TCP כותב אותם בנקודות הקצה ולא על הראוטרם עצמם, לכן לא יכול לשמור רוחב פס לחיבור מסוים וכו'. סיבה נוספת היא שמדובר בתנאים

משתנים, למרות שמנקודת המבט שלנו מדובר בשני hosts שמנסים לתקשר, במקביל יש hosts שחולקים איתנו ראוטרים בדרך.

Multiplexing

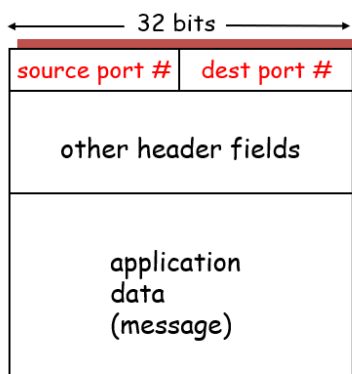
שקף 8

ל-host 2 יש שתי אפליקציות שרצות. host 1 ו-host 3 כל אחד רוצה לדבר עם אפליקציה אחרת. שכבת ה-transport מאפשרת להגיד לאיזו אפליקציה ההודעה שלנו שייכת. שכבת ה-IP נותנת אפשרות להגיע למחשב היעד, אבל בתוך המחשב יכולות להיות הרבה אפליקציות במקביל, וה-transport layer מוסיף מזהה שמאפשר להעביר את ההודעה לאפליקציה הספציפית. כדי לתמוך בזה מערכת ההפעלה נותנת מבנה שנקרא socket שנותן אבסטרקציה של קריאה וכתיבה. נבקש ממערכת ההפעלה להקצות socket ואז נכתוב ונקרא מה-socket. בדוגמה כאן כל אחד מהתהליכים האלה מיותר socket ובמקרה של P3 הוא כותב אליו, והוא מחובר ל-socket של P1 בשכבת ה-transport. אחר כך, P1 קורא את ההודעות מתוך ה-socket.



שקף 9

מבנה ההודעות (הסגמנטים). שכבת ה-transport מוסיפה header ויש לה שני מימושים. מה שמשותף להם זה שדה source port ושדה destination port. כך מתבצע Multiplexing לאפליקציות. כששולחים הודעה יש source IP addresses & destination IP addresses שאומרים מה כתובת המחשב השולח והמקבל, ועכשיו source port יזהה את התהליך השולח ו-destination port את התהליך המקבל. כשההודעה תגיע ליעד נוכל להסתכל על הפורט ויחליט לאיזה socket היא מתאימה, וה-OS תעביר ל-socket המתאים. הסגמנט יגיע ויכלול source port, אז כשעונים נוכל להשתמש בו בתור destination port כדי לענות.



TCP/UDP segment format

הדרך לייצר socket של UDP היא datagram, שמקבש לקשור אותה למספר port מסוים. מאחר שאין משמעות לחיבור ומה שמזהה socket ספציפי זה כתובת ה-IP של המכונה וה-port number שביציאת ה-socket קשרו אליו, אם אני בתור מי שיצר את ה-socket קורא לנקודה right ומבקש לשלוח לכתובת מסוימת, זה בעצם יעביר את ההודעה ל-host בצד השני, והוא יעביר את ההודעה לאפליקציה שקשרה את עצמה ל-port היעד שציינתי.

כשאנחנו עובדים מעל TCP קיים מושג של חיבור. מקודם רק קראנו וכתבנו הודעות (המשמעות היא שההודעה הגיעה ליעד ושם נקראו הבתים). כשעובדים מעל TCP צריך גם לייצר חיבור, מדובר בקשר persistent בין אפליקציה של השולח לאפליקציה של המקבל, ולכן אפשר לזהות חיבור TCP על ידי כל הפרמטרים:

Source & destination IP address וגם source & destination port address. שליחה וקבלה של הודעות יכולה להתבצע במקביל לשליחה וקבלה עם אותו port number אבל ההודעות שלו לא יתערבבו כי יש משמעות לחיבור שיצרנו. משמעות החיבור היא גם לוגית - אפשר לדבר על קצב שליחה מעל החיבור (בניגוד ל-UDP בו אין משהו שמלווה את התקשורת לאותו היעד - שולחים הודעה והיא אמורה להגיע ליעד). החיבור נותן לנו אפשרות לדבר על סדר ההודעות, קצב שליחה מעל החיבור וכו'.

המצב הכי סטנדרטי הוא שרת שמחכה ללקוחות שיתחברו אליו ואז ישלחו לו ברשות והוא יגיב. מקודם ב-UDP פשוט תפסנו port והתחלנו לקרוא הודעות, ב-TCP זה אחרת - השרת כן תופס port ומקשיב להתחיל לחיבורים. לפני שמתחילים לקרוא הודעות קוראים לפונקציה שנקראת listen, כלומר נתחיל להקשיב ללקוחות שרוצים להתחבר אלינו. כשלקוח כזה יגיע ויפתח חיבור המטרה היא להקצות משאבים לחיבור הזה, כלומר הקצאת buffer שיאפשר סידור של הודעות (הודעות יכולות להגיע עם בתים לא בסדר שהאפליקציה מוכנה לקרוא). נצטרך לשים את ההודעה שהקדימה את זמנה ב-buffer ונסדר אותן לפי sequence number. יוקצו גם משאבים נוספים (נדבר על flow control).

לרוב נקרא למי שיזם את יצירת החיבור קליינט, והשרת הוא הפסיבי שמחכה לחיבורים. כאמור השרת קורא ל-listen לעומת הקליינט שינסה להתחבר ל-host name על ידי יצירת socket תוך ציון מספר ה-port של השרת ושמו. מבחינת הלקוח לרוב לא משנה לו מאיזה port הוא ישלח את ההודעות, אז בדרך כלל בגלל שזה לא משנה נותנים ל-OS להקצות לנו port בצד של הלקוח. בצד של השרת לעומת זאת אי אפשר לעשות את זה, כי כל הלקוחות צריכים לדעת לאיזה port להתחבר.

ברגע שהלקוח מתחיל ליצור חיבור TCP ההודעה הראשונה שלו תגיע לשרת והוא יצטרך להחליט האם לקבל את החיבור או לא. קבלה תתבצע באמצעות accept. הפעולה של listen חוסמת, כלומר היא לא חוזרת,

ובכשהיא חוזרת זה אומר שיש חיבור חדש שצריך לעשות לו accept. מאחורי הקלעים, המטרה שלנו ביצירת חיבור היא יצירת sequence number שכל צד ידע לספור את הבתים שמתקבלים ולהקצות משאבים.

פרוטוקול TCP

פתיחת חיבור TCP באמצעות Three Way Handshake

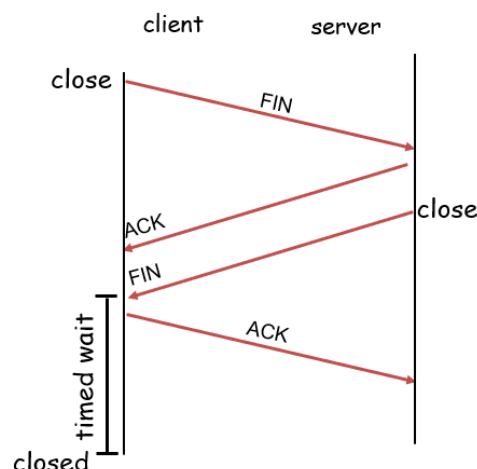
בהודעה הראשונה של הקליינט הוא שולח TCP SYN שמאותת לשרת שמטרתו לפתוח חיבור, עם sequence number שהלקוח בחר (מאיפה הוא מתחיל לספור את הבתים. מעט מוזר, הם לא מתחילים לספור מ-0 כמנגנון הגנה מפני מתקפות, לא נכנס לזה בקורס). אם השרת עושה accept הוא עונה ב-SYNACK שמשמעותו הכרה ב-SYN שנשלח, והוא שולח חזרה ללקוח. בנקודה זו גם השרת מקצה את ה-buffering לשרת שלו ובוחר initial sequence number.

בשלב האחרון הלקוח מאשר שהוא קיבל את ה-SYNACK של השרת ועונה ב-ACK משלו. לפעמים עם ה-ACK האחרון כבר ישלח מידע רלוונטי. כך שני הצדדים יודעים לסדר את המידע שמתקבל לפי הסדר שהשולח שלח אותם.

שקף 13

דיברנו על יצירת חיבור, נרצה גם לסגור אותו כשאין בו צורך - שחרור המשאבים שהקצינו עבורו ולא יהיו יותר בשימוש. מדובר בתהליך די דומה לחיבור. כאן בדוגמה הלקוח שולח את דגל ה-FIN, אבל זה לא חייב להיות כך, גם שרת יכול ליזום סגירה של חיבור.

נשלח FIN שאומר שהחיבור יכול להיסגר, הצד השני שולח FINACK, וסיום ב-ACK (שנשלח על ידי יוזם הסגירה). כשהיוזם מקבל FINACK הוא מבין שהשרת קיבל את הודעת ה-FIN, אבל אז הוא שולח הודעת ACK - איך ידע שהחתימה הסופית הגיעה? בדרך כלל TCP נכנס למצב timed wait, ה socket הופך למעין "זומבי", אי אפשר לכתוב אליו אבל עדיין מקשיבים לבדוק אם יש עוד הודעות שמגיעות. לכן נניח שאו שה-ACK בסוף יגיע או שיהיה timeout.



אז כאמור כל צד יכול לבקש לסגור socket ואז זה התהליך שקורה ברקע. כשכותבים קוד שמשתמש ברשת אחרי שהשרת מחזיר accept מערכת ההפעלה מקצה socket חדש לחיבור הזה. ברגע שנוצר חיבור האבסטרקציה הוא socket חדש שמיועד לאותו לקוח ואז אפשר לכתוב ללקוח הספציפי, זה הופך את התקשורת לנוחה. למשל, ליצור thread חדש ולהעביר לשם את הטיפול ב-ssocket החדש, כך מקבלים thread שמטפל בכל לקוח (בקשות מלקוח אחד לא משפיעות על האחר).

שקף 14

מתי נשתמש ב-UDP?

למה שנרצה להשתמש ב-UDP אם TCP מעניק כל כך הרבה? UDP זו גרסה בסיסית, barebones מעל IP שלא מספקת דבר מעבר ל-multiplexing. יש מקרים שבהם אי אפשר ליצור חיבור. DHCP עובד ב-broadcast, לא יודעים למי להתחבר אז אי אפשר ליצור חיבור TCP. במקרה כזה אין ברירה אלא שימוש ב-UDP. מקרים נוספים הם כאשר המנגנונים של TCP נותנים דברים לא שימושיים ויכולים ליצור בעיות, כמו למשל תקשורת של voice. אנחנו לא נרצה לחכות להודעות שלא מגיעות, הצד המקבל יכול לחכות הרבה זמן כי הלכו לאיבוד כמה מילישניות של הודעות, אבל TCP ידרוש שכל ההודעה תגיע. במקום זה, נרצה להמשיך לנגן את הקול בצד המקבל כי האוזן האנושית יודעת להשלים את החסר וכך מקבלים חווית משתמש טובה יותר. אופציה נוספת היא תקשורת קצרה, וה-handshake שדורש תקורה גבוהה לא מצדיק את זה, כמו למשל פרוטוקול DNS (בקשה של הודעה אחת ותשובה של הודעה אחת, אין כאן צורך בסידור ההודעה).

שקף 15

TCP Overview

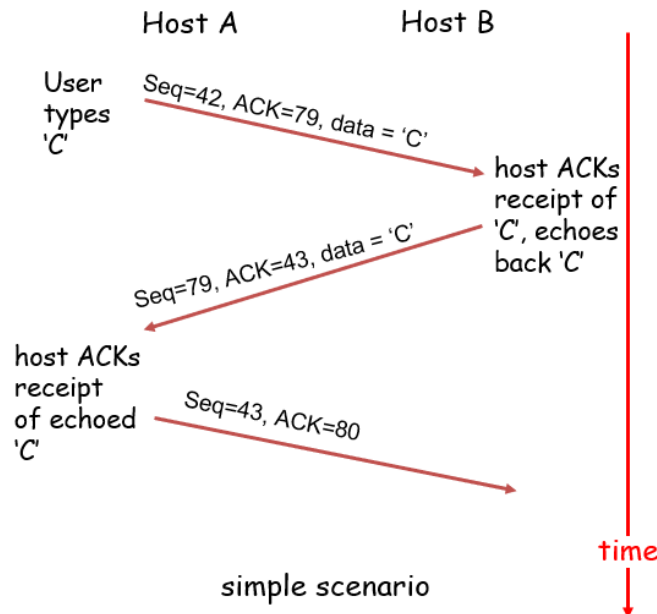
בעיקר לאורך הדיון ב-TCP כי יש שם יותר מנגנונים עליהם צריך לדבר. הפרוטוקול עובד מקצה לקצה, יש שולח אחד ומקבל אחד וביניהם קיים מושג של חיבור. הפרוטוקול אמין - מבטיח שהודעות יגיעו ושומר על הסדר (הסדר שהשולח שלח את הבתים זה הסדר בו המקבל יקבל אותם). מאפשר pipeline - לא חייבים לחכות לקבל אישור שהמידע הגיע לצד השני, אפשר לשלוח סגמנטים במקביל ובינתיים הם עוברים ברשת ויגיעו לצד השני - לא נחכה שזה יקרה. full duplex כלומר החיבור דו-כיווני, כל אחד מהצדדים יכול גם לקרוא וגם לכתוב. נדרשת לחיצת יד גם בהקמה וגם בסגירה.

שקף 16

Sequence #'s and ACKs

השימוש ב-sequence number וב-ACK מאפשר את התקשורת. sequence number אומר מה המספר הסידורי של דאטה שאנחנו שולחים. בדוגמה A host שולח הודעה עם sequence # 42, והדאטה הוא תו אחד. כל הודעת TCP מכילה ACK כלומר הגעה של מה שהגיע. ה-ACK מציין את ה-sequence הבא שהמקבל מצפה לקבל "קיבלתי עד לנקודה א". הטרימינולוגיה הזו חשובה, כי ה-ACK ב-TCP מאשר את מה שהגיע עד לאותה נקודה. אם קיבלנו הרבה הודעות אפשר בהודעת ACK אחת לאשר את ההגעה של כולן. בדוגמה נשלח ACK עם 43, כלומר הבית הבא שהוא מצפה לראות זה עם sequence # 43. המנגנון של ACK

בא בנוסף להודעה רגילה של TCP (אם יש ל-Host B משהו לשלוח). לכן שולח # sequence 79, ו-A בתגובה משיב לו שהוא מצפה לקבל את 80. כאן Host A לא שלח שום דאטה, אז ההודעה הבאה שלו גם תגיע עם # sequence 43.



שקף 17

TCP reliable data transfer

המטרה שלנו היא להשתמש ב-# seq וב-ACK כדי לייצר תקשורת אמינה (אובדנים, הודעות לא באותו סדר). מאחר שאנחנו עושים pipeline לסגמנטים (שולחים לפני שאנחנו יודעים שהצד השני קיבל) יכול להיות שיגיעו דברים לא באותו סדר. מנגנון ה-ACK שמאשר את כל מה שהיה עד עכשיו זו דרך אחת לטפל בזה.

אחד המנגנונים הבסיסיים של TCP כדי להבטיח אמינות זה שימוש ב-**retransmission timer**. ל-TCP יש טיימר שהוא שומר להודעה הכי ותיקה שלא התקבל עליה ACK וכשהטיימר פוקע הוא מניח שההודעה לא הגיעה (וגם לא תגיע) ולכן שולח אותה מחדש.

שני טריגרים לשליחה מחדש של הודעה

1. **אירועי timeout** – אם ה-retransmission timer פוקע, TCP יודע לשלוח מחדש את ההודעה. מדובר במשוב חוזר איטי ולא כל כך טוב.

2. **Duplicate ACKs** – כל הודעת TCP מכילה גם ACK. אם אני מקבל ACK כפולים, כלומר על אותו # seq כמה פעמים, אנחנו יודעים שהצד השני שולח ACK אבל משהו תוקע אותו, כלומר על ההודעות החדשות שמגיעות המקבל שולח ACK אבל בגלל שהוא נאגר (מאשר עד נקודה מסוימת) הוא לא יכול לאשר את ההודעה האחרונה שהתקבלה, אלא את האחרונה שהתקבלה לפי הסדר. כך נראה ACKs כפולים שיגרמו ל-TCP להחליט לשלוח מחדש מידע.

בתור התחלה נתעלם ממנגנון ה-ACK הכפול ונדבר רק על הטיימר.

שקף 18

TCP Sender Events

הטיימר הוא שעון שהשולח מתחזק. ברגע שהשולח ישלח הודעה הוא יחזיק טיימר אחד לחיבור ונתחיל להריץ אותו בעת שליחת ההודעה הוותיקה ביותר שעדיין לא קיבלה ACK. יש שתי אופציות:

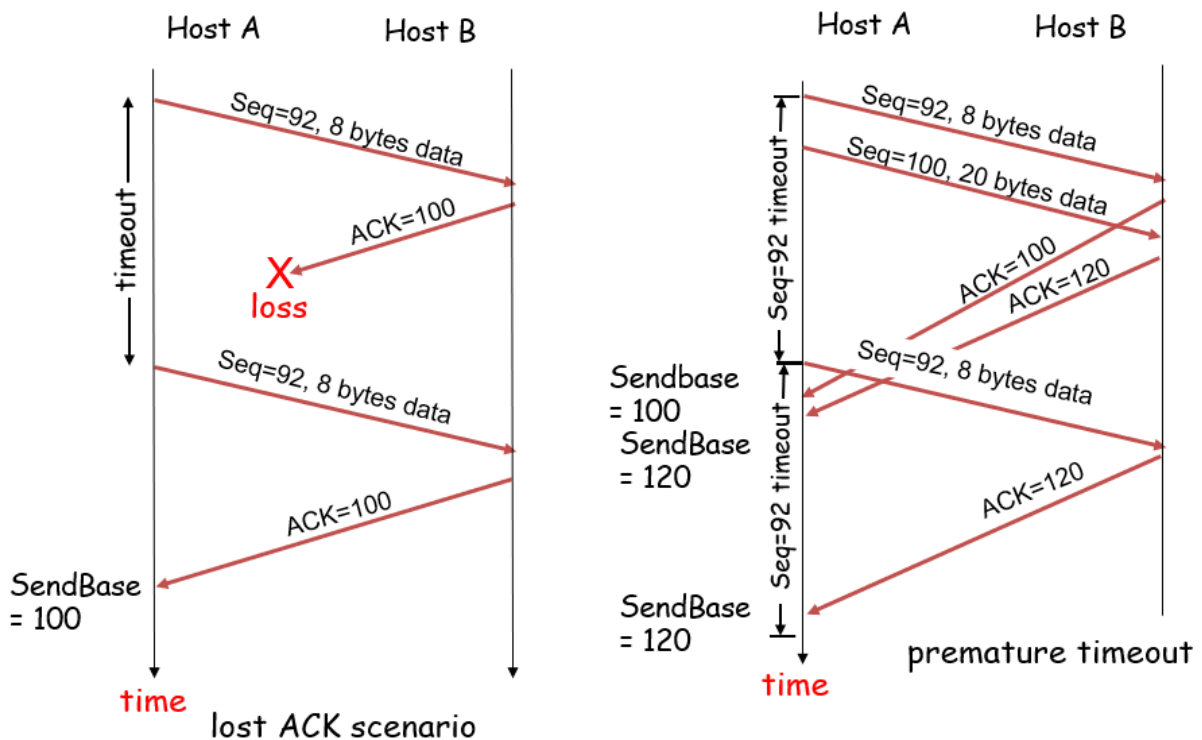
- הטיימר פוקע: נשלח מחדש את ההודעה שהטיימר שויך אליה
- קיבלנו ACK על ההודעה שהטיימר משויך לה: אם יש לנו הודעות outstanding שאנחנו לא יודעים מה גורלן (כלומר ה-ACK שחזר לא אישר את כל ההודעות ששלחנו עד כה, אלא רק חלק מהן) נאתחל את הטיימר מחדש, ואז חוזר חלילה.

שקף 19

קביעת הטיימר חשובה:

מצד אחד, הטיימר צריך להיות יחסי ל-latency שלוקח להודעות להגיע לצד השני (ה-roundtrip time). אם נקבע טיימר קצר מדי הטיימר כל הזמן יפקע: נשלח הודעה ובגלל שהטיימר קצר יותר מה-roundtrip time לא נספיק לקבל עליה ACK ואז נשלח שוב.

מצד שני, אם נקבע אותו להיות גדול מדי אנחנו נגיב מאוד לאט לבעיות, נחכה המון זמן לפני שנטפל באובדן מידע.



Reliable Transport

אנחנו כרגע מדברים על Layer 2. עד עכשיו הנחנו שאם אנחנו שולחים משהו הוא מגיע לצד השני באותו סדר ובלי הפרעות בדרך. עכשיו, נניח שכשאנחנו שולחים משהו ייתכן שההודעה לא תגיע (למשל התנגשות ב-Layer 2, תווך לא אידיאלי), וגם ייתכן שהודעות ששלחנו לא יגיעו באותו הסדר.

מה "המחיר" שצריך לשלם כדי לטפל בבעיות האלו? צריך רשת יציבה למרות הידיעה שמתחת יש שבבה לא אמינה.

פתרון אפשרי

המקבל יודא שמה שהגיע אליו תקין, אם השולח גילה שהייתה בעיה – ישלח שוב.

אבל אם לא נשלח ישירות ליעד הסופי אלא ל-switch, וה-switch הוא זה שנפל, לא נקבל את הודעת ה-ACK. צריך שהפקטות יאגרו במקרה של בעיה – ואת זה נגדיר היום. ההנחה היא שפקטות יכולות ליפול או להיהרס, יכולות להתעכב.. אנחנו נבנה פרוטוקולים שיטפלו בבעיה.

הערות והנחות:

- כל אחד יכול לשלוח ולקבל, אז אנחנו נתייחס אליהם כמכונה עם מצבים: שולח ומקבל.
- במקרה הכללי, הרשת לא מסונכרנת.
- נניח גם שאנחנו יכולים לזהות שגיאות.
- הפקטות יגיעו בסדר שבו נשלחו (לא נראה החלפות בסדר).

Stop & Wait Protocol

כאן נדבר על הפרוטוקול מנקודת המבט של Layer 2. השולח שולח פקטה ומתחיל טיימר. יש T_{out} שניות בהן הוא מחכה לקבל ACK/NACK קבלה/אי-קבלה.

הצד השולח

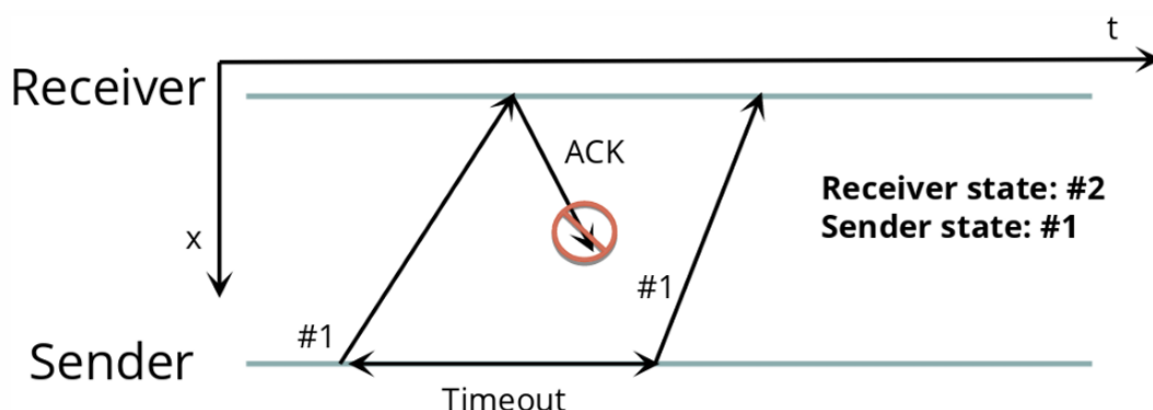
אם קיבל NACK לפני ה-timeout או שלא קיבל כלום: ישלח שוב את הפקטה ויאפס את הטיימר.
אם קיבל ACK: ישלח את הפקטה הבאה ויאפס את הטיימר.

הצד המקבל

אם קיבל פקטה בלי שגיאה: ישלח ACK.
אחרת: ישלח NACK או יתעלם.

ניסיון #1

האם מה שתיארנו מספיק? מה יכול להשתבש?

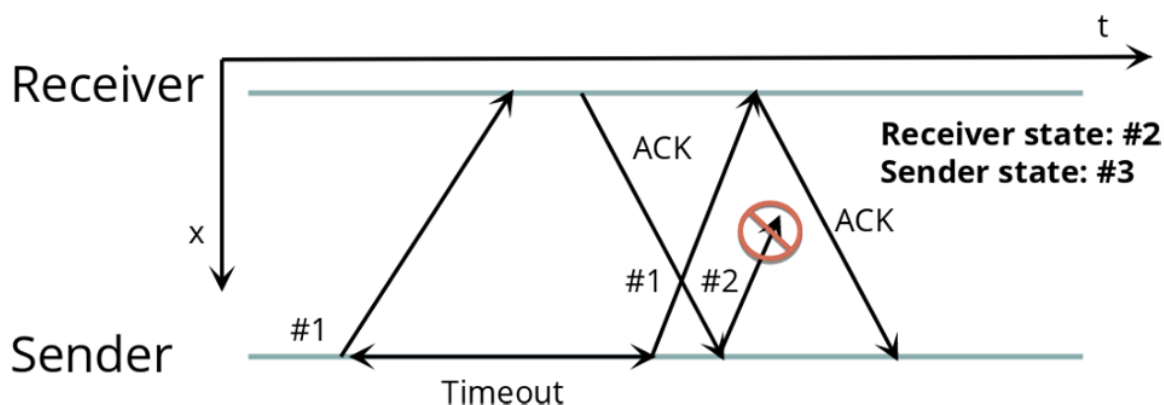


נסתכל על ציר הזמן, שולח ומקבל. השולח שלח את הפקטה הראשונה, המקבל הצליח לקבל ולכן שולח ACK. ה-ACK נאבד בדרך. מבחינת השולח, הצד השני לא קיבל את ההודעה לכן ישלח אותה שוב. המקבל יקבל שוב אותה פקטה, ויתייחס אליה כפקטה נוספת חדשה. מבחינת השולח – הפקטה האחרונה שהתקבלה היא #1, מבחינת המקבל הפקטה האחרונה שהתקבלה היא #2. נוצר פער בין השולח למקבל.

לכן, כשנשלח פקטה נסמן גם counter. אם המקבל יקבל שוב את פקטה מספר 1, הוא יבין שהייתה בעיה עם ה-ACK וישלח שוב.

ניסיון #2

האם עכשיו זה מספיק?



השולח עכשיו שולח פקטה, המקבל קיבל אז הוא שולח את ה-ACK. יכול להיות שלקח זמן, וה-ACK הגיע אחרי שפקע ה-timeout שהשולח הגדיר, לכן השולח ישלח את #1 פעם נוספת. עכשיו הוא קיבל את ה-ACK הראשון, לכן ישלח את הפקטה #2. נביח שהיא הלכה בדרך לאיבוד, ובינתיים ה-ACK השני שהמקבל שלח (על פקטה #1) הגיע, והשולח חושב שהצד השני קיבל את #2.

הפתרון פשוט – נוסף counter אצל המקבל. לא יהיה ACK כללי, אלא ACK על הודעה מספר X.

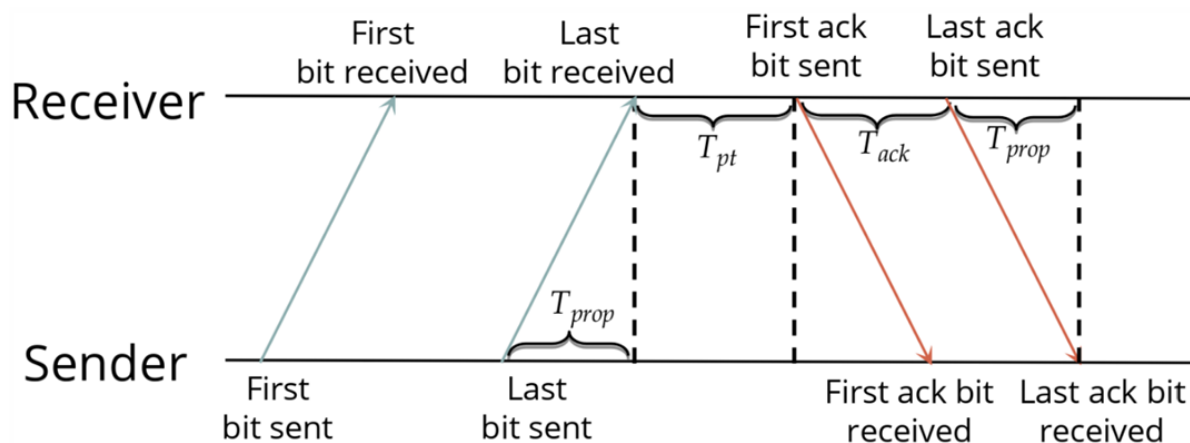
זה לא פותר את בעיית ה-timeout.

Stop & Wait – Timeouts

מה ה-Timeout ההגיוני לחכות? כשאנחנו שולחים פקטה מתייחסים לכמה גורמים, נציג חישוב ל-Timeout מינימלי. נניח שה-timeout מתחיל אחרי שליחת הביט האחרון של הפקטה. נרצה להסתכל על:

- גודל הפקטה
- גודל הודעת NACK/ACK
- Transmission rate (bandwidth)
- מרחק בין תחנות
- Propagation speed (T_{prop})
- Processing time (T_{pt}) בצד המקבל (לעודא את נכונות הפקטה). תמיד יהיה $0 <$ אלא אם נאמר אחרת.

נסתכל על המקרה הגרוע ביותר:



נקבל:

$$T_{out} \geq 2T_{prop} + T_{ack} + T_{pt}$$

דוגמה מספרית לחישוב Timeout

נתונים:

- Packet size: 2000 bit
- ACK/NACK size: 200 bit
- Transmission rate (BW): 10 Kbps
- Distance between stations: 1000 km
- Propagation speed: $2 \cdot 10^8 m/s$
- Processing time (T_{pt}): 0 sec

מה ה-timeout המינימלי שצריך עבור הפרוטוקול?

נחשב את T_{prop} :

$$T_{prop} = \frac{\text{distance}}{\text{propagation speed}} = \frac{10^6 m}{2 \cdot 10^8 m/s} = 5 \cdot 10^{-3} s$$

נחשב את T_{ack} :

$$T_{ack} = \frac{\text{ACK Size}}{\text{Transmission Rate}} = \frac{200 \text{ bits}}{10^4 b/s} = 2 \cdot 10^{-2} s$$

סה"כ:

$$T_{prop} = 5 \cdot 10^{-3} s$$

$$T_{ack} = 2 \cdot 10^{-2} s$$

$$T_{pt} = 0$$

Total timeout needed for the protocol to work:

$$T_{out} \geq 2T_{prop} + T_{ack} + T_{pt}$$

$$= 2 \cdot (5 \cdot 10^{-3}) + 2 \cdot 10^{-2} + 0$$

$$= 3 \cdot 10^{-2} s$$

אפשר כמובן גם T_{out} ארוך יותר, אבל זה יהיה בזבוז זמן.

Stop & Wait – Goodput

הנחות:

- ננצל את כל ה-BW
- נתעלם מנושא ה-NACK, אם עבר timeout נשלח שוב.
- נגדיר הסתברות לנפילת פקטות: $X \sim \text{Geo}((1-p)^2)$ (הריבוע הוא כי גם הפקטה הנשלחת יכולה ליפול וגם ה-ACK יכול ליפול)

$$\mathbb{E}[X] = \frac{1}{(1-p)^2} \text{ נשים לב:}$$

כעת נוכל לחשב את ה-goodput. תזכורת:

$$Goodput = \frac{Effective\ Transmission\ Time}{Total\ Transmission\ Time}$$

ובמקרה שלנו:

$$= \frac{T_{packet}}{\left(\frac{1}{(1-p)^2} - 1\right) (T_{packet} + T_{out})}$$

הזמן האפקטיבי הוא שליחת פקטות ולכן זה המונה. במכנה אנחנו שולחים בממוצע $\frac{1}{(1-p)^2}$ פעמים, אבל זה מתאר את המקרה שהצלחנו, ולכן 1- כדי לתאר את המקרה שבו זה לא הצליח. את זה נכפיל בכמות הזמן שלקח לפקטה להגיע ועוד הזמן שבו חיכינו בין שידורים.

אם נשים את T_{out} המינימלי כפי שחישבנו נקבל:

$$Goodput = \frac{T_{packet}}{\frac{1}{(1-p)^2} (T_{packet} + T_{out})} = \frac{T_{packet}(1-p)^2}{T_{packet} + T_{out}}$$

איפה הבעיה ברעיון שתיארנו? השולח מעביר את הפקטה שלו, ומחכה עד שהוא מקבל ACK. זה זמן שלא מנוצל, והוא פוגע ב-goodput.

Go Back N

אחרי ששלחנו את הפקטה הראשונה, בלי קשר לאם הגיעה או לא, נמשיך לשלוח פקטות. לא נשכח מכל מה ששידרנו, כדי שנוכל לטפל בשליחה חוזרת במקרה של שגיאה.

הגדרת הרעיון

השולח:

- מחזיק חלון בגודל N.
- שולח N פקטות בזו אחר זו.
- אם שעון ה-timeout פקע, שלח שוב את כל הפקטות מה-ACK האחרון הידוע.
- ימתין ל-**ACK(i)** המאשר את ההגעה של הפקטות i-1, 2,
- כשיקבל ACK ייזז את החלון כמה שאפשר כך שכל מה שמחוץ לחלון אושר על ידי ACK.

מקבל:

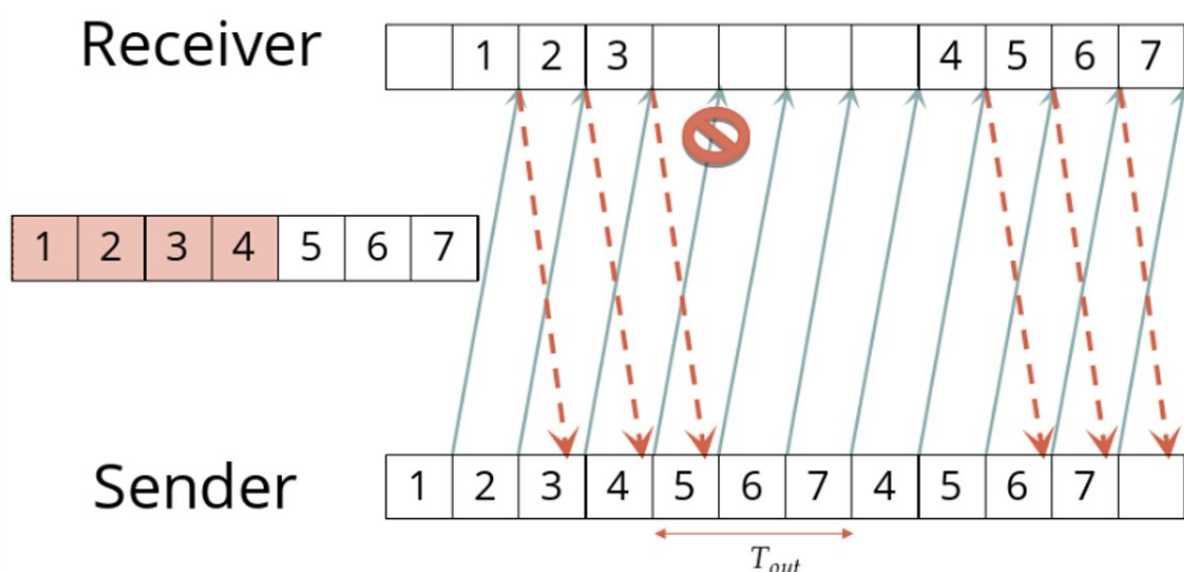
- מחזיק חלון בגודל פקטה בודדת.
- יודע מה הפקטה האחרונה שהוא קיבל, וכל פעם שמגיעה פקטה הוא בודק האם זה מספר הפקטה הבא שצפוי להגיע:
 - א. אם כן, שולח ACK על המספר הזה.
 - ב. אם לא, שולח ACK על מספר הפקטה הקודמת שקיבל.

טיימר

השולח משדר N פקטות. אם עבר ה-timeout והוא לא קיבל ACK על הפקטה הראשונה בחלון הוא יניח שהייתה בעיה וישלח שוב. אפשר לקצר לא בעזרת NACK אלא בעזרת ACK – כי אם ה-ACK על מספר שלא מתאים לפקטה האחרונה שנשלחה אז זה שקול ל-NACK (בחישובים נתעלם מהמקרה הזה).

דוגמה – בעיה בשליחת פקטה

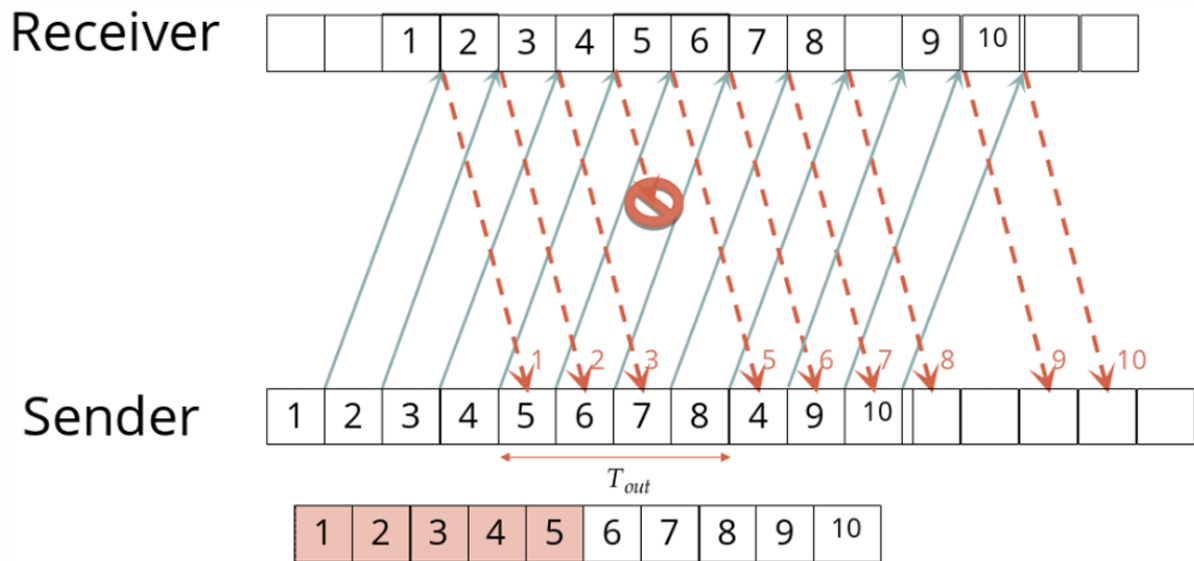
נניח שמחזיקים חלון בגודל 4, $T_{out} = 3$:



בכל פעם שמתקבל ACK נזיז את החלון צעדים בהתאם. נשים לב שהודעה 4 לא התקבלה ובזמן הזה המשכנו לשלוח את 5, 6, 7 – לא יכולנו להמשיך ל-8 כי נגמר החלון. ברגע שעבר T_{out} הטיימר פקע ולא קיבלנו אישור על 4. נתחיל לשלוח את הפקטות מתחילת החלון.

דוגמה – בעיה בקבלת ACK

נניח חלון בגודל 5, $T_{out} = 4$:

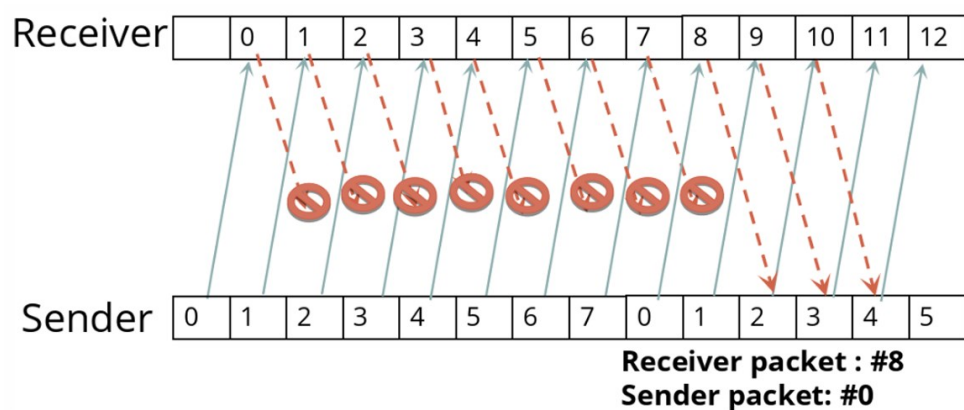


באן המשכנו לשלוח כי החלון אפשר את זה, אבל אז קיבלנו ACK(3) והחלון פקע. זה לא בהכרח אומר שהמקבל לא קיבל את 4, אבל השולח לא קיבל אישור על זה. ברגע שהחלון פקע השולח יתחיל מהנקודה האחרונה שלא אושרה, כלומר ישלח את 4, ובזמן השליחה הגיע האישור ACK(5). נשים לב ש-ACK הוא קומולטיבי, ולכן המשמעות של זה עבור השולח היא שהוא קיבל אישור שהמקבל **כן קיבל** את 4.

השולח מבין את זה, ובמקום להמשיך לשדר שוב 5,6,... הוא יחזור לשידור רגיל מהנקודה בה עצר בחלון בפעם האחרונה, כלומר 9, 10.. זה מראה שגם אם ACK אחד הלך לאיבוד זה לפעמים לא גורם נזק גדול, כי ה-ACK הבא יכול את האישור הקודם.

מה ה-tradeoff?

להחזיק חלון בגודל מסוים דורש משאבים. בדרך כלל המספר שיתאר איזו פקטה הגיעה יהיה בינארי ובגודל מסוים של ביטים. למשל עבור 3 ביטים נוכל לתאר 8 מספרים. תחת הנחה זו:



נניח שכל ה-ACKים נפלו בדרך. הזמן יעבור והשולח לא יקבל שום ACK ולכן יחזור לשדר את פקטה 0 מבלי לדעת שכל ה-7 הראשונות כבר התקבלו. כשהשולח יקבל את פקטה מספר 000 הוא יחשוב שמדובר בפקטה 8, כי הרי 8 בבינארית זה 1000 ויש רק 3 ביטים.

השולח יקבל ACK על 000 ויחשוב שזה אומר שפקטה 0 התקבלה, ולכן ימשיך לשדר כרגיל את 1, 2, 3, ... למרות שכל אלה כבר התקבלו. כך נוצר פער בין המקבל לשולח.

מה הפתרון לזה? עבור חלון בגודל N נצטרך N+1 מספרים, כלומר $\lceil \log_2(N + 1) \rceil$ ביטים. עבור חלון בגודל 8 נצטרך 4 ביטים.

שאלות לדוגמה

נתונים:

- Node A only sends data to B, B only ACKs
- No limit on buffers or sequence numbers
- The packet transmission (T_{packet}), propagation time (T_{prop}) and packet processing time (T_{pt}) are constant
- ACKs are delivered without any errors and $T_{ack} = 0$
- T_{out} is set to the minimal value that avoids unnecessary retransmissions
- $T_{out} = 2T_{prop} + T_{pt} + T_{ack}$
- Window size is equal to $T_{out} + 1$

נשנה מעט את הפרוטוקול:

המשדר A:

- כשיגלה על שגיאה ישלח בחזרה את הפקטה L פעמים.
- יפעיל את ה-timeout רק אחרי שהפקטה האחרונה נשלחה.
- ישלח את כל L הפקטות גם אם הוא קיבל באמצע ACK.

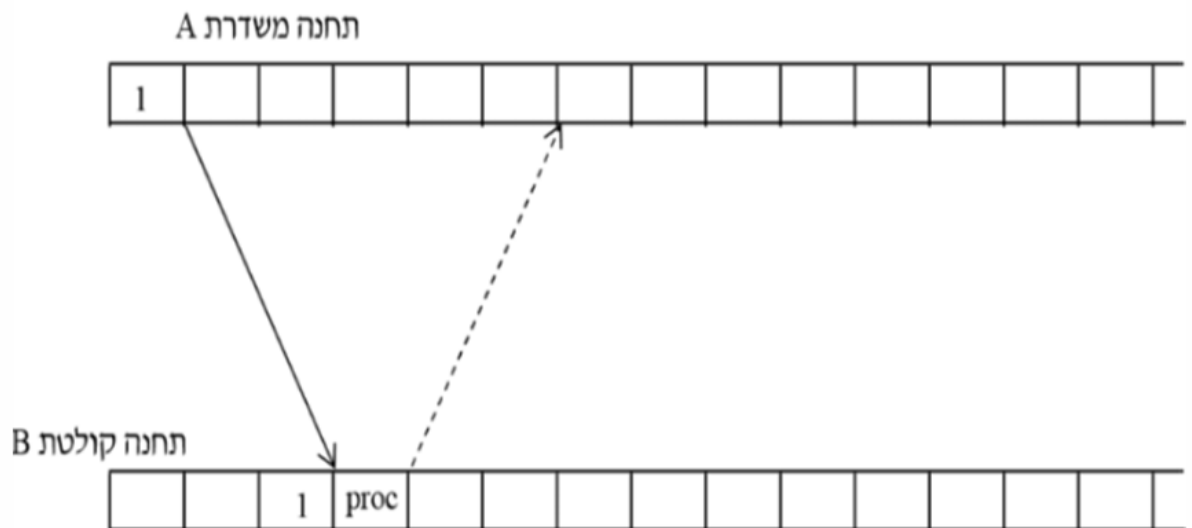
המקבל B:

- לא ישלח ACK על משהו שכבר שלח עליו.

שאלה 1

Complete the diagram when:

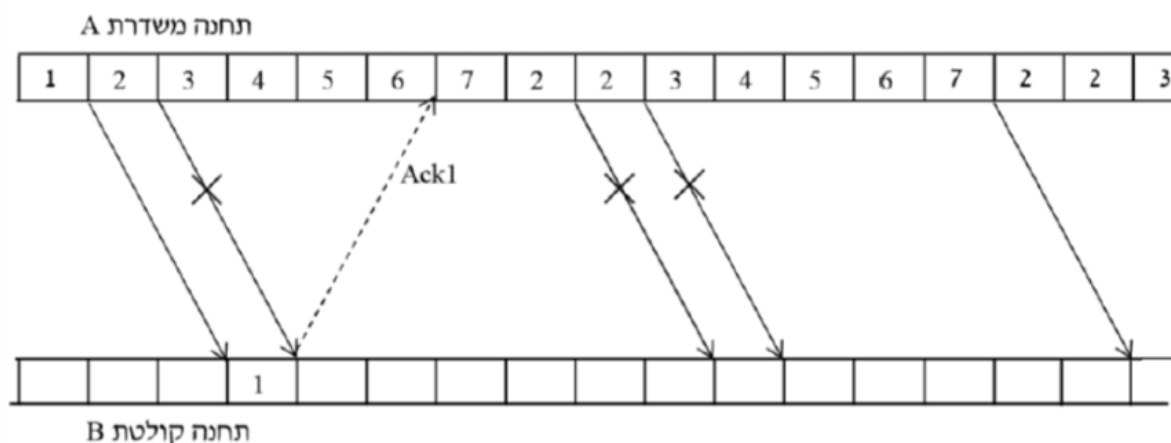
- $L = 2$, $T_{prop} = 2$, $T_{packet} = 1$ $T_{pt} = 1$
- The transmissions of packet #2 fail three times
- All other packets arrive successfully.



פתרון:

- $T_{out} = 2T_{prop} + T_{pt} + T_{ack} = 2 \cdot 2 + 1 + 0 = 5$
- Window size = 6

פתרון



A שלח את 1 עד 6 ואז קיבל אישור על 1, לכן המשיך לשלוח את 7. כרגע החלון נגמר ולא קיבלנו אישור על 2 (נתון ש-2 נופל 3 פעמים). עכשיו לפי הפרוטוקול A ישלח את 2 פעמיים, ושניהם יפלו. המשדר לא יודע שהם נופלים לכן ממשיך לשלוח שוב את כל שאר החלון: 3, 4, 5, 6, 7 אבל שוב נגמר החלון בלי לקבל ACK על 2. הוא ישלח פעמיים את 2 וימשיך הלאה 3, 4, 5, ... והפעם ה-2 התקבל.

שאלה 2

- For a given L , assume packets fail with probability p .
- What is the probability to succeed with exactly k retries (each retry consists of L packets)?

נשים לב שבכל פעם שאנחנו נכשלים, נשלח L פקטות.

בניסיון הראשון הסיכוי להצליח הוא $p_0 = 1 - p$.

לאחר מכן מגיעים שני ניסיונות שידור חוזרים:

בניסיון החוזר הראשון נשלח L פעמים. קודם כל צריך להיכשל פעם אחת ולכן נקבל p . אז הסיכוי להיכשל כל אחד מ- L הפעמים זה p^L , נהפוך את זה כדי לקבל את הסיכוי שהוא לא נכשל L פעמים לכן $1 - p^L$. סה"כ $p(1 - p^L)$

אם זה יצליח בניסיון החוזר השני: שלחנו פעם אחת וזה נכשל בהסתברות p , וגם L הניסיונות החוזרים נכשלו בהסתברות p^L , בסוף נכפיל בהסתברות להצליח.

אם זה יצליח בניסיון החוזר ה- k : p זה הכישלון הראשון, $(p^L)^{k-1}$ אלה $k-1$ הכישלונות הנוספים, והסיכוי להצליח הוא $1 - p^L$.

שאלה 3

- Assuming that:

① $T_{pt} = T_{ack} = 0$

- ② The receiver is ready to accept the packet

What is the expected time to successfully transmit a packet?

אנחנו רוצים את תוחלת הזמן עד לקבלת פקטה תקינה.

בהתחלה t_0 זה משך הזמן שלוקח לפקטה להיות משודרת: $t_0 = T_{packet}$

אם זה נכשל פעם אחת צריך לשדר פעם נוספת: $t_1 = T_{packet} + (T_{out} + L \cdot T_{packet})$, כי משדרים שוב, מחכים timeout ואז משדרים L ניסיונות חוזרים.

נסתכל על הניסיון ה- k : $t_k = T_{packet} + (T_{out} + L \cdot T_{packet}) \cdot k$, וזו כמות הזמן שאני אצטרך לחכות עד להגעת פקטה. נחשב תוחלת:

$$t_{avg} = \sum_{k=0}^{\infty} t_k \cdot p_k = T_{packet} + \frac{p(T_{out} + L \cdot T_{packet})}{1 - p^L}$$

(לא נתמקד בחישוב העמוק של התוחלת).

שאלה 4

קיבלנו בשאלה הקודמת את t_{avg} ונרצה לדעת מה ה-goodput – כמה מתוך סך הזמן ניצלנו לצורך שליחת פקטות.

$$a = \frac{T_{out} + T_{packet}}{T_{packet}} \quad Goodput = \frac{T_{packet}}{T_{avg}} = \frac{1 - p^L}{1 - p^L + p(a + L - 1)}$$

במונה יש זמן שידור פקטה, ובמכנה הזמן שמתעסקים בממוצע. לא התעסקנו בחישובים.

מה יהיה ה-goodput ב-Go Back N? ההבדל בין הפרוטוקול המקורי לחידוש בשאלה הוא שב-Go Back N פשוט $L=1$. לכן נקבל תשובה כללית:

$$Goodput_{GBN} = \frac{1 - p}{1 - p + pa}$$

ALOHA מדבר על זה שכמה תחנות מדברות אחת עם השנייה והורסים את התווך בגלל זה. כאן אנחנו בכלל לא בטוחים שהפקטה הגיעה לצד השני. מומלץ לעשות בדיקת שפיות כאשר $p = 0$ (לא נכשל לעולם, צריך לקבל goodput 1) או כאשר $p = 1$ (תמיד נכשל, צריך לקבל goodput 0). אפשר גם לבדוק מה קורה כש- $T_{out} \gg T_{packet}$ במקרה זה a גדול מאוד ואז כל ה-goodput ישאף ל-0.

Selective Repeat

בעיה ב-Go Back N

שמנו לב להרבה מקרים שבהם אנחנו חוזרים על שליחה של פקטות שאנחנו חושבים שלא הגיעו (למשל 1 התקבלה, 2 לא התקבלה, 3 התקבלה, 4 התקבלה.. במקרה הזה נתחיל לשלוח שוב את 2, 3, 4 למרות שהתקבלו). זו המוטיבציה לפרוטוקול הבא.

הגדרת הרעיון

התיקון לפרוטוקול הקודם שראינו, הוא שבמקרה בו רק פקטה 2 לא התקבלה, נזכור על אילו פקטות לא קיבלנו אישור ונשלח רק אותן פעם נוספת.

מה החיסרון בגישה הזו? החלון שתופס זיכרון יצטרך לשבת גם בצד המקבל. ב-Go Back N כל מה שהמקבל היה צריך לעקוב אחריו זה ההודעה האחרונה שהוא קיבל, כאן הוא יצטרך לעקוב אחרי כל ההודעות שקיבל (בפועל לא באמת על כולו, רק על חלון).

האלגוריתם

שולח:

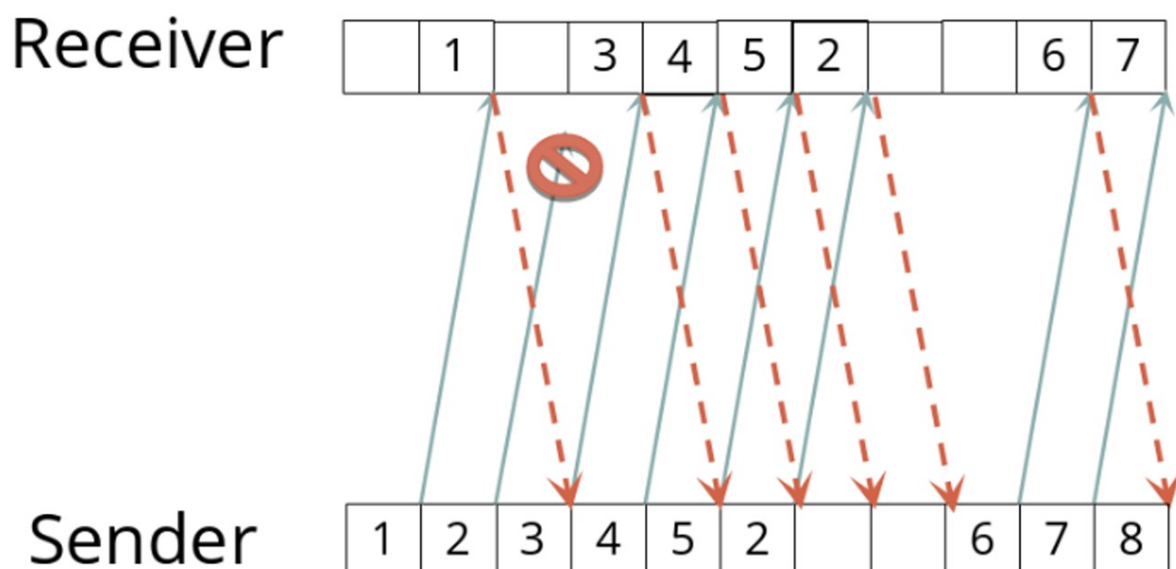
- מחזיק חלון בגודל N .
- שולח פקטות בזו אחר זו לפי הסדר.
- לכל פקטה מתחיל טיימר timeout ברגע סיום שליחת הביט האחרון של הפקטה.
- אם קיבל timeout על פקטה כלשהי בחלון, ישלח אותה שוב.
- קבלת **ACK(i)** מסמן אישור על הגעת הפקטה ה- i (לשים לב לשוני מ-Go Back N שהיה קומולטיבי).
- בעת קבלת ACK יזיז את החלון בהתאם, אבל לעולם לא יוציא מהחלון פקטה שלא קיבל עליה ACK, פקטה כזו תחסום את תחילת החלון.

מקבל:

- מחזיק חלון בגודל N .
- עונה ACK לכל פקטה שמתקבלת בחלון, גם אם היא לא בסדר הנכון.
- פקטות לא לפי הסדר בתוך טווח החלון יחכו בתוך באפר עד שפקטות שחסרות בסדר יתקבלו.
- כשכל הפקטות החסרות התקבלו, נעביר את כל הפקטות המסודרות לשכבה הבאה בהיררכיה ונזיז את החלון ל-slot של הפקטה הבאה שצפויה להתקבל (לפי # סידורי).

דוגמה

נניח חלון בגודל $T_{out} = 3, 4$.



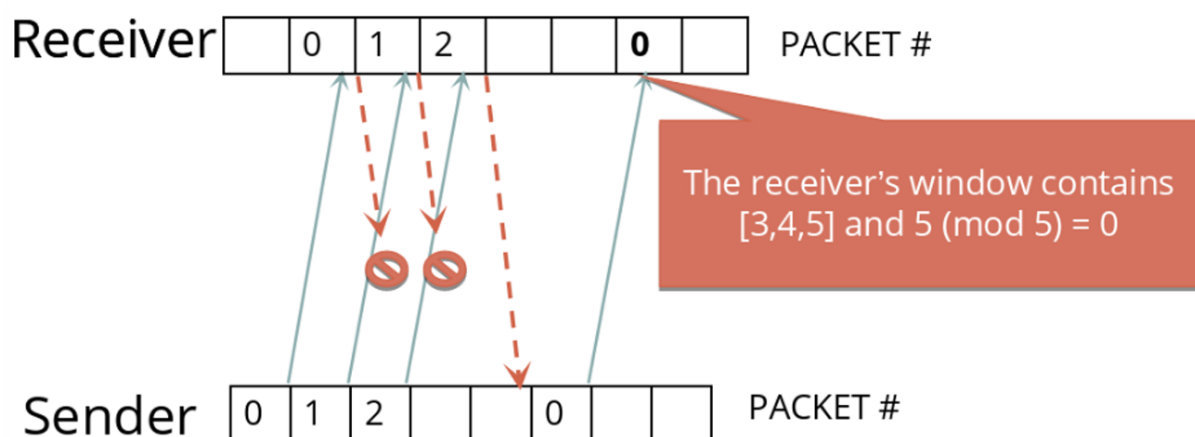
השולח שלח את פקטה 1, והיא מתקבלת (השולח עוד לא קיבל עליה ACK בינתיים, אבל זה בסדר כי עוד לא הגיע timeout). השולח מעביר את 2 והוא לא מגיע, שולח את 3 ועכשיו מגיע ACK על 1 לכן השולח יכול להזיז את החלון קדימה (להחזיק מקום עבור 2, 3, 4, 5). בזמן הזה 3 הספיק להגיע לצד המקבל, השולח מעביר את 4, 5, ואז מקבל ACK על 3, אבל השולח לא יכול להזיז את החלון כי אין אישור על 2. עכשיו גם פקע ה-timeout ולכן נשלח שוב את 2. בינתיים התקבל אישור על 4, 5. עד שלא מתקבל ACK על 2 אנחנו נמתין (זה לא חוסך זמן, אבל כן חוסך תווך).

אחרי שמקבלים ACK על 2 אפשר להזיז את החלון להודעה האחרונה שלא התקבל עליה אישור – אין כזו (קיבלנו כבר על 3, 4, 5) אז החלון ריק ואפשר להמשיך לשדר 6, 7, ...

גודל החלון

בדומה לבעיה שראינו ב-Go Back N, גם כאן יש משמעות לגודל החלון.

ייתכן מצב שבו בחלון בגודל 3 ו- $2N-1=5$ מספרים סידוריים נקבל:



כאן מהצד המקבל הוא שלח את 0, 1, 2 ולא התקבלו עדיין אישורים אז הוא ממתין. הגיע אישור על 2 ופקע ה-timeout לכן ישלח פעם נוספת. המקבל יחשוב שה-0 הוא 5 כי אין לו מספיק ביטים, ואז נוצר חוסר סנכרון בין המקבל לשולח.

נדרוש גודל חלון שהוא גדול או שווה $2N$ sequence numbers, כלומר $\lceil \log_2 2N \rceil$ ביטים. עכשיו יהיו לנו 6 sequence numbers בדוגמה הקודמת, ולא נתבלבל כמו קודם כי נזהה שזו הפקטה ה-0 ולא החמישית.

- Assume an “ideal” SR protocol:
 - Infinite-sized windows on both ends
 - Infinite bits for sequence numbers
 - Infinite-sized buffers on both ends
 - ACKs don't fail and T_{ack} is negligible
 - Packets fail with probability p

מה ה-goodput של הפרוטוקול?

$$Goodput = \frac{T_{packet}(1 - p) + 0 \cdot p}{T_{packet}} = 1 - p$$

כמות הפקטות הטובות שנשלח חלקי הזמן הממוצע שלוקח לשדר פקטה. במונה נשים את הזמן שלוקח לשדר פקטה כפול ההסתברות להצליח. נוסיף לזה את כמות הביטים שנרוויח מכשלון (0 כמובן) כפול ההסתברות להיכשל. נשארנו עם $1 - p$.

Selective Repeat vs. Go Back N

Criteria	Selective Repeat	Go-Back-N
Bandwidth utilization	Only retransmits lost packets	Might retransmit packets that already arrived successfully
Window sizes	Sender: N Receiver: N	Sender: N Receiver: 1
Sequence numbers	$2N$	$N + 1$
Out-of-order handling	Track which packets were received (out of order possible)	No sorting: receiver accepts packets only in order
Receiver storage	Store packets until missing/damaged packets are retransmitted	No need to keep undamaged packets (sender may resend)

החיסרון ב-Selective Repeat זה שהמקבל צריך להחזיק חלון ולא ביט בודד כמו ב-Go-Back-N, לפעמים זה קריטי. זה גם מכפיל את גודל ה-Sequence Numbers, מה שאומר שכמות הביטים שנשמור גדולה יותר. היתרון הבולט הוא שקבלת פקטות בסדר לא נכון לא ייזרקו, נדע לטפל בהן מה שמקדם מהר יותר את תהליך קליטת הפקטות.

- A sender (A) and a receiver (B) works with S&W with the following modification:
 - station A transmits a window of L packets, starts a timer and waits for all ACKs (for every packet in the transmitted window) before transmitting the next window
- In case any of the packets isn't ACKed – the entire window is considered failed and station A retransmits the entire window (after the timeout)
- If all packets are ACKed, station A will transmit the next window

העדכון הוא שהשולח מעביר חלון בגודל L פקטות, ולאחר מכן מפעיל טיימר ומחכה לכל ה-ACKs לפני שהוא מעביר את החלון הבא.

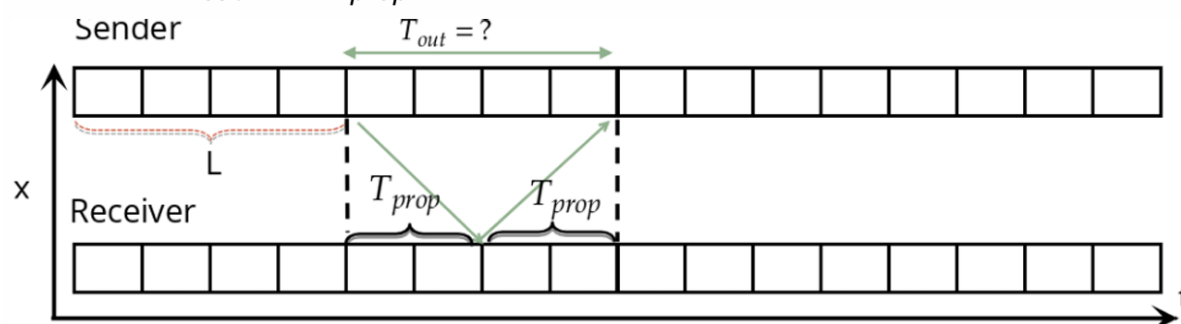
אם אחת מהפקטות לא קיבלה אישור בתום הזמן, החלון כולו ייחשב כנכשל ויישלח כולו מחדש. אחרת נעבור לחלון הבא.

נתון T_{prop} כלשהו, ו- T_{ack} , T_{pt} הם זניחים.

שאלה 1

מהו ה- T_{out} האופטימלי (בלי לבצע שידור נוסף)?

Answer: $T_{out} = 2T_{prop}$



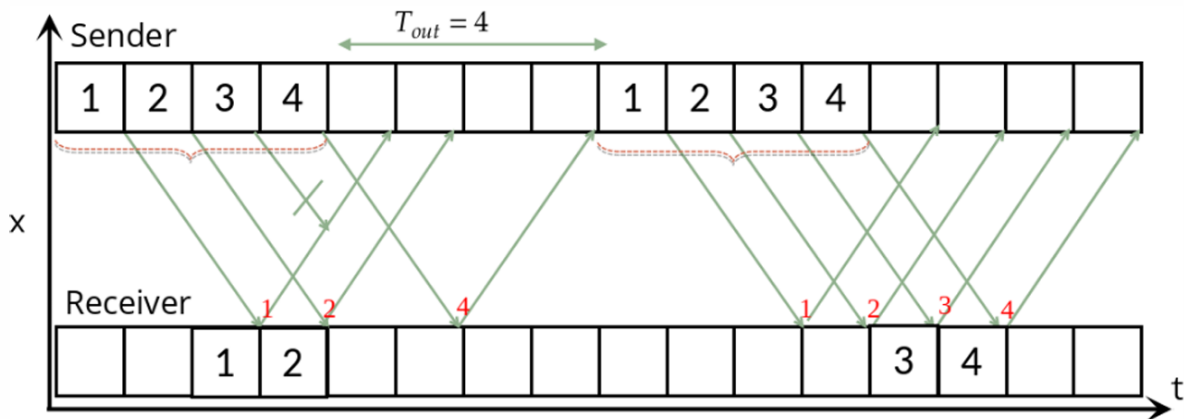
מתחילים את T_{out} מהרגע שסיימנו את השידור האחרון של החלון, ולכן מספיק $2T_{prop}$. מאחר שזמן ה- T_{ack} זניח, אפשר להגיד שכל ה-ACKs מתקבלים במקביל.

מעבשיו אפשר להניח $T_{prop} = 2 \text{ units}$, $L = 4$, $T_{out} = 2T_{prop}$.

שאלה 2

Question 2: Draw the packets and ACKs transmissions till packet #4 is successfully received, when packet #3 fails in its first transmission and all other packets in the window are successful (got ACKed)

פתרון:



מאחר ש- T_{prop} הוא 2 הוא יתקבל אצל המקבל בסלוט השלישי, 2 אחריו ו-3 כאמור לא מגיע. לאחר שעבר T_{out} השולח מבין שלא קיבל אישור על 3 ולכן יתחיל את השידור של כל החלון מחדש.

מה הסיכוי שחלון בגודל L ייכשל?

נחשב מה ההסתברות שפקטה כן תגיע: פקטה מגיעה בהסתברות $1 - p$. ההסתברות שכל החלון יגיע הוא $(1 - p)^L$. אנחנו רוצים את הסיכוי להיכשל ולכן ניקח את ההסתברות המשלימה: $1 - (1 - p)^L$.

שאלה 3

מה ה-Goodput? אפשר לפי (שידורים מוצלחים) ÷ (זמן שידור) או לפי (זמן שבו שידרתי בהצלחה) ÷ (כמה זמן ייקח לכל השידורים המוצלחים להגיע). נחשב לפי הדרך השנייה:

$$\begin{aligned} \eta &= \frac{\mathbb{E}[\text{Effective sending time}]}{L \cdot T_{packet} + 2t_{prop}} \\ &= \frac{Pr(\text{window failed}) \cdot 0 + Pr(\text{window succeeded}) \cdot L \cdot T_{packet}}{L \cdot T_{packet} + 2t_{prop}} \\ &= \frac{L \cdot T_{packet}(1 - p)^L}{L \cdot T_{packet} + 2t_{prop}} \end{aligned}$$

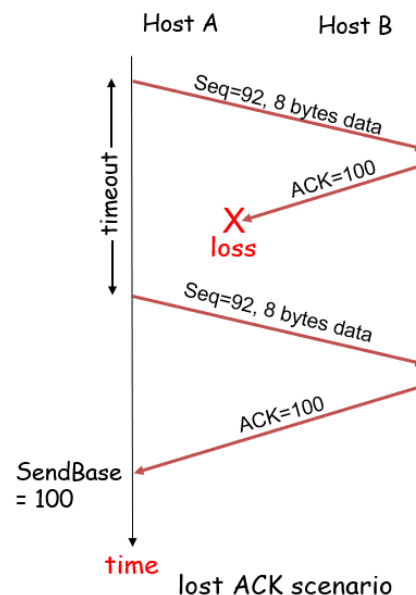
במכנה יש $2T_{prop}$ כי מחכים שהשידור האחרון יגיע כעבור T_{prop} והאישור על ההגעה יגיע כעבור T_{prop} נוסף.

התוחלת היא ההסתברות שהחלון יבשל כפול הרווח מכישלון (0), ועוד ההסתברות שהחלון יצליח להגיע כפול רווח הזמן מההצלחה (L כפול T_{packet}).

בשיעור שעבר דיברנו על TCP ואמרנו שהוא מייצר חיבור מקצה לקצה, אחד השירותים ש-TCP מציע זה שהוא מבטיח שמידע יגיע לצד השני ושהאפליקציה תקרא אותו בסדר שבו הוא נשלח (בזמן ש-IP הוא פרוטוקול best effort, כלומר לא מבטיח שהמידע יגיע לצד השני).

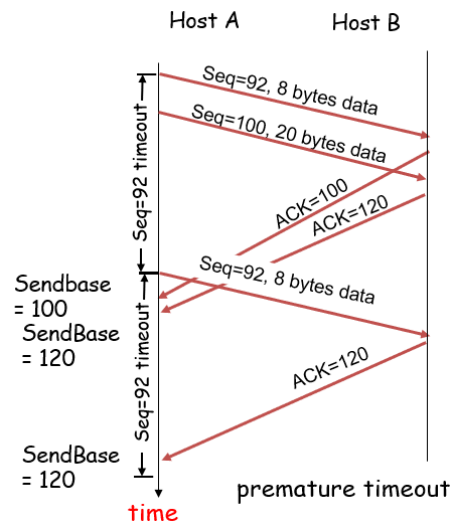
כשהשולח שולח הודעה הוא מצרף seq #. כשהמקבל מקבל מידע הוא שולח חזרה ACK שאומר מה הביט הבא שהוא אמור לקבל על אותו חיבור. בנוסף, השולח מתחיל טיימר (אם אין אחד שכבר רץ) שכשהוא פוקע הוא מחליט שההודעה שלו לא תקבל ACK יותר ולכן כנראה נפלה בדרך. הדרך המתוארת לא כל כך יעילה - הטיימר הוא פידבק איטי מדי ובפועל היינו רוצים לשלוח שוב מידע שהלך לאיבוד בצורה מהירה יותר. השולח עם כל הודעה שהוא מקבל שולח ACK שאומר מה הביט הבא שהוא מצפה לקבל, אם המקבל לא קיבל את אחד הסגמנטים הוא כל הזמן ישלח ACK על הסגמנט האחרון שהוא כן קיבל לפי הסדר - אצל השולח נראה ACKים כפולים ואז TCP ישתמש בזה כסיגנל לאובדן ויחליט לשדר מחדש.

כאן רואים את מנגנון הטיימר בפעולה ומה הבעיות איתו. בצד שמאל:



יש Host A ששולח סגמנט שמתחיל ב-92 seq # ושולח 8 בתים. Host B שולח לו ACK שאומר שהוא מצפה שההודעה הבאה תתחיל ב-100 seq #. אי אפשר להבדיל אם מה ששלחנו כן הגיע ולא קיבלנו ACK או שפשוט לא הגיע. בניח ש-A לא מחכה מספיק זמן, הטיימר פוקע ולכן שולח את ההודעה מחדש (למרות שהיא הגיעה).

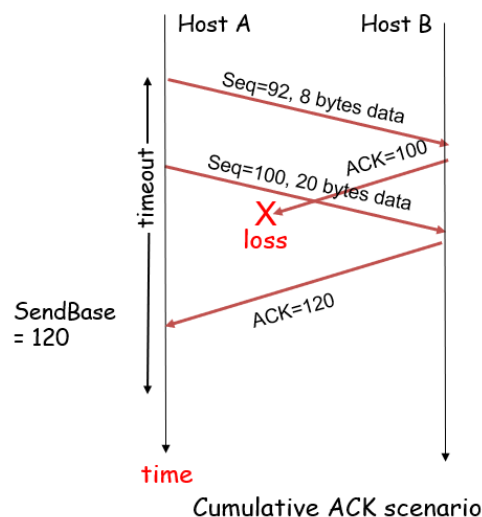
יש בצד ימין את Host B:



פרוטוקול TCP יכול לשלוח הודעות מאחת לשנייה בלי לחכות ל-ACK. בשביל תקשורת יעילה אנחנו לא נמתיין, ול-TCP יש אפשרות לשלוח אותן בצורה הזו. A שולח סגמנט ראשון עם 8 בתים ואז סגמנט נוסף עם 20 בתים. B מקבל את ההודעות ושולח ACK, אבל הטיימר של A פוקע מוקדם מדי (לפני הגעת ה-ACK הראשון על 100) ולכן מחליט לשדר אותו מחדש. A מחליט לשדר מחדש את הסגמנט הראשון ואז מתחיל טיימר מחדש (שוב). בזמן הזה הוא כבר מקבל ACK מההודעות הראשונות ולכן מתקדם, בהודעה הבאה יתחיל מ-120 seq#. זה שידור מחדש שיכולנו לחסוך עם טיימר גדול יותר.

שקף 20

נתאר מנגנון דומה אך עם שינוי: ה-ACK מאשר את כל מה שהגיע עד אותה נקודה, אפשר להבדיל אם ההודעה לא הגיעה או שה-ACK עליה לא הגיע:



ההודעה הראשונה הגיעה וה-ACK הלך לאיבוד. ב-pipeline נשלחת גם ההודעה השנייה (שמתקבלת וגם ה-ACK עליה מתקבל). מזה שהשולח קיבל ACK על 120 השולח יודע שגם ההודעה הראשונה הגיעה. זה ממזער את הבעייתיות שנובעת מההבדל בין הודעה שהלכה לאיבוד לבין ACK שהלך לאיבוד.

אם היינו מקבלים ACK על 92 זה נותן רמז לשולח שבקרוב נראה איך משתמשים בו.

Timeout

שקף 21

נתמקד בשאלה איך קובעים את ה-timeout. אם נקבע אותו להיות קצר מדי אז כל הזמן נשלח דברים מחדש וזה לא אידיאלי. אם נקבע אותו ארוך מדי אז זמן התגובה שלנו לשגיאות לא יהיה מספיק יעיל. הבעיה היא שאין round trip time קבוע לחיבור, זה מושפע ממסלולים על גבי האינטרנט שמשתנים, וכן מ-queueing delay בראוטר.

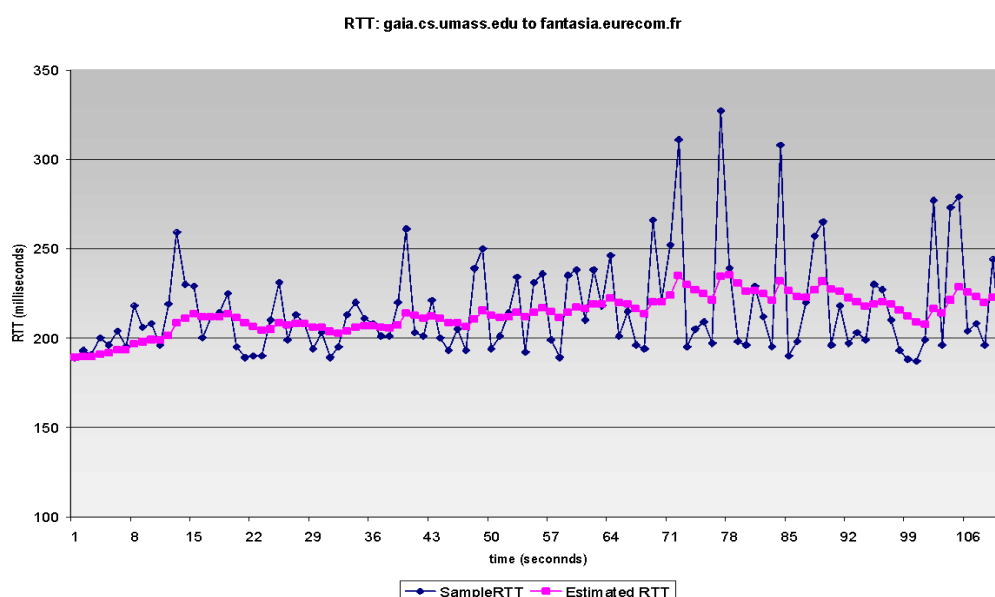
מה ש-TCP עושה זה לשערך round trip time בהתבסס על ההיסטוריה האחרונה. אנחנו מנסים להיות גם זולים במשאבים - לא נרצה להחזיק המון מדידות של החיבור ולמצע אותן, ומצד שני צריך משהו מספיק אקספרסיבי. נרצה לתת משקל יותר גבוה להיסטוריה שקרובה אלינו מתוך הנחה שהיא כנראה רלוונטית יותר.

שקף 22

הפרוטוקול מבצע ממוצע אקספוננציאלי של המדידות, יש משקולת למידע האחרון שהתקבל, ו-1 פחות זה היא המשקולת של ההיסטוריה, EstimatedRTT. בפעם הראשונה שלחנו סגמנט ושלחנו משקולת, זה יהיה הטריגר לעדכון של ה-EstimatedRTT. לרוב המשקולת היא $\frac{1}{8}$ ל-sample האחרון.

שקף 23

כאן רואים שה-estimated RTT חלק יותר מאשר ה-sampled:



שקף 24

יש לנו הערכה (ממוצע מוחלק) ל-RTT ונרצה לקחת קצת מעל זה (לקחת בדיוק זה לא מספיק טוב כי הרבה פעמים נקבל פידבק קצת אחרי). TCP מכיל משתנה נוסף שהוא devRTT - שערך השונות ב-RTT. גם השערך הזה הוא ממוצע אקספוננציאלי עם משקולת β שהיא $\frac{1}{4}$ והסטייה שהוא משערך זה כמה ה-sample RTT שונה או רחוק מה-estimated. ככל שהאסטימציה יותר גרועה נפצה על זה ב-devRTT גבוה יותר, ואז יהיה timeout interval גבוה יותר.

Duplicate ACKs

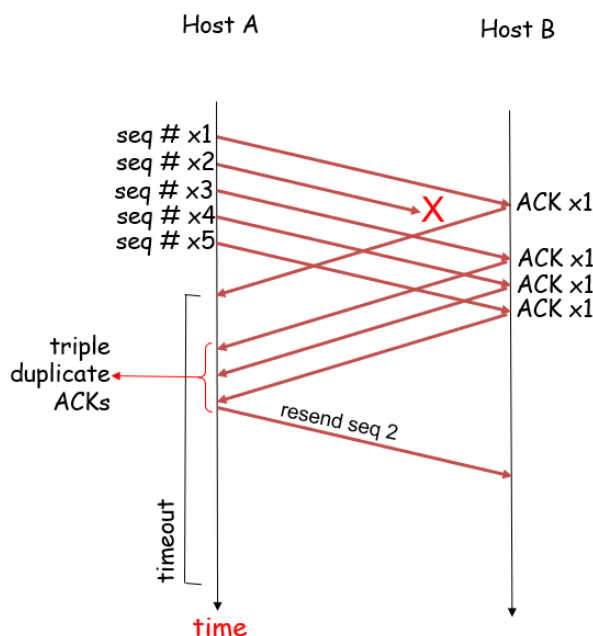
שקף 25

הצגנו את מנגנון ה-timeout של TCP, אבל כאמור הוא מנגנון די איטי בפידבק שלו (לא מגיב מהר להודעות). ראינו מקודם איך אפשר לעשות את זה מהר יותר - זה שקיבלנו ACK על משהו ישן אומר שהלכה לאיבוד הודעה בדרך.

זה המפתח למנגנון השני של TCP שנקרא Fast Retransmit (כי הוא מגיב מהר יותר ממנגנון ה-timeout). TCP מחפש ACK כפולים על הודעה מסוימת. זה יגיד שכל מה ששלחנו אחריה כולם יהיו על אותו מספר לפי ה-#seq האחרון שהתקבל. כש-TCP רואה 3 ACK כפולים הוא מחליט שההודעה שאחריה הלכה לאיבוד.

שקף 26

באן TCP שולח 5 הודעות ב-pipeline, לא מחכה לקבל ACK (נביח # seq בכפולות שוות):



B מקבל את ההודעה הראשונה ושולח עליה ACK, אבל לא מקבל את ההודעה השנייה. את השנייה הוא לא קיבל, אבל את השאר כן, לכן בשלב מסוים ימשיך לקבל ACK על ההודעה הראשונה. כשזה יקרה 3 פעמים הוא ידע לשלוח שוב את ההודעה השנייה, ואז כשיקבל עליה ACK הוא יבין שהכל התקבל (כי הוא כבר יהיה על המספר האחרון). יש באן שני מנגנונים שמתחרים - הטיימר מול מנגנון ה-ACK הכפול.

שקף 28

כמה הודעות אפשר לשלוח במקביל? הדרך הבטוחה ביותר היא לשלוח הודעה ואז לקבל עליה ACK ככה בסבב, אבל זה יגביל אותנו. מצד שני מוגבלים בתעבורה על פני ערוץ תקשורת. יכול להיות שאחד הראוטרים בדרך לא יכול להעביר מידע בקצב מאוד גבוה.

מנגנון Congestion Control של TCP אחראי לווסת את קצב השידור (כמה הודעות יכולות להישלח unACKED).

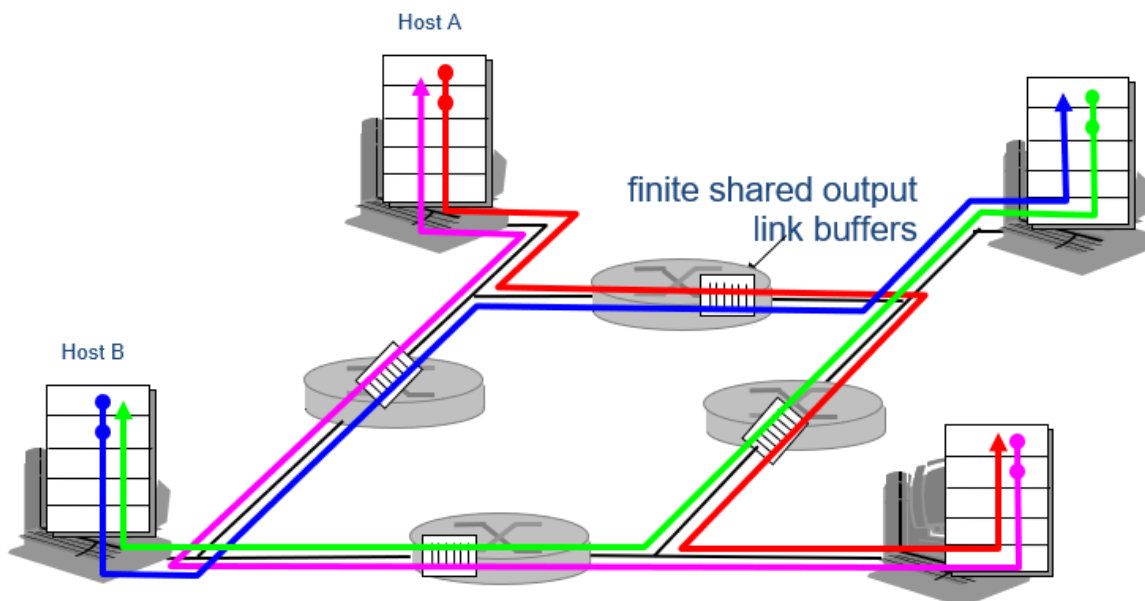
שקף 29

בעצם TCP מנסה להימנע ממצב של congestion - יש מקום כלשהו ברשת (אחד הראוטרים בדרך) שהוא מקבל יותר מדי מידע להעביר, יותר ממה שהוא מסוגל. יש שני דברים ש-congestion גורם להם:

1. **אובדן הודעות** - נדמיין שלראוטר יש תור של הודעות יוצאות שהוא אמור לשלוח ל-hop הבא. הבאפר מוגבל והוא מתישהו נגמר, לכן אין ברירה אלא לזרוק הודעות.
2. **latency גדל** - מאחר שהתורים גדלים בראוטרים בדרך, ה-queueing delay עולה ולכן RTT עולה.

שקף 30

דוגמה שממחישה למה הבעיה קשה (מהקשות בתקשורת). השולחים ברשת לא מסונכרנים, יש מערכת מבוזרת לגמרי עם אפליקציות שקמות ומסיימות לשלוח, והן לא מתואמות אחת עם השנייה. עדיין צריך בדרך כלשהי לחלק משאבים של הראוטרים בדרך באופן יחסית הוגן.



שקף 31

בסוף אובדן של פקטות. מה שיכול לקרות אם TCP לא עובד כמו שצריך זה שמנגנוני reliable transmit יפלו - הם יראו שהודעות אבדו וישלחו אותם מחדש אבל בכך מעמיסים את הרשת עוד יותר וגורמים לאובדן גדול יותר. יש לולאת פידבק שמחריפה את הבעיה, זה נקרא congestion collapse.

שקף 32

התקרית המתועדת הראשונה של עומס באינטרנט קרתה בין שתי מעבדות מחקר בקליפורניה שבאופן כללי החיבור שלהן עם throughput של 32 Kbps ירד ל-40 bps. בסביבות הזמן שבו זה קרה מספר המחשבים ברשת גדל משמעותית. זה מתאר למה הבעיה היא בעיה אקוטית וקשה באינטרנט. אותם מחשבים לא מתואמים ביניהם.

שקף 33

Flow Control

כאן יש את תיאור המנגנון היחיד באותה תקופה שנועד למנוע מהמקבל להיות תחת עומס. המקבל צריך buffer כדי לסדר את ההודעות לפי seq# למקרה שהגיעו לא לפי הסדר, לפני שיוכל לשלוח אותן לאפליקציה. בעצם המקבל גם מפרסם בהודעות שלו את ה-flow control window, מה גודל ה-buffer שנשאר לו בשביל לקבל הודעות בעתיד. זה אומר לשולח שאם תשלח יותר מ-X מידע לא יהיה לי איך להכיל אותו. המידע הזה יימסר על גבי ה-ACK וזה אומר לשולח איך להעביר את שארית המידע (ה-unACKed).

בעקבות האירוע הזה פיתחו את congestion control, הבינו שלא רק המקבל הוא ה-bottle neck אלא גם הראוטרם שבדרך עלולים להוות צוואר בקבוק.

שקף 34

הסיבה ש-flow control יחסית פשוט זה כי הוא תלוי רק בשולח ובמקבל, לעומת congestion control שלוקח בחשבון את כל הגורמים.

שקף 37

גישות לטיפול ב-Congestion Control

העולם של מנגנונים לביצוע congestion control מתחלק לשניים:

- **End-to-end congestion control**: חלק ממומשים בנקודות הקצה - השולח מקבל פידבק מהמקבל, ולפי אותו פידבק מחליט באיזה קצב שידור לשדר, הרשת שביניהם היא Blackbox, כמו TCP.
- **Network-assisted congestion control**: מנגנונים אחרים דורשים תמיכה של הראוטרם בדרך. למשל ראוטר בדרך יכול לזהות שהוא מתקרב לנקודה שבה הוא נהיה עמוס (תור יציאה הולך ומתארך) - כלומר לאחרונה קיבל מידע בקצב יותר מהיר ממה שהוא יכול לשדר החוצה. מנגנונים אלה נותנים אפשרות לאותה לשולח להוריד את הקצב.

No Congestion Control

שקף 38-40

נרחיב על מנגנוני end-to-end. עד עכשיו בקורס הזכרנו שני מימושים ל-transport layer. מימוש אחד בדמות UDP שמייצג class שלא מבצע congestion control. ב-UDP ההודעה פשוט נשלחת. אין מנגנון אצל השולח שמנסה לווסת את קצב השליחה.

יש מצבים שזה מה שעדיף לעשות. דוגמאות פופולאריות הן למשל Voice Over IP. יש קצב שידור שמצופה משיחה, אם נשלח יותר מהר זה לא יעזור (אנשים מדברים בקצב מסוים) ואם נשלח לאט מדי נגביר latency בין הצדדים. אפליקציות כאלה משתמשות ב-UDP ואם יש congestion ברשת או שהמקבל לא יכול לקבל את המידע, יהיה פשוט איבוד של פקטות וכשההודעה הבאה תגיע נתמודד עם החוסר.

Window-based Congestion Control

שקף 41

השיטה השנייה היא מנגנוני congestion control המבוססים על חלון שליחה. אמרנו שאפשר לשלוח הודעות ב-pipeline בלי לחכות ל-ACK. ככל שנשלח יותר קצב השידור גבוה יותר. חלון השליחה של מנגנון ה-congestion control זה כמה הודעות אפשר לשלוח בלי לקבל עליהן ACK.

נניח חלון בגודל W, כלומר אפשר לשלוח W הודעות ללא ACK. נניח שלאפליקציה תמיד יש מה לשלוח. עכשיו יש W סגמנטים באוויר שלא יודעים מה קרה איתם, מתישהו הם מתחילים להגיע למקבל. אחרי RTT נתחיל לקבל את ה-ACK. מה שקרה זה שהשולח מוגבל על ידי גודל החלון, הוא לא ניצל חלק מהזמן כי החלון התמלא.

איך נדע מה ה-W? כמה הודעות נכון לשלוח בכל RTT?

שקף 42

זאת החלטה שהשולח צריך לקבל, ההגבלה על קצב השידור ש-TCP אוכף על השולח זה

$$SourceRate = \frac{W \cdot MSS}{RTT} bps$$

(MSS = Maximum Segment Size)

אם נשלח פחות מדי זה בזבוז זמן כי אנחנו לא מנצלים את הרשת. אם W גדול מדי אז קצב השידור הוא יותר ממה שהרשת מסוגלת לעמוד בו, וזה יגורם ל-congestion.

שקף 43

בשקף זה מופיעים מימושים קלאסיים של TCP Congestion Control.

מה שמשותף לכל אותם מימושים של TCP (לכל מנגנוני ה-congestion control) זה שהם מחולקים לשני חלקים:

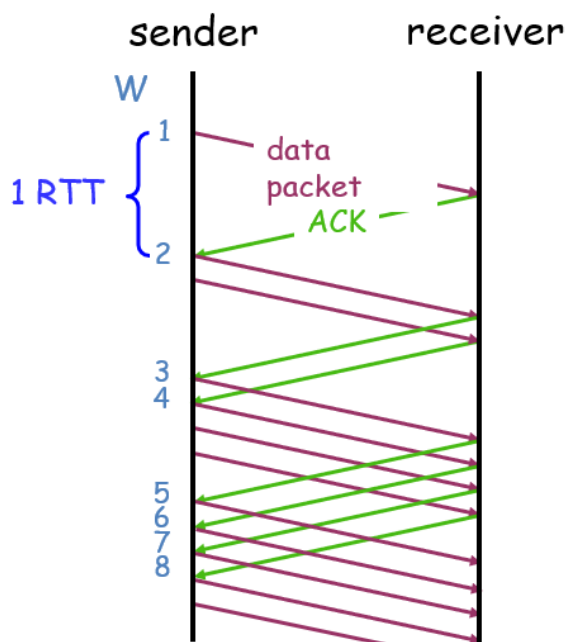
1. Slow Start (SS)
2. Congestion Avoidance (CA)

ברגע שקצב השידור עובר קצב מסוים SS Threshold החיבור עובר למצב השני של CA, ואז ההתנהגות של חיבור TCP הופכת להיות שונה.

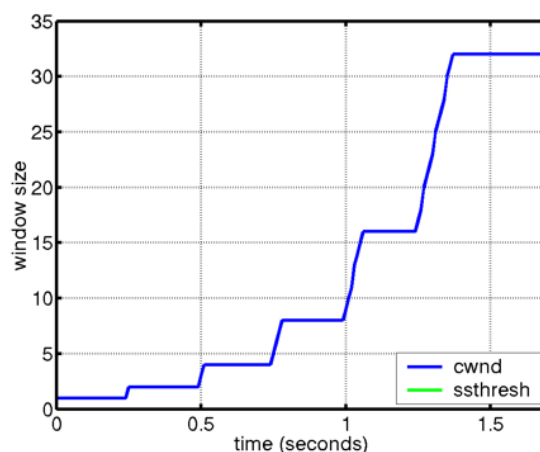
Slow Start (SS)

כל המנגנונים כמעט ולא נבדלים ב-SS, זה אומר שאנחנו מתחילים חיבור עם $W=1$ ועל כל ACK שנקבל אנחנו נגדיל את W ב-1. זה אומר שכל RTT בעצם W גדל פי 2. כשאנחנו חוצים רף מסוים שנקרא CA Threshold עוברים ל-CA.

כאן רואים דוגמה של התרחבות החלון לאורך זמן. עם הגעת ה-ACK השני החלון גם זז ימינה וגם גדל אז אפשר לשלוח עוד הודעות (4 לעומת 2 ב-RTT הקודם). האפקט של זה הוא שכל RTT אנחנו בעצם מכפילים את קצב השידור, גדילה אקספוננציאלית. מהר מאוד נגיע לקיבולת הערוץ.



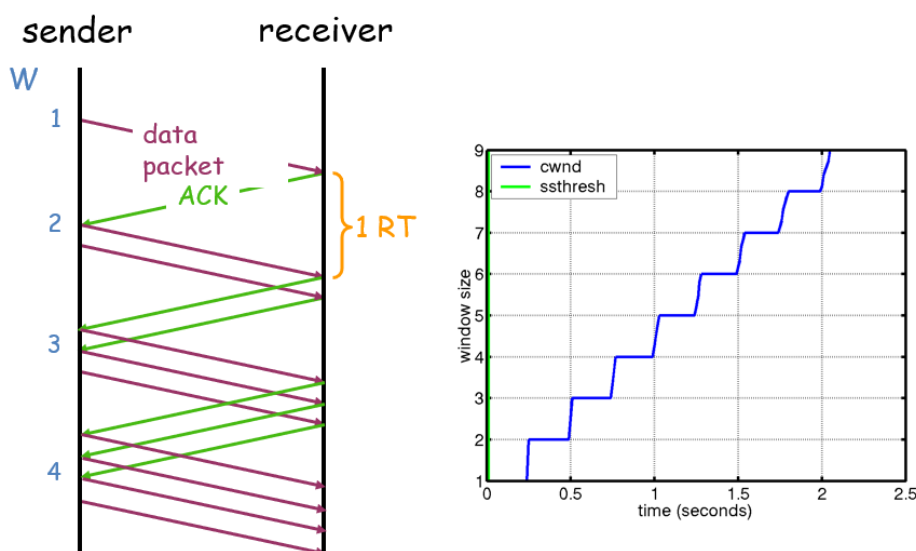
$$W \leftarrow W + 1 \text{ (for each ACK)}$$



מתישהו W יגיע לסף שהגדרנו ונעבור לשלב Congestion Avoidance שבו גדילת החלון הרבה יותר איטית.

על כל ACK שמתקבל נגדיל את החלון ב- $1/W$.

במצב CA עדיין נעלה את קצב השידור, אבל הרבה יותר לאט.



$$W \leftarrow W + 1 \text{ (for every } W \text{ ACKS)}$$

מנגנון שהציעו ב-88' שנקרא **TCP Tahoe**, הוא שב-CA נגדיל את קצב החלון לאט, אבל מתישהו עדיין נעבור את קיבולת הערוץ. כשזה קורה נוריד בצורה דרסטית את קצב השידור ונחזור ל-SS.

המנגנון אומר שאם הודעה הולכת לאיבוד זה סימן שהיה congestion (ראוטר בדרך היה עמוס, לא משנה איזה). נגיב לזה בכך שנוריד את קצב השידור שלנו. "הודעה הלכה לאיבוד" זה לפי ההגדרות של TCP לאובדן: אם ה-timer פקע או שקיבלנו 3 duplicate ACKs. הסקת המסקנות היא שהודעה הלכה לאיבוד כי שלחנו מהר מדי – צריך להוריד את קצב השידור.

בדוגמה כאן הודעה 4 לא הגיעה ולכן 5,6,7 נותנים ACKs כפולים על הודעה 3. במקרה הזה נוריד את קצב השידור:

Packets



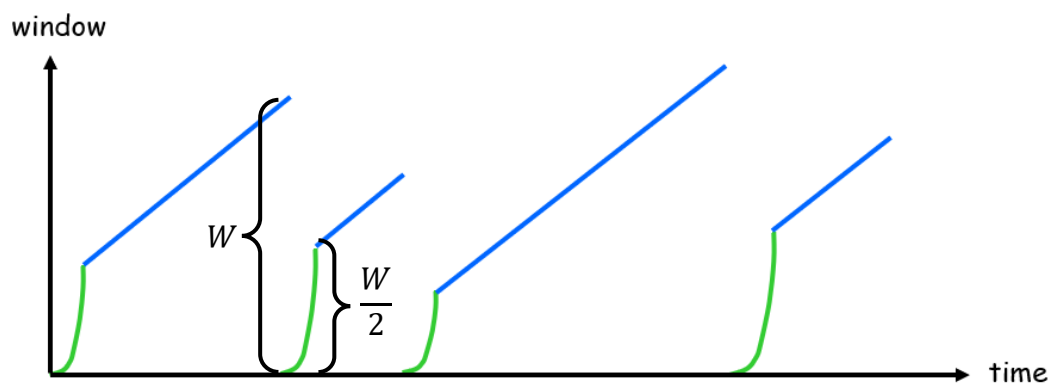
Acknowledgements



שקף 52

איך נגיב לאובדן לפי TCP Tahoe? מורידים אותו בצורה די דרסטית. מכוונים את $ssthresh$ להיות חצי מהחלון הנוכחי, ומקטינים את החלון להיות חזרה $W = 1$. עכשיו נגיע שוב למצב של SS כי $W < ssthresh$, אבל הפעם נעצור מוקדם יותר – בחצי החלון שעצרנו בו קודם.

שקף 50 (הוצג בסדר הפוך בהרצאה)



SS: Slow Start
CA: Congestion Avoidance

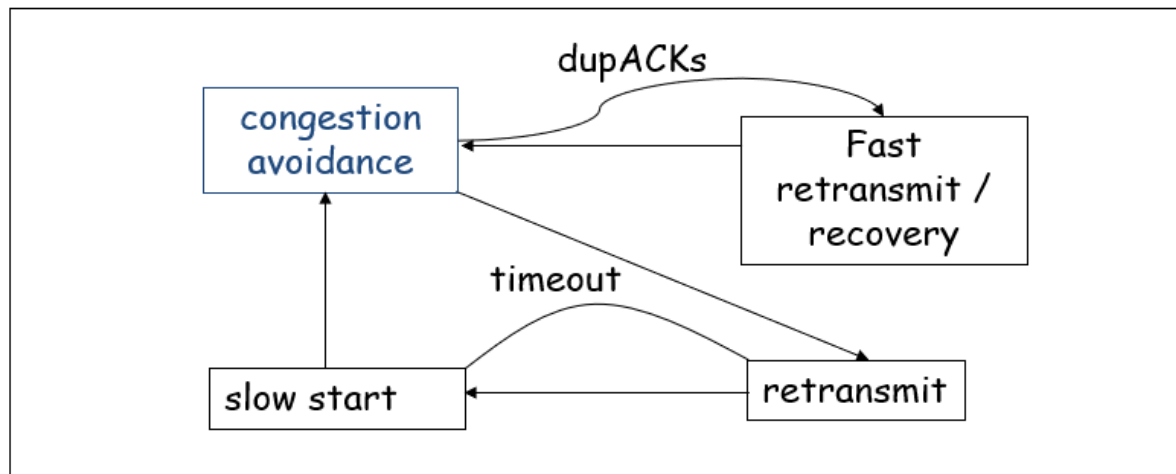
כאן מופיע גודל החלון כתלות בזמן. בכל פעם החלון יגדל אקספוננציאלית ואז לינארית, נקודות המעבר הם מצבים שבהם אבדה הודעה. הוספתי בגרף סימון שמראה את העדכון של חצי גובה החלון.

שקף 53

TCP Reno

השינוי הבא שעשו הוא **TCP Reno**. כאן לא מעוניינים להתחיל כל פעם עם $W=1$ מחדש ולהיכנס למצב של SS . האובזרווציה כאן היא שיכול להיות שיש הבדל בין מצב של ACK כפול לבין מצב של טיימר שפקע. אותם ACK כפולים נשלחים כתגובה למשהו (ראיה לזה שההודעות מגיעות, פשוט אחת הלכה לאיבוד). במקרה הזה לא צריך להסתכל על זה בעומס על הרשת אלא פשוט הודעה שהשתבשה.

העדכון באן מתייחס אחרת לכל אחד מהמצבים (פקיעת timer או dupACKs):



המנגנון נקרא Additive-Increase-Multiplicative-Decrease או בקצרה **AIMD**. משמעותו כשאנחנו ב-CA יש לנו additive increase בגודל החלון, אבל על 3 dupACKs נוריד את קצב השידור בצורה כפולית.

מציג מנגנון חדש שנקרא fast recovery: מקודם ב-TCP אמרנו שיש מנגנון של fast retransmit (לא מחכים ל-timeout, שולחים מיד כשנתקלים ב-3 dupACKs). TCP Reno משלב fast recovery שאומר אם קיבלתי 3 ACK כפולים אני אחצה את קצב השידור ב-2 ואמשיך הלאה. לעומת זאת, אם היה timeout יתנהג כמו Tahoe.

```

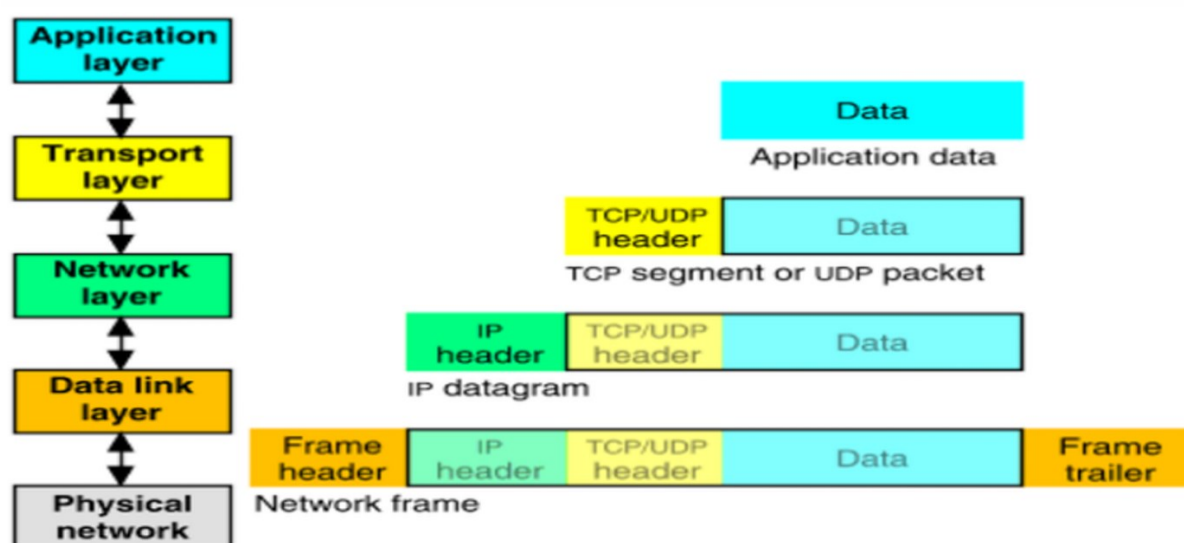
for every ACK {
    W += 1/W (Additive Increase)
}
for every 3 dupacks {
    ssthreshold = W/2
    W = W/2
    (Multiplicative Decrease)
}
for every timeout {
    ssthreshold = W/2
    Enter Slow Start (W = 1)
}
  
```


סיכום של TCP Reno. בכל ACK אנחנו מגדילים את החלון בצורה לינארית (additive increase). כשקיבלנו 3 ACK כפולים אנחנו נכוון את ssthresh להיות $W/2$ (כמו ב-Tahoe), רק שבמקום להחזיר $W = 1$ אנחנו חוצים אותו (multiplicative decrease). השוני הוא שבאן אנחנו **נשארים ב-CA**, כי עברנו את ה-threshold – אבל חצי מקצב השידור.

אם היה timeout נתנהג כמו Tahoe, גם חוצים את ssthreshold וגם נכנסים ל-SS עם $W = 1$.

שכבת ה-Transport

כפי שהזכרנו, בכל רמה יש data שנרצה לשלוח. שכבת האפליקציה מסתכלת על הדאטה בצורה האבסטרקטית ביותר, למשל לוקחים דף html ומעבירים אותו ל-transport layer לכרטיס הרשת, שם יתווסף לו header. שכבת ה-transport תוריד אותה ל-network layer שם יקבל header נוסף, יועבר ל-network layer הנכון ובתוך אותו רשת יצטרך להעביר לשכבת ה-data link שתעניק לו header נוסף – ובסוף לשכבה הפיזית.



נדבר על שני פרוטוקולים: TCP ו-UDP. אנחנו מוסיפים מורכבות על שכבה 3 כדי לאפשר שירות בין תהליכים בשכבת האפליקציה. הפרוטוקולים האלו הם Reliable (אם משהו נאבד בדרך נשחזר אותו, לעומת שכבה 3 שהיא "best effort" לא מבטיחה את מהימנות הדאטה וייתכן שפקטות כלל לא הגיעו, או הגיעו בסדר לא נכון).

נתמקד בנושאים של Connection Management, Flow Control, Congestion Control.

Logical Port

עד עכשיו ב-Layer 3 מחשב A רצה לדבר עם מחשב B. בפועל, יכולה להיות תקשורת גם בתוך הדפדפן ולבצע תקשורת במקביל. אנחנו נצטרך לדעת להבדיל איזו פקטה משויכת לאיזה תהליך, ולשם כך נועדו **Ports**.

בעבר Port היה חיבור פיזי לעולם החיצוני (בראוטר למשל), כאן ה-Port הוא לוגי בלבד, מחליטים על מספר 16-ביט, ובכל פורט אפשר לשים מזהה ייחודי לשיחה ספציפית מהמחשב שלי למחשב אחר. אם מקבלים פקטה חזרה אנחנו נדע גם לאיזה פורט לשייך אותה.

ב-UDP אנחנו נדבר על מצב connectionless, כלומר לא יוצרים שיחה בין שני קצוות אלא מבקשים זרימה של דאטה, וכל התקשורת תהיה מהצורה (destination IP, destination port).

ב-TCP יוצרים חיבור בין הקצוות. מה שיבדיל בין שיחה אחת לאחרת (בסוקטים שונים) יהיה גם source IP ו-source port number, ולכן ה-tuple הוא: (source IP, source Port, destination IP, destination port).

פונים לשרת ורוצים להתחיל לקבל הודעות, צריך לפתוח פורט (לוגי) שבו אנחנו מודיעים שאם שולחים הודעות לפורט הזה אז נשיב באותה שפה, זה נקרא binding. למשל אם יש שרת web והולכים לפורט מספר 80, השרת ידע שמה שביקשנו זה דף אינטרנט. יתכן שאותו מחשב גם משתמש בשירות VPN, אם רוצים לפעמים לשנות אותו אפשר לשנות את קבצי ה-HTML, בהתאם למקור ממנו הגיעה הבקשה נוכל להפריד מה אנחנו רוצים לעשות.

Binding זו פונקציה שיוצרת חיבור ואז דברים יקרו אם יגיעו פקטות לפורט. יש 1024 פורטים ששמורים לפרוטוקולים מוכרים: HTTP – 80, HTTPS – 443, SSH – 22, DNS – 53.

User Datagram Protocol – UDP

ההבדל מ-IP

אנחנו מעוניינים לשלוח הודעות IP למחשב אחר. לא נבטיח הרבה מעבר למה שכבר מובטח בפרוטוקול IP למעט החידושים הבאים:

- Multiplexing/Demultiplexing – אפשר לבצע כמה שיחות שונות. נסיף source and destination port numbers כדי לאפשר תקשורת מכוונת אפליקציה.
- Packet Integrity – ב-Layer הקודם היה checksum, אבל זה וידא רק את ה-headers ואנחנו רוצים לוודא גם את הדאטה עצמו.

לגבי source port – לא מאוד קריטי ב-UDP, אנחנו לא מצפים לקבל תשובה בחזרה (כי זה לא דו-שיח). אין משמעות ל-source port כי הוא יוכל להחזיר תשובה ל-port פנוי אחר. בכל זאת נשים לפעמים source port למשל עבור פרוטוקול DNS שנועדה להגן מפני DNS Poisoning.

Transmission Control Protocol – TCP

עקרונות הפרוטוקול

באן תהיה ממש תקשורת מבוססת connection כמו שיחה, ונבטיח יותר מאשר UDP:

- גם באן (כמו ב-UDP) יהיה checksum.
- מנגנון של Acknowledgements & Retransmission – אישור של הודעות ושליחה חוזרת שלהם.
- מספרים סידוריים כדי לזהות אובדן פקטות.

- Congestion Control – מה קורה כשמעמיסים על הרשת יותר מדי? נראה איך הפרוטוקול מסייע.
- Flow Control – למחשב המקבל יש גודל חלון, ואם פקטות מגיעות לא בסדר הנכון זה בסדר כל עוד עד שהחלון מקבל הוא מקבל את כל הפקטות החסרות.

התחלת Connection

מעוניינים להתחיל Session עם מישהו, מגדירים במפורש איך פותחים אותה ואיך סוגרים אותה. אנחנו לא יודעים במדויק מה יהיה ה-bandwidth או ה-delay מאחר שהם תלויים ברשת.

ה-ACKs שנקבל הם קומולטיביים (מצטברים) כמו ב-Go Back N.

שליחת דאטה ב-TCP

דרך אחת בה יכולנו לשלוח את הדאטה היא לשלוח ביט, לקבל אישור, להעביר ביט נוסף, לקבל אישור.. ברור לנו שהתקורה של ה-ACK גדולה מדי. אנחנו רוצים לארוז הרבה בתים יחד בשליחה אחת ולשלוח פקטות בזו אחר זו.

האם יש גודל מקסימלי? כן. למה שלא נשלח סגמנט בגודל של כל ההודעה? השכבות מתחת מכילות גודל פריים מקסימלי שהגדירו (MTU) ולא נוכל לחרוג ממנו. לכן גודל הפקטה מוגבלת על ידי גודל הפריים.

נגדיר מושגים:

MSS – עבור Maximal Segment Size, נמדד בביטים. מדובר בגודל המקסימלי של פקטה בשכבה 4.

MTU – עבור Maximal Transfer Unit, נמדד בביטים. מדובר בגודל המקסימלי של פריים בשכבה 2.

מה היחס בין שני הגדלים?

$$MSS < MTU$$

כי לפקטה בשכבה 4 מוסיפים גם תקורה נוספת, ואסור לנו לחרוג מגודל ה-MTU.

המחיר שכרוך ב-Reliability

אנחנו מגדירים מספר דרכים כדי לאפשר את האמינות בתקשורת:

- ביצוע checksum שמוודא שהחבילה הגיעה כמו שנשלחה.
- הודעה לשלוח על קבלת החבילה (ACK).
- העברת הדאטה לשכבה הבאה לפי הסדר.

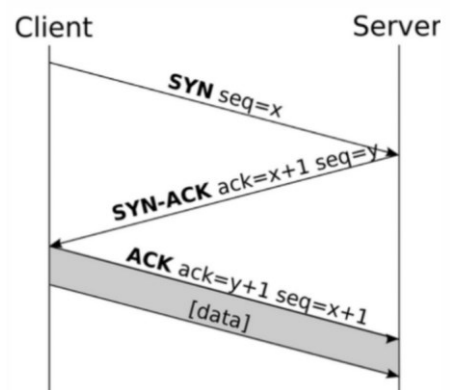
לפעמים נגיד שאנחנו לא **חייבים** את כל ההגנות הלאה, מתי בכל זאת נוכל להסתפק ב-UDP? שיחת Zoom, streaming.. לעומת זאת כשמורידים משהו מהאינטרנט או כשמקבלים מייל – אנחנו נעדיף לחכות שהדאטה יעבור וידוא שהוא הגיע בצורה תקינה.

פתיחת תקשורת TCP

אזהבים לחשוב על זה כמו דו-שיח בין שני מחשבים שביניהם יש צינור שעובר בו מידע. בפועל זה לא בדיוק כך, יש ראוטר ביניהם, שמעביר לספקיות אחרות ועוד ראוטרים, עד למחשב השני – אבל אנחנו רוצים להישאר בראייה שעל שכבה 4 בלבד. הבעיה באבסטרקציה הזו היא שלא נוכל לדעת מה העומסים.

בכל פעם נשלח הודעה עם מספר סידורי, ונקבל ACK על המספר הזה. צריך להסכים על מספר התחלתי בין הצדדים (לרוב לא מתחילים מ-0 כדי להקשות על זיוף בין הצדדים).

זה נקרא "לחיצת יד משולשת":



כשהמשתמש (יוזם השיחה) רוצה לדבר עם השרת (המאזין) הוא ישלח SYN יחד עם מספר סידורי x שמטרתו לסנכרן. השרת יקבל את זה בזמן שהוא יבצע listen וימתין. עם קבלת ההודעה הוא מבין שיש ניסיון ליזום תחילת התקשורת, לכן אם אפשרי יחזיר SYN-ACK עם מספר סידורי משלו y בתוספת ACK על ההודעה של הקליינט. הקליינט יחזיר ACK על ה- y , ומשם תתחיל תקשורת רציפה בין השניים.

[שאלה 1] מה קורה אם פקטות ה-SYN הולכו לאיבוד?

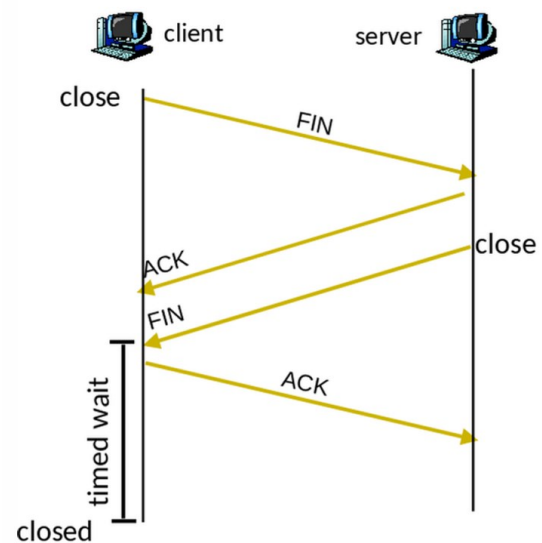
[תשובה] הצד השני לא יקבל כלום (ולכן כמובן לא ישיב כלום). מי ששלח את ה-SYN ימתין אבל בסוף יגיע timeout, הוא יבין שההודעה לא נענתה ולכן ינסה שוב. אם ה-SYN-ACK הוא זה שלא הגיע, מי שהמתין ל-ACK יחשוב שההודעה לא הגיעה ולכן ישלח אותה שוב, ואז הצד השני יבין שה-ACK שלו הלך לאיבוד וישלח אותו פעם נוספת.

[שאלה 2] איך אמורים להגדיר את ערך ה-timeout?

[תשובה] אין באמת דרך לדעת כמה רחוק נמצא הצד השני. לרוב בוחרים 3 או 6 שניות. זמן ה-timeout יתעדכן בצורה דינמית (מרחיבים במסגרת ההרצאות על הנושא).

סגירת תקשורת TCP

בסיום התקשורת היוזם שולח הודעת FIN שתיענה גם היא בהודעת ACK ואחריה FIN מצד השרת, ואז מסיימים ב-ACK של השולח.



זה לא מאוד קריטי כי אם אנחנו באמצע תקשורת ורוצים לסיים אפשר פשוט לכבות את המחשב וזה יסגור את התקשורת, אי אפשר להכריח שהתהליך הזה יקרה כך. ואמנם, לפעמים נרצה להמתין כדי לוודא שכל המידע ששלחנו הגיע.

TCP Sliding Window

התקשורת ב-TCP מבוססת על sliding window כדוגמת אלה שראינו (Selective Repeat, Go Back N). ברגע שהחלון התמלא ולא קיבלנו על ההודעות שבו ACK – לא המשכנו לפקטות הבאות אלא המשכנו לשלוח את אלה שאנחנו חושדים שהלכנו לאיבוד.

באן גודל החלון משתנה לפי הצורך, והוא מושפע משני גורמים:

- גודל ה-buffer של המקבל (Flow Control). מדובר במגבלה חומרית.
- העומס הנוכחי על הרשת (Congestion Control). אי אפשר לדעת בדיוק מהו, אבל נראה שיטות להערכה של המצב.

Flow Control

אנחנו לא יודעים מה מספר ההודעות שהמקבל יכול לאגור עד שהוא מתמלא ודורש את הפקטות שהלכו לאיבוד. אנחנו כן יודעים את כמות ה-ACKs שקיבלנו, ואת זה נסמן **rwnd** (עבור receiver window).

$$\text{Window size} = \text{LastByteSent} - \text{LastByteACKed} \leq \text{rwnd}$$

אם שלחנו בייט מספר 10 וקיבלנו ACK על בייט מספר 5, זה אומר שגודל החלון הוא לכל הפחות 5.

[שאלה] מה עושים אם גודל החלון הוא $rwnd = 0$?

[תשובה] נרצה פשוט להמשיך לשלוח את הפקטה הבאה כל כמה זמן בתקווה שמתישהו היא תתקבל, ואז כשנקבל ACK נוכל להגדיל את החלון. בסופו של דבר ה-buffer של המקבל יתפנה והשולח יוכל שוב להאיץ בקצב השליחה.

Congestion Control

עומס הרשת תלוי באיך שנחליט לשלוח את המידע שלנו, ככל שהרשת עמוסה יותר – יותר דברים יאבדו ברשת, ונוכל לדעת את זה רק לפי ה-ACK שנקבל (לכן אם היינו בעולם של UDP בלבד, לא הייתה דרך להבין מה רמת העומס ברשת).

ראינו בהרצאה דוגמאות ל-TCP Tahoe ו-TCP Reno, שמגדירים שני מצבים:

- Slow Start
- Congestion Avoidance

הרעיון הוא להתחיל עם קצב איטי אבל הוא יעלה בצורה מהירה, ואז להגיע למצב שבו אנחנו בקצב גבוה, אבל הוא יעלה בצורה איטית.

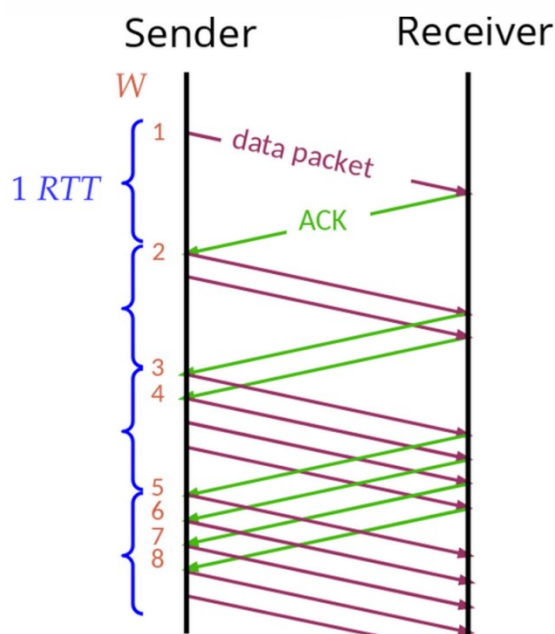
הרעיון

מחליטים על Slow Start Threshold (ssthresh). מה שגודל זה לא הפקטה שאנחנו שולחים אלא גודל החלון, הגרף שנראה יהיה קצב השליחה של הפקטות (נגזרת של כמות הפקטות שנשלחו).

תהיה שיטה לשינוי ה-ssthresh וגם את גודל ה-congestion window, כלומר כמה פקטות אנחנו מוכנים להחזיק באוויר (כמות הפקטות באוויר שאנחנו מאמינים שהרשת יכולה להחזיק).

הגדרת מצב (SS) Slow Start

1. נתחיל בחלון בגודל כלשהו, למשל $MSS = 1$.
2. עבור כל ACK שנקבל נבצע: $W = W + MSS$. נבחין שזה בפועל מכפיל את גודל החלון בכל RTT, כי ההגדלה מתבצעת עבור כל ACK, ועבשיו אנחנו מרשים פי 2 יותר ACK פוטנציאליים.
3. אם $W \geq ssthresh$ נכנס למצב של Congestion Avoidance (CA).



נשים לב שעל כל ACK אנחנו מגדילים את החלון ב-1, אז בכל RTT גודל החלון גדל פי 2.

הגדרת מצב Congestion Avoidance (CA)

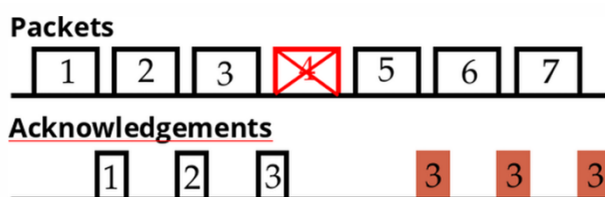
בשלב מסוים נזהה שיש כישלונות או כשאנחנו שולחים יותר מדי – זה שלב שבו נרצה להפסיק עם הגידול האקספוננציאלי של החלון. בשלב זה:

לכל N ACKs נגדיל את החלון ב-1. התוצאה היא גדילה איטית, הוספה של MSS אחד בכל RTT.

אובדן פקטות

מתי נרצה לשלוח שוב פקטה?

- כמובן, אחרי timeout.
- יש דרך לקצר את הזמן הזה – נסתכל על ACK שחזרו. נביח ששלחנו את 1,2,3 וקיבלנו עליהם ACK. המשכנו לשלוח 4,5,6,7 ואנחנו ממשיכים לקבל ACK על 3. ברגע שקיבלנו שלוש פעמים ACK על חבילה 3 – נבין שפקטה 4 אבדה.



הזיהוי הזה קורה מהר יותר מהזמן שהוגדר ל-timeout.

Fast Retransmission

המנגנון שהתחלנו לתאר נקרא Fast Retransmission:

- אחריו 3 ACK כפולים השולח ישלח את הפקטה הראשונה שלא קיבל עליה ACK.
- לאחר מכן, בהתאם לסוג ה-TCP שרץ (כמו למשל Reno או Tahoe) השולח יעבור ל-slow start או congestion avoidance, ישנה את ssthresh או כל דרך אחרת לפי איך שהגדיר.
- שינויים אלה אינם חלק מה-fast retransmission, אלא תגובה ל-ACK כפולים.

תגובת הפרוטוקולים השונים לאובדן פקטות

בעת אובדן פקטות, timeout או 3 ACK כפולים: גם Tahoe וגם Reno מגדירים $ssthresh = W/2$.

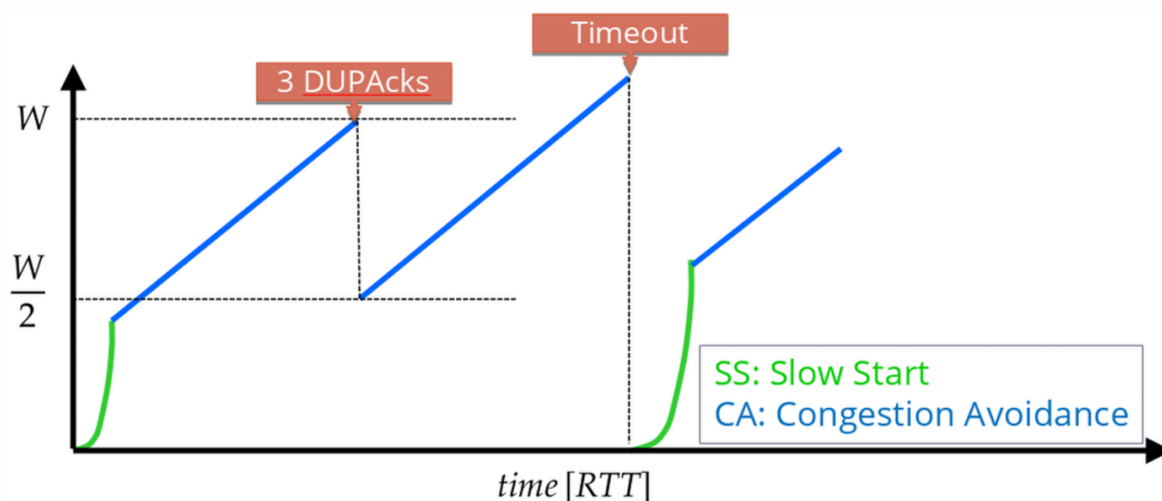
העדכונים האחרים משתנים בין הגישות:

	TCP Tahoe	TCP Reno
Timeout	1 Enter slow start 2 $W = 1$	1 Enter slow start 2 $W = 1$
3 Duplicate ACKs	1 Enter slow start 2 $W = 1$	1 $W \leftarrow W/2$ 2 Enter congestion avoidance

שני הפרוטוקולים מתנהגים זהה עבור timeout: נכנסים למצב SS ומעדכנים $W=1$.

אבל Reno מבדיל בין Timeout לבין duplicate ACKs (בעוד ש-Tahoe לא). Tahoe יעשה אותו דבר כמו שתיארנו, בעוד ש-Reno יעדכן $W \leftarrow W/2$ וייכנס למצב CA.

דוגמה לגרף ב-Reno:



דיברנו על שני פרמטרים, חלון המקבל (rwnd) וחלון ה-congestion (cwnd). איך נחליט מה יהיה החלון בפועל? נבחר את המינימום בין שניהם, כלומר גודל החלון בפועל הוא:

$$\text{LastByteSend} - \text{LastByteACKed} \leq \min\{\text{rwnd}, \text{cwnd}\}$$

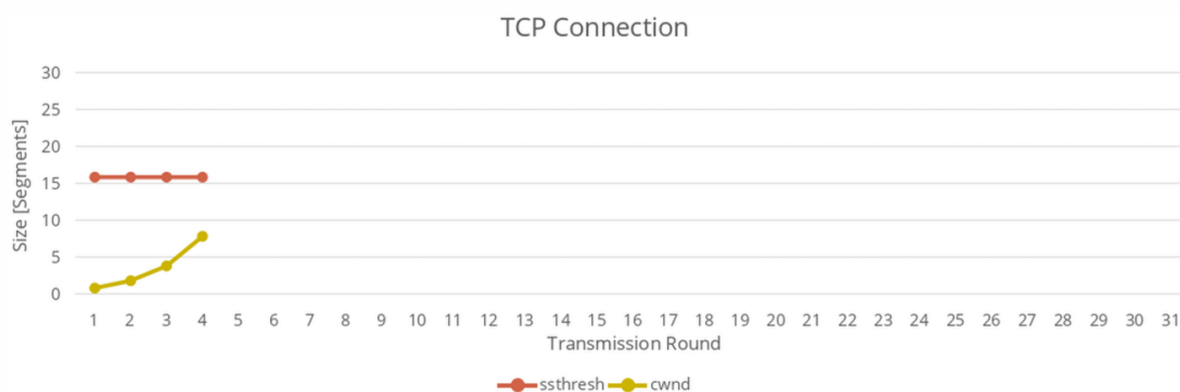
שאלות תרגול – TCP

לאורך השאלות נניח TCP Reno, ונצטרך להשלים את הדיאגרמות של ערכי cwnd כפונקציה של RTT.

שאלה 1

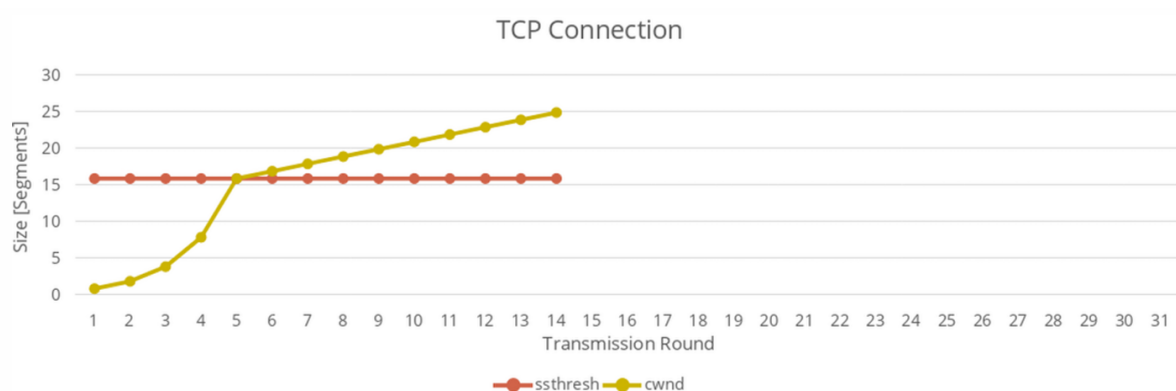
Assume that ssthresh=16 at the beginning.

What will be the next 10 points in the diagram, assuming all segments were ACKed?



פתרון:

נשים לב שבכל נקודת זמן גודל החלון מכפיל את עצמו, 1 ואז 2 ואז 4 – בסוף 8. אלו הנתונים. צריך לשרטט את 10 הנקודות הבאות כאשר אין איבוד בדרך. הנקודה הבאה בזמן 5 תהיה בדיוק 16, ב-ssthresh. נכנס למצב של CA ולכן נוריד את קצב גדילת החלון ל-1. מאותה נקודה נעלה לינארית:

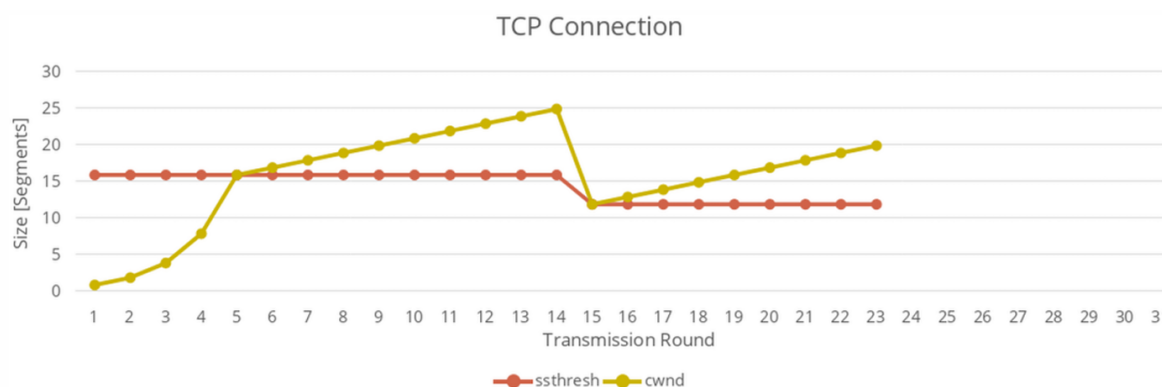


שאלה 2

מה יהיו 9 ההודעות הבאות, כשבזמן הבא קיבלנו ACK 3 כפולים?

פתרון

ב-Reno גודל החלון קטן פי 2, ונשארים במצב של CA. גודל החלון היה 25 לכן ירד ל-12 (עיגול מטה). מה יקרה ל-ssthresh? יעודכן להיות חצי החלון גם כן, כלומר 12 (לשים לב, לא מתעדכן להיות חצי מעצמו אלא חצי מגודל החלון).

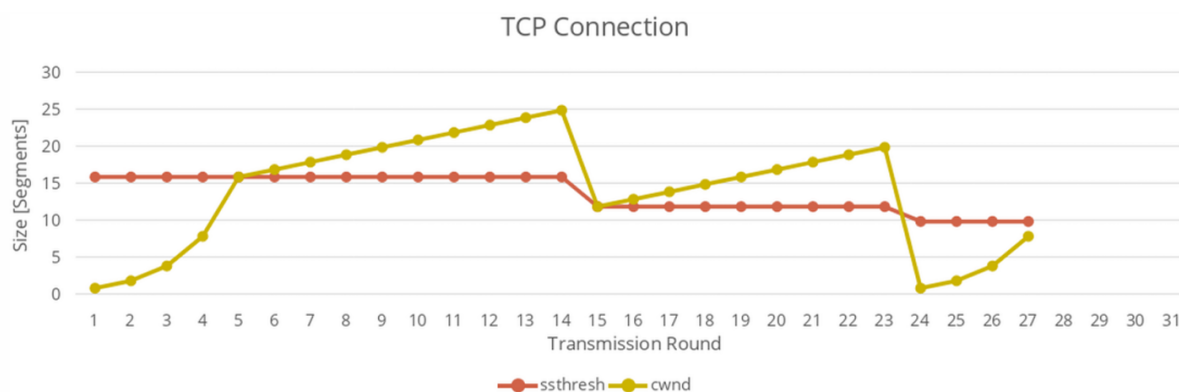


שאלה 3

השלימו את 4 הנקודות הבאות אחרי שהתקבל timeout.

פתרון:

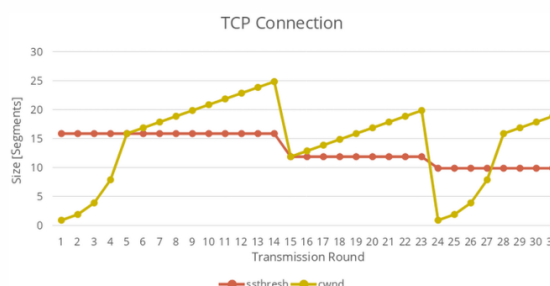
גודל ה-ssthresh יעודכן להיות חצי מגודל החלון הנוכחי כלומר 10, וגודל ה-cwnd יעודכן להיות 1:



שאלה 4

Given the following diagram, determine in which round each of the following segments were sent:

- Segment 6:
- Segment 33:
- Segment 40:
- Segment 70:

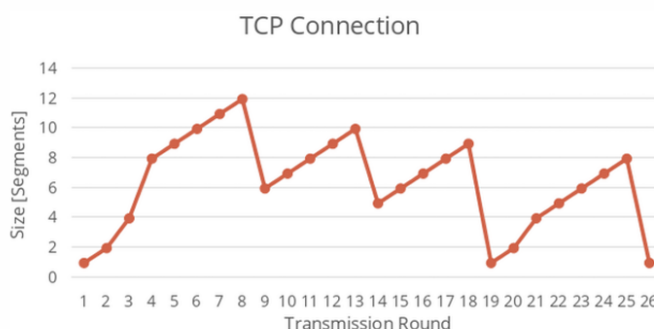


פתרון:

מתי פקטה מספר 6 נשלחה? נזכור שבאן לא רואים גרף של הפקטות אלא של הנגזרות, כלומר של גודל החלון. איך נדע מתי סגמנט 6 נשלח? סופרים את השטח מתחת לגרף, אבל לא נעשה אינטגרל אפשר פשוט לספור: בזמן 1 נשלחה פקטה מספר 1. בזמן 2 פקטות 2,3. בזמן 3 פקטות 4,5,6,7. כלומר פקטה 6 נשלחה בזמן 3. כך מגיעים לכל התשובות: 33 בסיבוב 6, 40 בסיבוב 6, 70 בסיבוב 8.

שאלה 5

בהינתן הגרף הבא:



איפה ה-ssthresh מתחיל? הערך ההתחלתי שלו הוא 8.

איפה קיבלנו timeout? בזמן 19, בזמן 26.

כמה duplicate ACKs קיבלנו? בזמן 9, בזמן 14.

כמה ערכים שונים יש ל-ssthresh? המספר מתעדכן בכל מקום שאנחנו רואים "נפילה" (כי ssthresh יתעדכן יחד עם גודל החלון). יש 4 נפילות, בתוספת הערך ההתחלתי = סה"כ 5 ערכים שונים.

Network Address Translation (NAT)

דיברנו שכשאנחנו רוצים לדבר עם מישהו אחר נצטרך לשלוח מעל IP ולהעביר דרך שכבות וכו', הסברנו את הסיבוך הכרוך בזה אבל שאנחנו אוהבים את השיטה הזו בכל זאת כי זה נוח ששינוי בשכבה אחת לא משפיע על השכבות האחרות.

כאן אנחנו נשבור את המודל הזה, ונשנה את התוכן של פקטה מתחתיו. הסיבה: בעבר כשהמציאו את האינטרנט היו מלא כתובות IP, אבל בהתחלה תפסו נתחים מהכתובות בצורה לא פרופורציונלית והיה צורך להתגבר על זה.

כשיש הרבה אנשים באותו רשת, לראוטר יש IP אמיתי, וכל מי שמתחתיו לא יהיה לו IP אמיתי, הכתובת של המחשבים בפנים יקבלו כתובת שבעצם הראוטר הוא זה שיודע לתרגם. זה עוזר קצת באבטחה, כי אתה לא יודע עם איזה מחשב דיברת (רק port), אבל הסיבה העיקרית היא לנצל כתובות IP.

Reserved IP Ranges

דיברנו לפני כמה תרגולים על רשת פנימית וחיצונית. כתובות IP פנימיות יהיו בשימוש רק בתוך ה-LAN, ולא בשימוש על גבי האינטרנט. לראוטר תהיה כתובת חיצונית, וכשאנחנו נשלח הודעה למחשב חיצוני, המחשב החיצוני יכיר רק את הכתובת של הראוטר שלך – ולא את הכתובת האמיתית שלך.

התרגום מתבצע בעזרת טבלה שבה מופיע source IP ו-source Port חיצוניים ופנימיים:

4L Protocol	Outside network		Inside private network	
	4Translated L port	Translate d IP	4Original L port	Local IP

הרשת החיצונית רואה IP & port, והרשת הפנימית רואה IP & port. את מספר ה-port שהראוטר מראה כלפי חוץ הוא פשוט המציא כדי לזכור איך לנתב בתוך הרשת שלו.

כל פקטה שמגיעה מבחוץ עוברת לראוטר, הראוטר מתרגם לפי הטבלה ומנתב ל-IP הפנימי.

שאלה

יש 3 מחשבים בתוך רשת, ונתונים ה-IP הפנימיים שלהם: 192.168.0.1, 192.168.0.2, 192.168.0.3 עבור המחשבים A, B ו-C בהתאמה. לכל אחד יש אפליקציות שהם רוצים להריץ. גם אם יש חפיפה בין ה-ports זה תקין. נתונה גם כתובת ה-IP החיצונית של הרשת: 132.58.58.2.

Question Consider a LAN with 3 computers: A, B and C.

- Computer A's IP is **192.168.0.1** and it runs two applications communicating with the internet with ports **1024** and **2000** over **TCP**
- Computer B's IP is **192.168.0.2** and it runs two applications communicating with the internet with ports **1000** and **2000** over **UDP**
- Computer C's IP is **192.168.0.3** and it runs one application communicating with the internet with port **1024** over **TCP**
- The network's assigned public IP is **132.58.38.2**

המטרה: למלא את טבלת ה-NAT כאשר ה-port שאנחנו מתרגמים מתחילים מ-5001:

4L Protocol	Outside network		Inside private network	
	Translated L4 port	Translated IP	4Original L port	Local IP
TCP	5001	132.58.38.2	1024	192.168.0.1
TCP	5002	132.58.38.2	2000	192.168.0.1
UDP	5003	132.58.38.2	1000	192.168.0.2
UDP	5004	132.58.38.2	2000	192.168.0.2
TCP	5005	132.58.38.2	1024	192.168.0.3

הפורטים הכתובים 5001, 5002 הם פיקטיביים ורק כלפי חוץ. מספר ה-port שמשודר כלפי חוץ עוזר לשרת ה-NAT לדעת לאיזו כתובת IP פנימית לנתב את המידע. בדוגמה הזאת ה-IP translated שמוצג החוצה אותו דבר, אבל זה לא בהכרח יהיה המצב תמיד.

הרצאה 12

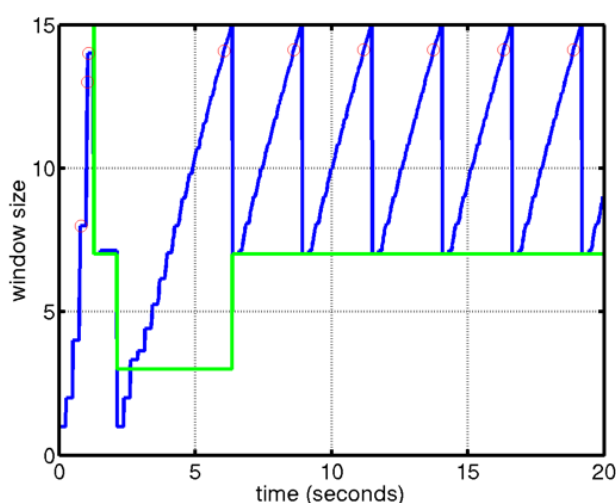
תזכורת

בהרצאה הקודמת דיברנו על Reno ועל איך הוא פותר את הבעיה שהייתה ב-Tahoe. בכל פעם ש-Tahoe זיהה הודעה שהלכה לאיבוד הוא נכנס חזרה ל-SS ללא תלות בסיבה לאובדן הבעיה. כאמור, TCP Reno מוסיף fast recovery שאומר שאם קיבלנו 3 duplicate ACKs נפעל אחרת מאשר כשהיה timeout.

מצגת 6 – TCP
שקף 55

Congestion Avoidance

Avoiding Slow Start with TCP Reno



בכחול אנחנו רואים את גודל החלון, ובירוק את ה-ss thresh. כל עיגול ירוק הוא אירוע וננסה להבין מהו לפי התנהגות הקווים. בהתחלה גידול אקספוננציאלי, 1 ואז 2 ואז 4, 8, ... מגיעים 3 אירועים ואז קצב השידור נחתך בחצי וה-ss thresh מגיע לאותו גובה: אנחנו מבינים שמדובר ב-3 ACK כפולים.

הקפיצה הבאה למטה היא timeout, ss thresh נחתך בחצי אבל אחריה שוב עליה אקספוננציאלית בגודל החלון, כלומר הגענו שוב ל-SS. בהמשך האירועים הם שוב 3 ACK כפולים – קצב שידור קטן בחצי ו-ss thresh יורד לגודל החלון (כלומר גם חצי מגובה החלון הקודם).

שקף 57

ראינו איך חיבור של TCP Reno מתנהג, אפשר לחשוב מה יהיה ה-throughput הממוצע שלו. מגיעים לזה שהוא מנצל בערך 75% מה-capacity.

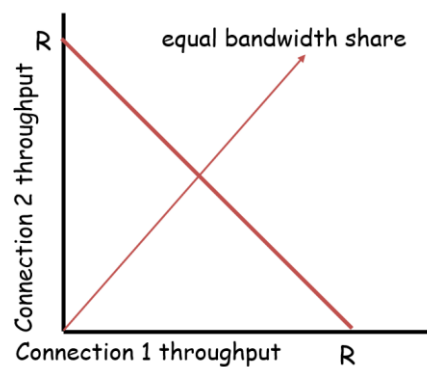
ככה מתנהג vanilla TCP, במציאות אנחנו כן יכולים להגיע למקסימום אבל זה לא בהכרח כי קצב השידור מקסימלי אלא כי יש בתוך הרשת באפרים וראוטרים והם מתמלאים. כשאנחנו חותכים את קצב השידור לחצי יש הודעות ברשת שעדיין מועברות מתוך הבאפרים, 75% זה כאשר הבאפרים ריקים.

שקף 58

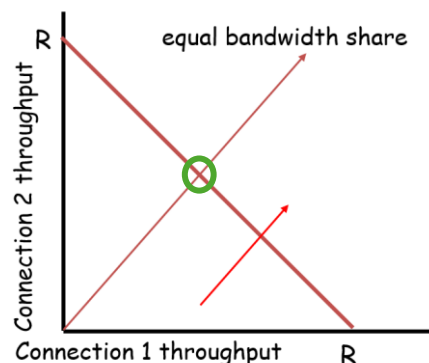
Fairness

אחד הנושאים המרכזיים ב-congestion control הוא נושא ה-fairness. יכול להיות שברשת יש המון שולחים במקביל וכל אחד מחליט את קצב השידור באופן מנותק מהאחרים. אידיאלית, היינו, רוצים חלוקה הוגנת (אם ה-bandwidth הוא R ויש k שולחים ברשת, נרצה קצב של R/k לכל אחד מהם). נזכור שאין סנכרון בין השולחים. מה שיפה ב-Reno הוא שהמנגנון של AIMD בעצם מביא אותנו למצב הוגן, ועבשיו נראה למה.

שקף 59

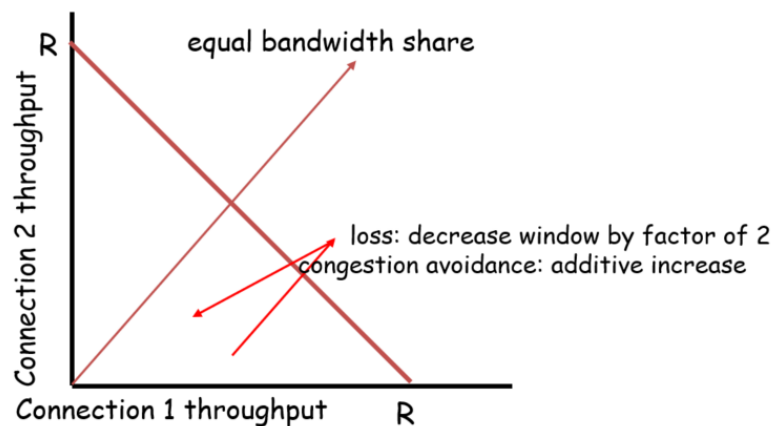


נחשוב על מצב ברשת עם שני שולחים, כל אחד מיוצג בציר (אנכי ואופקי). אם מנצלים את כל ה-BW בערוץ הסכום צריך להיות R , כלומר אנחנו חייבים להיות על הקו האדום העבה. מה קורה לשולחים?

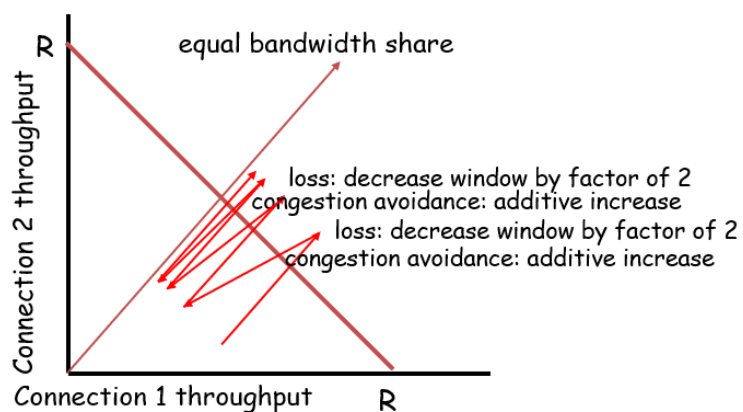


התחלנו מנקודה מסוימת, והיא אומר שהשולח הראשון (שמוצג על ידי ציר x) שולח מהר יותר מהשולח השני (שמוצג על ידי ציר y) כי זזנו יותר על ציר x מאשר על ציר y . אנחנו רוצים להגיע לנקודה החיתוך שמסומנת בעיגול ירוק – שם יש גם שוויון בין השולחים וגם ניצולת מקסימלית.

בשקצב השידור של שולח מספר 1 עולה מעל capacity, שני השולחים יקבלו אירוע של אובדן פקטות וקצב השידור יתחלק ב-2:



קצב השידור שוב יעלה, שוב אובדן, שוב חוצים, ...



כך מקבלים התכנסות למצב ההוגן מאחר שהירידה בקצב השידור כפליית – כי במונחים אבסולוטיים מי שמתנהג בצורה פחות הוגנת ומשדר מהר – יוריד את הקצב שלו יותר מהאחרים.

שקף 61

זה מצב שבו כולם ברשת שולחים באותה דרך, פותחים TCP שמשתמש ב-TCP Reno, אבל באינטרנט זה לא המצב. לפעמים יש שולחים שמשתמשים ב-UDP שם אין congestion control כלל, השולח מחליט את קצב השידור שלו ללא תלות בעומס הרשת.

דבר נוסף, יש לקוחות או שולחים שפותחים כמה חיבורים: למשל דפדפן רוצה להציג למשתמש את התוכן בצורה המהירה ביותר, ואז פותחים 10 חיבורים לשרת ומנסים להביא כמה שיותר, כך מקבלים חתיכה גדולה יותר מערוץ התקשורת המשותף.

שקף 62-63

במשך הרבה שנים הציעו דרכים שונות להמציא CA Mechanisms יותר ויותר טובים. כל טכניקה דורשת איזשהו סיגנל על העומס ברשת, Tahoe ו-Reno מבוססים על פקטות שהולכות לאיבוד, אבל לא תמיד זו הדרך הנכונה (למשל תקשורת לוויינית).

TCP Vegas

וריאנט יחסית מפורסם אבל לא נפוץ, שניסה להשתמש בשינוי ב-latency כאינדיקציה לעומס. לפני שהעמסנו יותר מדי את הרשת אנחנו נראה עלייה ב-latency של החיבור, הודעות שהיינו מקבלים עליהן ACK מהר עכשיו נמצאות ב-queue של הראוטר יותר זמן ויגיעו מאוחר יותר.

TCP Vegas אומר להקטין את החלון גם כשמזהים עלייה מעל ה-RTT באחוז מסוים:

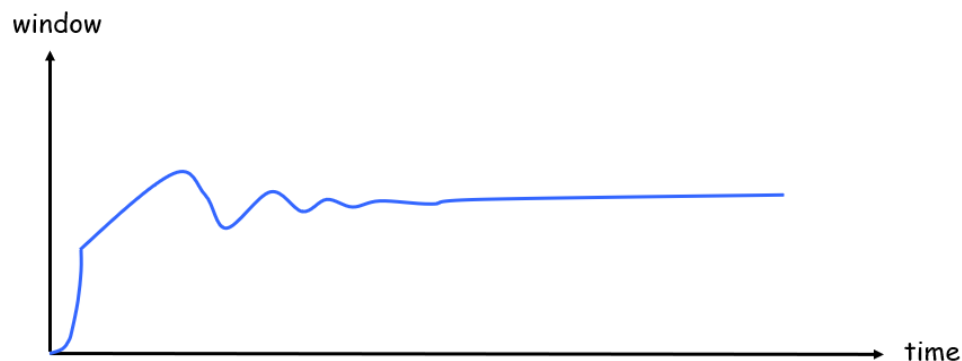
```
for every ACK {
  if  $W/RTT_{min} - W/RTT < \alpha$  then  $W++$ 
  if  $W/RTT_{min} - W/RTT > \beta$  then  $W--$ 
}
for every loss {
   $W = W/2$ 
}
```

זוכרים מה היה ה-RTT המינימלי (האופטימום), ואם הוא יחסית קרוב אליו מגדילים את החלון. אם הוא רחוק ממנו, מקטינים את החלון. זה לכאורה טריק חכם שאמור למנוע מאיתנו לאבד הודעה (שזה בזבזני, לוקח RT שלם לזהות שבכלל הייתה בעיה).

כשניסו אותו, זה לא עבד באינטרנט. האלגוריתם הוא latency based, מוריד את קצב השידור כשיש עליה ב-latency. כל השאר השתמשו ב-TCP Reno, ואכפת לאלגוריתם הזה מ-loss, כלומר כש-Vegas מוריד את קצב השידור מי שהשתמש ב-Reno זיהה שאפשר להעלות את ה-capacity. זה גרם ל-Vegas להוריד את קצב השידור שלו מאוד מהר ו"לפנות מקום" לכל השאר:

שקף 64

בתנאי מעבדה ההתכנסות נראית כך, ללא ירידות בחצי:



Queueing at Routers

שקף 68

מה קורה בראוטר? יש תור של הודעות שהולך ונבנה. כאמור לחלק מהאלגוריתמים אכפת מאובדן של הודעות כמו Reno ולאחרים מ-queueing delay כמו Vegas. אותם ראוטרים מסוגלים לתת אינדיקציה לשולח להוריד את קצב השידור כשהם בעומס, כאשר ה-queue שלו מתחיל להתמלא.

עד עכשיו הפידבק היה רק בין נקודות הקצה, עכשיו נראה טכניקות איך יכול להגיע פידבק מתוך הרשת.

שקף 70

Implicit Feedback

הדרך הפשוטה ביותר – פשוט לא לעשות כלום. כשהתור מלא זורקים את זנב ההודעות – drop-tail, שאין לו מקום. מתישהו השולח יבין שההודעות לא הגיעו בגלל העומס. הראוטר לא עושה שום דבר מיוחד להודיע לשולח ולכן היא implicit.

שקף 71

Active Queue Management

מדבר על שימוש בראוטר כדי לתת פידבק explicit לשולח. כשהתור הולך וגדל הוא יכול לשלוח הודעה לשולח שמתריעה לו שהיא תחת עומס. ב-header של IP יש מקום שמיועד לזה. בדרך כלל הפיצ'ר הזה לא בשימוש. אם כן מפעילים אותו זה בדרך כלל בתוך רשתות מנוהלות ולא על גבי האינטרנט.

מצגת 7 – BGP

Interdomain Routing

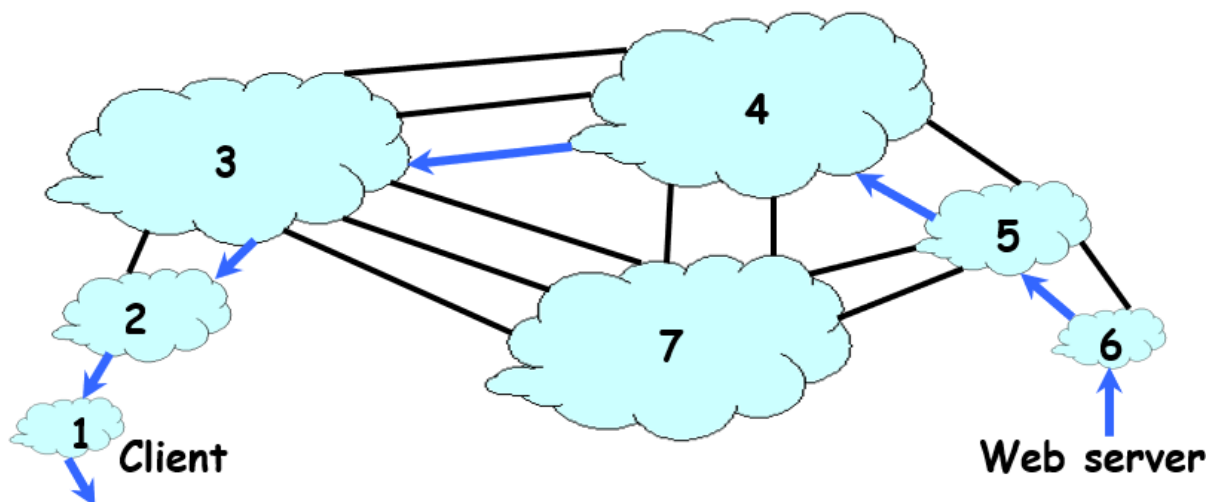
שקף 3

ראינו אלגוריתמים ל-routing בקורס. מה שהיה אינהרנטי בכולם זה שהם כולם ניסו למצוא את המסלול הקצר ביותר ממקום א' ל-ב', וזה מתאים לניתוב תעבורה בתוך רשת מסוימת. האינטרנט כולו מורכב מהרבה רשתות ("רשת של רשתות") שכל אחת מהן יכולה להיות מנוהלת באופן שונה. אנחנו צריכים למצוא מסלול שיביא את ההודעות שלנו מקצה אחד לקצה השני, וכדי למצוא את המסלול הזה אנחנו לא יכולים להסתמך על אלגוריתמים שראינו שלא מתחשבים באלמנטים מסוימים בדרך עליהם נרחיב.

שקף 4

Autonomous Systems

נגדיר את הגופים השונים בדרך. כל רשת נקראת (AS) Autonomous System והיא נקראת כך כי היא פועלת בצורה אוטונומית. בין הרשתות האלו התיאום של הניתוב קורה באמצעות פרוטוקול משותף שנקרא BGP.



בגרף הזה כל ענן מחובר לעננים אחרים, החיבור הוא פיזי אבל גם מתאר יחס עיסוקי. AS מסוים מחובר ל-AS אחר כי יש להם בעולם הפיזי הסכם ביניהם (למשל 5 אולי הספקית של 6, כלומר 6 משלמת ל-5). מבחינת BGP הוא לא עובד כדי למצוא מסלול לכתובות IP בודדות אלא ל-*prefixes*. למשל הכתובת 12.0.0.0/8 מציינת שיש 8 ביטים קבועים, יש מישהו שבלוק הכתובות שלו יוכל להכריז שה-*IP prefix* הזה הוא שלו ב-BGP וכולם צריכים להבין את המסלול איך להגיע ל-*prefix* הזה. כשמישהו שולח הודעה עם *dest IP* ספציפי, אז הניתוב יהיה לפי ה-*prefix* הכי ספציפי שמכיל את אותה כתובת IP. אם אף אחד אחר לא מכריז משהו שהבית העליון שלו הוא 12 הניתוב יהיה לפי מי שאמר שהוא הבעלים של 12/8.

שקף 5

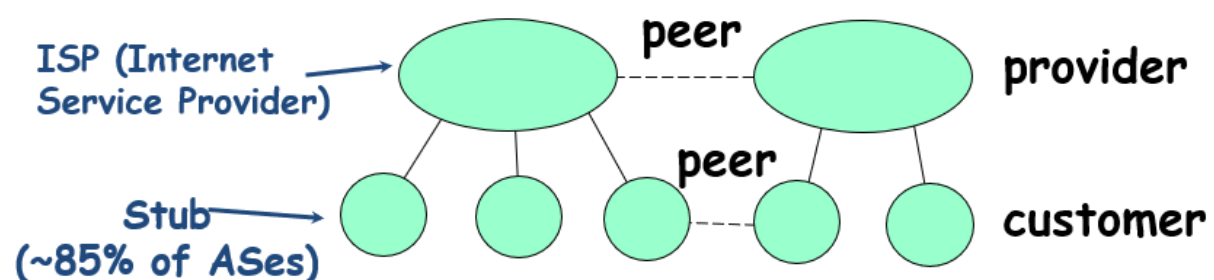
AS Numbers (ASNs)

לכל AS כזה משויך מספר 32-ביט, גם גופים גדולים וגם קטנים מקבלים כזה (הארוורד 11, העברית 378,...).

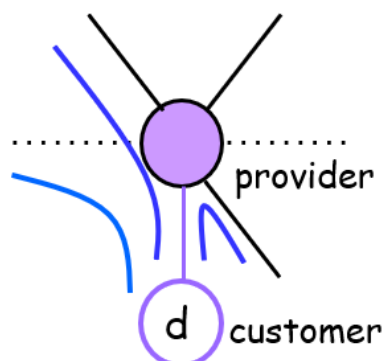
שקף 6

The Commercial Internet

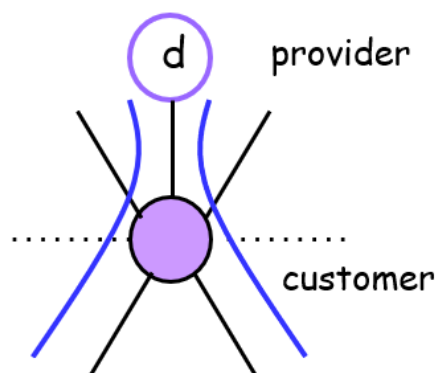
ה-AS חותמים על הסכמים ביניהם שאומרים מי מחובר למי. נוח לנו לסווג את היחסים לשניים: לקוח וספק (AS קטן מחובר למישהו גדול שמעביר את ההודעות שלו החוצה), או שזה שני גופים באותו גודל שמעבירים את התעבורה אחד לשני, אבל יעבירו אחד לשני כמות הודעות בערך זהה, ללא תשלום (כדי ששני הצדדים ירוויחו). השאלה כרגע היא איך הודעה שאנחנו שולחים מנותבת בין הארגונים שמרכיבים את האינטרנט. כמעט כולם באינטרנט הם *stubs* כלומר ללא לקוחות.



Traffic to the customer



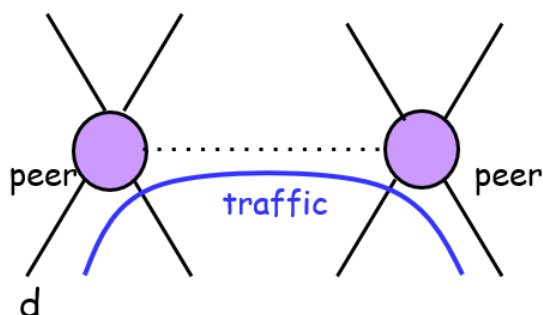
Traffic from the customer



בדוגמה הזו יש לנו לקוח וספק, והלקוח משלם לספק כדי שיחבר אותו לאינטרנט. הספקית אחראית להעביר אליו תעבורה שממוענת לו, וגם להעביר הלאה את מה שהוא מעוניין לשלוח.

Peering Relationship

Traffic to/from the peer and its customers



זו אופציה נוספת, לינק בין peers. הספקיות מחוברות בהסכם ביניהן בלי לשלם, ומרוויחות מהלקוחות שמעבירים ביניהם.

Tier 1 ISPs

נהוג לסווג את ה-AS באינטרנט לפי מבנה היררכי שנוצר מאותם יחסים. אם מישהו לקוח של איזשהי ספקית, בסוף בסוף יש כמה גופים שאין להם אף ספקית, והם נקראים Tier 1 (מחוברים בהסכמי peer בינם לבין עצמם). אין הרבה כאלה, בין 10 ל-20 (למשל AT&T).

Tier 2 ISPs

מתחתיהם יש providers שהם מצד אחד לקוחות של מישהו אחר, אבל גם להם יש לקוחות משל עצמם שמשלמים על תעבורה.

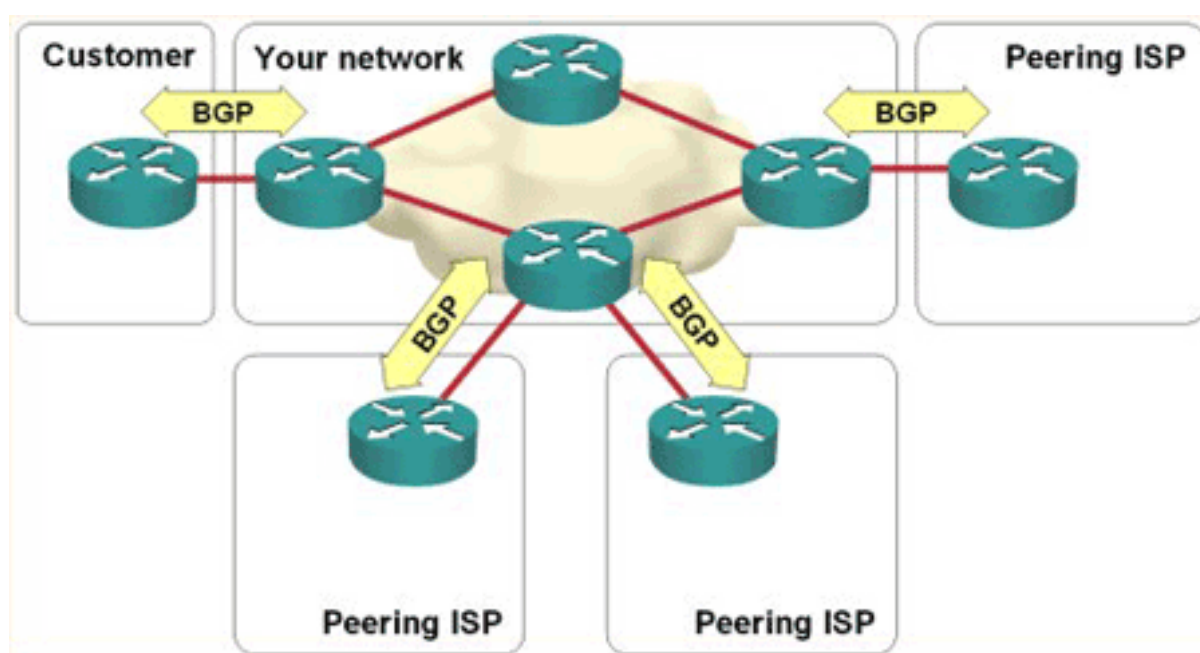
Stub ASes

בקצה נמצאים גופים שהם לא ספקית של אף אחד. יש רשתות קטנות (אוניברסיטה למשל היא stub) אבל יש גם חברות משמעותיות שהעסק שלהן הוא לא להעביר תוכן של אחרים אלא לספק תוכן, כמו למשל Google.

שקף 12

Interdomain Routing

נתמקד בניתוב בתוך הרשת. הנה דוגמה לרשת שבתוכה ראוטרים:



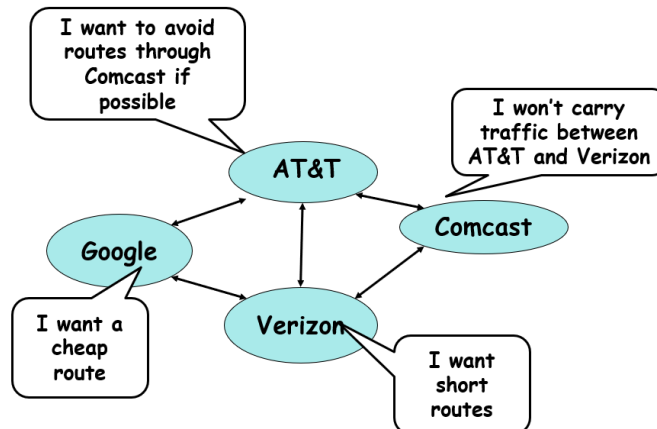
כל רשת מריצה אלגוריתם בעצמה כדי להחליט איך לנתב הודעות בתוך הרשת, אבל הרשת שלי מחוברת לרשתות אחרות על ידי border router – רגל אחת שלו מחוברת לארגון שלי ורגל אחרת לארגון אחר. כל אותם border router מדברים בפרוטוקול שנקרא BGP שנועד לקבוע את הניתוב ברמת ה-AS.

שקף 13

נעשה הפרדה: Intradomain – כשדיברנו על ניתוב עד עכשיו לזה התכווננו, כלומר בתוך אותו AS. עכשיו נדבר על ניתוב Interdomain, כלומר בין AS שונים.

כשמדברים על Interdomain Routing אכפת לנו להתחשב בדברים ש-AS מסוים ירצה לעשות בניתוב שלו, וזה מושפע משכניו. כדי שהאינטרנט יתקיים, חייב שהפרוטוקול שנשתמש בו לניתוב יוכל לתת לנו את המידע שיאפשר לנו לקבוע סדר עדיפויות במסלולים (לאו דווקא האחד עם הכי פחות ראטרים בדרך).

נקודה חשובה היא ש-BGP **לא אומר** ניתוב בדרך הקצרה ביותר. לכל אחת יש רצונות משלו:

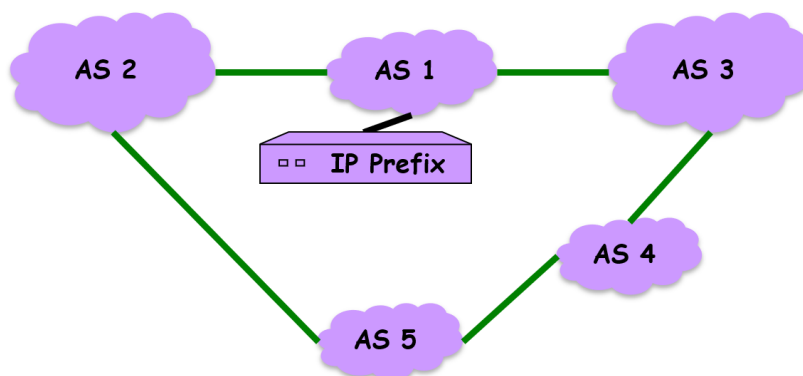


אנחנו צריכים שהפרוטוקול שאחראי על ניתוב בין AS-ים יוכל לתמוך ברצונות, וזה מה ש-BGP יעשה.

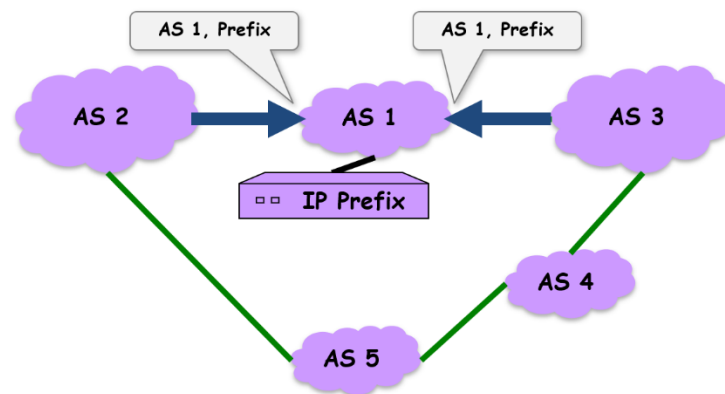
Border Gateway Protocol – BGP

BGP אחראי על הניתוב באינטרנט, נחשב לדבק של האינטרנט כי מחייב גופים שונים להשתמש באותו הפרוטוקול.

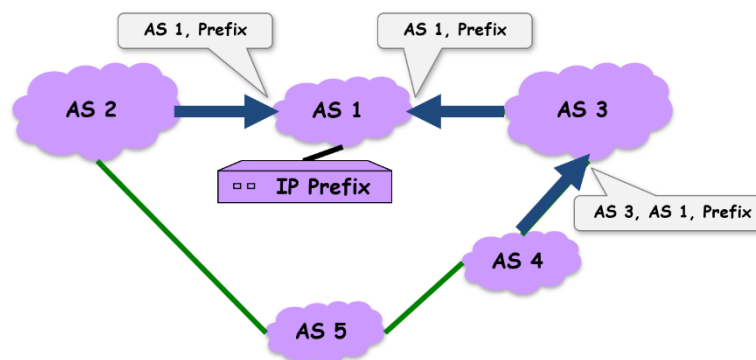
האינטרנט מחולק ל-AS שמזהים על ידי ASN. AS הוא הגוף האוטונומי, והוא יכול להיות בעלים של IP Prefix (כחלק מרכוש הארגון). בדוגמה הבאה ל-AS 1 יש פרפיקס משלו:



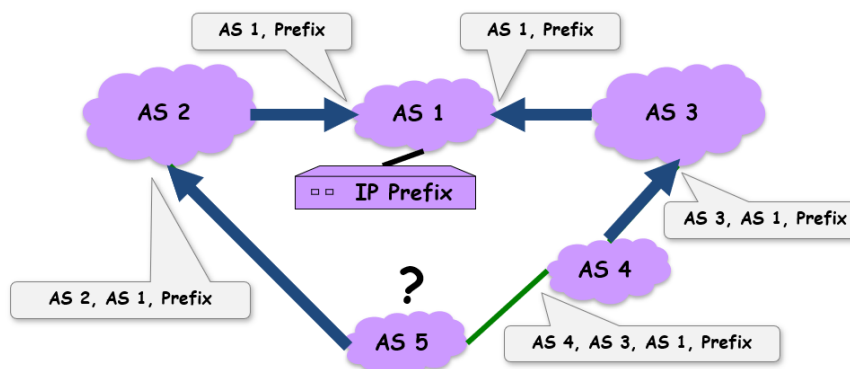
אז ה-border router של אותו ארגון יכולים להכריז את אותו פרפיקס, כלומר AS1 שולח לשכנים שלו דרך ה-border routers שלו מסלול שאומר שבדי להגיע ל-IP Prefix שלי תגיעו לכאן. אותם שכנים מקבלים את המסלול וצריכים להחליט מה לעשות איתו (לדחות אותו, לקבל אותו ולהחליט להשתמש במסלול שקיבלו ממשיהו אחר, להשתמש בו ולשלוח לחלק מהשכנים שלהם...) התהליך הזה קורה hop-by-hop וכל AS בצורה בלתי תלויה באחרות מפעיל את ה-policy שלו על מה לעשות עם הפרסומים שהוא מקבל.



במקרה הזה, AS2 ו-AS3 החליטו להשתמש במסלול הזה כדי לשלוח הודעות שלהן לפרפיקס הזה. כשמישהו מ-AS3 ישלח הודעה שה-IP dest שלה נמצא בתוך הפרפיקס הזה הן יעברו ל-AS1. בנוסף, נראה עכשיו ש-AS3 החליט לשלוח את הפרסום גם לשכן שלו AS4 – והוא בתורו מקבל את המסלול:



עכשיו גם 2 וגם 4 מחליטים לפרסם הלאה את אותו פרסום ל-5. כל אחד מוסיף את עצמו למסלול:

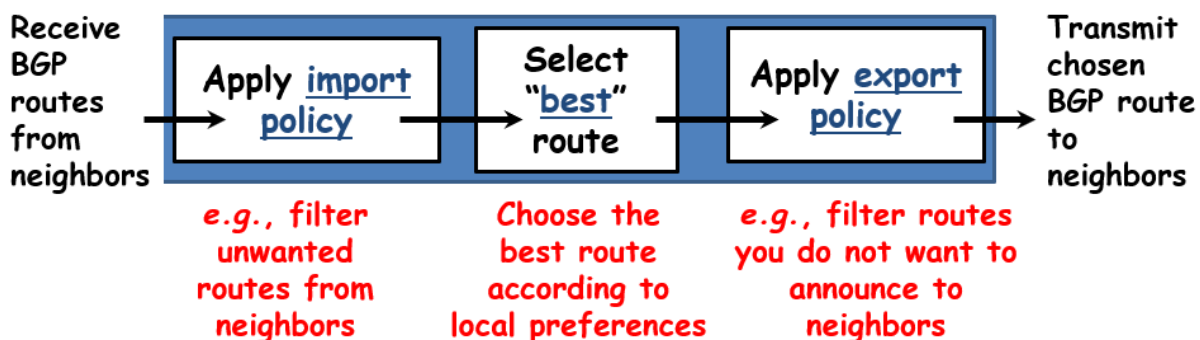


כש-5 מגיע לקבל את ההחלטה הוא רואה את כל הווקטור, לא רק את מספר ה-hops (כמו בפרוטוקולים שראינו ב-intradomain, בתוך AS). זה נותן לנו אפשרות להפעיל פוליסות יותר מורכבות. 5 קיבל שני פרסומים וצריך להחליט לאיפה לשלוח traffic שממוען לאותו prefix. אי אפשר לדעת ללא קונטקסט מה 5 יבחר – זה תלוי באילוצים שלו.

שקף 18

כשראוטר מקבל פרסום על נתיב חדש ב-BGP הוא צריך להחליט מה לעשות איתו, וזה קורה בכמה שלבים:

1. **מפעיל import policy** – אם יש פרסום שעובר דרך AS שהוא בוחר לא לעבור דרכו ("התעבורה שלי לא תעבור במתחרה/מדינת אויב..."). זה מתאפשר בזכות העובדה ש-BGP חושף את כל המסלול.
2. **בוחר מסלול הכי טוב מבין מה שהוא קיבל לאותו IP Prefix** – יכול להיות מצב שקיבלנו מסלול ל-prefix אחד, ועוד מסלול ל-prefix שמוכל בראשון נשמור את שניהם. אבל, אם קיבלנו שני נתיבים לאותו prefix נבחר מה עדיף לנו והוא יהיה המסלול הפעיל.
3. **מחליט איך לנתב את הפרסום הלאה** – יכול להיות שיחליט לשלוח לכל השכנים או לחלקם (אם למשל הם לקוחות שלו). יכול להיות גם שלא ישלח לאף אחד כי אין לקוחות והשכן היחיד האחר הוא provider (ואז אם נפרסם משהו שקיבלנו מ-provider 1 ל-provider 2, הם יכולים להחליט להעביר דברים דרכי בשני הכיוונים – וזה קורה לפעמים בטעות).



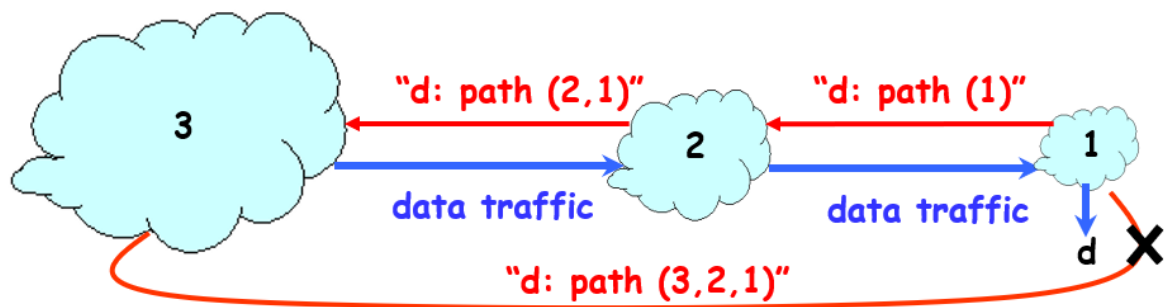
שקף 19-21

נקודה מרכזית I

בניתוב BGP עובדים ב-IP Prefix. נתב לא מקבל הודעה איך להגיע לכתובת מסוימת אלא אל IP Prefix. נחשיב את זה שיש לנו שני מסלולים לאותו prefix אם שניהם מגיעים בדיוק לאותו prefix ולא "מוכל ב". אם מסלול לאחד ומסלול נוסף לאחד שמכיל אותו – נשמור את שניהם. ההחלטה לנתב לכל prefix מתרחשת באופן בלתי תלוי באחרים. אם קיבלנו 16/ וגם 4/ שמוכל בו אפשר להחליט על hops שונים.

נקודה מרכזית II

בניתוב שלנו אנחנו רואים את כל המסלול. כשראוטר BGP רוצה לקבל החלטה הוא מודע לכל המסלול שההודעה שהוא ישלח יעבור בה. הוא מחליט מה ה-hop הבא, אבל יודע מה קורה גם הלאה (שונה ממה שראינו עד כה).



1 מכריז שהוא הבעלים של prefix d, הוא רוצה לקבל הודעות IP שה-IP שלהן הוא כתובת בתוך ה-prefix הזה, ולכן מכריז את זה ב-BGP. 2 עושה את אותו הדבר, מחליט להעביר ל-3 את ההודעה ואומר לו שהמסלול ל-1 עובר דרכו.

כשמישהו שולח הודעת IP שה-IP שלה נמצא בתוך ה-prefix d, אם הוא היה מתוך AS3 הנתב שלו יבין: "זה נמצא בתוך ה-prefix d שה-hop הבא שלי אליו הוא 2". בכחול אנחנו רואים איך ה-traffic עובר, וזה בכיוון הנגדי להודעות ה-BGP.

נקודה מרכזית III

מאחר שאנחנו מודעים לכל נתיב, אפשר לממש דברים מורכבים יחסית כדי לתעדף מסלולים מסוימים מעל אחרים. Node מסוים יכול לתעדף מסלול ארוך יותר כדי להימנע מלהעביר מידע דרך node אחר.

אנחנו יודעים שאפשר ליצור forwarding table ולבחור אחד מבין Distance Vector או Link State, אבל זה בעיקר ברשתות קטנות. נדרש פרוטוקול שיאפשר גישור בין רשתות גדולות.

[The Structure of the Internet](#)

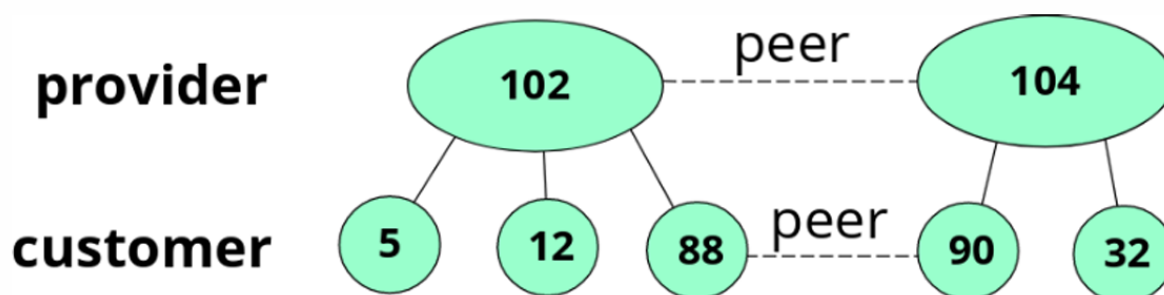
כשמסתכלים על רשתות גדולות יש דברים שצריך להתייחס אליהם כמו מדיניות (לא נרצה להעביר מידע דרך מדינה מסוימת), הסכמים כספיים..

Autonomous System – או בקיצור AS, מדובר על כל רשת כמו למשל בזק, Verizon. כל אחת תהיה מאופיינת על ידי מספר ייחודי, AS Number או בקיצור ASN.

לכל AS יש רצונות משלו (הכי זול, הכי קצר, שלא יעבור דרך מתחרה..). בניגוד למה שלמדנו עד עכשיו בשלקחנו את המסלול הכי מהיר או הכי קצר, כאן השיקולים לבחירת המסלול לא תמיד יהיו ידועים לנו (משתנים בין ASes שונים). לרוב השיקולים יהיו כלכליים.

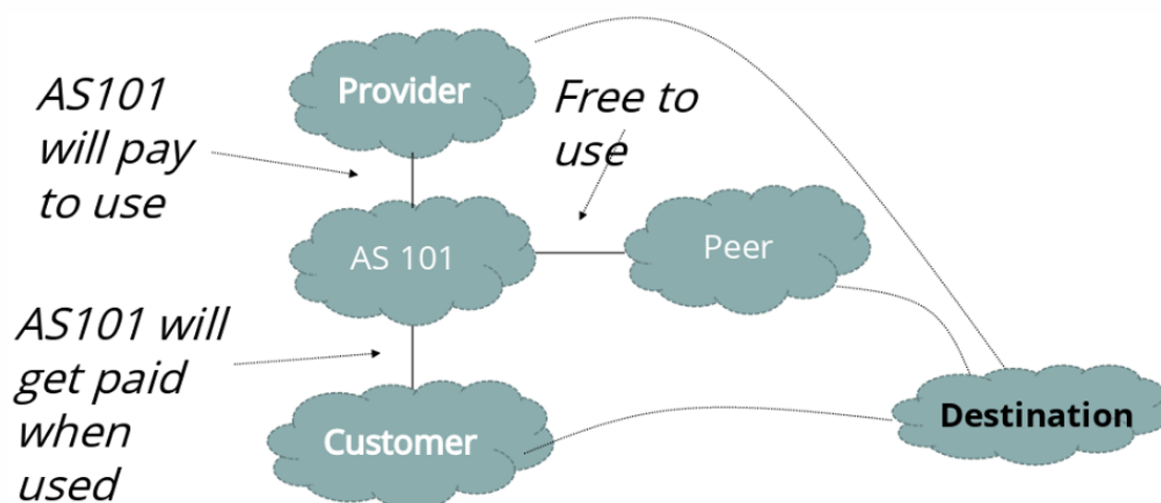
[סוגי קישורים](#)

יש כמה AS והם מחוברים אחד לשני ביחסים שונים:



הם יכולים להיות ביחס **ספק-לקוח**, הלקוח יכול לתת שירות ליחידות קצה. הספק מנתב בין לקוחות שונים.

מצד שני הם יכולים להיות **peers** גם בין ספקים וגם בין לקוחות (כלומר קשר בין משתמשים באותה רמה). מדובר בהסכם שבו הם לא משלמים אחד לשני אבל מסכימים על תעבורת מידע ביניהם. למה זה חינוך בעצם? מדובר באינטרס של שני הצדדים – כדי להיות AS טוב, ספק (כמו בזק למשל) ירצה לספק גישה לכל האינטרנט ולכן יעשה הסכמים גם אם הם לא מניבים לו כסף באופן ישיר. גם אם כבר AS מחובר לכל האינטרנט, להוסיף יתירות עוזר במזעור עומסים ולהיות עמיד בפני תקלות.

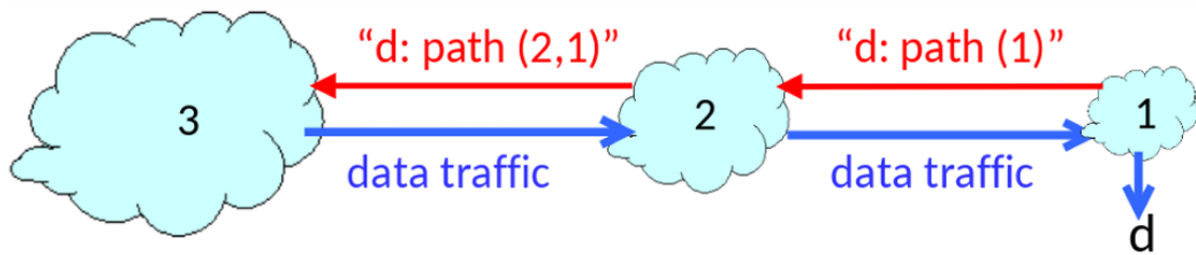


דרגות של AS

1. Tier 1 – יש כ-10 כאלה, כולם בהסכם peer עם כל השאר. אם הודעה מגיעה ל-AS ב-Tier 1, ההודעה תגיע לכל האינטרנט.
 2. Tier 2 – יש כ-1,000 כאלה. ספקים שנותנים שירותים ליחידות קצה ואחרות (אזור גיאוגרפי, מדינות) אבל אין להם הסכם עם כל שאר העולם. הם משלמים ל-AS ב-Tier 1 כדי להיות מקושרים לשאר.
 3. Stub – 85% מה-ASes (למשל רשת האוניברסיטאות). הם קונים שירות מ-Tier 1 או Tier 2 ודרכם מקבל גישה לכל שאר האינטרנט, ויעביר את המידע ליחידות הקצה (לא ל-AS אחרים).
- לבית פרטי למשל, אפשר להתחבר לאינטרנט דרך Tier 2 Stub, וגם דרך Tier 1, הכל תלוי מחיר (בישראל למשל אין אפשרות להתקשר ישירות מול Tier 1).

IP Prefix

כשאנחנו מסתכלים על IP כלשהו $k/2^k$, ה- k זה גודל ה-subnet (בכמה ביטים אפשר לשחק, ככל שיש יותר ביטים יש לנו יותר כתובות להעניק למשתמשים). אם יש לנו IP שנרצה לשלוח אליו מידע, נסתכל איזה AS שייך ל-subnet הזה ונשלח אליו. כשיש כמה אופציות, למשל רוצים לשלוח למישהו שמתחיל ב-240.161 ויש לנו מסלול גם אל 240 וגם אל 240.161, נשתמש במסלול הכי מדויק שזמין לנו.

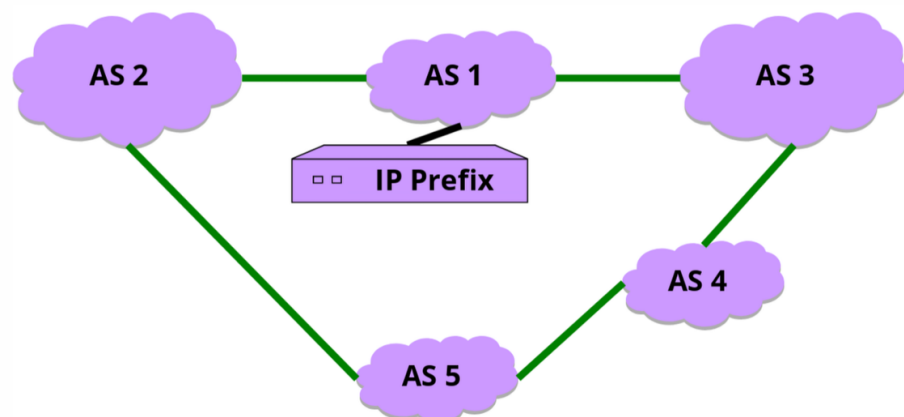


בפרוטוקולים קודמים ראינו בטבלת המיפויים רק את הקודקוד הבא (ואת המשך הדרך לא זכרנו), כאן אנחנו נשמור את כל הדרך – כדי לאפשר קבלת החלטות לפי אילוצים מורכבים יותר.

תכונות BGP

כדי להגיע ליעד משתמשים ב-BGP, פרוטוקול מבוסס על AS מפרסם את כל ה-IP שאפשר להגיע אליהם דרכו. כל AS מחליט איך לתעדף את המסלולים שלו, יבנה את המסלול (לא רק הקפיצה הבאה – מסלול שלם) ואז יפרסם אותו הלאה, ויעביר לשכנים שלו שאפשר להגיע לאותו IP דרכו באמצעות המסלול שבחר.

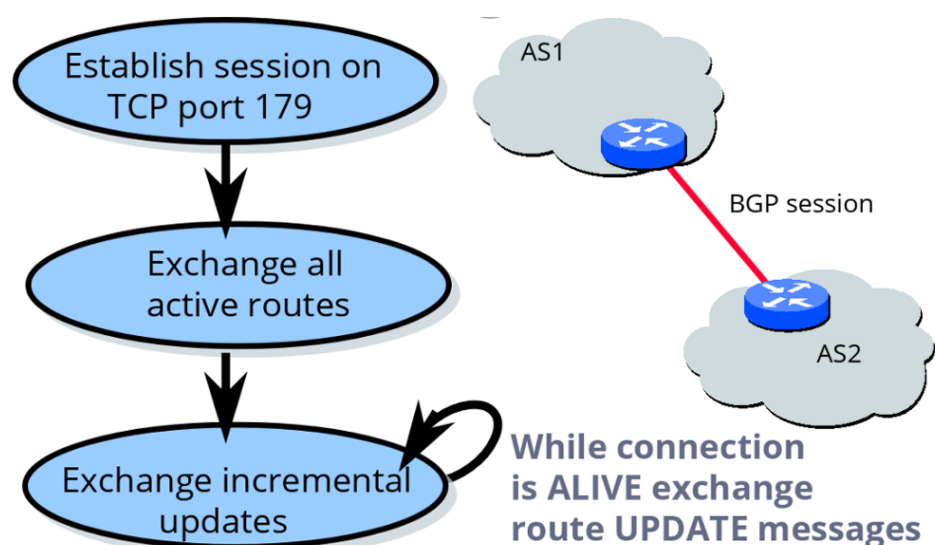
הערה: בין אותן 2 נקודות אפשר לבחור מסלול נכנס ששונה ממסלול יוצא.



נניח שיש לנו באמצע מחשב קצה/ראוטר (כל דבר רק לא AS) שמחובר ל-AS1 (שמחובר בהתאם לשרטוט לשאר). ראו פירוט על דוגמה זו [כאן](#).

דרך פעולת BGP

1. קבלת מסלולי BGP מהשכנים.
2. הפעלת Import Policy (פילטור מסלולים שאנחנו לא מעוניינים לעבור בהם – תלוי דרישות).
3. בחירת מסלול מיטבי (מבין אלו שעברו את הסינון על פי העדפות מקומיות – מודל עסקי).
4. הפעלת Export Policy (לסנן מסלולים שלא מעוניין לפרסם דרכו).
5. העברת מסלולי BGP לשכנים.



יש לנו AS1 ו-AS2 והם רוצים להיות מחוברים ביניהם (למשל בזק ופרטנר). AS1 פותח סשן TCP דרך port 179 (פורט ייעודי ל-BGP). יעביר את המסלולים שהוא מוכן לפרסם, ושני ה-AS ישתפו מסלולים זה עם זה. העניין קורה בצורה איטרטיבית כל כמה זמן ויעדכנו זה את זה במסלולים חדשים.

אפשר לפרסם מסלולים חדשים אם קיבלנו AS חדש שאנחנו מוכנים להעביר דרכו, ואפשר גם להסיר מסלולים (אם למשל היה בהם שיבוש או שנגמר חוזה מסוים).

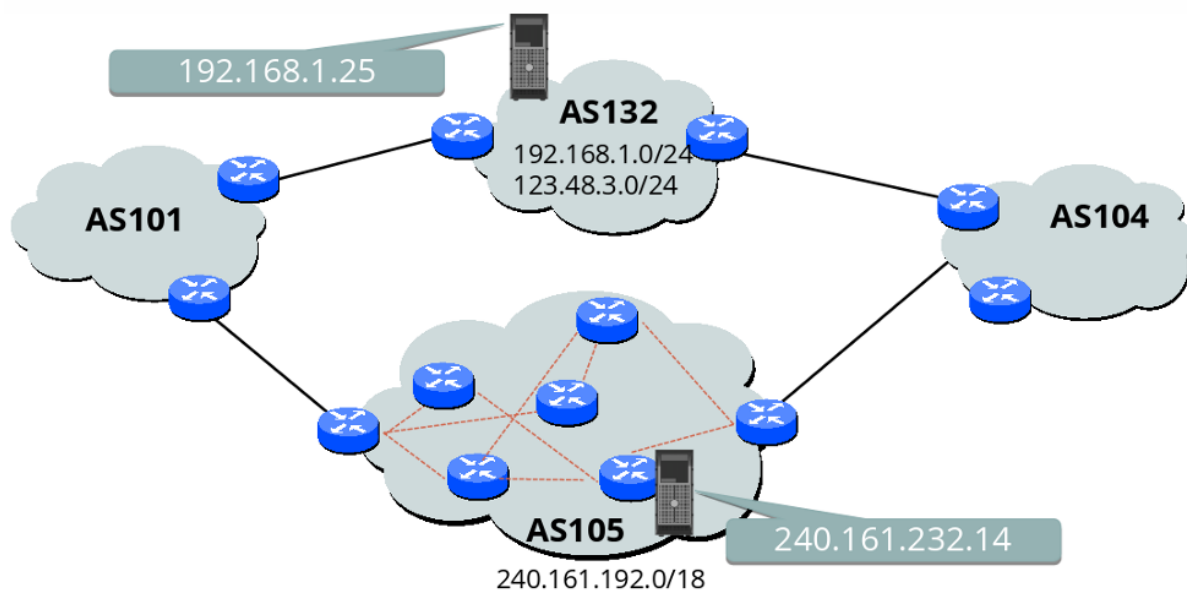
שאלות

[שאלה 1] איך אפשר לוודא שלא יהיו מעגלים?

[תשובה] יש לנו את כל המסלול בניגוד לפרוטוקולים קודמים. אם ה-AS שלנו נמצא על המסלול הזה זה יגרום למעגל ופשוט לא נשתמש בו.

[שאלה 2] האם BGP הוא distance vector או link state?

[תשובה] אף אחד מהם. בשניהם אנחנו מסתכלים רק על next-hop לעומת BGP שבו כל המסלול חשוף. ב-BGP אין צורך באף שיטה לבדיקת מעגלים.



[שאלה] איך נגיע מ-240.161.232.14 אל 192.168.1.25?

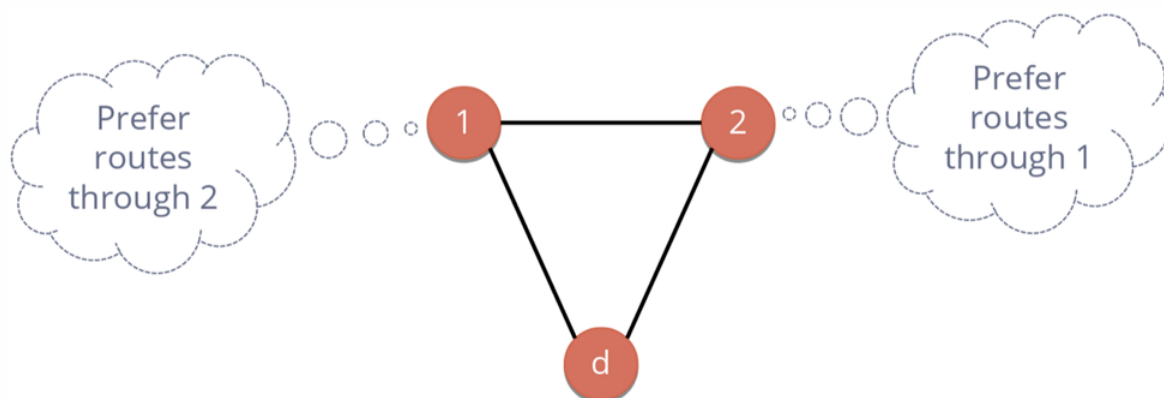
[תשובה] קודם כל, צריך להגיע לאחד מבין edge routers, שהם מחוברים ל-ASes edge routers. לשם כך משתמשים ב-Internal Border Gateway Protocol (iBGP) – הוא מגדיר איך מתקשרים בתוך אותו AS. אחר כך, כדי לצאת החוצה משתמשים ב-Interior Gateway Protocol (IGP), מסתכל על איך להגיע ליחידת קצה של אותה רשת BGP. משם יהיה פרוטוקול BGP כרגיל.

BGP Stability

יכול להיות שיהיה ללקוח כלשהו כמה העדפות, זה לא אומר שנעשה את המסלול שעשינו בפעם שעברה – בדיוק כלל זה נוגע לתחום כלכלי. ככלל, מסלולים יכולים להשתנות (חוזים נגמרים, מסלולים נופלים).

דוגמה

A **stable** BGP state is a state when no AS would like to change its routes

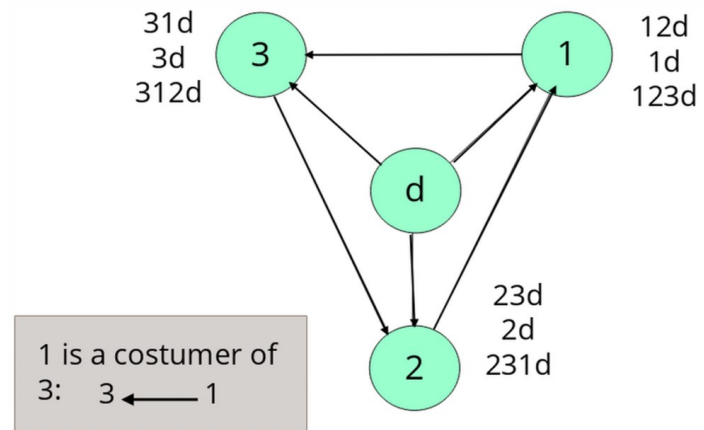


BGP Stability – האם אחרי זמן התייצבות כלשהו יהיה מסלול אחיד שמוסכם על כל האינטרנט כדי להגיע

ממחשב אחד לאחר?

בדוגמה נניח שכל AS מעדיף להעביר את ההודעה שלו ל-d דרך השני. כאן d מפרסם שהוא זמין ואז 2 מודיע ל-1 שיש מסלול דרכו ל-d וכך זה נשאר, אף אחד לא משנה את המסלול שלו, הרשת במצב stable BGP.

האם זה תמיד המצב? לא.



משמעות הסימון של החץ הוא כמו במקרא "1 הוא לקוח של 3". כאן יש 4 ASes (1,2,3,d). כל אחד מפרסם את המסלולים שהוא מעדיף לעשות, והם רשומים ליד כל קודקוד לפי סדר עדיפויות. כאן לא תהיה יציבות.

הוכחה:

נניח בשלילה שיש יציבות. נסתכל על 1:

- אם הוא בחר את 12d אז מכריחים את 2 להיות 2d. אם 2 הוא כזה, אז 3 לא יכול לעשות 31d (כי 1 מעביר ל-2) אז יצטרך לעשות 3d. אבל המסלולים האלה לא יתייצבו. אם נסתכל על 2 הוא מעדיף בעדיפות ראשונה לעשות 23d, והוא גם רואה ש-3 עושה 3d כלומר המסלול הזה אפשרי, ולכן ישנה את המסלול, אבל אז ל-1 שוב תהיה בעיה כי אין יותר מסלול 12d (המסלול הופך להיות 123d וזה כבר בעדיפות נמוכה יותר מ-1d). לכן 1 יבחר את 1d.
- אם בוחרים את 1d אז חוזרים ל-3, הוא יכול לעשות את הבחירה הכי טובה שלו שהיא 31d. שוב חוזרים ל-2, הוא יכול לעשות 2d, ושוב חזרנו ל-1...
- הוכחה דומה מתרחשת אם בוחרים את 123d. אין התייצבות.

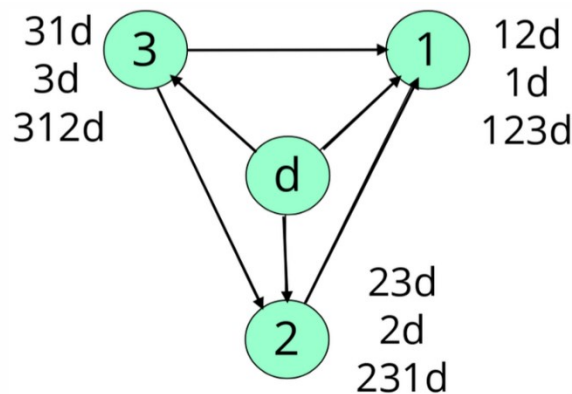
אם כך האינטרנט היה עובד – דברים לא היו עובדים כמו שצריך. לשם כך נדרש פתרון.

BGP Safety

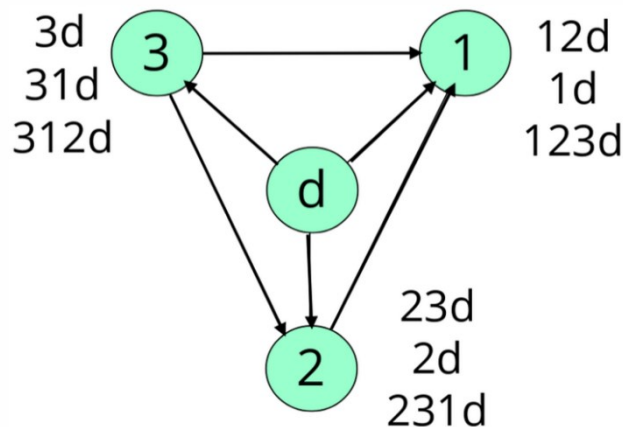
יש משפט שלא נוכיח. אם יש יותר ממצב יציב אחד של BGP, אז הרשת אינה BGP safe.

אפשר להבטיח BGP Safety ברשת אם כל התנאים הבאים מתקיימים:

1. **Topology Condition**: נדרוש שלא יהיה מסלול מעגלי ("A לקוח של B לקוח של C לקוח של A")
 2. **Preference Condition**: לתעדף פרסום מסלולים שעוברים דרך הלקוח שלי, ולא דרך הספק או ה-peer שלי.
 3. **Export Condition**: את המסלולים שלמדנו מהספק ומה-peers נפרסם רק ללקוחות שלנו.
- בדוגמה הקודמת, איזה מהתנאים לא התקיימו? 1 לקוח של 3, 3 לקוח של 2, 2 לקוח של 1 – יש מעגל. נתקן:

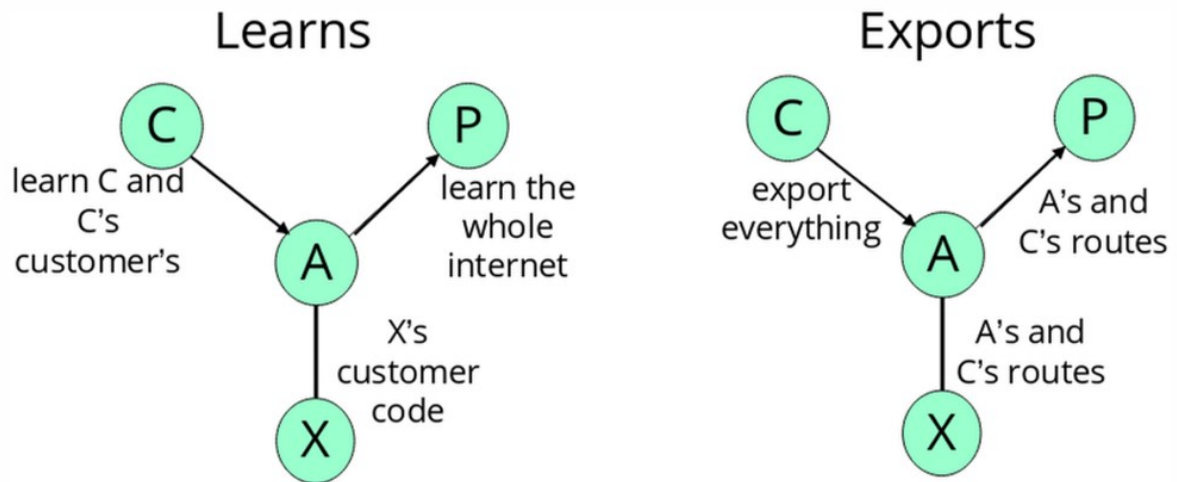


גם כאן יש בעיה: אצל AS3 לא מתקיים תנאי 2, הוא מתעדף מסלול שעובר דרך הספק שלו (1). אז נהפוך שעדיפות ראשונה תהיה 3d ושנייה תהיה 31d:



עכשיו בגלל התנאי השלישי Export Conditions 3 יספר על המסלול רק ללקוחות שלו, כלומר ל-1 ול-2 (הערה: זה מה שאמרו בתרגול, אבל לדעתי זה הפוך מאיך שהגדירו במצגת של התרגול, אמרו $1 \rightarrow 3$ משמעותו 1 לקוח של 3, וכאן החץ הפוך).

2 יבחר את המסלול שהכי טוב לו שזה 23d, ו-1 יבחר את 1d (כי אין מסלול 12d שידוע לו).



באן A הוא AS, X הוא Peer, P הוא ספק, C הוא לקוח.

מה הוא שומע? מ-X הוא לומד על הלקוחות של X, מהלקוח הוא לומד על C והלקוחות שלו, ומהספק הוא שומע על כל הלקוחות מכל העולם.

מה הוא מספר? ללקוח שלו הוא מספר הכל (מספק לו שירות אינטרנט), לספק שלו הוא מספר רק את המסלולים של A ושל C (כדי ש-P לא ינצל אותו ליותר מזה), ול-X הוא מספר על כל המסלולים של A ושל C (גם, כדי ש-X לא ינצל אותו מעבר עם מסלולים דרך P, וזה יעלה ל-A כסף).

הרצאה 13

בשיעור הזה נמשיך את הדיון על BGP, ונסתכל על בעיות בפרוטוקול.

[תזכורת](#)

ראוטר BGP הוא הראוטר שמחבר AS לשאר ה-AS השונים. כשראוטר BGP מקבל הודעה על נתיב מסוים הוא עובר בכמה שלבים, בודק האם המסלול החדש תואם את ה-import policy שלו (שכנים שהוא לא רוצה לקבל מהם מסלולים). אחר כך, מדג את המסלול ביחס למסלולים אחרים שהוא מכיר ובוחר את המסלול הטוב ביותר כדי להעביר את התעבורה שלו ליעד. לבסוף, בוחר האם לייצא את המסלול לשכנים שלו (או חלקם). ראינו 3 נקודות עיקריות.

[מצגת 7](#)

A Closer Look at BGP

[Layers](#)

[שקף 22](#)

זה פרוטוקול שמי שמריץ אותה זה הראוטר, ולכן הוא רץ מעל מודל השכבות, זה אפליקציה שקובעת את הנתיב לפיו המידע יעבור ברשת. כשראוטרים מדברים ב-BGP אנחנו בעצם מדברים בשכבת האפליקציה מעל כל השכבות.

[Incremental Protocol](#)

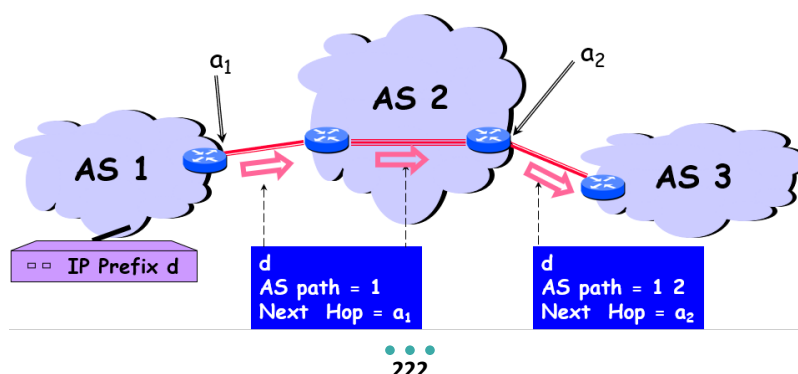
[שקף 23-24](#)

אמרנו ש-BGP רץ מעל TCP, אז צריך לדעת לאן להתחבר. BGP רץ מעל פורט 179, כלומר נשלח SYN לפורט הזה כשנרצה להתחבר. כשהחיבור נוצר הם מחליפים ביניהם את כל ה-export routes שלו. מאותה נקודה נעבוד בצורה incremental, כל ראוטר שמתעדכן לו מסלול יכול להעביר הודעת update, או למשוך מסלול שהוא נתן לאחד השכנים (מכל סיבה).

[BGP Route Update](#)

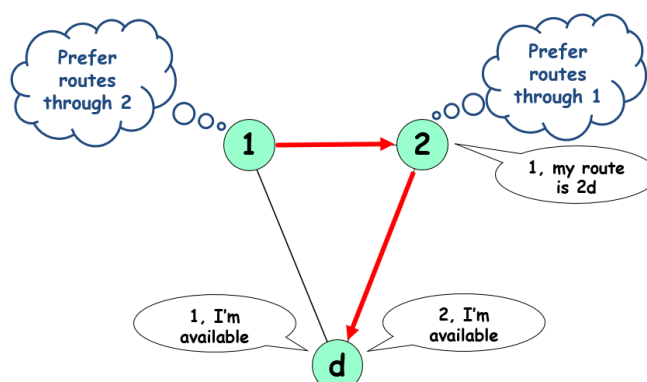
[שקף 25](#)

הודעות BGP update מכילות את ה-prefix, את המסלול AS Path (כלומר רצף AS שמי שמקבל את העדכון יעברו דרכו) ולבסוף next stop IP address. הודעות BGP מחלחלות מהיעד החוצה, אבל כשמסתכלים על הדרך שיש לעשות נצטרך ללכת בדרך אחורה (לעשות reverse).



באן יש מצב פשוט של 3 ASes, ל-1 ול-2 יש העדפות שונות. 1 מעדיף מסלולים שעוברים דרך 2, ו-2 מעדיף מסלולים שעוברים דרך 1. מה יקרה כש-d ישלח עדכוני BGP, לאן המערכת תתכנס?

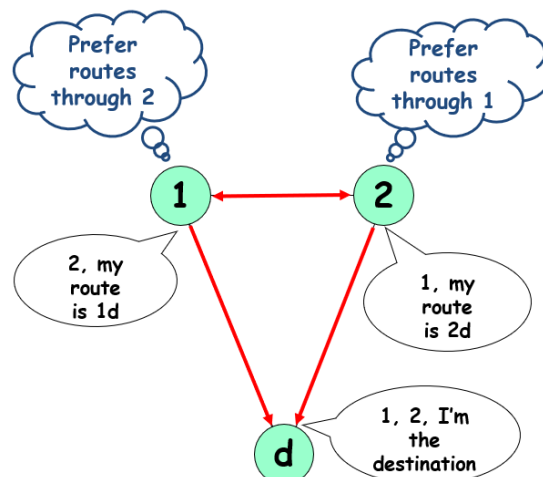
קודם כל d שולח ל-2 שאפשר לעבור דרכו, ואז 2 מעדכן את 1 על המסלול החדש, ולכן גם 1 מעדכן את המסלול שלו. כש-d יגיד ל-1 שאפשר לעבור דרכו, 1 לא ישנה כלום כי הוא מעדיף בכל מקרה לנתב דרך 2 ולכן הרשת תתכנס - הגענו ל-stable state.



אם הנתיבים מתחילים להשתנות זה יפגע לנו בביצועים, אז כן נרצה להגיע ל-stable state.

Instability

נניח ש-d הודיע ל-1 ול-2 באותו הזמן שאפשר לנתב דרכו. עכשיו כל אחד מהם יספר על המסלול שלהם ל-d, ולכן יעדיפו להעביר זה לזה, אבל הגענו למצב בו אף אחד מהם לא מתייצב על מסלול, כל הזמן המסלול ישתנה. זה מקשה על מעקב. יכול להיות שראוטר ה-BGP כל הזמן יחליפו עדכונים, כי הם כל הזמן ילמדו מסלולים חדשים, כלומר ישלחו זה לזה הודעות וזה יוצר עומס. כמו כן, נקבל TCP reordering כל הזמן, וזה יגרום לירידה בביצועים (ב-UDP בכלל הודעות לא יגיעו בסדר הנכון).



שקף 31-32

התכונה שאנחנו שואפים לקבל היא BGP Safety - התכנסות המסלולים למצב יציב שלא יזוז ממנה, ובצורה מהירה. יש כמה mechanisms שמטרתן לעזור לרשת להתכנס (אלה פאצ'ים שנועדו להוביל את הרשת להימנע מהמצב שראינו קודם):

1. יתנו עונש לנתב שמשנה את המסלול שלו לעתים תכופות, יעדיפו אותו פחות.
 2. לא רוצים שישלחו עדכונים על אותו prefix לעיתים תכופות. מדובר בהיוריסטיקה שמונעת עומס של הודעות BGP ברשת.
- לא תמיד המנגנונים האלה פעילים, לפעמים אפקטיבי ולפעמים פחות. המציאות היא שהאינטרנט לא במצב רע במובן הזה - המסלולים די יציבים (מסלולים ל-prefixes פופולאריים יציבים בסדר גודל של ימים).

למה האינטרנט יציב?

שקף 33-36

מדובר באובזרבציה. כשניסו לנתח השורה התחתונה שהתקבלה היא בגלל מניעים כלכליים מקבלים מבנה ברשת שגורם ל-BGP להתכנס. לא אמורים להיות מסלולים עם cycle כי זה מצב לא הגיוני כלכלית.

אם הראוטר שלי (בתור AS) מקבל שני מסלולים, נעשה קונפיגורציה שתתעדף לעבור דרך הלקוחות ולא דרך הספקים (כי ללקוח שלי אני לא משלם, לספק כן). אם למדנו מסלול מספק כלשהו, כנראה לא נרצה לייצא אותו לספקים אחרים (כי אז יעבירו את התעבורה דרכי, ואני לקוח שלהם - אשלם פעמיים גם על התעבורה שלי וגם על שלהם).

Gao-Rexford Conditions for BGP Safety

המבנה הזה באינטרנט נוצר מיחסים כלכליים. זה הוביל שני חוקרים Gao ו-Rexford לנסח 3 תנאים שגורמים לרשת להתכנס למצב מוגדר:

1. אין מעגלים של לקוחות וספקים.
2. אעדיף מסלולים שלמדנו מלקוחות על פני מסלולים שלמדנו מספקים או peers.
3. אספר לשכנים שלי על מסלולים שקיבלתי מהספקית או מ-peers רק אם הם לקוחות שלי.

סיכום

שקף 39

ב-BGP אנחנו לומדים את כל המסלול שההודעה שלנו אמורה לעבור, ומשתמשים בזה כדי לקודד לתוך הראוטר סט עדיפויות יחסית מורכב. ראינו ש-BGP לא מבטיח בטיחות, אבל שבפועל זה כן קורה. ראינו תנאים מספיקים שמובילים את הרשת שלנו למצב יציב.

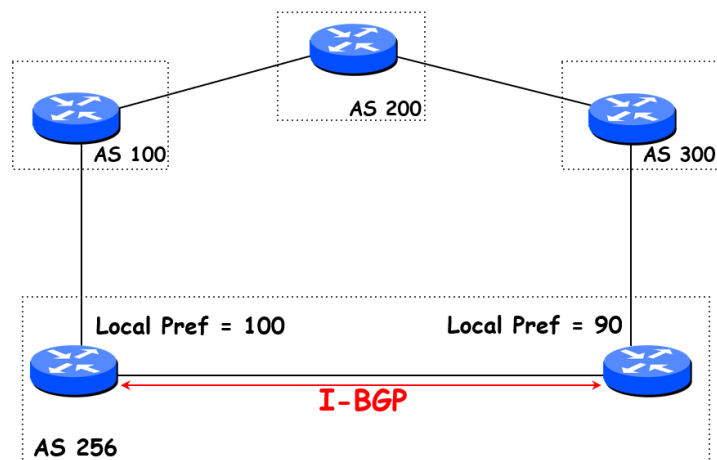
שקף 40-41

החלק שמחבר בין ASes נקרא Inter. אבל בתוך AS הוא לא מיוצג תמיד על ידי ראوتر BGP בודד, אלא יכול להיות מורכב על ידי כמה שמחברים אותו לכל מיני שכנים.

iBGP

שקף 42

ל-BGP יש פרוטוקול iBGP שמטרתו להודיע לשאר ראוטרי ה-BGP של הרשת על המסלול החדש שהתקבל. ל-AS יש ראוטרים פנימיים רגילים ויש לפעמים כמה ראוטרים BGP. המטרה היא לסנכרן את כל ראוטרי ה-BGP בתוך אותו AS.



בציור יש 4 AS שונים (100,200,300) ועוד AS (256) עם שני נתבי BGP שכל אחד מהם מחובר לנתב אחר. האדמיניסטרטור קינפג סדר עדיפויות ביניהם. יתעדפו מסלולים מ-AS 100 על פני AS 300. הנתבים נמצאים באותו AS, אז האדמין מחבר אותם אחד לשני כדי שהם יוכלו להתחבר בסשן iBGP. כשאחד מהם מקבל דיווח על מסלול חדש הוא מעביר את הדיווח ב-iBGP לנתב השני.

[Joining iBGP and IGP information](#)

שקף 43-45

השם הכולל לפרוטוקולי ניתוב סטנדרטיים נקראים IGP, הניתוב הפנימי ל-AS (שם אפשר להשתמש ב-RIP, OSPF וכן הלאה). קודם כל, מטרת העל היא כשמקבלים פרסומי BGP לשייך מסלול ל-prefix מסוים דרך אחד מנתבי ה-BGP. מה שזה אומר - מהי נקודת היציאה מה-AS שלנו לשכן, שדרכו ההודעות ימשיכו לעבור ליעד? זה התפקיד של iBGP.

אם קיבלנו הודעה לאחד מנתבי ה-BGP ואנחנו רוצים להוציא אותה לנתב BGP אחר, בתוך ה-AS הניתוב עומד להיווצר עם אחד מפרוטוקולי הניתוב שכבר ראינו (RIP, OSPF). בתוך הרשת האדמיניסטרטור מקנפג את המשקולות. למשל באיור שני האדומים הם נתבי BGP, והגיע מסלול שאחד מהם יודע שצריך להיות מנותב לשני, הוא יעביר באמצעות iBGP.

שקף 46

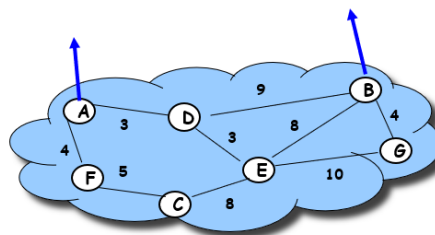
מה התמונה הבוללת? אם אני AS וקיבלתי כל מיני מסלולים מראוטרי BGP שמחברים אותי לשכנים - מאיפה נשלח את המידע? בעצם יכול להיות מצב שאחד מנתבי ה-BGP יש לו מסלול עדיף בגלל local prefs או אורך מסלול מבחינת כמות AS שנדרשים כדי להגיע ליעד.. בדוגמה הזו יש שני נתבים שיש להם מסלולים שקולים.

שקף 47-48

כל אחד ינתב את התעבורה שלו לפי נקודת היציאה הקרובה ביותר על פי ההגדרות של IGP: שני נתבי ה-BGP ידברו ב-iBGP ויסכימו על highest local pref, אם יש אחד כזה נשתמש בו. אחרת, אם קיבלנו תיקו נשתמש ב-shortest AS path-.

• Hot-potato routing:

- Each router selects the closest egress point
- ... based on IGP link weights



סיכום

שקף 49

בעצם AS הוא ארגון עצמאי שהוא מנהל אותה באופן נפרד ויכול להחליט על הפרוטוקולים הפנימיים שלה. הוא יכול להתחבר לשכנים דרך יותר מדריך יציאה אחת על ידי נתבי BGP. באמצעות iBGP נסנכרן את נקודות היציאה האלו, והניתוב בתוך הרשת קורה באמצעות iGP.

BGP Security

שקף 50-51

כמו הרבה פרוטוקולים, BGP לא נוצר בצורה בטוחה - והוא חשוף לבעיות אבטחה.

בעיה #1 - BGP Session Security

שקף 52-55

מידע יכול לעבור דרך תוקף שמשפיע על איך המידע מנותב, או לנתק בין יעדים. אפיק התקיפה הראשון הוא תקיפת חיבור ה-TCP של שני נתבי ה-BGP. אם התוקף מצליח לשנות את המידע על החיבור, הוא ישפיע על הדרך בה הפקטות עוברות.

יש כל מיני דרכים לנצל את זה:

1. להקשיב על תעבורת ה-TCP, להבין איך כל AS עומד לנתב את התעבורה שלו, ואולי ללמוד כך איפה בדאי להקשיב או לפרוס ציוד שיעזור בהמשך.

2. אפשר לזרוק חלק מההודעות, לגרום לפקטות לא להגיע ליעדן.

3. לגרום להם לסגור את החיבור - ב-TCP יש הודעת Reset שאפשר לשלוח כשמישהו נתקל בבעיה ופשוט ללכת (זו לא סגירה בטוחה בניגוד ל-FIN ו-FIN ACK). את הודעת ה-reset אפשר לשלוח גם בלי seq#, ולכן תוקף יוכל להזריק reset ולסגור את החיבור, ובעצם לנתק שני ראוטרי BGP.

הפתרונות מתמקדים בזיהוי אנומליות, אותנטיקציה של הודעות. התקשורת רוב הזמן די צפויה ובזיהוי תקלה אפשר להזעיק אדמיניסטרטור.

בעיה #2 - מניפולציית BGP

שקף 56-57

כשאנחנו מפרסמים מידע ב-BGP, אין אותנטיקציה למידע. כדי להגיע ל-IP prefix מסוים אפשר להגיד שניתן לעבור דרכי, גם אם זה לא נכון. עכשיו יכול להיות יותר מפרסום BGP אחד שמסתובבים באינטרנט, וחלקם לא נכונים.

ליוטיוב יש IP prefix והם מפרסמים אותו ב-BGP. זו דוגמה יחסית מפורסמת: בשנת 2008 יצאה הודעה ממשרד התקשורת הפקיסטני הוציא הנחיה לפיה יש לחסום מספר כתובות IP כשהמטרה הייתה לחסום את יוטיוב פנימית בתוך פקיסטן (קישור לסרטון ספציפי פוגעני). בפועל, אחת ספקיות התקשורת בפקיסטן קינפגה את הראוטרים שלה לא נכון והכריזה כתובת לא נכונה. בניתוב, ה-most specific prefix הוא זה שקובע, וכשהספקית הכריזה את ה-24/ כל התעבורה לכתובות האלה הגיעה אליהם מכל העולם והיה outage של כמה שעות ליוטיוב. BGP לא נותן את הכלים להגן מפני זה, כל אחד מודיע על ה-prefixes ששייכים לו, אבל מה שקרה זה שכל התעבורה הגיעה ל-Pakistan Telecom.

זה נקרא prefix hijacking - מי שאינו הבעלים הלגיטימיים של prefix מסוים מכריז עליו כשלו. כשהוא עושה את זה, יש פרסום מתחרה ב-BGP, ומאחר שב-BGP אין מנגנון אותנטיקציה, אך אחד בדרך לא יכול להבין את מהימנות המידע רק מהודעת ה-BGP, ולכן יעבוד לפי אותם הנחות שהוגדרו לו (למשל לעבור דרך לקוחות קודם וכו'). ההשפעה היא שמקבלים את התעבורה של התוקף על פני הנתיב הנכון.

קשה לדעת במצבים כאלה האם זה misconfiguration או התקפה מכוונת, ולכן גם קשה להפנות אצבע מאשימה.

סיכום: יש פעמים בהן התוקף יכול לעשות announce רק על תת-prefix, הוא יהיה היחיד ויקבל את התעבורה אליו. בדרך הוא יאבד את המסלול אל היעד. ההתקפה הופכת את התוקף למקבל המידע בלי דרך אל היעד

האמיתי. זה מהר מאוד יגרור בעיה בשירות. ההתקפה מנטרלת גם את התוקף, והיא מאוד visible - כולם מאבדים את הקישוריות ליעד.

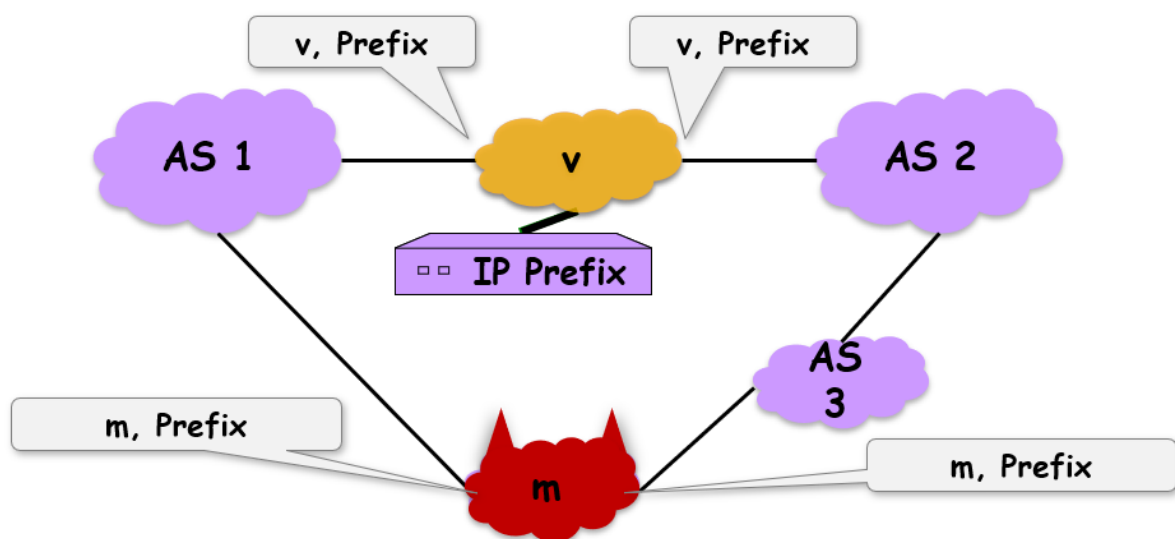
[הגנה מפני prefix hijack](#)

שקף 63-64

ה-best common practice הוא להסתכל על מי שנתן את ההודעה. אם זה לקוח אנחנו לא מצפים ממנו להכריז על שירות תוכן גדול. זה לא חלק מהפרוטוקול, יותר כמו מקרים נקודתיים שלא אמורים לקרות. כל יצרו סט חוקים היוריסטיים שמזערו את הבעיה.

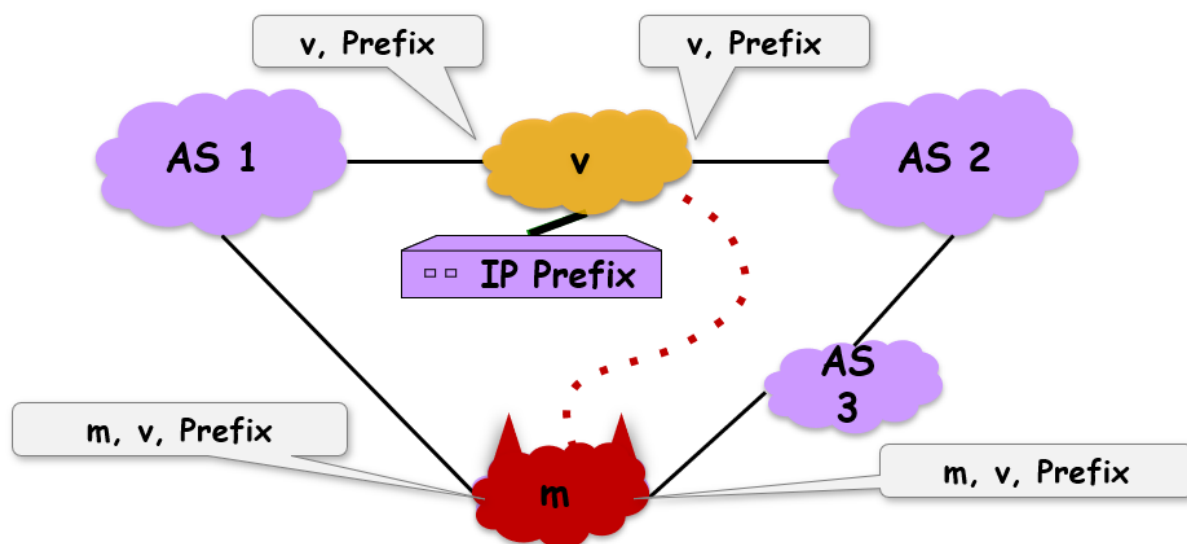
הפתרון האמיתי: **Origin Authenticating**.

יש db שמכיל את הבעלים של כל Prefix. אני אחראי על בלוק כתובות מסוים, ואם מישהו אחר מכריז עליו זה או כי הסכמנו לו להכריז בשמינו, או שצריך לדחות את ההכרזה. כל מי שמשתמש ב-DB כדי לבדוק מסלולים יוכל לדחות את חלקם כשמישהו אחר מפרסם את המסלולים כשייכים לו.



היום הפתרון הזה מיושם. כמעט כל ה-IP prefixes רשומים, וחלק מה-AS התחילו לאכוף, מדובר באימות מקור ה-prefix.

בעיה עם זה היא שזה לא מספיק כדי למנוע את כל ההתקפות. אם התוקף רואה ש-v כבר כתב שהוא הבעלים של prefix ב-db, אז m המרושע יכול לשקר ולהגיד שהוא מחובר ל-v, לפרסם מסלול שלא קיים ואולי להצליח לעבוד על חלק מה-AS, זה נקרא RPKI.



יש פתרונות שהציעו להתמודד עם זה, אבל הפתרונות לא מגנים לגמרי, בבדים, וכוללים רכישה של חומרה חדשה.